



# Guidewire InsuranceSuite™ for Guidewire Cloud

## Data Model Configuration

Release: Latest



© 2026 Guidewire Software, Inc.

For information about Guidewire trademarks, visit <https://www.guidewire.com/legal-notices>.

Guidewire Proprietary & Confidential — DO NOT DISTRIBUTE

Product Name: Guidewire InsuranceSuite for Guidewire Cloud

Product Release: Latest

Document Name: Data Model Configuration

Document Revision: 16-May-2026

# Contents

|                                                                   |           |
|-------------------------------------------------------------------|-----------|
| <b>Support.....</b>                                               | <b>9</b>  |
| <b>Part 1</b>                                                     |           |
| <b>Data model configuration.....</b>                              | <b>11</b> |
| <b>1 Working with the Data Dictionary.....</b>                    | <b>13</b> |
| What is the Data Dictionary?.....                                 | 13        |
| What can you view in the Data Dictionary?.....                    | 14        |
| Using the Data Dictionary.....                                    | 14        |
| Property colors.....                                              | 15        |
| Object attributes.....                                            | 15        |
| Property attributes and tags.....                                 | 16        |
| Entity subtypes.....                                              | 17        |
| Data field types.....                                             | 17        |
| Virtual properties on data entities.....                          | 18        |
| <b>2 The application data model.....</b>                          | <b>19</b> |
| What is the data model?.....                                      | 19        |
| The data model in Guidewire application architecture.....         | 20        |
| The base data model.....                                          | 20        |
| Working with dot notation.....                                    | 20        |
| Overview of data entities.....                                    | 21        |
| Data entity metadata files.....                                   | 21        |
| The extensions properties file.....                               | 23        |
| Working with entity definitions.....                              | 24        |
| Search for an existing entity definition.....                     | 24        |
| Data entities.....                                                | 24        |
| Data entities and the application database.....                   | 25        |
| Application database tables.....                                  | 26        |
| Data objects and scriptability.....                               | 28        |
| XML root elements and related base data object types.....         | 29        |
| <delegate> elements and related data object types.....            | 30        |
| <entity> elements and related data object types.....              | 33        |
| <extension> elements and related data object types.....           | 40        |
| <nonPersistentEntity> elements and related data object types..... | 42        |
| <subtype> elements and related data object types.....             | 43        |
| <viewEntity> elements and related data object types.....          | 46        |
| <viewEntityExtension> elements and related data object types..... | 48        |
| Data entity subelements.....                                      | 49        |
| <array>.....                                                      | 50        |
| <column>.....                                                     | 53        |
| <edgeForeignKey>.....                                             | 58        |
| <events>.....                                                     | 61        |
| <foreignkey>.....                                                 | 62        |
| <fulldescription>.....                                            | 65        |
| <implementsEntity>.....                                           | 65        |

|          |                                                                               |           |
|----------|-------------------------------------------------------------------------------|-----------|
|          | <implementsInterface>.....                                                    | 67        |
|          | <index>.....                                                                  | 67        |
|          | <onetoone>.....                                                               | 69        |
|          | <remove-index>.....                                                           | 70        |
|          | <tag>.....                                                                    | 71        |
|          | <typekey>.....                                                                | 71        |
|          | Interactions of ignoreForEvents attribute with data entity relationships..... | 73        |
| <b>3</b> | <b>Working with associative arrays.....</b>                                   | <b>77</b> |
|          | Overview of associative arrays.....                                           | 77        |
|          | Associative array mapping types.....                                          | 78        |
|          | Scriptability and associative arrays.....                                     | 78        |
|          | Setting array member values.....                                              | 79        |
|          | Subtype mapping associative arrays.....                                       | 79        |
|          | Working with array values by using subtype mapping.....                       | 80        |
|          | Typelist mapping associative arrays.....                                      | 82        |
|          | Working with array values by using typelist mapping.....                      | 83        |
|          | Example: Mapping an entity to entities of a different type.....               | 85        |
|          | Constant mapping associative arrays.....                                      | 86        |
|          | Working with array values by using constant mapping.....                      | 86        |
| <b>4</b> | <b>Modifying the base data model.....</b>                                     | <b>89</b> |
|          | Planning changes to the base data model.....                                  | 89        |
|          | Overview of data model extensions.....                                        | 89        |
|          | Strategies for extending the base data model.....                             | 90        |
|          | What happens if you change the data model?.....                               | 91        |
|          | Naming restrictions for extensions.....                                       | 92        |
|          | Defining a new data entity.....                                               | 92        |
|          | Create a new entity.....                                                      | 93        |
|          | Default properties on a new entity.....                                       | 93        |
|          | Extending a base configuration entity.....                                    | 94        |
|          | Create a new extension file.....                                              | 95        |
|          | Working with attribute and element overrides.....                             | 95        |
|          | Overriding data type attributes.....                                          | 97        |
|          | Overriding nested subelements.....                                            | 98        |
|          | Extending the base data model.....                                            | 98        |
|          | Creating a new delegate object.....                                           | 99        |
|          | Extending a delegate object.....                                              | 101       |
|          | Defining a subtype.....                                                       | 107       |
|          | Defining a reference entity.....                                              | 108       |
|          | Define an entity array.....                                                   | 108       |
|          | Implementing a many-to-many relationship between entity types.....            | 109       |
|          | Extending an existing view entity.....                                        | 110       |
|          | Removing objects from the data model.....                                     | 112       |
|          | Remove an extension entity type.....                                          | 112       |
|          | Remove an extension to an entity type.....                                    | 113       |
|          | Marking personally identifiable information (PII) .....                       | 114       |
|          | Implications of modifying the data model.....                                 | 114       |
|          | Are changes to a data model extension valid?.....                             | 114       |
|          | Writing code to support changes to the data model.....                        | 115       |
|          | Strategies for handling extension removal.....                                | 115       |
|          | Troubleshooting modifications to the data model.....                          | 115       |
|          | Deploying data model changes to the application server.....                   | 116       |

|                                                                                           |            |
|-------------------------------------------------------------------------------------------|------------|
| Entity best practices.....                                                                | 116        |
| Observe entity declaration naming conventions.....                                        | 116        |
| Add a suffix to entity extensions.....                                                    | 117        |
| Use singular words for field names except for arrays.....                                 | 117        |
| Add ID as a suffix to column names for foreign keys.....                                  | 118        |
| <b>5 Example: Creating assignable entities.....</b>                                       | <b>119</b> |
| Creating an assignable extension entity type.....                                         | 119        |
| Step 1: Create extension entity type UserAssignableEntity_Ext.....                        | 119        |
| Step 2: Create assignment class NewAssignableMethodsImpl.....                             | 121        |
| Step 3: Test assignment of your extension entity instance.....                            | 122        |
| Making your extension entity eligible for round-robin assignment.....                     | 123        |
| Step 1: Extend the AssignmentState entity types.....                                      | 124        |
| Step 2: Subclass class AssignmentType.....                                                | 126        |
| Step 3: Create class UserAssignableEntityExtEnhancement.....                              | 127        |
| Step 4: Test round-robin assignment.....                                                  | 128        |
| Creating an assignable extension entity type that uses foreign keys.....                  | 129        |
| Step 1: Create extension entity type TestClaimAssignable_Ext.....                         | 129        |
| Step 2: Add foreign keys to assignable entity types.....                                  | 130        |
| Step 3: Create new assignment type for extension entity type TestClaimAssignable_Ext..... | 131        |
| Step 4: Create enhancement class TestClaimAssignableExtEnhancement.....                   | 132        |
| Step 5: Create delegate class TestClaimAssignableMethodsImpl.....                         | 133        |
| Step 6: Add the necessary permission for the extension entity type.....                   | 134        |
| Step 7: Test assignment of the assignable entity type.....                                | 136        |
| <b>6 Data types.....</b>                                                                  | <b>139</b> |
| Working with data types.....                                                              | 139        |
| Using data types.....                                                                     | 140        |
| Guidewire-reserved data types.....                                                        | 140        |
| Database data types.....                                                                  | 140        |
| Data types and database vendors.....                                                      | 141        |
| Defining a data type for a property.....                                                  | 141        |
| The data types configuration file.....                                                    | 141        |
| <...DataType>.....                                                                        | 142        |
| Deploying modifications to the data types configuration file.....                         | 142        |
| Customizing base configuration data types.....                                            | 143        |
| The Length attribute.....                                                                 | 143        |
| The Precision and Scale attributes.....                                                   | 143        |
| The Validator attribute.....                                                              | 143        |
| List of customizable data types.....                                                      | 144        |
| The data type API.....                                                                    | 145        |
| Retrieving the data type for a property.....                                              | 145        |
| Retrieving a particular data type in Gosu.....                                            | 145        |
| Retrieving a data type reflectively.....                                                  | 146        |
| Using the IDataType methods.....                                                          | 146        |
| Define a new data type.....                                                               | 146        |
| Defining a new tax identification number data type.....                                   | 147        |
| Step 1: Register the data type.....                                                       | 147        |
| Step 2: Implement the IDataTypeDef Interface.....                                         | 148        |
| Step 3: Implement the data type aspect handlers.....                                      | 149        |
| <b>7 Field validation.....</b>                                                            | <b>153</b> |
| Field validator definitions.....                                                          | 153        |
| <FieldValidators>.....                                                                    | 154        |

|          |                                                                           |     |
|----------|---------------------------------------------------------------------------|-----|
|          | <ValidatorDef>.....                                                       | 154 |
|          | Modifying field validators.....                                           | 157 |
|          | Using <column-override> to modify field validation.....                   | 157 |
| <b>8</b> | <b>Working with typelists</b> .....                                       | 159 |
|          | What is a typelist?.....                                                  | 159 |
|          | Terms related to typelists.....                                           | 160 |
|          | Typelists and typecodes.....                                              | 160 |
|          | Typelist definition files.....                                            | 161 |
|          | Different kinds of typelists.....                                         | 162 |
|          | Internal typelists.....                                                   | 162 |
|          | Extendable typelists.....                                                 | 163 |
|          | Custom typelists.....                                                     | 163 |
|          | Working with typelists in Studio.....                                     | 163 |
|          | The typelist editor.....                                                  | 164 |
|          | Entering typecodes.....                                                   | 166 |
|          | Typekey fields.....                                                       | 167 |
|          | Example: Typekey field.....                                               | 168 |
|          | Removing or retiring a typekey.....                                       | 170 |
|          | Remove a typekey.....                                                     | 170 |
|          | Retire a typekey.....                                                     | 170 |
|          | Typelist filters.....                                                     | 171 |
|          | Static filters.....                                                       | 171 |
|          | Create a static filter.....                                               | 172 |
|          | Creating a static filter using categories.....                            | 173 |
|          | Creating a static filter using includes.....                              | 174 |
|          | Creating a static filter using excludes.....                              | 175 |
|          | Dynamic filters.....                                                      | 176 |
|          | Creating a dynamic filter.....                                            | 177 |
|          | Typecode references in Gosu.....                                          | 179 |
|          | Mapping typecodes to external system codes.....                           | 179 |
|          | Qualified and unqualified typelist extensions.....                        | 180 |
|          | Typelist best practices.....                                              | 180 |
|          | Observe typelist naming conventions.....                                  | 181 |
|          | Add a suffix to new typelists and typecode extensions.....                | 181 |
|          | Do not rely on implicit conversion of typecodes to strings.....           | 181 |
| <b>9</b> | <b>Customizing the database upgrade</b> .....                             | 183 |
|          | Creating custom upgrade version checks and version triggers.....          | 183 |
|          | Modifying tables using a custom version trigger.....                      | 184 |
|          | Changing the nullability of subtype columns after a database upgrade..... | 185 |
|          | Version checks.....                                                       | 185 |
|          | Order of execution.....                                                   | 185 |
|          | Create a custom version check or version trigger.....                     | 186 |
|          | IDatamodelUpgrade API examples.....                                       | 187 |
|          | BeforeUpgradeVersionTrigger Structure.....                                | 187 |
|          | AfterUpgradeVersionTrigger Structure.....                                 | 188 |
|          | Altering columns to match the data model using a version trigger.....     | 188 |
|          | Modify a single column using a version trigger.....                       | 188 |
|          | Modify multiple columns using a version trigger.....                      | 189 |
|          | Alter a non-nullable column to nullable using a version trigger.....      | 189 |
|          | Creating columns using a version trigger.....                             | 189 |
|          | Dropping columns using a version trigger.....                             | 191 |

|                                                                            |     |
|----------------------------------------------------------------------------|-----|
| Renaming columns using a version trigger.....                              | 191 |
| Setting a column value for a specific subtype using a version trigger..... | 191 |
| Adding a subtype to an extension entity using a version trigger.....       | 191 |
| Creating tables using a version trigger.....                               | 192 |
| Renaming tables using a version trigger.....                               | 193 |
| Deleting rows using a version trigger.....                                 | 193 |
| Inserting rows using a version trigger.....                                | 193 |
| Inserting data selected from another table using a version trigger.....    | 193 |
| Updating rows using a before upgrade version trigger.....                  | 194 |
| Upgrading archived or quote store entities using a version trigger.....    | 194 |
| Incremental upgrade.....                                                   | 196 |





# Support

For assistance, visit the Guidewire Community.

**Guidewire customers**

<https://community.guidewire.com>

**Guidewire partners**

<https://partner.guidewire.com>



# Data model configuration

The data model is the description of the database representation of all entities and other elements that are part of a Guidewire application.



# Working with the Data Dictionary

Guidewire provides the *Data Dictionary* to help you understand the data model. The *Data Dictionary* is a detailed set of linked documentation in HTML format. These linked HTML pages contain information on all the data entities and typelists that make up the current data model.

## Related concepts

“What is the Data Dictionary?” on page 13

“What can you view in the Data Dictionary?” on page 14

“Using the Data Dictionary” on page 14

## What is the Data Dictionary?

The *Data Dictionary* documents all the entities and typelists in your application installation. Provided that you regenerate it following any customizations to the data model, the dictionary documents both the base data model and your extensions to it. Using the *Data Dictionary*, you can view information about each entity, such as fields and attributes on it.

You must manually generate the *Data Dictionary* after you install the application. Guidewire strongly recommends that you perform this task as part of the installation process. Also, as you extend the data model, it is important that you regenerate the *Data Dictionary* as needed in order to view your extensions to the data model.

To generate the *Data Dictionary*, run the following command from the application installation directory:

```
gwb genDataDictionary
```

The current version of the *Data Dictionary* is stored in the following directory:

```
build/dictionary/data/
```

To view the *Data Dictionary*, open the following file:

```
build/dictionary/data/index.html
```

As an option, you can generate the *Data Dictionary* in XML format with associated XSD files. Use the generated XML and XSD files to import the *Data Dictionary* into third-party database design tools.

## Related concepts

“Regenerating the data and security dictionaries”

## What can you view in the Data Dictionary?

**Note:** If you use a third-party tool to edit configuration files, Guidewire recommends that you work with one that fully supports UTF-8 file encoding. If the editing tool does not handle UTF-8 characters correctly, it can create errors that you then see in the Guidewire *Data Dictionary*. This is not an issue with the *Data Dictionary*. It occurs only if the third-party tool cannot handle UTF-8 values correctly.

After you open the *Data Dictionary* (at `build/dictionary/data/index.html`), Guidewire presents you with multiple choices. You can choose to view either **Data Entities**, **Data Entities (Database View)**, **Data Entities (Migration View)**, or **Typelists**.

The standard, database, and migration views of data entities are similar but not identical. You use each for a different purpose. In general:

- Use the *standard view* to view a full set of entities associated with the application and the columns, typekeys, arrays, and foreign keys associated with each entity.
- Use the *database view* to view all persistent, non-virtual entities associated with the application and the persistent columns, typekeys, arrays, and foreign keys associated with each persistent entity.
- Use the *migration view* to assist you in converting data from a legacy application. This view provides a subset of the information in the standard view of the application entities that is more useful for those working on the conversion of legacy data.
- Use the *typelists view* to view all typelists associated with the application and the typecodes associated with each typelist.

### The migration view of the Data Dictionary

The standard *Data Dictionary* view separates out entity subtypes from the main entity supertype. In brief, a *supertype* relates to a *subtype* in a parent-child relationship. For example, if a **Contact** data entity is the supertype, then **Person** and **Company** are examples of its subtypes. Thus, an entity subtype inherits the characteristics of its supertype and adds individual variations particular to it.

This separation into supertype and subtype is not particularly useful for data conversion (the process of importing data into InsuranceSuite from an external legacy application). Therefore, the migration view of the *Data Dictionary* differs from the standard view in the following respects:

- The migration view displays subtype fields interspersed with supertype fields. For example:
  - `fieldA`
  - `fieldB` (only for subtype `XYX`)
  - `fieldC` (only for subtype `DFG`)
  - `fieldD`
- The migration view does not show virtual fields or virtual arrays.
- The migration view does not show non-loadable columns. For example, it does not show `createUserID` or `createTime`.
- The migration view omits any non-persistent entities.
- The migration view omits entities that are persistent but non-loadable. For example, **Group** is not loadable. Therefore, the migration view does not display it.

### Related concepts

“Using the Data Dictionary” on page 14

## Using the Data Dictionary

You use the *Data Dictionary* to do the following:

- To determine what a field means that you see in a data view definition.
- To see what fields are available to add to a view, or to use in rules, or to export in an integration template, and more.
- To view the list of options for an associated typekey field.

You navigate the dictionary like a web site, with links leading you to associated pages. You can use the **Back** and **Forward** controls of your browser to take you to previously visited pages. Within the *Data Dictionary*, you have the option to navigate to three **Data Entities** views or the **Typelists** view.

If you click on one of the **Data Entities** views, a left-side pane is displayed listing entities in the application. Then, on the right side, a pane is displayed that shows the details of the selected item in the left-side pane.

If you click on the **Typelists** view, a left-side pane is displayed listing all of the typelists in application. Then, on the right-side, a pane is displayed that shows the typecodes for the selected typelist in the left-side pane.

Within the details of an object, you can follow links to related objects or view the allowed values for a typelist.

#### Related concepts

- “Property colors” on page 15
- “Object attributes” on page 15
- “Entity subtypes” on page 17
- “Data field types” on page 17
- “Virtual properties on data entities” on page 18
- “What is a typelist?” on page 159
- “What can you view in the Data Dictionary?” on page 14

## Property colors

An examination of the *Data Dictionary* shows properties in green, blue, and red. These colors have the following meanings:

| Color | Meaning                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Green | <p>The object property is part of the Guidewire base configuration. The object definition file exists in Studio in the following locations:</p> <ul style="list-style-type: none"> <li>• <b>config &gt; configuration &gt; Metadata</b></li> <li>• <b>config &gt; configuration &gt; Extensions</b></li> </ul>                                                                                                                                                                                                                                                                           |
| Blue  | <p>The object property is defined in an extension file, either by Guidewire or as a user customization. The object definition file exists in Studio in the following location:</p> <ul style="list-style-type: none"> <li>• <b>config &gt; configuration &gt; Extensions</b></li> </ul> <p>It is possible for Guidewire to define a base object in the <b>Metadata</b> folder and then to extend the object using an extension entity in the <b>Extensions</b> folder.</p>                                                                                                               |
| Red   | <p>Occasionally, it is possible to see a message in red in the <i>Data Dictionary</i> that states:</p> <p><i>This entity is overwritten by the application during staging.</i></p> <p>This message indicates that the application auto-populates a table or column's staging table equivalent. Do not attempt to populate the table yourself as the loader import process overwrites the staging table during import.</p> <p>See also the description of the <code>overwrittenInStagingTable</code> attribute in “&lt;entity&gt; elements and related data object types” on page 33.</p> |

## Object attributes

An object in the ClaimCenter data model can have a number of special attributes assigned to it. These attributes describe the object (or entity) further. You use the *Data Dictionary* to see what these are. For example, in ClaimCenter the Transaction entity has the attributes **Abstract**, **Editable**, **Extendable**, **Keyed**, **Loadable**, **Sourceable**, **Supertype**, and **Versionable**.

The following list describes the possible attributes:

| Attribute          | Description                                                                                                                                                                                                                                                                                                                                                              |
|--------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Abstract</b>    | The entity is a supertype. However, all instances of it must be one of its subtypes. That is, you cannot instantiate the supertype entity itself. An abstract entity is appropriate if the supertype serves only to collect logic or common fields. The abstract entity is appropriate because, in this case, it is not expedient for the supertype to exist on its own. |
| <b>Editable</b>    | The related database table contains rows that you can edit. An <b>Editable</b> table manages additional fields that track the immediate status of an entity in the table. For example, the table tracks who created the entity, the creation time, who last modified the entity, and the modification time.                                                              |
| <b>Exportable</b>  | <i>Unused.</i>                                                                                                                                                                                                                                                                                                                                                           |
| <b>Extendable</b>  | It is possible to extend the entity with additional custom fields added to it.                                                                                                                                                                                                                                                                                           |
| <b>Final</b>       | It is not possible to subtype this entity. However, you can extend the entity by adding fields to it.                                                                                                                                                                                                                                                                    |
| <b>Keyed</b>       | The entity has a related database table that has a primary key. Each row in a <b>Keyed</b> table has an integer primary key named ID. The application manages these IDs internally, and the application ensures that no two rows in a keyed table have the same ID. You can also associate an external unique identifier with each row in a table.                       |
| <b>Loadable</b>    | It is possible to load the entity through the use of staging tables.                                                                                                                                                                                                                                                                                                     |
| <b>Sourceable</b>  | The entity links to an external source. Each row in a table for a <b>Sourceable</b> entity has additional fields to identify the external application and to store the ID of the <b>Sourceable</b> entity in the external application.                                                                                                                                   |
| <b>Supertype</b>   | The entity has a single table that represents multiple types of entities called subtypes. Each subtype shares application logic and a majority of its fields. Each subtype can also define fields that are particular to it.                                                                                                                                             |
| <b>Temporary</b>   | The entity is a temporary entity created as part of an upgrade or staging table loading. The application deletes the entity after the operation is complete.                                                                                                                                                                                                             |
| <b>Versionable</b> | The entity has a version number that increases every time the entity changes. The application cache uses the version number to determine if updates have been made to an entity.                                                                                                                                                                                         |

To view the definition of a particular attribute, click the small question mark (?) by the attribute name in the attribute list in the *Data Dictionary*.

## Property attributes and tags

### Property attributes

A property in the ClaimCenter data model can have a number of special attributes assigned to it. These attributes describe the property further. You use the *Data Dictionary* to see what these are. For example, the Transaction entity has a property named **Claim**. The attributes of this property are **export as id**, **exportable**, **loadable**, **non-null**, and **writable**.

The following list describes the possible attributes:

| Attribute                      | Description                                                                                                                                                                                                                                                                                                   |
|--------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>database column:</b> [name] | The corresponding database column for the property has a name of [name]. This attribute is present if [name] is a name other than the name of the property.                                                                                                                                                   |
| <b>default:</b> [value]        | This attribute is often present for a property that is associated with a column or a typekey. When the attribute is present, [value] represents the default value for the property.                                                                                                                           |
| <b>exportable</b>              | <i>Unused.</i>                                                                                                                                                                                                                                                                                                |
| <b>export as id</b>            | <i>Unused.</i>                                                                                                                                                                                                                                                                                                |
| <b>loadable</b>                | The property is loadable by way of the staging tables.                                                                                                                                                                                                                                                        |
| <b>localized</b>               | The property contains a <localization> subelement. This subelement causes a localization table to be created during the next database upgrade. A localization table stores localized values for a column for every language other than the default application language. See the <i>Globalization Guide</i> . |



| Attribute                  | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>non-null</b>            | The property cannot contain null values.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>overwritten on load</b> | The property contains an <code>overwrittenInStagingTable</code> attribute with a value of <code>true</code> . InsuranceSuite uses the property and a corresponding staging table during the load of staging table data to operational tables. When the <code>overwrittenInStagingTable</code> attribute has a value of <code>true</code> , the loader import process overwrites the staging table corresponding the property. See “Staging tables” on page 28 and <i>Plugins, Prebuilt Integrations, and SOAP APIs</i> . |
| <b>triggers validation</b> | The property contains a <code>triggersValidation</code> attribute with a value of <code>true</code> . When this scenario exists and an array, a foreign key, or a one-to-one entity on the property changes, InsuranceSuite initiates validation on the property. See “<array>” on page 50, “<foreignkey>” on page 62, and “<onetoone>” on page 69 for more information about the effects of the <code>triggersValidation</code> attribute when arrays, foreign keys, and one-to-one entities change.                    |
| <b>unique index</b>        | The property participates in one or more unique indexes.                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <b>virtual property</b>    | The application creates no physical database column for the property.                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
| <b>writable</b>            | The user can write the property with Gosu or the <code>IDataObjectAPI</code> interface.                                                                                                                                                                                                                                                                                                                                                                                                                                  |

To view the definition of a particular attribute, click the small question mark (?) by the attribute name in the attribute list in the *Data Dictionary*.

### Property tags

A property in the data model can have a number of tags assigned to it. These tags further describe the property.

You use the *Data Dictionary* to see what these tags are. For example, the *Data Dictionary* indicates whether the *PersonalData* obfuscation tag is set on the various fields that use the tag. If the tag has a value, the *Data Dictionary* indicates this value as well. One such value for the *PersonalData* tag is `ObfuscateDefault`.

## Entity subtypes

If you look at *Contact* in the *Data Dictionary*, for example, you see that data dictionary lists a number of subtypes. For certain objects, you can think of the object in several different ways:

- As a generic object. That is, all contacts are similar in many ways.
- As a specific version or subtype of that object. For example, you would want to capture and display different information about companies than about people.

The application creates *Contact* object subtypes by having a base set of shared fields common to all contacts and then extra fields that exist only for the subtype.

The application also looks at the subtype as it decides which fields to show in the application interface. You can check which subtype a contact is by looking at its subtype field (for example, in a Gosu rule or class).

## Data field types

You can use the *Data Dictionary* to view the type of each object field. The following list describes some of the possible field types on an object:

| Type          | Description                                                                                                                                                                                                                                              |
|---------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>array</b>  | Represents a one-to-many relationship. For example, consider the relationship between a contact and a set of addresses. There is no actual column in the database table that maps to the array. The application stores this information in the metadata. |
| <b>column</b> | As the name specifies, it indicates a column in the database. Inside a column field, you can add tag subtypes to point out additional information about the field. The <i>Data Dictionary</i> displays such tags and their values.                       |

| Type                    | Description                                                                                                                                       |
|-------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>foreign key</b>      | References a keyable entity. For example, Policy has a foreign key (AccountID) to the related account on the policy, found in the Account entity. |
| <b>typekey</b>          | Represents a discrete value picked from a particular list, called a typelist.                                                                     |
| <b>virtual property</b> | Indicates a derived property. The application does not store virtual properties in the physical database.                                         |

**Note:** Inside any field of the data model aside from virtual properties, you can add tags and optional corresponding values to point out additional information about the field. The *Data Dictionary* displays such tags and values.

## Virtual properties on data entities

The *Data Dictionary* lists certain entity properties as *virtual*. The application does not store virtual properties in the physical database. Instead, it derives a virtual property through a method. The value returned from the method is typically dynamic, such as a calculation, a concatenation of other fields, or a pointer to the most recently added member of an array.

For example, if you view the Account entity in the *Data Dictionary* (for PolicyCenter), you see the following next to the **AccountContactRoleSubtypes** field:

```
Derived property returning gw.api.database.IQueryResult (virtual property)
```

### Examples

The following examples illustrate some of the various ways that Guidewire applications determine a virtual property. The following examples use Guidewire ClaimCenter for illustration.

#### Virtual property based on a foreign key

Claim.BenefitsDecisionReason is a virtual property that pulls its value from the cc\_claimtext table, which stores ClaimText.ClaimTextType = BenefitsDecisionReason. It returns a mediumtext value. The other fields in cc\_claimtext and cc\_exposuretext work in a similar fashion.

#### Virtual property based on an associated role

Claim.claimant is a virtual property that retrieves the Contact associated with the Claim having the ClaimContactRole of claimant. The virtual property returns a Person value.

#### Virtual property based on a typelist

Contact.PrimaryPhoneValue is a virtual property that calculates its return value based on the value from Contact.PrimaryPhone. It retrieves the telephone number stored in the field represented by that typekey. The telephone number can be one of the following:

- Contact.HomePhone
- Contact.WorkPhone
- Person.CellPhone

It returns a phone value.

# The application data model

The application *data model* comprises the persistent data objects, called *entities*, that the application manages in the database.

## Related concepts

- “What is the data model?” on page 19
- “Overview of data entities” on page 21
- “XML root elements and related base data object types” on page 29

## Related reference

- “Data entity subelements” on page 49

## What is the data model?

At its simplest, the Guidewire data model is a set of XML-formatted metadata definitions of entities and typelists.

---

**Entities** An *entity* defines a set of fields for information. You can add the following kinds of fields to an entity:

- Column
- Type key
- Array
- Foreign key
- Edge foreign key

---

**Typelists** A *typelist* defines a set of code/value pairs, called *typecodes*, that you can specify as the allowable values for the type key fields of entities. Several levels of restriction control what you can modify in typelists:

- **Internal typelists** – You cannot modify internal typelists because the application depends upon them for internal application logic.
- **Extendable typelists** – You can modify this kind of typelist according to its schema definition.
- **Custom typelists** – You can also create custom typelists for use on new fields on existing entities or for use with new entities.

---

The application loads the metadata of the data model on start-up. The loaded metadata instantiates the data model as a collection of tables in the application database. Also, the loaded metadata injects Java and Gosu classes in the application server to provide a programmatic interface to the entities and typelists in the database.

## The data model in Guidewire application architecture

Guidewire applications employ a metadata approach to data objects. The application uses metadata about application domain objects to drive both database persistence objects and the Gosu and Java interfaces to these objects.

This architecture provides enormous power to extend Guidewire application capabilities. Typically, you alter enterprise-level software applications through customization, wherein you change the behavior of the software by editing the code itself. In contrast, a Guidewire application uses XML files that provide default behavior, permissions, and objects in the base configuration. You change the behavior of the application by modifying the base XML files and by creating Gosu business rules, classes, enhancements, and other objects.

### The base data model

The application data model specifies the entities, fields, and other definitions that comprise a default installation of an InsuranceSuite application.

For example, the PolicyCenter data model defines a `PolicyPeriod` entity and several fields on it, such as `AccountNumber`, `IssueDate`, and `Status`.

The application lets you change its data model to accommodate your business needs. You make your changes to the data model by modifying existing XML files and adding new ones. The application stores your files that change the data model in the following application directory:

```
modules/configuration/config/extensions
```

However, you always access and edit the data model files indirectly through the **configuration > config > Extensions** folder in Studio. Do not edit the XML files directly from the file system yourself.

Changes that you make to the data model are called *data model extensions*. For example, you can extend the data model by adding new fields to the `User` entity, or you can declare entirely new entities. The complete data model of your application installation comprises the base application data model model and any data model extensions that you make.

---

**WARNING:** Do not attempt to modify any files other than those in the `modules/configuration` directory. Any attempt to modify files outside of this directory can prevent the application from starting.

---

### Working with dot notation

Many places within the application require knowledge of fields within the application data model, especially while you configure the application. For example, code in a business rule, class or enhancement may need to check the *owner* of an assignable object. As an alternative, code may need to check the date and time of object creation. The application provides an easy and consistent method of referring to fields within the data model, using relative references based on a *root object*.

A root object is the starting point for any field reference. If you run Gosu rules on a claim for example, the claim is the root object, and you can access anything that relates to this claim. On the other hand, if you run an assignment rule for an activity, the activity is the root object. In this case, you have access to fields that relate to the activity, including the claim associated with the activity.

Guidewire applications use *dot notation* for relative references. For example, assume that your code has `Claim` as the root object. For a simple reference to a field on the claim such as `LossDate`, use:

```
claim.LossDate
```

However, suppose that you want to reference a field on an entity that relates to the claim, such as the policy expiration date. You must first describe the path from the claim to the policy, then describe the path from the policy to the expiration date of the policy:

```
claim.Policy.ExpirationDate
```

## Overview of data entities

Data entities are the high-level business objects used by the application. For example:

### ClaimCenter

Entities include Claim, Exposure, and Policy

### PolicyCenter

Entities include Policy, PolicyLine, and Coverage

### BillingCenter

Entities include TroubleTicket and Disbursement

An entity serves as the root object for data views, rules, Gosu classes, and most other data-related areas of the application. Guidewire defines a set of data objects in the base configuration from which it derives all other objects and entities. For many of the Guidewire base entities, you can also create entity extensions that enhance the base entities and provide additions required to support your particular business needs. In some cases, you can even define entirely new entities.

## Data entity metadata files

You define data entities through XML elements in the entity metadata definition files. The root element of an entity definition specifies the kind of entity and any attributes that apply. Subelements of the entity element define entity components, such as columns, or fields, and foreign keys.

---

**WARNING:** Do not modify any of the base data entity definition files (those in the `modules/configuration/config/metadata` directory) by editing them directly. You can view these files in read-only mode in Guidewire Studio in the **configuration > config > Metadata** folder.

---

To better understand the syntax of entity metadata, it is sometimes helpful to look at the data model and its metadata definition files. The application uses separate metadata definition files for entity declarations and extensions to them.

The base metadata files are available in Studio in the following location: **configuration > config > Metadata**

The extension metadata files are available in Studio in the following location: **configuration > config > Extensions**

The file extensions of metadata definition files distinguish their type, purpose, and contents.

| File type | Purpose                   | Contains                                                                                                                    | More information                                                                                    |
|-----------|---------------------------|-----------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|
| .dti      | Data Type Info            | A single data type definition.                                                                                              | "Data types" on page 139                                                                            |
| .eti      | Entity Type Information   | A single Guidewire or custom entity declaration. The name of the file corresponds to the name of the entity being declared. | "XML root elements and related base data object types" on page 29                                   |
| .eix      | Entity Internal eXtension | A single Guidewire entity extension. The name of the file corresponds to the name of the Guidewire entity being extended.   | Internal definition; see .etx                                                                       |
| .etx      | Entity Type eXtension     | A single Guidewire or custom entity extension. The name of the file corresponds to the name of the entity being extended.   | "<extension> elements and related data object types" on page 40<br>"<viewEntityExtension> elements" |

| File type | Purpose                     | Contains                                                                                                                        | More information                          |
|-----------|-----------------------------|---------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------|
|           |                             |                                                                                                                                 | and related data object types” on page 48 |
| .tti      | Typelist Type Info          | A single Guidewire or custom typelist declaration. The name of the file corresponds to the name of the typelist being declared. | “Working with typelists” on page 159      |
| .tix      | Typelist Internal eXtension | A single Guidewire typelist extension. The name of the file corresponds to the name of the Guidewire typelist being extended.   | “Working with typelists” on page 159      |
| .ttx      | Typelist Type eXtension     | A single Guidewire or custom typelist extension. The name of the file corresponds to the name of the typelist being extended.   | “Working with typelists” on page 159      |

The type of a metadata definition file determines what you can store and whether you can modify its contents.

| File type | Location                                       | Files are modifiable |
|-----------|------------------------------------------------|----------------------|
| .dti      | configuration > config > datatypes             | No                   |
| .eti      | configuration > config > Extensions > Entity   | Yes                  |
|           | configuration > config > Metadata > Entity     | No                   |
| .eix      | configuration > config > Metadata > Entity     | No                   |
| .etx      | configuration > config > Extensions > Entity   | Yes                  |
| .tti      | configuration > config > Extensions > Typelist | Yes                  |
|           | configuration > config > Metadata > Typelist   | No                   |
| .tix      | configuration > config > Metadata > Typelist   | No                   |
| .ttx      | configuration > config > Extensions > Typelist | Yes                  |

## The Metadata folder

The Studio **Metadata** folder contains the metadata definition files for entities that comprise the application data model.

A **Metadata** folder contains the following metadata definition file types:

- **Declaration files** – Versions of metadata definition files with extensions \*.eti and \*.tti.
- **Internal extension files** – Versions of metadata definition files with extensions \*.eix or \*.tix.

For an example, the data model includes the following metadata definition files that collectively define the Address entity type.

| File version | Metadata location                          | File purpose                       |
|--------------|--------------------------------------------|------------------------------------|
| Address.eti  | configuration > config > Metadata > Entity | Entity definition                  |
| Address.eix  | configuration > config > Metadata > Entity | Extension to the entity definition |

At runtime, the application merges the .eti and .eix versions of the Address definition file to create a complete Address entity type.

## The Extensions folder

The **configuration > config > Extensions** folder contains your data model definitions that extend the data model. The application considers the base definitions in **configuration > config > Metadata** first, and then applies the definitions in the **Extensions** folder to them. This lets you create an entity extension that overrides any Guidewire entity extensions.

**Note:** You cannot extend an entity that is in an .eti declaration file in the **Extensions > Entity Studio** folder. In addition, you cannot extend an extension at all.

## Example of Activity metadata and extension files

The data model includes the following metadata definition files that collectively define the Activity entity.

| File         | Location                                   | Purpose                            |
|--------------|--------------------------------------------|------------------------------------|
| Activity.eti | configuration > config > Metadata > Entity | Entity definition, not modifiable. |
| Activity.eix | configuration > config > Metadata > Entity | Entity extension, not modifiable.  |

To extend the Activity entity, create the following extension file through Guidewire Studio.

| File         | Location                                     | Purpose                  |
|--------------|----------------------------------------------|--------------------------|
| Activity.etx | configuration > config > Extensions > Entity | Custom entity extension. |

**WARNING:** Use only Guidewire Studio to create data model definition files. Use of Studio assures that the files reside in the correct location.

## The extensions properties file

In general, when you change the data model configuration files, you want the application to detect those changes and upgrade the database. The application detects data model changes using the following:

### checksum

Each time the application server starts, it records a checksum of the configuration files. If the previously recorded checksum and the current checksum differ, that indicates the files have changed.

### version property

The **configuration > config > Extensions** folder in Guidewire Studio contains an `extensions.properties` file that contains the numeric property `version`. After you update the configuration files, increment the `version` property to a new number that represents the changes. You can also increase the version number to force a database upgrade, even if the configuration files are unchanged.

**Note:** The version number must only increase in value. If it's set to a lower value than previously recorded, the server fails to start.

Upon startup, the server examines the checksum and the version for changes from the last recorded values. Server behavior depends on whether the values are different or the same:

| Checksum  | Version     | Server behavior                   |
|-----------|-------------|-----------------------------------|
| same      | same        | Starts with no database upgrade.  |
|           | incremented | Starts and upgrades the database. |
| different | same        | Fails to start.                   |
|           | incremented | Starts and upgrades the database. |

**Note:** When the server mode is set to dev, setting the `version` property is optional. The server upgrades the database whenever the file checksum has changed, regardless of the version. This allows you to make multiple

changes during development, and then designate them as a new version when finished. See "Server modes" in the *InsuranceSuite Server Administration*.

## Working with entity definitions

In working with entity definitions, you typically want to perform the following operations:

- Search for an existing entity definition
- Create a new entity definition
- Extend an existing entity definition

This section describes procedures for each operation.

**Note:** Guidewire strongly recommends that you verify your entity definitions at the time that you create them. To do so, right-click the entity in the **Project** window, and then click **Validate**. The verification process highlights any issues with a data model definition, enabling you to correct any issues.

## Search for an existing entity definition

### Procedure

1. In Guidewire Studio, press **Ctrl+N**.
2. In the **Enter class name** dialog, start typing the name of the entity that you want to find. Studio displays a list of matching entries that begin with the character string that you type.
3. In the list, click the name of the entity definition that you want to view.

Pay attention to the class type. For example, if you type **Activity**, Studio displays a list that includes all components whose name contains that text. Look for the one that ends with **.eti**.

### Results

Studio opens the file in the appropriate editor.

## Data entities

XML metadata files define all data entities in the data model. The **datamodel.xsd** file defines the elements and attributes that you can include in the XML metadata files. You can view a read-only version of the **datamodel.xsd** file in Guidewire Studio:

1. In Guidewire Studio, press **Ctrl+Shift+N** to search for a file.
2. In the search window, type **datamodel.xsd**.
3. Select the matching file to open it.

---

**WARNING:** Do not attempt to modify **datamodel.xsd**. That might invalidate your installation and prevent it from starting thereafter.

---

The **datamodel.xsd** file defines the following:

- The set of allowable or valid data entities
- The attributes associated with each data entity
- The allowable subelements on each data entity

All entity definition files must correspond to the definitions in **datamodel.xsd**.

Using XML files, Guidewire defines a data entity as a root element in an XML file that bears the name of the entity. For example, Guidewire declares the **Activity** entity type with the following **Activity.eti** file:

```
<?xml version="1.0"?>
<entity xmlns="http://guidewire.com/datamodel"
```



```

    desc="An activity is a instance of work assigned to a user and belonging to a claim."
    entity="Activity"
    exportable="true"
    extendable="true"
    platform="true"
    table="activity"
    type="retireable">
...
</entity>

```

At application server start up, the application loads the XML definitions of the data entities into the application database.

## Data entities and the application database

Each data entity is defined as a root XML element in the file that bears its name. For example, the Activity data entity is defined in Activity.eti:

```

<entity xmlns="http://guidewire.com/datamodel"
  entity="Activity"
  ...
  type="retireable">
...
</entity>

```

Notice that for the base configuration Activity object, the application sets the type attribute to retireable. The type attribute that determines how the application manages the data entity in the database. For example:

- If a data entity has a type value of `versionable`, the application stores instances of the entity in the database with a specific ID and version number.
- If a data entity has a type value of `retireable`, the application stores instances of the entity in the database and preserves the instances even when you *retire*, or hide them. When you retire specific instances, the application does not display them in the interface. However, the database still stores them until the application archives them or expressly deletes them with a special function.

---

**IMPORTANT:** For each data entity in the data model and for each entity type that you declare, the application automatically generates a field named `ID` that is of data type `key`. An `ID` field is the internally managed primary key for the object. Do not attempt to create entity fields of type `key`. The `key` type is for Guidewire internal use only. Guidewire also reserves the exclusive use of the following additional data types: `foreignkey`, `typekey`, and `typelistkey`.

---

The following table lists the possible values for the entity type attribute. Use only those type attributes marked for general use to create or extend an data entity. Do not attempt to create or extend an entity with a type attribute marked for internal-use.

| Type attribute | Usage                                  | Description                                                                                                                                                                                                                                                                                                                                                                     |
|----------------|----------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| editable       | General use                            | An editable entity is a versionable entity. The application automatically stores the version number of an editable entity. In addition to the standard versionable attributes of version and ID, an editable entity has the following additional attributes: <ul style="list-style-type: none"> <li>• CreateUser and CreateTime</li> <li>• UpdateUser and UpdateTime</li> </ul> |
| effdated       | PolicyCenter only<br>General use       | An effdated entity is an editable entity. effdated entities have effective date fields, meaning start and end dates. An effdated entity is a member of an effective dated graph, rooted at an effdatedbranch entity. The application auto-splits the date fields during editing in some modes.                                                                                  |
| effdatedbranch | PolicyCenter only<br>Internal use only | An effdated branch entity is a retireable entity. effdatedbranch entities define the entity type of the root of a tree that contains effdated entities.<br><br>Guidewire recommends not to attempt to create an entity with a type attribute of effdatedbranch.                                                                                                                 |

| Type attribute    | Usage                                  | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-------------------|----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| effdatedcontainer | PolicyCenter only<br>Internal use only | An effdatedcontainer entity is a retireable entity. effdatedcontainer entities define the entity type that has branch children. Guidewire recommends not to attempt to create an entity with a type attribute of effdatedcontainer.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| joinarray         | Internal use only                      | A joinarray entity works in a similar manner as a versionable entity. Guidewire recommends not to use this entity type. Use versionable instead.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| keyable           | Internal use only                      | All entities are keyable entities, except for those entities with an explicit nonkeyable entity type. Moreover, all entity types other than the nonkeyable entity type have the keyable entity type as an ancestor.<br><br>A keyable entity has an ID. You can delete keyable entities from the database.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| nonkeyable        | Internal use only                      | Do not use.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| retireable        | General use                            | <p>A retireable entity is an editable entity. retireable entities are the most common type of entity. Most, but not all, base entities are of this type.</p> <p>After the application adds an instance of a retireable data entity to the database, the application never deletes the instance. Instead, the application retires the instance. For example, if you select a retireable instance in a list view and then click <b>Delete</b>, the application preserves the instance in the database. However, the application inserts an integer in the Retired column for the row that represents the instance. Any non-zero value in the Retired column indicates that the application considers the instance retired.</p> <p>For retireable entities, the following fields are created automatically:</p> <ul style="list-style-type: none"> <li>• ID and PublicID</li> <li>• CreateUser and CreateTime</li> <li>• UpdateUser and UpdateTime</li> <li>• Retired</li> <li>• BeanVersion</li> </ul> <p>These are the same fields as those the application creates for editable entities, with the addition of the Retired property.</p> <p><b>IMPORTANT</b> Although it is extremely common for a base entity to be retireable, it is not required. You cannot assume this to be the case. Always check the <i>Data Dictionary</i> to determine whether an entity can be retired.</p> <p><b>Note:</b> The preservation of retireable entity instances terminates when an archiving process or an application-specific process deletes the entity instances.</p> |
| versionable       | General use                            | A versionable entity is a keyable entity. versionable entities have a version and an ID. Entities of this type can detect concurrent updates. It is possible to delete entities of this type from the database.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |

## Application database tables

For every entity type in the data model, a table is created in the application database. For example, PolicyCenter creates a Policy table to store information about the Policy object.

In the application database, you can identify an entity or extension table by the following prefix:

|                              |                                                              |
|------------------------------|--------------------------------------------------------------|
| <b>BillingCenter</b><br>bc_  | Entity table – one for each entity in the base configuration |
| <b>ContactManager</b><br>ab_ |                                                              |
| <b>ClaimCenter</b><br>cc_    |                                                              |
| <b>PolicyCenter</b>          |                                                              |

|                               |                                                                                                 |
|-------------------------------|-------------------------------------------------------------------------------------------------|
| pc_                           |                                                                                                 |
| <b>BillingCenter</b><br>bcx_  | Extension table – one for each custom extension added to the <b>extensions</b> folder in Studio |
| <b>ContactManager</b><br>abx_ |                                                                                                 |
| <b>ClaimCenter</b><br>ccx_    |                                                                                                 |
| <b>PolicyCenter</b><br>pcx_   |                                                                                                 |
|                               |                                                                                                 |

**Note:** It is possible to create non-persistent entities. These are entities or objects that you cannot save to the database. Guidewire discourages the use of non-persistent entities in favor of Plain Old Gosu Objects (POGOs), instead. See “<nonPersistentEntity> elements and related data object types” on page 42 for more information.

Besides entity tables, InsuranceSuite creates the following types of tables in the database:

| Table type       | More information                     |
|------------------|--------------------------------------|
| shadow tables    | “Shadow tables” on page 27           |
| staging tables   | “Staging tables” on page 28          |
| temporary tables | “Temporary (temp) tables” on page 28 |

## Shadow tables

A shadow table stores a copy of data from a main table for testing purposes. Every entity table potentially has a corresponding shadow table. Shadow tables in the database have one of the following prefixes:

|                                |                       |
|--------------------------------|-----------------------|
| <b>BillingCenter</b><br>bct_   | Entity shadow table   |
| <b>ContactManager</b><br>abt_  |                       |
| <b>ClaimCenter</b><br>cct_     |                       |
| <b>PolicyCenter</b><br>pct_    |                       |
|                                |                       |
| <b>BillingCenter</b><br>bctt_  | Typelist shadow table |
| <b>ContactManager</b><br>abtt_ |                       |
| <b>ClaimCenter</b><br>cctt_    |                       |
| <b>PolicyCenter</b><br>pctt_   |                       |
|                                |                       |

Shadow tables provide a way to quickly save and restore test data. All GUnit tests, including those that you write yourself, use shadow tables automatically. You cannot prevent GUnit tests from using shadow tables. GUnit tests use shadow tables according to the following process:

1. GUnit copies data from the main application tables to the shadow tables to create a backup of your test data.

2. GUnit runs your tests.
3. GUnit copies data backed up data in shadow tables to the main tables to restore a fresh copy of your test data for subsequent tests.

To start a test server, set its `server.running.tests` system property to `true` either explicitly or programmatically. In addition, set the server environment to one that uses your test database. Upon server startup, if shadow tables do not already exist, the application drops the database, recreates it, and then creates the shadow tables. The application creates shadow tables only in an empty database.

## Staging tables

The application generates a staging table for any entity that is marked with an attribute of `loadable="true"`. The `loadable` attribute is `true` by default in the base configuration. A staging table largely parallels the main entity table except that:

- The application replaces foreign keys by `PublicID` objects of type `String`.
- The application replaces `typecode` fields by `typekey` objects of type `String`.

After you load data into these staging tables, you run the command line tool `table_import` to bulk load the staging table data into the main application database tables. See the *InsuranceSuite System and Internal Tools* for information on how to use this command.

---

**IMPORTANT:** Some data types, for example, `Entity`, contain an `overwrittenInStagingTable` attribute. If this attribute is set to `true`, then do not attempt to populate the associated staging table yourself because the loader import process overwrites this table.

---

In the application database, you can identify a staging table by the following prefix:

### **BillingCenter**

`bcst_`

### **ContactManager**

`abst_`

### **ClaimCenter**

`ccst_`

### **PolicyCenter**

`pcst_`

## Temporary (temp) tables

The application generates a temporary table for any entity that is marked with an attribute of `temporary="true"`. Do not confuse a temporary table with a shadow table, they are not synonymous. InsuranceSuite uses temporary tables as work tables during installation or upgrade only. InsuranceSuite does not use them if the server is running in standard operation.

Unfortunately, it is easy to forget to clear up these tables if they are no longer needed. Therefore, it is quite possible for an application to have several of these temporary tables remaining even though the upgrade triggers that used them are long gone.

In the application database, temporary tables look like any other entity table except that temporary tables are almost always empty.

## Data objects and scriptability

*Scriptability* is the ability of code to *set* (write) or *get* (read) a scriptable item such as a property (column) on an entity. To do so, you set the following attributes:

- `getterScriptability`
- `setterScriptability`

The following table lists the different types of scriptability:

| Type                      | Description                                                             |
|---------------------------|-------------------------------------------------------------------------|
| <code>all</code>          | Exposed in Gosu, wherever Gosu is valid, such as in rules and PCF files |
| <code>doesNotExist</code> | Not exposed in Gosu                                                     |
| <code>hidden</code>       | Not exposed in Gosu                                                     |

If you do not specify a scriptability annotation, scriptability defaults to `all`.

**IMPORTANT:** There are subtle differences in how entities and fields marked as `doesNotExist` and `hidden` are treated. However, these differences relate to internal code. For your purpose, these two annotations behave in an identical manner, meaning any entity or field that uses one of these annotations does not show in Gosu code. In general, there is no need for you to use either one of these annotations.

## Scriptability behavior on entities

If you set `setterScriptability` at the entity level but you also set the value to `hidden` or `doesNotExist`, then the application does not generate constructors for the entity. In essence, you cannot create a new instance of the entity in Gosu. Within the data model, you can set the following scriptability annotation on `<entity>` objects:

| Object                      | Set (write) | Get (read) |
|-----------------------------|-------------|------------|
| <code>&lt;entity&gt;</code> | Yes         | No*        |

\* `<entity>` does not contain a `getterScriptability` attribute.

## Scriptability behavior on fields (columns)

If you set `setterScriptability` at the field level, then the value that you set controls the writability of the associated property in Gosu. Within the data model, you can set the following scriptability annotation on fields on `<entity>` objects:

| Field                               | Set (write) | Get (read) |
|-------------------------------------|-------------|------------|
| <code>&lt;array&gt;</code>          | Yes         | Yes        |
| <code>&lt;column&gt;</code>         | Yes         | Yes        |
| <code>&lt;edgeForeignKey&gt;</code> | Yes         | Yes        |
| <code>&lt;foreignkey&gt;</code>     | Yes         | Yes        |
| <code>&lt;onetoone&gt;</code>       | Yes         | Yes        |
| <code>&lt;typekey&gt;</code>        | Yes         | Yes        |

## XML root elements and related base data object types

The base configuration recognizes a set of XML root elements. These elements match by name a set of base data object types. All objects exist as instances of one of the base data object types or as instances of a subtype of a base object type. The following table lists the XML root elements for the base data object types:

| Data object type      | Extension | Folder                  | More information                                                          |
|-----------------------|-----------|-------------------------|---------------------------------------------------------------------------|
| <delegate>            | .eti      | metadata,<br>extensions | "<delegate> elements and related data object types" on page 30            |
| <entity>              | .eti      | metadata,<br>extensions | "<entity> elements and related data object types" on page 33              |
| <extension>           | .etx      | extensions              | "<extension> elements and related data object types" on page 40           |
| <nonPersistentEntity> | .eti      | metadata,<br>extensions | "<nonPersistentEntity> elements and related data object types" on page 42 |
| <subtype>             | .eti      | metadata,<br>extensions | "<subtype> elements and related data object types" on page 43             |
| <viewEntity>          | .eti      | metadata                | "<viewEntity> elements and related data object types" on page 46          |
| <viewEntityExtension> | .etx      | extensions              | "<viewEntityExtension> elements and related data object types" on page 48 |

**IMPORTANT:** There are additional data object types that Guidewire uses for internal purposes. Do not attempt to create or extend a data entity that is not listed in the previous table.

#### Related concepts

"<delegate> elements and related data object types" on page 30

#### Related reference

"<entity> elements and related data object types" on page 33

"<extension> elements and related data object types" on page 40

"<nonPersistentEntity> elements and related data object types" on page 42

"<subtype> elements and related data object types" on page 43

"<viewEntity> elements and related data object types" on page 46

"<viewEntityExtension> elements and related data object types" on page 48

## <delegate> elements and related data object types

The <delegate> XML root element defines delegate data object types. A delegate data object type is a collection of properties and behaviors. Moreover, a delegate contains an interface and a default implementation of that interface. A delegate can also add its own columns to the tables of data object types that implement the delegate. This type of delegation enables a data object type to implement an interface while delegating the implementation to the delegate.

You often use a delegate so that objects can share code. The delegate implements the shared code rather than each class implementing copies of common code. Thus, a delegate is an entity associated with an implemented interface that multiple parent entities can reuse.

Guidewire defines delegates in data model metadata files. You can extend existing delegates that are marked as extendable. You can also create your own custom delegates.

The application stores all database columns on the delegate entity in the implementing entity.

#### Attributes of <delegate>

The <delegate> element contains the following attributes.

| <delegate> attribute | Description      | Default |
|----------------------|------------------|---------|
| extendable           | <i>Internal.</i> | false   |

| <b>&lt;delegate&gt; attribute</b> | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | <b>Default</b> |
|-----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| name                              | <i>Required.</i>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | None           |
| platform                          | <i>Internal.</i> Do not use. The only real effect is to change the location in which the table appears in a data distribution report.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | false          |
| requiresType                      | Specifies that the delegate must be implemented by an entity of a general type, such as <code>nonkeyable</code> , <code>retireable</code> or <code>versionable</code> .<br><br>Possible values for the <code>requiresType</code> attribute are the values of the <code>type</code> attribute on the <code>&lt;entity&gt;</code> element. Some of the general types extend others. For example, <code>editable</code> extends <code>versionable</code> , and <code>versionable</code> extends <code>keyable</code> . An entity can implement the delegate if the implementor is the specified type or one that extends it. For example, if the <code>requiresType</code> attribute is <code>keyable</code> , then an implementing entity could have type <code>keyable</code> , <code>versionable</code> , or <code>editable</code> . |                |
| setterScriptability               | See “Data objects and scriptability” on page 28 for information.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | None           |

### Subelements of <delegate>

The `<delegate>` element contains the following subelements.

| <b>&lt;delegate&gt; subelement</b> | <b>Description</b>                                                                                                                                                                                                                                                       |
|------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| column                             | See “<column>” on page 53.                                                                                                                                                                                                                                               |
| datetimeordering                   | <i>Internal.</i>                                                                                                                                                                                                                                                         |
| dbcheckbuilder                     | <i>Internal.</i>                                                                                                                                                                                                                                                         |
| foreignkey                         | See “<foreignkey>” on page 62.                                                                                                                                                                                                                                           |
| fulldescription                    | See “<fulldescription>” on page 65.                                                                                                                                                                                                                                      |
| implementsEntity                   | See “<implementsEntity>” on page 65.                                                                                                                                                                                                                                     |
| implementsInterface                | See “<implementsInterface>” on page 67.                                                                                                                                                                                                                                  |
| index                              | See “<index>” on page 67.                                                                                                                                                                                                                                                |
| monetaryamount                     | Handles monetary amounts. The <code>&lt;monetaryamount&gt;</code> subelement is a compound data type that stores its values in two separate database columns: a <code>&lt;money&gt;</code> column type and a <code>typekey</code> to the <code>Currency</code> typelist. |
| typekey                            | See “<typekey>” on page 71.                                                                                                                                                                                                                                              |

## Implementing delegate data object types

To implement most delegate data object types, you add the following to an entity definition or extension.

```
<implementsEntity name="SomeDeLegate"/>
```

For example, in the base configuration, the `Group` entity implements the `Validatable` delegate by using the following:

```
<entity entity="Group" ... >
  <implementsEntity name="Validatable"/>
  ...
</entity>
```

It is possible for an entity to implement multiple delegates, just as a Gosu or Java class can implement multiple interfaces.

See also

- “<implementsEntity>” on page 65
- “Creating a new delegate object” on page 99.
- For a discussion of working with delegates in Gosu classes, see the *Gosu Reference Guide*.

## Delegate object types that you cannot implement directly

There are some delegates that you cannot implement directly through the use of the <implementsEntity> element. They are:

- Versionable
- KeyableBean
- Editable
- Retireable
- EffDated (PolicyCenter)
- EffDatedBranch (PolicyCenter)
- EffDatedContainer (PolicyCenter)

These special delegates are implicitly added to an entity if you set the type attribute on the entity to one of these values. Therefore, do not use the <implementsEntity> element to specify one of these delegates. Instead, use the type attribute on the entity declaration. The basic syntax looks similar to the following:

```
<entity name="SomeEntity" ... type="SomeDelegate">
```

For example, in the base configuration, the Group entity also implements the Retireable delegate by setting the entity type attribute to retireable.

```
<entity entity="Group" ... type="retireable">
  <implementsEntity name="Validatable"/>
  ...
</entity>
```

Also, it is not possible to explicitly implement the EventAware delegate. The application automatically adds this delegate to any entity that contains an <events> element.

## Archiving delegates in the base configuration must not be changed

---

**IMPORTANT:** Guidewire must enable archiving for the archiving functionality to be available.

---

To support the archive process, certain Guidewire entities in the base configuration implement one or more of the following delegates:

- RootInfo
- Extractable
- OverlapTable

---

**WARNING:** Do not change or remove the archiving delegates on Guidewire entities in the base configuration. Otherwise, the server may not start due to errors in the archiving domain graph.

---

## Recommendations for using delegates

Guidewire recommends that you use delegates in three scenarios.

### Implementing a common interface

Guidewire recommends that you use a delegate if you want both of the following:



- If you want to have multiple entities implement the same interface
- If you want most of the implementations of the interface to be common

The application defines a number of delegates in the base configuration. For example:

- Assignable
- Editable
- Validatable
- ...

To determine the list of base configuration delegate entities, search the **configuration > config > Metadata > Entity** folder for files that contain the following text:

```
<delegate
```

### Subtyping without single-table inheritance

Guidewire recommends that you create a delegate entity rather than define a supertype entity if you do not want to store subtype data in a single table. The application stores information on all subtypes of a supertype entity in a single table. This can create a table that is extremely large and extremely wide. This is true especially if you have an entity hierarchy with a number of different subtypes that each have their own columns. Using a delegate avoids this single-table inheritance while preserving the ability to define the fields and behavior common to all the subtypes in one place.

Guidewire recommends that you consider carefully before making a decision on how to model your entity hierarchy.

### Using entity polymorphism

Guidewire recommends that you create a delegate entity if you want to use polymorphism on class methods. For core InsuranceSuite classes defined in Java, you cannot override these class methods on its Gosu subtypes. However, you can push all methods and behaviors that can possibly be polymorphic into an interface, rather than the Java superclass. You can then require that all implementers of the delegate implement that interface (the `<implementsEntity>`) through the use of the `delegate requires` attribute. This delegate usage permits the use of polymorphism and enables delegate implementations to share common implementations on a common superclass.

## <entity> elements and related data object types

The `<entity>` XML root element defines many—if not most—of the application entities.

### Attributes of <entity>

The `<entity>` element contains the following attributes.

| <entity> attribute | Description                                                                                                                                                                                                                                                                                                                                                                                 | Default |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| abstract           | If true, you cannot create an instance of the entity type at run time. Instead, you must declare a subtype entity with <code>abstract=false</code> , which you can instantiate. Any of the generated code is abstract.                                                                                                                                                                      | false   |
| admin              | Determines whether you can reference the entity from staging tables: <ul style="list-style-type: none"> <li>Entity X has <code>admin="true"</code>. Suppose that you have another, loadable table Y that has a foreign key to X. Then at the time you load the staging table for Y, you can load public IDs that specify entities of type X that are already in the main tables.</li> </ul> | false   |

| <entity> attribute                     | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | Default            |
|----------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|
|                                        | <ul style="list-style-type: none"> <li>Entity X has <code>admin="false"</code>. Any Y that you load into a staging table must specify an X that is being loaded into the staging table for X at the same time.</li> </ul> <p>This is important because it allows the staging table loader to do less checking at load time. For example:</p> <ul style="list-style-type: none"> <li>If <code>admin="false"</code>, then the staging table loader merely has to check that all public IDs in <code>ccst_y</code> specify valid entries in <code>ccst_x</code>.</li> <li>If <code>admin="true"</code>, then the staging table loader has to check that all public IDs in <code>ccst_y</code> specify a valid entry in <code>ccst_x</code>. It must also check that all public IDs in <code>ccst_y</code> specify a valid entry in <code>cc_x</code>, the main table.</li> </ul> |                    |
| <code>autoSplit</code>                 | <i>PolicyCenter only.</i> Use with <code>effDated</code> entities, whether the application auto-splits the entity on a slice edit.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | <code>true</code>  |
| <code>cacheable</code>                 | <i>Internal.</i> If set to <code>false</code> , then Guidewire prohibits entities of this type and all its subtypes from existing in the global cache.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | <code>true</code>  |
| <code>consistentChildren</code>        | <p><i>Internal.</i> If set to <code>true</code>, then the application generates a consistency check and a loader validation that tries to ensure that links between child entities of this entity are consistent. Guidewire enforces the constraint only while loading data from staging tables. You can detect violations of the constraint on data committed to entity tables after the fact by running a consistency check.</p> <p><b>IMPORTANT</b> Guidewire does not enforce <code>consistentChildren</code> constraints at bundle commit.</p>                                                                                                                                                                                                                                                                                                                           | <code>false</code> |
| <code>desc</code>                      | A description of the purpose and use of the entity.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | <code>None</code>  |
| <code>displayName</code>               | <p><i>Optional.</i> Creates a more human-readable form of the entity name. You can access this name using the following:</p> <p><code>entity.DisplayName</code></p> <p>If you do not specify a value for the <code>DisplayName</code> attribute, then the <code>entity.DisplayName</code> method returns the value of the <code>entity</code> attribute instead. If you subtype an entity that has a specified display name, then the <code>entity.DisplayName</code> method returns the name of the subtype key.</p>                                                                                                                                                                                                                                                                                                                                                         | <code>None</code>  |
| <code>edgeTable</code>                 | For the purposes of archiving, whether the entity has the semantics of an <code>edgeForeignKey</code> table.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | <code>false</code> |
| <code>effDatedBranchType</code>        | <i>PolicyCenter only.</i> Relevant only for entities of type <code>effDated</code> . This value defines the type of the root of the tree in which the <code>effDated</code> entity exists. The only allowed value is <code>PolicyPeriod</code> .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | <code>None</code>  |
| <code>effDatedBranchesField</code>     | <i>Internal. PolicyCenter only.</i> Relevant only for <code>Policy</code> entities. This value defines the field that refers to the branches of this container.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | <code>None</code>  |
| <code>effDatedContainerField</code>    | <i>Internal. PolicyCenter only.</i> Relevant only for <code>PolicyPeriod</code> entities. This value defines the field that refers to the container of this branch.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | <code>None</code>  |
| <code>effDatedRegistryTableName</code> | <i>Internal. PolicyCenter only.</i> Relevant only for <code>Policy</code> entities. This value defines the name of the table that stores the <code>effdated</code> table registry for this container entity.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | <code>None</code>  |

| <entity> attribute         | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | Default            |
|----------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|
| entity                     | <p><i>Required.</i> The name of the entity. You use this name to access the entity in data views, rules, and other application areas.</p> <p>This attribute has no maximum or minimum length in and of itself. However, if the <code>final</code> attribute is set to <code>false</code>, the entity has an associated typelist containing as typecodes the names of the entity subtypes. These typecodes have a maximum length of 50 characters. Hence, the <code>entity</code> attribute for all non-final entities has an implicit maximum length of 50.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | None               |
| exportable                 | <i>Unused.</i>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                    |
| extendable                 | <i>Internal.</i> If <code>true</code> , it is possible to extend this entity.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | <code>true</code>  |
| final                      | <p>If <code>true</code>, you cannot subtype the entity. If <code>false</code>, you can define subtypes using this entity as the supertype.</p> <p><b>IMPORTANT</b> If you define this incorrectly, the application generates an error message upon resource verification and the application server refuses to start. The application generates the verification error in the following situations:</p> <ul style="list-style-type: none"> <li>• If you attempt to subtype an entity that is marked as <code>final</code> that exists in the metadata folder in Guidewire Studio.</li> <li>• If you attempt to subtype an entity that is marked as <code>final</code> that exists in the extensions folder in Studio.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | <code>true</code>  |
| generateInternallyIfAbsent | <i>Internal.</i> Do not use.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | <code>false</code> |
| ignoreForEvents            | <p>The <code>ignoreForEvents</code> attribute indicates whether other entities pass over the instant entity when determining the origin of an event. If the value of <code>ignoreForEvents</code> is <code>true</code>, the application ignores the entity in this context.</p> <p>If the <code>&lt;events&gt;</code> element is not present in an entity, that entity does not generate events. Moreover, that entity does not generate events when you create, modify, or delete it. Thus, it makes no sense for the application to examine either that entity or entities that reference it when the product searches for the source of an event.</p> <p>Nonetheless, even if an entity generates no events, whenever you modify it, the application still examines that entity and all entity instances that reference it. The scope of this examination encompasses the non-event-generating entity as well as arrays, foreign keys, and edge foreign keys that reference it. If the application finds an entity instance that both generates events and references the modified, non-event-generating entity, the product generates a <code>Changed</code> event for that entity instance. This examination and event generation process results in unnecessary and long chain queries.</p> <p>Guidewire offers two solutions to the resultant performance challenge. You can avoid the examination and event generation altogether if you set the <code>ignoreForEvents</code> attribute to <code>true</code> on the non-event-generating entity. In this case, the application ignores the non-event-generating entity as well as references to it when an event occurs.</p> <p>As an alternative, you can avoid the examination and event generation in some cases while allowing the process in others. To effect this selective avoidance, set the <code>ignoreForEvents</code> attribute to <code>true</code> not on the non-event-generating entity. Rather, set it to <code>true</code> only on event-generating entity instances both that reference the entity and that you want the application</p> | <code>false</code> |

| <entity> attribute        | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | Default |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
|                           | <p>to ignore. Such referencing entity instances might be arrays, foreign keys, or edge foreign keys.</p> <p>In this case, when you modify the non-event-generating entity, the application does not ignore the entity altogether. When you modify the entity, event-generating entity instances that reference the entity and on which the ignoreForEvents attribute is false still generate events. Instead, the application ignores only the referencing entity instance on which you set the ignoreForEvents attribute to true.</p> <p>Thus, if you want no events triggered at all when you modify a non-event-generating entity, set the ignoreForEvents attribute on that entity to true. If you want some events avoided and some triggered, take this step only on event-generating entities that reference the non-event-generating entity and that you want to ignore.</p> <hr/> <p><b>IMPORTANT:</b> With the ignoreForEvents attribute, the application rules out specific entities or activities for event generation. The attribute eliminates events where they are not needed. Hence, the attribute results in improved performance. Set the ignoreForEvents attribute to true as appropriate to ensure optimal performance. This performance improvement is especially important for any entity shared among graph entities, such as Claim, Policy, or Account.</p> <hr/> |         |
| instrumentationTable      | <i>Internal.</i>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | false   |
| loadable                  | If true, you can load the entity through staging tables.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | true    |
| lockable                  | <p><i>Internal.</i> If set to true, the application adds a lock column (lockingcolumn) to the table for this entity. The application uses this to acquire an update lock on a row. The most common use is on objects in which it is important to implement safe ordering of messages. In that case, the entity that imposes the safe ordering needs to be lockable.</p> <p><b>IMPORTANT</b> Guidewire strongly recommends that you do not use this locking mechanism.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | false   |
| overwrittenInStagingTable | <p><i>Internal.</i> If true and the entity is loadable, the loader process auto-populates the staging table during import.</p> <p><b>IMPORTANT</b> If set to true, do not attempt to populate the table yourself, because the loader import process overwrites this table.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | false   |
| platform                  | <i>Internal.</i> Do not use. The only real effect is to change the location in which the table appears in a data distribution report.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | false   |
| priority                  | The priority of the corresponding subtype key. This value is meaningful only for entities participating in a subtype hierarchy, which can be either the <subtype> entities or the root <entity>.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | -1      |
| readOnly                  | <p><i>Optional.</i> The typical use of read-only entities is for tables of reference data that you import as administrative data and then never touch again.</p> <p>You can add a read-only entity only to a bundle that has the allowReadOnlyBeanChanges() flag set on its commit</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | None    |

| <entity> attribute  | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | Default |
|---------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
|                     | <p>options. That means that inserting, modifying or deleting a read-only entity requires one of these special bundles.</p> <p>You cannot set bundle commit options from Gosu. Therefore, you cannot modify these entities from Gosu, unless some Gosu-accessible interface gives you a special bundle. The administrative XML import tools use such a special bundle. However, Guidewire uses these tools internally only in the PolicyCenter product model.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |         |
| setterScriptability | See “Data objects and scriptability” on page 28.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | None    |
| subpackage          | Subpackage to which the class corresponding to this entity belongs.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | None    |
| table               | <p><i>Required.</i> The name of the database table in which the application stores the data for this entity. The application automatically prefixes table names with <code>xx_</code> for base entities and <code>xxx_</code> for extension entities, where <code>xx</code> is the product code (for example, <code>cc</code> for ClaimCenter).</p> <p>Guidewire recommends the following table naming conventions:</p> <ul style="list-style-type: none"> <li>• Do not begin the table name with any product-specific extension.</li> <li>• Use only unaccented, lowercase Roman letters and the underscore character.</li> <li>• Prefix the table name with <code>test_</code> if you use the table solely for testing.</li> <li>• Because the application automatically prefixes extension table names with <code>xxx_</code>, if you run into limits on the length of the table name, you can consider removing the <code>_Ext</code> suffix from the table name.</li> </ul> <p>The table name has no minimum allowable length so long as the name is not an empty string. Guidewire enforces the following restrictions on the maximum allowable length of the table name:</p> <ul style="list-style-type: none"> <li>• <code>loadable="true"</code> — maximum of 25 characters</li> <li>• <code>loadable="false"</code> — maximum of 26 characters</li> </ul> | None    |
| temporary           | <p><i>Internal.</i> If true, then this table is a temporary table that the application uses only during installation or upgrade.</p> <p>The application deletes all temporary tables after it completes the installation or the upgrade.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | false   |
| type                | <i>Required.</i> See “Overview of data entities” on page 21 for a discussion of data entity types.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | None    |
| typelistTableName   | <p>The typelist that the application generates when you create a non-final entity has an associated database table. If you do not specify a value for the <code>typelistTableName</code> attribute, then the application uses the name of the entity as the table name for the subtype typelist.</p> <p>However, the application places a restriction of 25 characters on the length of the database table name. You use the <code>typelistTableName</code> attribute to specify the database table name for the typelist if an entity name is too long to become a valid typelist table name.</p> <p>It is not valid to use this attribute with entity types marked as final.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | None    |

| <entity> attribute | Description                                                                                                                        | Default |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------|---------|
| validateOnCommit   | <i>Internal.</i> Do not use. If true, the application validates this entity during a commit of a bundle that contains this entity. | true    |

---

**IMPORTANT:** Do not modify internal entity attributes. This instruction applies even in the case of custom entities.

---



---

**IMPORTANT:**

If you create a non-final <entity> element, then the application automatically creates a typelist with the same name as the resulting entity. The typelist allows you to keep track of the subtypes of that entity. The application creates this typelist whether or not a <subtype> element exists that refers to the non-final <entity> element.

Suppose that you change a final <entity> element to a non-final <entity> element. Suppose further that you already have a typelist with the same name as the resulting entity. In this case, you must change the name of the preexisting typelist beforehand. Otherwise, the automatic typelist generation cannot proceed, and you will receive an error.

---

### Subelements of <entity>

The <entity> element contains the following subelements.

| <entity> subelement       | Description                                                                                                                                                                                               |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| array                     | See "<array>" on page 50.                                                                                                                                                                                 |
| checkconstraint           | <i>Internal.</i>                                                                                                                                                                                          |
| column                    | See "<column>" on page 53.                                                                                                                                                                                |
| customconsistencycheck    | <i>Internal.</i>                                                                                                                                                                                          |
| datetimeordering          | <i>Internal.</i>                                                                                                                                                                                          |
| dbcheckbuilder            | <i>Internal.</i>                                                                                                                                                                                          |
| edgeForeignKey            | See "<edgeForeignKey>" on page 58.                                                                                                                                                                        |
| events                    | Indicates that the entity raises events and that the entity is aware of events. See "<events>" on page 61.                                                                                                |
| foreignkey                | See "<foreignkey>" on page 62.                                                                                                                                                                            |
| fulldescription           | See "<fulldescription>" on page 65.                                                                                                                                                                       |
| implementsEntity          | See "<implementsEntity>" on page 65.                                                                                                                                                                      |
| implementsInterface       | See "<implementsInterface>" on page 67.                                                                                                                                                                   |
| index                     | See "<index>" on page 67.                                                                                                                                                                                 |
| jointableconsistencycheck | <i>Internal.</i>                                                                                                                                                                                          |
| monetaryamount            | Handles monetary amounts. The <monetaryamount> subelement is a compound data type that stores its values in two separate database columns: a <money> column type, and a typekey to the Currency typelist. |
| onetoone                  | See "<onetoone>" on page 69.                                                                                                                                                                              |
| remove-index              | See "<remove-index>" on page 70.                                                                                                                                                                          |
| searchColumn              | See "<entity> elements and related data object types" on page 33                                                                                                                                          |

| <entity> subelement     | Description                                                                                                                                                                                                                                                                                                                 |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| searchTypekey           | Defines a search denormalization typekey in the database. The denormalization copies the value of a column on another table into a typekey field on the denormalizing table. You must link the tables through a foreign key. The purpose of this denormalization is to avoid costly joins in performance critical searches. |
| tableAugmenter          | <i>Internal.</i>                                                                                                                                                                                                                                                                                                            |
| typekey                 | See “<typekey>” on page 71.                                                                                                                                                                                                                                                                                                 |
| validateTypekeyinset    | <i>Internal.</i>                                                                                                                                                                                                                                                                                                            |
| validateTypekeynotinset | <i>Internal.</i>                                                                                                                                                                                                                                                                                                            |

**IMPORTANT:** Do not modify internal entity subelements even in the case of custom entities.

### The <searchColumn> subelement

The <searchColumn> subelement on <entity> defines a search denormalization column in the database. The denormalization copies the value of a column on another table into a column on the denormalizing table. You must link the tables through a foreign key. The purpose of this denormalization is to avoid costly joins in performance-critical searches.

The use of search denormalization columns adds overhead to updates, as does any denormalization. Guidewire recommends that you only use these columns if there is an identifiable performance problem with a search that is directly related to the join between the two tables.

**Note:** It is possible to have a <searchColumn> subelement on the <extension> and <subtype> elements as well.

The <searchColumn> element contains the following attributes.

| <searchColumn> attribute | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                              | Default |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| columnName               | Name to use for the database column corresponding to this property. If you do not specify a value, then the application uses the name value instead.<br><br>The columnName attribute must be no more than 30 characters in length. It allows only unaccented Roman letters, numbers, and the underscore character. The first character of the columnName attribute must be a letter. Although the underscore character is allowable here, Guidewire discourages its use. | None    |
| deprecated               | If true, then the application marks the item as deprecated in the <i>Data Dictionary</i> and places a Deprecated annotation on it in the <i>Guidewire Studio API Reference</i> .<br><br>If you deprecate an item, use the description to explain why.<br><br>For more information, see “Data entity subelements” on page 49.                                                                                                                                             | false   |
| desc                     | Description of the intended purpose of this column.                                                                                                                                                                                                                                                                                                                                                                                                                      | None    |
| name                     | <i>Required.</i> Name of the column on the table <i>and</i> the field on the entity. The name value maps to the accessor and mutator methods of a field on the entity, not the actual private member field. For example, name maps to setName and getName, not the private _name member field.<br><br>Column names must contain letters only. A column name cannot contain an underscore.                                                                                | None    |
| sourceColumn             | <i>Required.</i> Name of the column on the source entity, whose value this column copies. The sourceColumn must not name a localized column.                                                                                                                                                                                                                                                                                                                             | None    |

| <searchColumn> attribute | Description                                                                                                                                                                                                                                                                                                                           | Default |
|--------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| sourceForeignKey         | <i>Required.</i> Name of a foreign key field on this entity, which refers to the source entity for this search denormalization column. The sourceForeignKey must not be importable against existing objects.                                                                                                                          | None    |
| sourceSubtype            | Optional name of the particular subtype on which the source column is defined. If not specified, then the application assumes that the source column to exist on the entity referred to by the source object. However, you must specify this value if the sourceColumn is on a subtype of the entity referred to by sourceForeignKey. | None    |

For example, suppose that you set the following attribute definitions:

- searchColumn – *MyDenormColumn*
- sourceForeignKey – *Source*
- sourceColumn – *SourceField*

This declaration says:

Copy the value of *SourceField* on the object pointed to by the foreign key named *Source* into the field named *MyDenormColumn*. The application automatically populates the column as part of bundle commit, staging table load, and database upgrade.

If you need to denormalize a field on a subtype of the entity referred to by the foreign key, then you can specify the optional sourceSubtype attribute.

As with linguistic denormalization columns, you cannot access the value of these search denormalization columns in memory. The value is only available in the database. Thus, you can only access the value through a database query.

It is possible to make a query against a search denormalization column that is a denormalization of a linguistic denormalization column. In that case, the query generator knows not to wrap the column values in the linguistic denormalization function. This preserves the optimization that linguistic denormalization columns provide.

It is important to understand that search denormalization columns specify one column only — the column that you specify with the sourceColumn attribute. So, if you want to denormalize both a column and its linguistic denormalization, then you need two separate search denormalization columns. However, in this case, you typically would just want to denormalize the linguistic denormalization column. You would only want to denormalize the source column if you wanted to support case-sensitive searches on it.

Search denormalization columns can only specify <column> or <typekey> fields.

## <extension> elements and related data object types

The <extension> XML root element defines extension data object types. An extension data object type extends an already existing data object type or entity. Guidewire defines extensions in data model metadata files.

See also

- For information on how to extend the base data objects, see “Modifying the base data model” on page 89.

### Attributes of <extension>

The <extension> element contains the following attributes.

| <extension> attribute | Description                                                                                                                                                                                                                                                 | Default |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| entityName            | <i>Required.</i> This value must match the file name of the entity that it extends.<br><br>The application generates an error at resource verification if the value that you set for the entityName attribute in an extension does not match the file name. | None    |



### Subelements of <extension>

The <extension> element contains the following subelements.

| <extension> subelement  | Description                                                                                                                                                                                                                                                                                                                 |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| array                   | See “<array>” on page 50.                                                                                                                                                                                                                                                                                                   |
| array-override          | Use to override, or flip, the value of the triggersValidation attribute of an <array> element definition on a base data object. See “Working with attribute and element overrides” on page 95 for details.                                                                                                                  |
| column                  | See “<column>” on page 53.                                                                                                                                                                                                                                                                                                  |
| column-override         | Use to override certain very specific attributes of a base data object. See “Working with attribute and element overrides” on page 95 for details.                                                                                                                                                                          |
| description             | A description of the purpose and use of the entity.                                                                                                                                                                                                                                                                         |
| edgeForeignKey          | See “<edgeForeignKey>” on page 58.                                                                                                                                                                                                                                                                                          |
| edgeForeignKey-override | Use to override certain very specific attributes of a base data object.                                                                                                                                                                                                                                                     |
| events                  | See “<events>” on page 61.                                                                                                                                                                                                                                                                                                  |
| foreignkey              | See “<foreignkey>” on page 62.                                                                                                                                                                                                                                                                                              |
| foreignkey-override     | Use to override, or flip, the value of the triggersValidation attribute of a <foreignkey> element definition on a base data object. See “Working with attribute and element overrides” on page 95 for details.                                                                                                              |
| implementsEntity        | See “<implementsEntity>” on page 65.                                                                                                                                                                                                                                                                                        |
| implementsInterface     | See “<implementsInterface>” on page 67.                                                                                                                                                                                                                                                                                     |
| index                   | See “<index>” on page 67.                                                                                                                                                                                                                                                                                                   |
| internalonlyfields      | <i>Internal.</i>                                                                                                                                                                                                                                                                                                            |
| monetaryamount          | Handles monetary amounts. The <monetaryamount> subelement is a compound data type that stores its values in two separate database columns: a <money> column type and a typekey to the Currency typelist.                                                                                                                    |
| onetoone                | See “<onetoone>” on page 69.                                                                                                                                                                                                                                                                                                |
| onetoone-override       | Use to override, or flip, the value of the triggersValidation attribute of a <onetoone> element definition on a base data object. See “Working with attribute and element overrides” on page 95 for details.                                                                                                                |
| remove-index            | See “<remove-index>” on page 70.                                                                                                                                                                                                                                                                                            |
| searchColumn            | See “<entity> elements and related data object types” on page 33.                                                                                                                                                                                                                                                           |
| searchTypekey           | Defines a search denormalization typekey in the database. The denormalization copies the value of a column on another table into a typekey field on the denormalizing table. You must link the tables through a foreign key. The purpose of this denormalization is to avoid costly joins in performance critical searches. |
| typekey                 | See “<typekey>” on page 71.                                                                                                                                                                                                                                                                                                 |
| typekey-override        | Use to override certain specific attributes, or fields, of a <typekey> element definition on a base data object. See “Working with attribute and element overrides” on page 95 for details.                                                                                                                                 |

## <nonPersistentEntity> elements and related data object types

The <nonPersistentEntity> XML root element defines non-persistent entity data object types. A non-persistent entity data object type defines a temporary entity instance or object. The application creates and uses this object only during the time that the server is running. When the server shuts down, it discards the object data. It is not possible to commit a non-persistent entity object to the database.

Guidewire defines non-persistent entities in data model metadata files.

**Note:** You cannot subtype a persistent entity with a non-persistent entity.

### Guidewire recommendations for non-persistent entities

As a general rule, Guidewire recommends that you do not create new non-persistent entities. In addition, do not extend existing non-persistent entities unless instructed to do so. For example, the base configuration contains non-persistent entities that you can extend to specify new search criteria.

A major issue with non-persistent entities is that they do not interact well with data bundles. For example, passing a non-persistent entity instance to a PCF page is generally a bad idea because it generally does not work in the manner that you expect.

A non-persistent entity instance has to live in a bundle and can only live in one bundle. Therefore, passing it to one context removes it from the other context. Even worse, it is possible that in passing the non-persistent entity instance from one context to another, the object loses any nested arrays or links associated with it. Thus, it is possible to lose parts of the entity graph as the non-persistent entity instance moves around.

In situations where you want the behavior of a non-persistent entity, use a Gosu class instead. For example:

- If you want the behavior of a non-persistent entity in web services, create a Gosu class. Then, expose the class as a web service. Do not rely on non-persistent entities and entity serialization.
- If you want a field that behaves as a column, specify a data type through the use of annotations. For example, you could use this technique for nonnegativeinteger column behavior. After specifying the data type, add the wanted data type behavior to properties on Gosu classes. For information on how to associate data types with object properties using the annotation syntax, see “Defining a data type for a property” on page 141.

### Attributes of <nonPersistentEntity>

The <nonPersistentEntity> element contains the following attributes.

| <nonPersistentEntity> attribute | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | Default |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| abstract                        | If true, you cannot create an instance of the entity type at runtime. Instead, you must declare a subtype entity with <code>abstract=false</code> , which you can instantiate. Any of the generated code is abstract.                                                                                                                                                                                                                                                                              | false   |
| desc                            | A description of the purpose and use of the entity.                                                                                                                                                                                                                                                                                                                                                                                                                                                | None    |
| displayName                     | <i>Optional.</i> Creates a more human-readable form of the entity name. You can access this name using the following:<br><br><code>entity.displayName</code><br><br>If you do not specify a value for the <code>displayName</code> attribute, then the <code>entity.displayName</code> method returns the value of the entity attribute, instead. If you subtype an entity that has a specified display name, then the <code>entity.displayName</code> method returns the name of the subtype key. | None    |
| entity                          | <i>Required.</i> The name of the entity. You use this name to access the entity in data views, rules, and other application areas.                                                                                                                                                                                                                                                                                                                                                                 | None    |
| exportable                      | <i>Unused.</i>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |         |
| extendable                      | If true, it is possible to extend this entity.                                                                                                                                                                                                                                                                                                                                                                                                                                                     | true    |

| <b>&lt;nonPersistentEntity&gt;<br/>attribute</b> | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | <b>Default</b> |
|--------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| final                                            | If true, you cannot subtype the entity. If false, you can define subtypes using this entity as the supertype.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | true           |
| platform                                         | <i>Internal.</i> Do not use. The only real effect is to change the location in which the table appears in a data distribution report.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | false          |
| priority                                         | The priority of the corresponding subtype key. This value is meaningful only for entities -1 participating in a subtype hierarchy, which can be either the <subtype> entities or the root <entity>.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |                |
| typelistTableName                                | <p>If you create a non-final entity, then the application automatically creates a typelist to keep track of the subtypes of that entity. That typelist has an associated database table. If you do not specify a value for this attribute, then the application uses the name of the entity as the table name for the subtype typelist.</p> <p>However, InsuranceSuite places a restriction of 25 characters on the length of the database table name. You use this attribute to specify the database table name for the typelist if an entity name is too long to become a valid typelist table name.</p> <p>It is not valid to use this attribute with entity types marked as final.</p> | None           |

### Subelements of <nonPersistentEntity>

The <nonPersistentEntity> element contains the following subelements.

| <b>&lt;nonPersistentEntity&gt;<br/>subelement</b> | <b>Description</b>                                                                                                                                                                                       |
|---------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| array                                             | See "<array>" on page 50.                                                                                                                                                                                |
| column                                            | See "<column>" on page 53.                                                                                                                                                                               |
| edgeForeignKey                                    | See "<edgeForeignKey>" on page 58.                                                                                                                                                                       |
| foreignkey                                        | See "<foreignkey>" on page 62.                                                                                                                                                                           |
| fulldescription                                   | See "<fulldescription>" on page 65.                                                                                                                                                                      |
| implementsEntity                                  | See "<implementsEntity>" on page 65.                                                                                                                                                                     |
| implementsInterface                               | See "<implementsInterface>" on page 67.                                                                                                                                                                  |
| monetaryamount                                    | Handles monetary amounts. The <monetaryamount> subelement is a compound data type that stores its values in two separate database columns: a <money> column type and a typekey to the Currency typelist. |
| onetoone                                          | See "<onetoone>" on page 69.                                                                                                                                                                             |
| typekey                                           | See "<typekey>" on page 71.                                                                                                                                                                              |

## <subtype> elements and related data object types

The <subtype> XML root element defines an entity type that is a subtype of another entity. The subtype entity has all of the fields and elements of its supertype. The subtype entity can also have additional fields and elements. Guidewire defines subtype entities in the data model metadata files.

The application does not associate a separate database table with a subtype. Instead, the application stores all subtypes of a supertype in the table of the supertype and resolves the entity to the correct subtype based on the value of the Subtype field. To accommodate this, the application stores all fields of a subtype in the database as nullable columns—even the ones defined as non-nullable. However, if you define a field as non-nullable, then the metadata service enforces this for all data operations.

You can only define a subtype for any entity that has its `final` attribute set to `false`. The application automatically creates a Subtype field for non-final entities.

### Attributes of <subtype>

The <subtype> element contains the following attributes:

| <subtype> attribute              | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | Default            |
|----------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------|
| <code>abstract</code>            | If true, you cannot create an instance of the entity type at runtime. Instead, you must declare a subtype entity with <code>abstract=false</code> , which you can instantiate. Any of the generated code is <code>abstract</code> .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | <code>false</code> |
| <code>desc</code>                | A description of the purpose and use of the subtype.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | None               |
| <code>displayName</code>         | <p><i>Optional.</i> Occasionally in the user interface, you want to display the subtype name of subtyped entity instances. Use the <code>displayName</code> attribute to specify a <code>String</code> to display as the subtype name. You can access this name using the following:</p> <pre>entity.DisplayName</pre> <p>If you do not specify a value for the <code>displayName</code> attribute, then the application displays the name of the entity. The entity name is often not user-friendly. For a description of the <code>displayName</code> attribute, see “&lt;entity&gt; elements and related data object types” on page 33.</p>                                                                                                                | None               |
| <code>entity</code>              | <i>Required.</i> This attribute has a maximum length of 50.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |                    |
| <code>final</code>               | If true, you cannot subtype the entity. If false, you can define subtypes using this entity as the supertype.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | <code>false</code> |
| <code>platform</code>            | <i>Internal.</i> Do not use. The only real effect is to change the location in which the table appears in a data distribution report.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | <code>false</code> |
| <code>priority</code>            | The priority of the corresponding subtype key. This value is meaningful only for entities participating in a subtype hierarchy, which can be either the <subtype> entities or the root <entity>.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | -1                 |
| <code>readOnly</code>            | <p><i>Optional.</i> The typical use of read-only entities is for tables of reference data that you import as administrative data and then never touch again.</p> <p>You can add a read-only entity only to a bundle that has the <code>allowReadOnlyBeanChanges()</code> flag set on its commit options. That means that inserting, modifying or deleting a read-only entity requires one of these special bundles.</p> <p>You cannot set bundle commit options from Gosu. Therefore, you cannot modify these entities from Gosu, unless some Gosu-accessible interface gives you a special bundle. The administrative XML import tools use such a special bundle. However, Guidewire uses these tools internally only in the PolicyCenter product model.</p> | None               |
| <code>setterScriptability</code> | See “Data objects and scriptability” on page 28 for information.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | None               |
| <code>supertype</code>           | <i>Required.</i>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |                    |

### Subelements of <subtype>

The <subtype> element contains the following subelements.

| <subtype> subelement         | Description                |
|------------------------------|----------------------------|
| <code>array</code>           | See “<array>” on page 50.  |
| <code>checkconstraint</code> | <i>Internal.</i>           |
| <code>column</code>          | See “<column>” on page 53. |

| <subtype> subelement      | Description                                                                                                                                                                                                                                                                                                                 |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| customconsistencycheck    | <i>Internal.</i>                                                                                                                                                                                                                                                                                                            |
| datetimeordering          | <i>Internal.</i>                                                                                                                                                                                                                                                                                                            |
| dbcheckbuilder            | <i>Internal.</i>                                                                                                                                                                                                                                                                                                            |
| edgeForeignKey            | See "<edgeForeignKey>" on page 58.                                                                                                                                                                                                                                                                                          |
| events                    | See "<events>" on page 61.                                                                                                                                                                                                                                                                                                  |
| foreignkey                | See "<foreignkey>" on page 62.                                                                                                                                                                                                                                                                                              |
| fulldescription           | See "<fulldescription>" on page 65.                                                                                                                                                                                                                                                                                         |
| implementsEntity          | See "<implementsEntity>" on page 65.                                                                                                                                                                                                                                                                                        |
| implementsInterface       | See "<implementsInterface>" on page 67.                                                                                                                                                                                                                                                                                     |
| index                     | See "<index>" on page 67.                                                                                                                                                                                                                                                                                                   |
| jointableconsistencycheck | <i>Internal.</i>                                                                                                                                                                                                                                                                                                            |
| monetaryamount            | Handles monetary amounts. The <monetaryamount> subelement is a compound data type that stores its values in two separate database columns: a <money> column type, and a typekey to the Currency typelist.                                                                                                                   |
| onetoone                  | See "<onetoone>" on page 69.                                                                                                                                                                                                                                                                                                |
| searchColumn              | See "<entity> elements and related data object types" on page 33.                                                                                                                                                                                                                                                           |
| searchTypekey             | Defines a search denormalization typekey in the database. The denormalization copies the value of a column on another table into a typekey field on the denormalizing table. You must link the tables through a foreign key. The purpose of this denormalization is to avoid costly joins in performance critical searches. |
| tableAugmenter            | <i>Internal.</i>                                                                                                                                                                                                                                                                                                            |
| typekey                   | See "<typekey>" on page 71.                                                                                                                                                                                                                                                                                                 |
| validateTypekeyinset      | <i>Internal.</i>                                                                                                                                                                                                                                                                                                            |
| validateTypekeynotinset   | <i>Internal.</i>                                                                                                                                                                                                                                                                                                            |

## Subtypes and typelists

After you define a new subtype, the application automatically adds that entity type to the associated entity typelist. This is true, even if the application marks that typelist as *final*.

For example, suppose that you define an *Inspector* entity as a subtype of *Person*.

```
<?xml version="1.0"?>
<subtype xmlns="http://guidewire.com/datamodel" desc="Professional inspector" displayName="Inspector"
  entity="InspectorExt"
  supertype="Person">
  <column name="InspectorLicenseExt" type="varchar" desc="Inspector's business license number">
    <columnParam name="size" value="30"/>
  </column>
</subtype>
```

Notice that while *InspectorExt* is subtype of *Person*, *Person*, itself, is a subtype of *Contact*. The application automatically adds the new *InspectorExt* type to the *Contact* typelist. This is true, even though the application marks the *Contact* typelist as *final*.

To see this change:

- In the *Data Dictionary*, you must restart the application server.

- In the Contact typelist in Studio, you must restart Studio.

#### See also

- “Defining a subtype” on page 107

## <viewEntity> elements and related data object types

The <viewEntity> XML root element defines view entity data object types. A view entity data object type is a logical view of entity data. You can use a view entity to enhance performance during the viewing of tabular data. A view entity provides a logical view of data for an entity of interest to a list view. A view entity can include paths from the root or primary entity to other related entities.

For example, from the ClaimCenter ActivityView, you can specify a column with a Claim.ClaimNumber value. The Activity entity is the primary entity of the ActivityView. The Activity entity has a corresponding view entity called ActivityView.

Unlike a standard entity, a view entity does not have an underlying database table. The application does not persist view entities to the database. Instead of storing data, a view entity restricts the amount of data that a database query returns. A view entity does not represent or create a *materialized view*, which is a database table that caches the results of a database query.

Queries against a view entity type are actually run against the normal entity table in the database, as specified by the primaryEntity attribute of the view entity definition. The query against a view entity automatically adds any joins necessary to retrieve view entity columns if they include a bean path. However, access to view entity columns is not possible when constructing the query.

A view entity improves the performance on frequently used pages that list entities. Like other entities, you can subtype a view entity.

For example, the **My Activities** page uses a view entity, the ActivityDesktopView, which is a subtype of the ActivityView.

Because the application can export view entity types, it generates SOAP interfaces for them.

**Note:** If you create or extend a view entity that references a column that is of type="currencyamount", then you must handle the view entity extension in a particular manner. See “Extending an existing view entity with a currency column in ClaimCenter” on page 111 for details.

Guidewire defines view entities in the data model metadata files.

### Attributes of <viewEntity>

The <viewEntity> element contains the following attributes:

| <viewEntity> attribute | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | Default |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| abstract               | If true, you cannot create an instance of the entity type at runtime. Instead, you must declare a subtype entity with abstract=false, which you can instantiate. Any of the generated code is abstract.                                                                                                                                                                                                                                                                                | false   |
| desc                   | A description of the purpose and use of the entity.                                                                                                                                                                                                                                                                                                                                                                                                                                    | None    |
| displayName            | <p><i>Optional.</i> Creates a more human-readable form of the entity name. You can access this name using the following:</p> <pre>viewEntity.displayName</pre> <p>If you do not specify a value for the displayName attribute, then the <code>entityDisplayName</code> method returns the value of the entity attribute, instead. If you subtype an entity that has a specified display name, then the <code>entity.displayName</code> method returns the name of the subtype key.</p> | None    |

| <b>&lt;viewEntity&gt;<br/>attribute</b> | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | <b>Default</b> |
|-----------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| entity                                  | <i>Required.</i> Name of this viewEntity object.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | None           |
| exportable                              | <i>Unused.</i>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |                |
| extendable                              | If true, it is possible to extend this entity.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | true           |
| final                                   | If true, the entity definition is final and you cannot define any subtypes for it. If false, then you can define a subtype using this entity as the supertype.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | true           |
| platform                                | <i>Internal.</i> Do not use. The only real effect is to change the location in which the table appears in a data distribution report.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | false          |
| primaryEntity                           | <i>Required.</i> The primary entity type for this viewEntity object. The primary entity must be keyable. See “Data entities and the application database” on page 25 for information on keyable entities.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | None           |
| priority                                | For supertypes and subtypes, the priority of the corresponding subtype key.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | -1             |
| showRetiredBeans                        | Whether to show retired beans in the view.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | None           |
| supertypeEntity                         | <i>Optional.</i> The name of supertype of this entity.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | None           |
| typelistTableName                       | <p>If you create a non-final entity, then the application automatically creates a typelist to keep track of the subtypes of that entity. That typelist has an associated database table. If you do not specify a value for this attribute, then the application uses the name of the entity as the table name for the subtype typelist.</p> <p>However, the application places a restriction of 25 characters on the length of the database table name. You use this attribute to specify an alternate database table name for the typelist if an entity name is too long to become a valid typelist table name.</p> <p>It is not valid to use this attribute with entity types marked as final.</p> | None           |

### Subelements of <viewEntity>

The <viewEntity> elements contain the following subelements:

| <b>&lt;viewEntity&gt; subelement</b> | <b>Description</b>                                                                                                                                                                                           |
|--------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| computedcolumn                       | Specifies a column with row values that Guidewire computes while querying the database. For example, the values of a computed column might be the sum of the values from two database columns (col1 + col2). |
| computedtypekey                      | Specifies a typekey that has some type of transformation applied to it during querying from the database.                                                                                                    |
| fulldescription                      | See the discussion following the table.                                                                                                                                                                      |
| implementsInterface                  | See “<implementsInterface>” on page 67.                                                                                                                                                                      |
| viewEntityColumn                     | Represents a column in a viewEntity table.                                                                                                                                                                   |
| viewEntityLink                       | Uses to access another entity through a foreign key. Typically, you use this value within the application interface to create a link to that entity.                                                         |
| viewEntityName                       | Represents an entity name column in a viewEntity table. An entity name is a string column that contains the name of an entity that is suitable for viewing in the user interface.                            |
| viewEntityTypekey                    | Represents a typekey column in a viewEntity table.                                                                                                                                                           |

The *Data Dictionary* uses the fulldescription subelement. The following example illustrates how to use this element:

```
<fulldescription>
  <![CDATA[<p>Aggregates the information needed to display one activity row (base entity for all other
```

```
activity views).</p>]]>
</fulldescription>
```

The other subelements all require both a name and path attribute. The following code illustrates this:

```
<viewEntityName name="RelActAssignedUserName" path="RelatedActivity.AssignedUser"/>
```

Specify the path value relative to the `primaryEntity` on which you base the view.

The `computedcolumn` takes a required expression attribute and an additional, optional function attribute. The following is an example of a `computedcolumn`:

```
<computedcolumn name="Amount" expression="{1}" paths="LineItems.Amount" function="SUM"/>
```

The expression for this column can take multiple column values `{column_num}` passed from the user interface. For example, a valid expression is `{1} - {2}`, where `{1}` represents the first column and `{2}` represents the second column. The function value must be an SQL function that you can apply to this expression. The following are legal values:

- SUM
- AVG
- COUNT
- MIN
- MAX

**Note:** If the SQL function aggregates data, the application applies an SQL group automatically.

## <viewEntityExtension> elements and related data object types

The `<viewEntityExtension>` XML root element defines view entity extension data object types. You use the view entity extension entity to extend the definition of a `viewEntity` entity. See “<viewEntity> elements and related data object types” on page 46 for more information on view entities. Guidewire defines view entity extension data object types in the data model metadata files.

### Attributes of <viewEntityExtension>

The `<viewEntityExtension>` element contains the following attributes:

| <viewEntityExtension> attribute | Description                                                                                                                                                                                                                                                                                                                          | Default |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| entityName                      | <p><i>Required.</i> This value must match the file name of the <code>viewEntityExtension</code> that it extends.</p> <p>The application generates an error at resource verification if the value set that you set for the <code>entityName</code> attribute for a <code>viewEntityExtension</code> does not match the file name.</p> | None    |

### Subelements of <viewEntityExtension>

The `<viewEntityExtension>` element contains the following subelements:

| <viewEntityExtension> subelement | Description                                                                                                                                                                                                                 |
|----------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| computedcolumn                   | Specifies a column with row values that Guidewire computes while querying the database. For example, the values of a computed column might be the sum of the values from two database columns ( <code>col1 + col2</code> ). |



| <code>&lt;viewEntityExtension&gt;</code><br>subelement | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |
|--------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>computedTypekey</code>                           | Specifies a typekey that has some type of transformation applied to it during querying from the database.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| <code>description</code>                               | A description of the purpose and use of the entity.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <code>viewEntityColumn</code>                          | Represents a column in a <code>viewEntity</code> table. The <code>viewEntityColumn</code> element contains a path attribute that you use to define the entity path for the column: <ul style="list-style-type: none"> <li>The path attribute definition cannot traverse arrays.</li> <li>The path attribute is always relative to the primary entity on which you base the view.</li> </ul> <b>Note:</b> If you reference a column of type <code>currencyamount</code> , you must also define the <code>currencyProperty</code> specified in the original column definition on the <code>viewEntity</code> entity. See “Extending an existing view entity” on page 110 for an example of this. |
| <code>viewEntityLink</code>                            | Uses to access another entity through a foreign key. Typically, you use this value within the user interface to create a link to that entity.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| <code>viewEntityName</code>                            | Represents an entity name column in a <code>viewEntity</code> table. An entity name is a string column that contains the name of an entity that is suitable for viewing in the user interface.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
| <code>viewEntityTypekey</code>                         | Represents a typekey column in a <code>viewEntity</code> table.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |

#### Important caution

Guidewire strongly recommends that you not create a view entity extension—`viewEntityExtension`—that causes traversals into revisioned (effdated) data. Doing so has the possibility of returning duplicate rows if any revisioning in the traversal path splits an entity.

Instead, try one of the following:

- Denormalize the desired data onto a non-effdated entity.
- Add domain methods to the implementation of the View entity.

## Data entity subelements

This topic describes the subelements that you can use in metadata definition files. These subelements are:

- `<array>`
- `<column>`
- `<edgeForeignKey>`
- `<events>`
- `<foreignkey>`
- `<fulldescription>`
- `<implementsEntity>`
- `<implementsInterface>`
- `<index>`
- `<onetoone>`
- `<remove-index>`
- `<tag>`
- `<typekey>`

### Subelements for internal use only

Do not use the following entity subelements. Guidewire uses these subelements for internal purposes only.

- `<aspect>`
- `<checkconstraint>`
- `<customconsistencycheck>`
- `<datetimeordering>`
- `<dbcheckbuilder>`
- `<jointableconsistencycheck>`
- `<tableAugmenter>`
- `<validateapikeyinset>`
- `<validateapikeynotinset>`

### The deprecated attribute

The deprecated attribute applies to the following subelements:

- `<array>`
- `<column>`
- `<componentref>`
- `<edgeForeignKey>`
- `<foreignkey>`
- `<onetoone>`
- `<searchColumn>`
- `<typekey>`

The `deprecated="true"` attribute does not alter the database in any way. Instead, the deprecated attribute marks a data field as deprecated in the *Data Dictionary* and places a Deprecated annotation on the field in the *Guidewire Studio API Reference*. The deprecated attribute supports organizations that want to remove a field in a two-phase process.

In the first phase, you add the deprecated attribute to the field subelement. Studio indicates the field is deprecated whenever Gosu code references the field. During this first phase, developers work to remove the deprecated field from their code. The second phase occurs after developers remove all occurrences of the deprecated field.

In the second phase, you drop the field from the entity definition. In some cases, Guidewire will drop the column from the database automatically to synchronize the physical database with your revised data model. In most cases however, the DBA must alter the database with SQL statements run against the database to synchronize the database with your revised data model.

Guidewire generally recommends against using the deprecated attribute and the two-phase removal process. If you deprecate a field, Studio signals to the development team that the field is no longer used. The DBA does not receive this information. Over time, with a number of deprecated fields, the DBA manages an ever larger amount of unused information in the physical database. To avoid managing unused data, Guidewire strongly recommends that you keep your physical database and the data model of your application synchronized by dropping unused fields instead of deprecating them.

## `<array>`

An array defines a set of additional entities of the same type to associate with the main entity. For example, a `Claim` entity includes an array of `Document` entities.

## Attributes of &lt;array&gt;

The <array> element contains the following attributes:

| <array> attribute   | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | Default |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| arrayentity         | <i>Required.</i> The name of the entity that makes up the array.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | None    |
| arrayfield          | <i>Optional.</i> Name of the field in the array table that is the foreign key back to this table. However, you do not need to define a value if the array entity has exactly one foreign key back to this entity.<br><br>Note that even if you define only one foreign key explicitly, additional foreign keys may be created implicitly. For example, CreateUserID is automatically added to an editable entity. In that case, arrayfield would be required because there is more than one foreign key.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | None    |
| cascadeDelete       | If the cascadeDelete attribute is true, the array container is retireable, and you delete the array container; then the application deletes (or retires) the array elements also. If this scenario applies but the array container is not retireable, then the application deletes the array elements.<br><br><b>IMPORTANT:</b> Whether the cascadeDelete attribute is true or false, if the entity containing the array is not retireable, deleting an instance of that entity also deletes the objects inside the array. This behavior occurs even if the array objects themselves are retireable in order to preserve the array objects' referential integrity. For this reason, Guidewire recommends that you avoid placing an array of retireable objects inside an entity that is not retireable.                                                                                                                                                                                                                                                                                                                                                                                                                                             | false   |
| deprecated          | If true, then the application marks the item as deprecated in the <i>Data Dictionary</i> and places a Deprecated annotation on it in the Guidewire Studio API Reference.<br><br>If you deprecate an item, use the description to explain why.<br><br>See “Data entity subelements” on page 49 for more information.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | false   |
| desc                | A description of the purpose and use of the array.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | None    |
| exportable          | <i>Unused.</i>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |         |
| getterScriptability | See “Data objects and scriptability” on page 28 for information.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | all     |
| ignoreForEvents     | This attribute has complex, non-intuitive interactions with data entity relationships. See “Interactions of ignoreForEvents attribute with data entity relationships” on page 73 for information on how to use and configure it.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | false   |
| name                | <i>Required.</i> The name of the property corresponding to this array.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | None    |
| owner               | If true, then the entity containing the array owns the objects in the array. The owner attribute has the following behavior: <ul style="list-style-type: none"> <li>• If true (an <i>owned</i> array), then the method <code>isFieldChanged</code> returns true if a child element of the array has changed. If false (a <i>non-owned</i> array), then the method <code>isFieldChanged</code> returns false if a child element of the array has changed. In either case, <code>isFieldChanged</code> returns true if any child elements are added to or removed from the array, regardless of the value of the owner attribute.</li> <li>• If true and you delete the owning object, the application deletes (or retires) the array items as well. In this case, the application deletes the array items notwithstanding a value of false for the cascadeDelete attribute. However, see the important note following this description.</li> <li>• If true, then if you update the contents of the array, then the application considers the owner as updated as well. This behavior can result in side effects. For example, notwithstanding a value of false for the triggersValidation attribute, the behavior can trigger validation.</li> </ul> | false   |

| <array> attribute   | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | Default |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
|                     | <ul style="list-style-type: none"> <li>If true, then child array elements are copied along with the parent when the parent copied, such as with the copy method.</li> <li>If true, the effects of the owner attribute override the effects of false values for the cascadeDelete and triggersValidation attributes.</li> </ul> <hr/> <p><b>IMPORTANT:</b> Whether the owner attribute is true or false, if the entity containing the array is not retireable, deleting an instance of that entity also deletes the objects inside the array. This behavior occurs even if the array objects themselves are retireable in order to preserve the array objects' referential integrity. For this reason, Guidewire recommends that you avoid placing an array of retireable objects inside an entity that is not retireable.</p> <hr/> <p><b>Note:</b> The owner attribute must have a value of true if the corresponding array permits concurrent changes to its values. The application applies an optimistic locking approach for concurrent changes to array values. A true value for the owner attribute results in the application notifying the array of concurrent changes or additions to the array children. The notification is in the form of a concurrent data change exception (CDCE) in the user interface.</p> |         |
| requiredmatch       | <p>One of the following values</p> <ul style="list-style-type: none"> <li>all – There must be at least one matching row in the array for every row from this table. For example, there must be at least one check payee for every check.</li> <li>none – There is no requirement for matching rows.</li> <li>nonretired – There must be at least one matching row for every non-retired row from this table.</li> </ul>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | None    |
| setterScriptability | See “Data objects and scriptability” on page 28 for information.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | all     |
| triggersValidation  | <p>Whether changes to the entity pointed to by this array trigger validation. Changes to the array that trigger validation include:</p> <ul style="list-style-type: none"> <li>The addition of an object to the array.</li> <li>The removal of an object from the array.</li> <li>The modification of an object in the array.</li> </ul> <p>See the discussion of this attribute that follows this table.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               | false   |

If set to true, the triggersValidation attribute can trigger additional processing. Exactly what happens depends on several different factors:

- If the parent entity for the array is validatable, any modification to the array triggers the execution of the Preupdate and Validation rules on the parent entity. Validation occurs whenever the application attempts to commit a bundle that contains the parent entity. For an entity to be validatable, it must implement the Validatable delegate.
- If the parent entity has preupdate rules, but no validation rules, then the application executes the preupdate rules on the commit bundle. This is the case only if configuration parameter UseOldStylePreUpdate is set to true, which is the default. If UseOldStylePreUpdate is set to false, the application invokes the IPreUpdateHandler plugin on the commit bundle instead. Then, the application executes the logic defined in the plugin on the commit bundle.
- If the parent entity has validation rules, but no preupdate rules, then the application executes the validation rules on the commit bundle.
- If the parent entity has neither preupdate nor validation rules then the following occurs:
  - In the case of UseOldStylePreUpdate=true, the application does nothing.
  - In the case of UseOldStylePreUpdate=false, the application calls the IPreUpdateHandler plugin on the commit bundle.

- In any case, in ClaimCenter, any processing of the commit bundle excludes the Closed and Reopened validation rules.

### Subelements of <array>

The <array> element contains the following subelements:

| <array> subelement | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| array-association  | <p>This subelement contains the following attributes:</p> <ul style="list-style-type: none"> <li>• hasContains (default = false)</li> <li>• hasGetter (default = true)</li> <li>• hasSetter (default = false)</li> <li>• valueField (default = ID)</li> </ul> <p>It also contains the following subelements of its own, each of which can exist, at most, one time:</p> <ul style="list-style-type: none"> <li>• constant-map</li> <li>• subtype-map</li> <li>• typelist-map</li> </ul> <p>See “Typelist mapping associative arrays” on page 82 for more information.</p> |
| fulldescription    | See “<fulldescription>” on page 65.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| link-association   | <p>This subelement contains the following attributes:</p> <ul style="list-style-type: none"> <li>• hasGetter (default = true)</li> <li>• hasSetter (default = false)</li> <li>• valueField (default = ID)</li> </ul> <p>It also contains the following subelements of its own, each of which can exist, at most, one time:</p> <ul style="list-style-type: none"> <li>• constant-map</li> <li>• subtype-map</li> <li>• typelist-map</li> </ul> <p>See “Subtype mapping associative arrays” on page 79 for more information.</p>                                           |
| tag                | See “<tag>” on page 71.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |

## <column>

The <column> element defines a single-value field in the entity.

**Note:** For a discussion of <column-override>, see “Working with attribute and element overrides” on page 95 for details.

### Attributes of <column>

The <column> element contains the following attributes:

| <column> attribute          | Description                                                                                                                                                                                                                                                                   | Default |
|-----------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| allowInitialValueForUpgrade | Internal use.                                                                                                                                                                                                                                                                 | false   |
| autoincrement               | The name of a database sequence used as the source of values for the column. This attribute is applicable only for integer columns, and only for implementations where the application relies on the database for the sequence. Do not use the same sequence in more than one | None    |

| <column> attribute  | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Default |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
|                     | autoincrement column. There can be at most one autoincrement column per table.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |         |
| columnName          | <p><i>Optional.</i> If specified, the application uses this value as the column name of the corresponding database column. If you do not specify a columnName value, then the application uses the value of the name attribute for the database column name.</p> <p>The columnName attribute must be no more than 30 characters in length. It allows only unaccented Roman letters, numbers, and the underscore character. The first character of the columnName attribute must be a letter. Although the underscore character is allowable here, Guidewire discourages its use.</p> <p><b>IMPORTANT</b> All column names on a table must be unique within that table. Otherwise, Guidewire Studio™ displays an error if you verify the resource and the application server fails to start.</p> | None    |
| createhistogram     | <p>Whether to create a histogram on the column during an update to the database statistics.</p> <p><b>Note:</b> It is possible to override this attribute on an existing column in an extension (*.etx) file using the &lt;column-override&gt; element. You can use the override to turn off an existing histogram or to create one that did not previously exist.</p> <p>This change does not take effect during an upgrade. The change occurs only if you regenerate statistics for the affected table by using the Guidewire maintenance_tools command.</p> <p><b>See also</b></p> <ul style="list-style-type: none"> <li>• “Working with attribute and element overrides” on page 95</li> <li>• <i>InsuranceSuite System and Internal Tools</i></li> </ul>                                  | false   |
| default             | <p>Default value given to the field during new entity creation.</p> <p><b>Note:</b> For columns with the datetime data type, see <i>Plugins, Prebuilt Integrations, and SOAP APIs</i> for the syntax to use for the value of the default attribute.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | None    |
| deprecated          | <p>If true, then the application marks the item as deprecated in the <i>Data Dictionary</i> and places a Deprecated annotation on it in the Guidewire Studio API Reference.</p> <p>If you deprecate an item, use the description to explain why.</p> <p>For more information, see “Data entity subelements” on page 49.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | false   |
| desc                | A description of the purpose and use of the field.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | None    |
| exportable          | <i>Unused.</i>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |         |
| getterScriptability | See “Data objects and scriptability” on page 28 for information.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | all     |
| ignoreforevents     | <p>If you change, add, or remove an entity X that does not generate events, then the application searches for all event-generating entity instances that specify X. If the application finds any of these event-generating entity instances, it generates Changed events for those entity instances.</p> <p>To determine what entities reference a non-event-generating entity, the application examines the foreign keys and arrays that point to the entity. However, if you set ignoreForEvents to true on an entity that references the non-event-generating entity, then the application ignores that link as it determines what entities specify another entity.</p>                                                                                                                      | false   |

| <column> attribute                     | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | Default |
|----------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
|                                        | <ul style="list-style-type: none"> <li>At the entity level, the <code>ignoreForEvents</code> attribute means changes to (or addition or removal of) this entity do not cause Changed events to fire for any other entity.</li> <li>At the column level, the <code>ignoreForEvents</code> attribute means changes to this column do not cause the application to generate events.</li> </ul>                                                                                                                                                                                       |         |
| <code>loadable</code>                  | If true, you can load the field through staging tables. A staging table can contain a column mapping to the field.                                                                                                                                                                                                                                                                                                                                                                                                                                                                | true    |
| <code>loadedByCallback</code>          | <i>Internal.</i> If true, then the loading code does not use a default value or report a warning if the column is nullable without a default.                                                                                                                                                                                                                                                                                                                                                                                                                                     | false   |
| <code>name</code>                      | <p><i>Required.</i> The name of the column on the table and the field, or property, on the entity. The application uses this value as the column name <i>unless</i> you specify a <code>columnName</code> attribute. Use this name to access the column in data views, rules, and other application areas.</p> <p><b>IMPORTANT</b> All column names on a table must be unique within that table. Otherwise, Studio displays an error if you verify the resource and the application server fails to start.</p>                                                                    | None    |
| <code>nullok</code>                    | <p>Whether the column can contain null values.</p> <p>In general, this attribute is always true, as many tables include columns that do not require a value at different points in the process.</p>                                                                                                                                                                                                                                                                                                                                                                               | true    |
| <code>overwrittenInStagingTable</code> | <p><i>Internal.</i> If true and the entity is loadable, the loader process auto-populates the staging table during import.</p> <p><b>IMPORTANT</b> If set to true, do not attempt to populate the table yourself as the loader import process overwrites this table.</p>                                                                                                                                                                                                                                                                                                          | false   |
| <code>required</code>                  | Whether the column is required to be non-null upon initial construction of instances of the entity. See the <i>Gosu Reference Guide</i> .                                                                                                                                                                                                                                                                                                                                                                                                                                         | false   |
| <code>scalable</code>                  | Whether this value scales as the effective and expired dates change. This attribute applies only to number-type values. For example, you cannot scale a <code>varchar</code> . Also, it only applies to effective dated types.                                                                                                                                                                                                                                                                                                                                                    | false   |
| <code>setterScriptability</code>       | See “Data objects and scriptability” on page 28 for information.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | all     |
| <code>soapnullok</code>                | <i>Unused.</i>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    |         |
| <code>supportsLinguisticSearch</code>  | <p>Applies only to columns of <code>varchar</code>-based data types.</p> <ul style="list-style-type: none"> <li>If true, case insensitive searches can be performed on the denormalized version of the column.</li> <li>If false, searches are binary and cannot be performed on a denormalized version of the column.</li> </ul> <p>You cannot use this attribute with an encrypted column.</p> <p>The setting of this attribute does not affect locale or language used in searches.</p> <p>See also “Including data from subentities”.</p>                                     | false   |
| <code>type</code>                      | <p><i>Required.</i> Data type of the column, or field. In the base configuration, Guidewire defines a number of data types and stores their metadata definition files (*.dti) in under <b>configuration &gt; config &gt; datatypes</b>.</p> <p>Each metadata definition <i>file name</i> is the name of a specific data type. You use one of these data types as the <code>type</code> attribute on the &lt;column&gt; element. Thus, the list of valid values for the <code>type</code> attribute is the same as the set of .dti files in the application datatypes folders.</p> | None    |

| <column> attribute | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           | Default |
|--------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
|                    | <p>Each metadata definition also defines the <i>value type</i> for that data type. The value type determines how the application treats that value in memory.</p> <p>The name of the data type is not necessarily the same as the name of its value type. For example, for the <code>bit</code> data type, the name of the data type is <code>bit</code> and the corresponding value type is <code>java.lang.Boolean</code>. Similarly, the data type <code>varchar</code> has a value type of <code>java.lang.String</code>.</p> <p>The <code>datetime</code> data type is a special case. The application persists this data type in the application database using the <i>database</i> data type <code>TIMESTAMP</code>. This corresponds to the value type <code>java.util.Date</code>. In other words:</p> <ul style="list-style-type: none"> <li>• A column whose type is <code>datetime</code> is represented in memory as instances of <code>java.util.Date</code>.</li> <li>• This type of value is stored in the database as <code>TIMESTAMP</code>.</li> </ul> <p><b>Note:</b> Guidewire does not support the <code>unlimiteddecimal</code> data type in a persistent data model configuration.</p> <p><b>See also</b></p> <ul style="list-style-type: none"> <li>• “Data types” on page 139</li> <li>• “Customizing base configuration data types” on page 143</li> <li>• “Define a new data type” on page 146</li> </ul> |         |

### Subelements of <column>

The <column> element contains the following subelements:

| <column> subelement          | Description                                 | Default |
|------------------------------|---------------------------------------------|---------|
| <code>columnParam</code>     | See “<columnParam> subelement” on page 56.  | None    |
| <code>fulldescription</code> | See “<fulldescription>” on page 65.         | None    |
| <code>localization</code>    | See “<localization> subelement” on page 58. | None    |
| <code>tag</code>             | See “<tag>” on page 71.                     | None    |

## <columnParam> subelement

You use the <columnParam> element to set parameters that a column type requires. The type attribute of a column determines which parameters you can set or modify by using the <columnParam> subelement. You can determine the list of parameters that a column type supports by looking up the type definition in its `.dti` file.

For example, if you have a `mediumtext` column, you can determine the valid parameters for that column by examining file `mediumtext.dti`. This file indicates that you can modify the following attributes of a `mediumtext` column:

- `encryption`
- `logicalSize`
- `trimwhitespace`
- `validator`

Because you cannot modify the base configuration data type declaration files, you cannot see these files in Guidewire Studio. To view these files, navigate to the directory `modules/configuration/config/datatypes`.



The following example, from `Account.eti` in `PolicyCenter`, illustrates how to use this subelement to define certain column parameters.

```
<?xml version="1.0"?>
<entity xmlns="http://guidewire.com/datamodel"
  desc="An account is ..."
  entity="Account"
  ...
  table="account"
  type="retireable">
  ...
  <column desc="Business and Operations Description."
    name="BusOpsDesc"
    type="varchar">
    <columnParam name="size" value="240"/>
  </column>
  ...
</extension>
```

### Parameters that you can define by using <columnParam>

The following list describes the parameters that you can define by using <columnParam>, depending on which parameters are listed as valid in the `.dti` file of the data type.

| Parameter                | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|--------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| countryProperty          | Name of a property on the owning entity that returns the country to use for localizing the data format for this column.                                                                                                                                                                                                                                                                                                                                                                                                                      |
| createSpatialIndex       | Applies to columns with a spatial point data field. If the <code>createSpatialIndex</code> parameter has the default value of <code>true</code> in a column of an extension or custom entity, the application creates an index for spatial point data. If the parameter has a value of <code>false</code> , the application does not create such an index. In this case, the application avoids creating an index regardless of whether the parameter is in a <columnParam> subelement of a <column> element or a <column-override> element. |
| currencyProperty         | Name of a property on the owning entity that returns the currency for this column.                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| encryption               | Whether the application stores this column in encrypted format. This only applies to text-based columns.<br><br>Guidewire allows indexes on encrypted columns, or fields. However, because Guidewire stores encrypted fields as encrypted in the database, you must encrypt the input string and search for an exact match to it.                                                                                                                                                                                                            |
| exchangeRateProperty     | Name of a property on the owning entity that returns the exchange rate to use during currency conversions.                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| extensionProperty        | The name of a property on the owning entity whose value contains the phone extension.                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| logicalSize              | The size of this field in the user interface. You can use this value for <code>String</code> columns that do not have a maximum size in the database, such as <code>CLOB</code> objects. If you specify a value for the <code>size</code> parameter, then the <code>logicalSize</code> value must be less than or equal to the value of that parameter.                                                                                                                                                                                      |
| phoneCountrycodeProperty | The name of a property on the owning entity whose value contains the country with which to validate and format values.                                                                                                                                                                                                                                                                                                                                                                                                                       |
| precision                | The <i>precision</i> of the field. Precision is the total number of digits in the number. The <code>precision</code> parameter applies only if the data type of the field allows a precision attribute.                                                                                                                                                                                                                                                                                                                                      |
| scale                    | The <i>scale</i> of the field. Scale is the number of digits to the right of the decimal point. The <code>scale</code> parameter applies only if the data type of the field allows a scale attribute.                                                                                                                                                                                                                                                                                                                                        |
| secondaryAmountProperty  | Name of a property on the owning entity that returns the secondary amount related to this currency amount column.                                                                                                                                                                                                                                                                                                                                                                                                                            |
| size                     | Integer size value for columns of type <code>TEXT</code> and <code>VARCHAR</code> . This parameter specifies the maximum number of characters, not bytes, that the column can hold.                                                                                                                                                                                                                                                                                                                                                          |

| Parameter      | Description                                                                                                                                                                                                                                                              |
|----------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                | <b>WARNING</b> The database upgrade utility automatically detects definitions that lengthen or shorten a column. For shortened columns, the utility assumes that you wrote a version check or otherwise verified that the change does not truncate existing column data. |
| trimwhitespace | Applies to text-based data types. If true, then the application automatically removes leading and trailing white space from the data value.                                                                                                                              |
| validator      | The name of a ValidatorDef in fieldvalidators.xml. See “<ValidatorDef>” on page 154.                                                                                                                                                                                     |

**Note:** See “Overriding data type attributes” on page 97 for an example of using a nested <columnParam> subelement within a <column-override> element to set the encryption attribute on a column.

## <localization> subelement

**IMPORTANT:** Guidewire must enable archiving for the archiving functionality to be available.

For a discussion of the column <localization> element with examples on how to use it, see the *Globalization Guide*.

### Attributes of <localization>

The <localization> element contains the following attributes:

| <localization> attribute | Description                                                                                                         | Default |
|--------------------------|---------------------------------------------------------------------------------------------------------------------|---------|
| extractable              | Whether the localized data entity is marked as Extractable for archiving.                                           | false   |
| nullok                   | <i>Required.</i> Whether null values are allowed.                                                                   | None    |
| overlapTable             | Whether the localized data entity implements the OverlapTable delegate and is marked as OverlapTable for archiving. | false   |
| tableName                | The table name of the localized data table.                                                                         | None    |
| unique                   | <i>Required.</i> Whether values must be unique.                                                                     | false   |

## <edgeForeignKey>

You use the <edgeForeignKey> element to define a reference to another entity, in a manner similar to the “<foreignkey>” element. However, you use an edge foreign key in place of a standard foreign key to break a cycle of foreign keys in the data model. Guidewire defines this element in the data model metadata files as the <edgeForeignKey> XML subelement.

## The data model and circular references

A chain of foreign keys can form a cycle, also known as a circular reference, in the data model. As an example of a circular reference, entity type A has a foreign key to entity type B, and B has a foreign key to A. Circular references can occur with more extensive chains of foreign keys, such as A refers to B, which refers to C, which refers to A. The data model does not permit circular foreign keys reference, because the application can't determine a safe order for committing the entity instances in a circular reference to the database.

For example, entity types A and B have foreign key references to each other. The foreign keys create a circular reference. Suppose that a bundle contains a new instance of A and a new instance of B. The circular reference would cause a foreign key constraint to fail upon committing the bundle. If the application commits A before B is committed and in the database, a constraint failure occurs on the foreign key from A to B. The converse order of committing B before A causes a similar failure.

An edge foreign key in place of a standard foreign key resolves circular references so the application can determine a safe order for committing the entity instances within a cycle. An edge foreign key from A to B introduces a new,

hidden associative entity with a foreign key to A and a foreign key to B. The edge foreign key associates A and B without establishing foreign keys in the database directly between them. With an edge foreign key, the application can safely first commit new object A, then new object B, and finally the edge foreign key instance.

## Edge foreign keys in entity database tables

Unlike a standard foreign key, an edge foreign key does not correspond to an actual column in the database table of an entity type. Nor does an edge foreign key implement a database foreign key constraint. However, the *Data Dictionary* labels edge foreign keys as standard foreign keys. In Gosu code, you access edge foreign keys in the same manner that you access standard foreign keys.

## Edge foreign keys and associative database tables

An edge foreign key creates an *associative table* in the database. An associative table is essentially a table of foreign keys relationships. An associative table associates other database tables with each other but holds no other essential business data itself.

The associative table that implements an edge foreign key has two columns:

- OwnerID
- ForeignEntityID

If entity instance A has an edge foreign key to entity type B, the application creates a row in the edge foreign key table. The value in the row for OwnerID points to A and the value for ForeignEntityID points to B.

Every time you traverse, or dereference, the edge foreign key, the application loads the join array.

- If the array is of size 0, then the value of the edgeForeignKey is null.
- If the array is of size 1, then the application follows the ForeignEntityID on the row.

---

**IMPORTANT:** When you change a foreign key to an edge foreign key, you must write an upgrade trigger to populate the edge table for existing data. Otherwise, the associative edge table will remain empty even upon a database upgrade. Moreover, when you reference the edge foreign key, you will receive a null value.

---

## Edge foreign keys in Gosu

In Gosu code, edge foreign keys work in a manner similar to standard foreign keys. Just like a standard foreign key, you can query an edge foreign key and get and set its attributes.

## Edge foreign keys and performance

An edge foreign key has more performance issues than a standard foreign key, because the application must manage a separate table for the relationship. Queries must join an extra table. Nullability constraints in the database do not work with edge foreign keys, so you must enforce nullability constraints with extra Gosu code the you develop.

## Edge foreign keys and archiving

---

**IMPORTANT:** Guidewire must enable archiving before the archiving functionality is available.

---

If you add an edge foreign key to an entity that is part of the archive domain graph, set the extractable attribute on the edge foreign key to true. The value true causes the application to add the Extractable delegate to the associative table generated for the edge foreign key. If the extractable attribute of an edge foreign key on an entity that is part of the archive domain graph is not set to true, the server will not start. In addition, the entity type that the edge foreign key relates to must implement the Extractable delegate.

## When to use edge foreign keys

Use an edge foreign key only to avoid circular foreign key references in the data model. Circular foreign key references can prevent the application from determining a safe order for committing the entity instances in a circular reference to the database.

Use an edge foreign key instead of a standard foreign key in the following situations:

- An entity type has self-referencing foreign keys, including foreign keys between subtypes.
- A Group entity must specify its parent group.
- Entity type A has a foreign key to B, and entity type B has a foreign key to A.
- Cycles that involve more than two entity types.
- The primary member of an array requires a foreign key to its owner.

**Note:** `<edgeForeignKey>` is a valid subelement for all entity types, including entity extension types.

## Attributes of `<edgeForeignKey>`

The `<edgeForeignKey>` element contains the following attributes.

| <code>&lt;edgeForeignKey&gt;</code> attribute | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | Default |
|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| <code>createHistogram</code>                  | Whether to create a histogram on the column during an update to the database statistics.<br><br><b>Note:</b> It is possible to override this attribute on an existing column in an extension (*.etx) file using the <code>&lt;column-override&gt;</code> element. You can use the override to turn off an existing histogram or to create one that did not previously exist.<br><br>This change does not take effect during an upgrade. The change occurs <i>only</i> if you regenerate statistics for the affected table by using the Guidewire <code>maintenance_tools</code> command.<br><br><b>See also</b> <ul style="list-style-type: none"> <li>• “Working with attribute and element overrides” on page 95</li> <li>• <i>Administration Guide</i></li> </ul> | false   |
| <code>deprecated</code>                       | If true, then the application marks the item as deprecated in the <i>Data Dictionary</i> and places a Deprecated annotation on it in the Guidewire Studio API Reference.<br><br>If you deprecate an item, use the description to explain why.<br><br>For more information, see “Data entity subelements” on page 49.                                                                                                                                                                                                                                                                                                                                                                                                                                                 | false   |
| <code>desc</code>                             | A description of the purpose and use of the edge foreign key.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | None    |
| <code>edgeTableName</code>                    | The name of the edge table entity. If you do not specify a name, then the application creates one automatically by taking the <code>edgeTableName</code> attribute and capitalizing the first letter. If the <code>edgeTableName</code> contains an underscore, then all text up to and including the first underscore is discarded, and the first letter after the underscore is capitalized.                                                                                                                                                                                                                                                                                                                                                                       | None    |
| <code>edgeTableName</code>                    | <i>Required.</i> The name of the edge, or join array, table to create. As with other entity table names, the <code>edgeTableName</code> does not require a prefix such as <code>pc_</code> .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | None    |
| <code>exportable</code>                       | <i>Unused.</i>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |         |
| <code>exportasid</code>                       | <i>Unused.</i>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |         |
| <code>extractable</code>                      | Whether the edge entity is marked Extractable for archiving.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | false   |

| <edgeForeignKey> attribute      | Description                                                                                                                                                                                                                                                            | Default |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| fkentity                        | <i>Required.</i> The entity to which this foreign key points.                                                                                                                                                                                                          | None    |
| getterScriptability             | See “Data objects and scriptability” on page 28 for information.                                                                                                                                                                                                       | all     |
| ignoreForEvents                 | This attribute has complex, non-intuitive interactions with data entity relationships. See “Interactions of ignoreForEvents attribute with data entity relationships” on page 73 for information on how to use and configure it.                                       | false   |
| importableagainstexistingobject | If true and the entity is importable, or loadable, then the value in the staging table can be a reference to an existing object. This reference is the publicID of a row in the source table for the referenced object.                                                | true    |
| loadable                        | If true, then the application creates a staging table for the edge table.                                                                                                                                                                                              | false   |
| loadedByCallback                | <i>Internal.</i> If true, then the loading code does not use a default value or report a warning if the column is nullable without a default.                                                                                                                          | false   |
| name                            | <i>Required.</i> Specifies the name of the property on the entity.                                                                                                                                                                                                     | None    |
| nullok                          | Whether the column can contain null values. This value is meaningless for edgeForeignKey objects.                                                                                                                                                                      | true    |
| overlapTable                    | Whether the edge entity implements the OverlapTable delegate and is marked OverlapTable for archiving.                                                                                                                                                                 | false   |
| overwrittenInStagingTable       | <i>Internal.</i> If true and the edge table is loadable, the loader process auto-populates the staging table during import.<br><br><b>IMPORTANT</b> If set to true, do not attempt to populate the table yourself, as the loader import process overwrites this table. | false   |
| setterScriptability             | See “Data objects and scriptability” on page 28 for information.                                                                                                                                                                                                       | all     |
| soapnullok                      | <i>Unused.</i>                                                                                                                                                                                                                                                         |         |

## Subelements of <edgeForeignKey>

| <edgeForeignKey> subelement | Attributes | Description                         |
|-----------------------------|------------|-------------------------------------|
| fulldescription             | None       | See “<fulldescription>” on page 65. |
| tag                         | None       | See “<tag>” on page 71.             |

## <events>

If the <events> element appears within an entity, that entity and its descendant entities raise events. The names of the standard events are XXAdded, XXChanged, and XXRemoved, where XX is the name of the entity declaring the <events> element. A subtype or descendant of an event-aware entity can override the names of the standard events by declaring its own <events> element. The descendant can also add additional events.

If the <events> element does not appear in an entity or any of its ancestors, that entity does not raise any events.

**Note:** This element is not valid for a nonPersistentEntity.

Guidewire defines this element in the data model metadata files as the <events> XML subelement. There can be at most one <events> element in an entity. However, you can specify additional events through the use of <event> subelements. For example:

```
<events>
  <event>
    ...
  </event>
</events>
```

**Note:** The application automatically adds the EventAware delegate to any entity that contains the <events> element. This addition makes such an entity aware of events that it raises and that other entities raise.

### Attributes of <events>

There are no attributes on the <events> element.

### Subelements of <events>

The <events> element contains the following subelements.

| <events> subelement | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|---------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| event               | <p>Defines an additional event to fire for the entity. Use multiple &lt;event&gt; elements to specify multiple events. This subelement contains the following attributes:</p> <ul style="list-style-type: none"> <li>description (required = true)</li> <li>name (required = true)</li> </ul> <p>The &lt;event&gt; element requires each of these attributes.</p> <p>Note that the event name is exposed as a public static constant field. For example, you might have an event named PersonalDataPurge on the ABContact data entity. Such an event is exposed publicly as a constant field on the ABContact class. The public class-level String constant is ABContact.PERSONALDATAPURGE_EVENT.</p> |

## <foreignkey>

The <foreignkey> element defines a foreign key reference to another entity.

### Attributes of <foreignkey>

The <foreignkey> element contains the following attributes.

| <foreignkey> attribute | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | Default |
|------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| columnName             | <p><i>Optional.</i> If specified, the application uses this value as the column name of the corresponding database column. If you do not specify a columnName value, then the application uses the value of the name attribute for the database column name.</p> <p>The columnName attribute must be no more than 30 characters in length. It allows only unaccented Roman letters, numbers, and the underscore character. The first character of the columnName attribute must be a letter. Although the underscore character is allowable here, Guidewire discourages its use.</p> <p><b>Note:</b> As a common and recommended practice, use the suffix ID for the column name. For example, for a foreign key with the name Claim, set the columnName to ClaimID.</p> <p>Guidewire does not require that you use an ID suffix on names of foreign key columns. However, Guidewire strongly recommends that you adopt this practice to help you analyze the database and identify foreign keys.</p> <p><b>IMPORTANT</b> All column names on a table must be unique in that table. Otherwise, Guidewire Studio™ displays an error if you verify the resource, and the application server fails to start.</p> | None    |
| createConstraint       | If true, the database creates a foreign key constraint for this foreign key.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | true    |
| createbackingindex     | If true, the database automatically creates a backing index on the foreign key. true<br>If set to false, the database does not create a backing index.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |         |

| <foreignkey> attribute          | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | Default |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| createhistogram                 | <p>Whether to create a histogram on the column during an update to the database statistics.</p> <p><b>Note:</b> It is possible to override this attribute on an existing column in an extension (*.etx) file using the &lt;column-override&gt; element. You can use the override to turn off an existing histogram or to create one that did not previously exist.</p> <p>This change does not take effect during an upgrade. The change occurs only if you regenerate statistics for the affected table by using the Guidewire maintenance_tools command.</p> <p><b>See also</b></p> <ul style="list-style-type: none"> <li>“Working with attribute and element overrides” on page 95</li> <li>InsuranceSuite System and Internal Tools</li> </ul> | false   |
| deprecated                      | <p>If true, then the application marks the item as deprecated in the <i>Data Dictionary</i> and places a Deprecated annotation on it in the <i>Guidewire Studio API Reference</i>.</p> <p>If you deprecate an item, use the description to explain why.</p> <p>For more information, see “Data entity subelements” on page 49.</p>                                                                                                                                                                                                                                                                                                                                                                                                                  | false   |
| desc                            | A description of the purpose and use of the field.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | None    |
| existingreferencesallowed       | <p>If the following attributes are set to false, which is not the default:</p> <ul style="list-style-type: none"> <li>loadable</li> <li>importableagainstexistingobject</li> </ul> <p>then, the value in the staging table can only be a reference to an existing object.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                       | true    |
| exportable                      | Unused.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |         |
| exportasid                      | Unused.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |         |
| fkentity                        | Required. The entity to which this foreign key refers.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | None    |
| getterScriptability             | See “Data objects and scriptability” on page 28 for information.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | all     |
| ignoreForEvents                 | This attribute has complex, non-intuitive interactions with data entity relationships. See “Interactions of ignoreForEvents attribute with data entity relationships” on page 73 for information on how to use and configure it.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | false   |
| importableagainstexistingobject | If true and the entity is importable (loadable), then the value in the staging table can be a reference to an existing object. (This is the public ID of a row in the source table for the referenced object.)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      | true    |
| includeIdInIndex                | <p>If true, then include the ID as the last column in the backing index for the foreign key.</p> <p>This is useful if the access pattern in one or more important queries is to join to this table through the foreign key. You can then use the ID to probe into a referencing table. The only columns that you need to access from the table are this foreign key, and the retired and ID columns.</p> <p>In that case, adding the ID column to the index creates a covering index and eliminates the need to access the table.</p>                                                                                                                                                                                                               | false   |
| loadable                        | If true, you can load the field through staging tables. A staging table can contain a column for the public ID of the referenced entity.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | true    |
| loadedByCallback                | Internal. If true, then the loading code does not use a default value or report a warning if the column is nullable without a default.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | false   |

| <foreignkey> attribute    | Description                                                                                                                                                                                                                                                                                     | Default |
|---------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| name                      | <i>Required.</i> Specifies the name of the property on the entity. In ClaimCenter, if the <foreignkey> element is in an event-generating custom entity and points to an Exposure or ServiceRequest entity, the name attribute for the element must be Exposure or ServiceRequest, respectively. | None    |
| nonEffDated               | <i>PolicyCenter only. Internal.</i> This applies only to a foreign key that points to an effdated entity. If a foreign key points to an effdated entity and this attribute is true, then the application creates a real foreign key between the source and target entities.                     | false   |
| nullok                    | Whether the field can contain null values.                                                                                                                                                                                                                                                      | true    |
| overwrittenInStagingTable | <i>Internal.</i> If true (and the table is loadable), it indicates that the loader process auto-populates the staging table during import.<br><b>IMPORTANT</b> If set to true, do not attempt to populate the table yourself because the loader import process overwrites this table.           | false   |
| required                  | Whether the foreign key is required to be non-null upon initial construction of instances of the entity. See the <i>Gosu Reference Guide</i> .                                                                                                                                                  | false   |
| setterScriptability       | See “Data objects and scriptability” on page 28 for information.                                                                                                                                                                                                                                | all     |
| soapnullok                | <i>Unused.</i>                                                                                                                                                                                                                                                                                  |         |
| triggersValidation        | Whether changes to the entity referred to by this foreign key trigger validation.                                                                                                                                                                                                               | false   |

#### Subelements of <foreignkey>

The <foreignkey> element contains the following subelements.

| <foreignkey> subelement | Attributes | Description                         |
|-------------------------|------------|-------------------------------------|
| fulldescription         | None       | See “<fulldescription>” on page 65. |
| tag                     | None       | See “<tag>” on page 71.             |

## Entity properties for foreign keys between effdated entities in PolicyCenter

The foreign key of an entity having the effdated type matches a different property on the entity to which the foreign key refers than most foreign keys. Most foreign keys refer to and match on the value of another object's ID property. By contrast, the application compares the foreign key, the BranchValue property, and the SliceDate property of an effdated entity with four properties of a referenced entity. These four comparisons result in the source entity instance of the foreign key matching at most one target entity instance.

The target entity properties in the four comparisons are FixedID, BranchValue, EffectiveDate, and ExpirationDate. The comparisons for the FixedID and BranchValue properties on the source and target entities require exact matches. The comparison for the EffectiveDate property on the target entity requires the value of the SliceDate property on the source entity to be greater than or equal to the value of the EffectiveDate property on the target entity. The comparison for the ExpirationDate property requires the value of the SliceDate property to be less than the value of the ExpirationDate property.

The application restricts effdated entity instances with the same FixedID and BranchValue property values from having overlapping effective date ranges or EffectiveDate and ExpirationDate pairs. As a result, through the sequence of property comparisons, the source entity instance of the foreign key will match no more than one target entity instance.

The following table provides information about these target entity properties as well as the SliceDate property for the source entity:



| Property       | Type    | Meaning                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|----------------|---------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| FixedID        | Integer | Identifies a single object across effective date periods, both: <ul style="list-style-type: none"> <li>• Within the same period but with different effective dates</li> <li>• Across multiple periods but in one contractual policy period</li> </ul> This value must be non-null.                                                                                                                                                                                                                                                                                                                                                           |
| BranchValue    | Integer | <p><b>Note:</b> At the database layer, BranchValue is the BranchID column.</p> <p>Foreign key to the entity instance that contains this entity instance. Within the same branch, all entities must share the same BranchValue value. This value must be non-null. This is a read-only property.</p> <p>From Gosu, the foreign key property appears as the BranchValue property, although the database column name for that property is actually BranchID.</p>                                                                                                                                                                                |
| EffectiveDate  | Date    | Date the entity instance becomes effective.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| ExpirationDate | Date    | Date the entity instance expires or is no longer effective.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| SliceDate      | Date    | <p>The slice date is the current view, or slice, of the branch (and its entities) in effective time. If the slice date is null, the branch is being viewed/edited in window mode. Any edits made with the slice date set are made in that effective time, splitting the entity instance if necessary. This is a read-only property. To get this PolicyPeriod at a different slice date, use the <code>getAsOf</code> method.</p> <p><b>Note:</b> The application never persists the slice date value itself in the database. The slice date property exists as a special property on the in-memory entity instance that Gosu can access.</p> |

See also

- *Application Guide*

## <fulldescription>

The application uses the `fulldescription` subelement to populate the *Data Dictionary*. For example:

```
<fulldescription>
  <![CDATA[<p>Aggregates the information needed to display one activity row
    (base entity for all other activity views).</p>]]>
</fulldescription>
```

## <implementsEntity>

The `<implementsEntity>` subelement specifies that an entity implements the specified delegate. Guidewire calls an entity an *implementor* of a delegate if the entity specifies the delegate in an `<implementsEntity>` subelement.

**IMPORTANT:** Do not change the delegate that a Guidewire base entity implements by creating an extension entity that includes an `<implementsEntity>` subelement. The application generates an error if you do.

If a delegate definition includes the optional `requires` attribute, then the implementor must provide an `adapter` attribute on its `<implementsEntity>` subelement. The `adapter` attribute specifies the name of a Java or Gosu type that implements the interface that the delegate definition specifies in its own `requires` attribute.

For example, the PolicyCenter base configuration defines a `Cost` delegate as follows:

```
<?xml version="1.0"?>
<delegate ... name="Cost" requires="gw.api.domain.financials.CostAdapter">
  ...
</delegate>
```

The base configuration defines a `BACost` entity that includes an `<implementsEntity>` subelement, which specifies delegate with `name="Cost"`. Therefore, the `BACost` entity is an implementor of the `Cost` delegate.

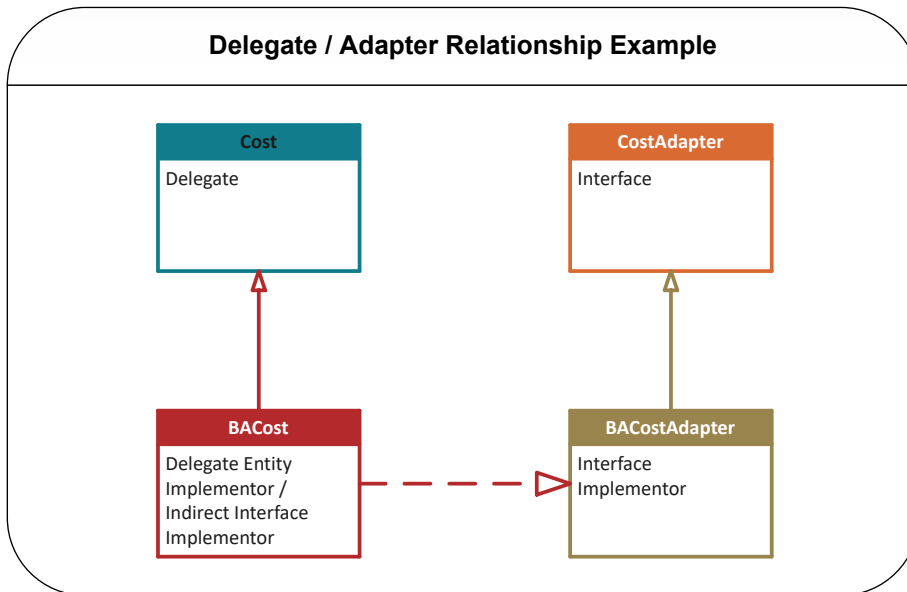
```
<?xml version="1.0"?>
<entity ... entity="BACost" ... >
```

```

...
<implementsEntity name="Cost" adapter="gw.lob.ba.financials.BACostAdapter" />
...
</entity>

```

The Cost delegate requires an implementation of the CostAdapter interface. So in its adapter attribute, the BACost entity specifies a BACostAdapter class, which implements the CostAdapter interface that the Cost adapter specifies in its requires attribute. The following diagram illustrates the relationship between the Cost delegate and the BACostAdapter adapter class:



Follow these rules for defining entities that implement delegates:

- If you specify a value for the requires attribute in a delegate, then implementers of the delegate must specify an adapter attribute in their definitions. The adapter attribute must specify the name of a Java or Gosu type that implements the interface specified by the requires attribute in delegate definition.
- If you do not specify a value for the requires attribute in a delegate, then implementers of the delegate must not specify an adapter attribute their definitions.

### The Extractable Delegate

**IMPORTANT:** Guidewire must enable archiving before the archiving functionality is available.

Entities that are part of the archive domain graph must implement the Extractable delegate. For example, if you create a custom subtype of Contact, then the custom subtype must implement the Extractable delegate.

If you add an edge foreign key to an entity that is part of the archive domain graph, set the extractable attribute on the edge foreign key to true. The value true causes the application to add the Extractable delegate to the associative table generated for the edge foreign key.

### Attributes of <implementsEntity>

The <implementsEntity> element contains the following attributes.

| <implementsEntity> subelement | Description                                                                |
|-------------------------------|----------------------------------------------------------------------------|
| name                          | <i>Required.</i> The name of the delegate that this entity must implement. |

### Subelements of <implementsEntity>

There are no subelements on the <implementsEntity> subelement.

## <implementsInterface>

The <implementsInterface> subelement specifies that an entity implements the specified interface. This element defines two attributes: an interface (iface) attribute and an implementation (impl) attribute.

For example, the PolicyCenter base configuration defines the BACost entity with the following <implementsInterface> subelement:

```
<entity ... entity="BACost" ...>
  ...
  <implementsInterface
    iface="gw.lob.ba.financials.BACostMethods"
    impl="gw.lob.ba.financials.BACostMethodsImpl"/>
</entity>
```

The BACostMethods interface has getter methods that any class that implements this interface must provide. The getter methods are coverage, state, and vehicle. By including the <implementsInterface> subelement, the BACost entity lets you use getter methods on instances of the BACost entity in Gosu code.

```
var cost : BACost
var cov    = cost.Coverage
var state  = cost.State
var vehicle = cost.Vehicle
```

### Attributes of <implementsInterface>

The <implementsInterface> element contains the following attributes.

| <implementsInterface> attribute | Description                                                                                                                                                      |
|---------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| iface                           | <i>Required.</i> The name of the interface that this data object must implement.                                                                                 |
| impl                            | The name of the class or subclass that implements the specified interface, if any. Optional when declared on abstract entities or delegates; required otherwise. |

### Subelements on <implementsInterface>

There are no subelements on the <implementsInterface> subelement.

## <index>

The <index> element defines an index on the database table used to store the data for an entity. Guidewire defines this element in the data model metadata files as the <index> XML subelement. This element contains a required subelement, which is <indexcol>.

The <index> element instructs the application to create an index on the physical database table. This index is in addition to those indexes that the application creates automatically.

An index improves the performance of a query search within the database. It consists of one or more fields that you can use together in a single search. You can define multiple <index> elements within an entity, with each one defining a separate index. If a field is already part of one index, you do not need to define a separate index containing only that field.

For example, ClaimCenter frequently searches non-retired claims for one with a particular claim number. Therefore, the Claim entity defines an index containing both the Retired and ClaimNumber fields. However, another common search uses just ClaimNumber. Since that field is already part of another index, a separate index containing only ClaimNumber is unnecessary.

A column used in an index cannot have a length of more than 1000 characters.

---

**IMPORTANT:** In general, the use of a database index has the possibility of reducing update performance. Guidewire recommends that you add a database index with caution. In particular, do not attempt to add an index on a column of type CLOB or BLOB. If you do so, the application generates an error message upon resource

verification. Regardless of the column type, add a database index only in the application, not outside of the application.

### Attributes of <index>

The <index> element contains the following attributes.

| <index> attribute    | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          | Default |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| desc                 | A description of the purpose and use of the index.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | None    |
| expectedtobecovering | If true, it indicates that the index covers all the necessary columns for a table that is to be used for at least one operation. For example, search by name is one such operation.<br><br>Thus, if true, it indicates that there is to be no table lookup. In this case, use the desc attribute to indicate which operation that is.                                                                                                                                                                                                                                                                                                                                                                | false   |
| name                 | <i>Required.</i> The name that identifies the index in the data model. The maximum length for the index name is 18 characters. Note that this is not the same as the physical database index name; for details of the physical database index name, see "Index naming convention" in <i>InsuranceSuite Database Administration</i> .<br><br><b>IMPORTANT</b> For <subtype> definitions, all index names must be unique between the subtype and supertype. In other words, do not duplicate an index name between the subtype definition in the <b>extensions</b> folder and its supertype in the <b>metadata</b> folder. Otherwise, the application generates an error on resource verification.     | None    |
| trackUsage           | If true, track the usage of this index.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | true    |
| unique               | Whether the values of the index are unique for each row.<br><br><b>Note:</b><br><br>When you set the unique attribute to true for an index on a subtype entity, the application generally sets the attribute to false upon index generation. In this case, the database does not define the index as unique. Rather, the application uses an integrity check to ensure that you do not add duplicates through the staging tables and a consistency check to highlight past duplicates.<br><br>Exceptional cases arise if the unique attribute is true and either the index consists of only a single column or the index has an ID column. In these cases, the database defines the index as unique. | false   |
| verifyInLoader       | If true, then the application runs an integrity check for unique indexes before loading data from the staging tables.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | true    |

### Subelements of <index>

The <index> element contains the following subelements.

| <index> subelement | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | Default |
|--------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| forceindex         | Use to force the application to create an index if running against a particular database.<br><br>This subelement is useful because the index generation algorithm can throw away some declared indexes as being redundant. In some cases, the application can require one or more of those indexes to work around an optimization problem.                                                                                                                                             | None    |
| indexcol           | <i>Required.</i> Defines a field that is part of the index. You can specify multiple <indexcol> elements to define composite indexes. This subelement contains the following attributes: <ul style="list-style-type: none"> <li>keyposition – <i>Required.</i> The position of the field within the index. The first position is 1.</li> <li>name – <i>Required.</i> The column name of the field. This name can be a column, foreignkey, or typekey defined in the entity.</li> </ul> | None    |

| <index><br>subelement | Description                                                                                                                                                                                                                                         | Default |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
|                       | <ul style="list-style-type: none"> <li>sortascending – If true, the default, then the sort direction is ascending. If false, then the sort direction is descending.</li> </ul> <p>The column cannot have a length of more than 1000 characters.</p> |         |

## <onetoone>

The <onetoone> element defines a single-valued association to another entity that has a one-to-one cardinality. Guidewire defines this element in the data model metadata files as the <onetoone> XML subelement. A one-to-one element functions in a similar manner to a foreign key in that it makes a reference to another entity. However, its purpose is to provide a reverse pointer to an entity or object that is pointing at the <onetoone> entity, through the use of a foreign key.

For example, entity A has a foreign key to entity B. If an instance of B can have at most one instance of A, then you can declare the <onetoone> element on B, to provide simple access to the associated A. In essence, using a <onetoone> element creates an *array-of-one*, with at most one element. Zero elements are also possible. As a best practice, make the foreign key non-nullable.

Note that you can declare a <onetoone> element only if the following conditions are true:

- The foreign key is backing only the <onetoone> and nothing else.
- The foreign key is either non-nullable or the entity declaring it is non-retireable.

If the above conditions are not true, then a consistency check is created instead.

**Note:** The application labels <onetoone> elements in the *Guidewire Data Dictionary* as foreign keys. You access these elements in Gosu code in the same manner as you access foreign keys.

### Attributes of <onetoone>

The <onetoone> element contains the following attributes.

| <onetoone> attribute                                                                                                                                                                                                                                                                                        | Description                                                                                                                                                                                                                                                                                                  | Default |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| cascadeDelete                                                                                                                                                                                                                                                                                               | If true, then the application deletes the entity to which the <onetoone> element points if you delete this entity.                                                                                                                                                                                           | false   |
| deprecated                                                                                                                                                                                                                                                                                                  | If true, then the application marks the item as deprecated in the <i>Data Dictionary</i> and places a Deprecated annotation on it in the Guidewire Studio API Reference.<br>If you deprecate an item, use the description to explain why.<br>For more information, see “Data entity subelements” on page 49. | false   |
| desc                                                                                                                                                                                                                                                                                                        | A description of the purpose and use of the field.                                                                                                                                                                                                                                                           | None    |
| exportable                                                                                                                                                                                                                                                                                                  | <i>Unused.</i>                                                                                                                                                                                                                                                                                               |         |
| fkentity                                                                                                                                                                                                                                                                                                    | <i>Required.</i> The entity to which this foreign key points.                                                                                                                                                                                                                                                | None    |
| getterScriptability                                                                                                                                                                                                                                                                                         | See “Data objects and scriptability” on page 28 for information.                                                                                                                                                                                                                                             | all     |
| ignoreForEvents                                                                                                                                                                                                                                                                                             | This attribute has complex, non-intuitive interactions with data entity relationships. See “Interactions of ignoreForEvents attribute with data entity relationships” on page 73 for information on how to use and configure it.                                                                             | false   |
| <b>IMPORTANT:</b> Do not confuse the definition of the ignoreForEvents attribute for the <onetoone> element with the definition of the attribute of the same namesake in <entity> and <array> elements. Although they share the same name, the two ignoreForEvents attributes are near opposites in effect. |                                                                                                                                                                                                                                                                                                              |         |

| <b>&lt;onetoone&gt; attribute</b> | <b>Description</b>                                                                                                                                                                                                                                                                                                                                                                              | <b>Default</b> |
|-----------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------|
| linkField                         | <i>Optional.</i> Specifies the foreign key field that points back to this object.                                                                                                                                                                                                                                                                                                               | None           |
| name                              | <i>Required.</i> Specifies the name property on the entity.                                                                                                                                                                                                                                                                                                                                     | None           |
| nullok                            | Whether the field can contain null values.                                                                                                                                                                                                                                                                                                                                                      | true           |
| owner                             | If true, this entity owns the linked object (the object to which the <onetoone> element points): <ul style="list-style-type: none"> <li>• If you delete the owning object, then the application deletes the linked object as well.</li> <li>• If you update the object pointed to by the &lt;onetoone&gt; element, then the application considers the owning object updated as well.</li> </ul> | false          |
| setterScriptability               | See “Data objects and scriptability” on page 28 for information.                                                                                                                                                                                                                                                                                                                                | all            |
| triggersValidation                | Whether changes to the entity pointed to by this entity trigger validation.                                                                                                                                                                                                                                                                                                                     | false          |

### Subelements of <onetoone>

The <onetoone> element contains the following subelements.

| <b>&lt;onetoone&gt; subelement</b> | <b>Description</b>                  | <b>Default</b> |
|------------------------------------|-------------------------------------|----------------|
| fulldescription                    | See “<fulldescription>” on page 65. | None           |
| tag                                | See “<tag>” on page 71.             | None           |

## <remove-index>

The <remove-index> element defines the name of a database index that you want to remove from the data model. It is valid for use with the following data model elements:

- <entity>
- <extension>

You can use this element to safely remove a non-primary key index if it is one of the following:

- non-unique
- unique and contains an ID column

Guidewire performs metadata validation to ensure that the <remove-index> element removes only those indexes that fall into one of these categories.

**Note:** Adding, removing, or changing indexes results in changes to server performance. Before making modifications to indexes, consult with your database administrator or Guidewire representative. Include performance testing in your modification plan.

**Note:** The <remove-index> element does not remove spatial point indexes. To remove a spatial point index for an extension entity, override a spatial point <column> element with a createSpatialIndex parameter having a value of false.

See “Overriding data type attributes” on page 97 for an example of using a nested <columnParam> subelement within a <column-override> element to set an attribute on a column. Use the same pattern to set the createSpatialIndex parameter to false and to remove a spatial point index for an extension entity. See also “<columnParam> subelement” on page 56 for a description of the createSpatialIndex parameter.

### The index is non-unique

You can safely remove a non-primary key index with the unique attribute set to false. In general, these are indexes that Guidewire provides for performance enhancement. It is safe to remove these kinds of indexes.

### The index is unique and contains an ID column

You can safely remove a non-primary key index with the unique attribute set to `true` if that index also includes ID as a key column. An ID column is already unique by itself, and so a multicolumn index that includes the ID column does not enforce a uniqueness condition. Thus, it is safe to remove these kinds of indexes.

For example, the `WorkItem` entity contains the following index definition:

```
<index desc="Covering index to speed up checking-out of work items and they involve search on status"
  name="WorkItemIndex2" unique="true">
  <indexcol keyposition="1" name="status"/>
  <indexcol keyposition="2" name="Priority" sortascending="false"/>
  <indexcol keyposition="3" name="CreationTime"/>
  <indexcol keyposition="4" name="ID"/>
</index>
```

Although the unique attribute is set to `true`, you can safely remove this index because the index definition contains an ID column; in this example, `keyposition="4"`.

### Attributes of <remove-index>

The `<remove-index>` element contains the following attributes.

| <code>&lt;remove-index&gt;</code> attribute | Description                                                                                       | Default |
|---------------------------------------------|---------------------------------------------------------------------------------------------------|---------|
| name                                        | Name of the index to remove. The index name is defined by the <code>&lt;index&gt;</code> element. | None    |

### Modifying an index

In many cases, you want to modify an existing database index. In that case, use the `<remove-index>` element to remove the index, and then add a new index with the same name that contains the desired characteristics.

## <tag>

The `<tag>` element defines a data model annotation. You can use predefined tags to add your own metadata to the data model.

The file `modules/configuration/config/metadata/tags.lst` specifies the list of valid tags that you can use. You cannot add a new tag to this file.

At most one of each tag can exist on a field at one time. Extensions can add, but not override, tags on existing fields.

### Attributes of <tag>

The `<tag>` element contains the following attributes.

| <code>&lt;tag&gt;</code> attribute | Description                           | Default |
|------------------------------------|---------------------------------------|---------|
| name                               | <i>Required.</i> The name of the tag. | None    |
| value                              | The value of the tag.                 | None    |

### Use of the <tag> subelement in archiving

One of the common uses for the `<tag>` subelement is adding the element to a foreign key to assist in resolving certain archive domain graph issues. See *Data Archiving*.

## <typekey>

The `<typekey>` element defines a field for which a typelist defines the values. Guidewire defines this element in the data model metadata files as the `<typekey>` XML subelement.

**Note:** For information on typelists, typekeys, and keyfilters, see “Working with typelists” on page 159.

## Attributes of <typekey>

The <typekey> element contains the following attributes.

| <typekey> attribute       | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        | Default |
|---------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| columnName                | <p><i>Optional.</i> If specified, the application uses this value as the column name of the corresponding database column. If you do not specify a <code>columnName</code> value, then the application uses the value of the <code>name</code> attribute for the database column name.</p> <p>The <code>columnName</code> attribute must be no more than 30 characters in length. It allows only unaccented Roman letters, numbers, and the underscore character. The first character of the <code>columnName</code> attribute must be a letter. Although the underscore character is allowable here, Guidewire discourages its use.</p> <p><b>IMPORTANT</b> All column names on a table must be unique within that table. Otherwise, Guidewire Studio displays an error if you verify the resource and the application server fails to start.</p> | None    |
| createhistogram           | <p>Whether to create a histogram on the column during an update to the database statistics.</p> <p><b>Note:</b> It is possible to override this attribute on an existing column in an extension (*.etx) file using the &lt;column-override&gt; element. You can use the override to turn off an existing histogram or to create one that did not previously exist.</p> <p>This change does not take effect during an upgrade. The change occurs <i>only</i> if you regenerate statistics for the affected table by using the Guidewire <code>maintenance_tools</code> command.</p> <p><b>See also</b></p> <ul style="list-style-type: none"> <li>“Working with attribute and element overrides” on page 95</li> <li><i>InsuranceSuite System and Internal Tools</i></li> </ul>                                                                     | false   |
| default                   | The default value given to the field during new entity creation.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | None    |
| deprecated                | <p>If true, then the application marks the typekey as deprecated in the <i>Data Dictionary</i> and places a Deprecated annotation on it in the <i>Guidewire Studio API Reference</i>.</p> <p>If you deprecate a typekey, use the <code>description</code> attribute (<code>desc</code>) to explain why.</p> <p>For more information, see “Data entity subelements” on page 49.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | false   |
| desc                      | A description of the purpose and use of the field.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | None    |
| exportable                | <i>Unused.</i>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |         |
| getterScriptability       | See “Data objects and scriptability” on page 28 for information.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | None    |
| loadable                  | If true, then you can load the field through staging tables. A staging table can contain a <code>true</code> column, as a <code>String</code> , for the code of the typekey.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |         |
| loadedByCallback          | <i>Internal.</i> If true, then the loading code does not use a default value or report a warning false if the column is nullable without a default.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |         |
| name                      | <i>Required.</i> Specifies the name of the property on the entity                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | None    |
| nullok                    | Whether the field can contain null values.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | true    |
| overwrittenInStagingTable | <p><i>Internal.</i> If true and the typekey is loadable, the loader process auto-populates the typekey in the staging table during import.</p> <p><b>IMPORTANT</b> If set to true, do not attempt to populate the typekey yourself because the loader import process overwrites this typekey.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  | false   |
| required                  | Whether the typekey is required to be non-null upon initial construction of instances of the entity. See the <i>Gosu Reference Guide</i> .                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         | false   |
| setterScriptability       | See “Data objects and scriptability” on page 28 for information.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   | None    |



| <typekey> attribute | Description                                                                                                                                                                                | Default |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| soapnullok          | <i>Unused.</i>                                                                                                                                                                             |         |
| typefilter          | The name of a filter associated with the typelist. See “Static filters” on page 171 for additional information.                                                                            | None    |
| typelist            | <i>Required.</i> The name of the typelist from which this field gets its value.<br><b>See also</b> <ul style="list-style-type: none"> <li>“Working with typelists” on page 159.</li> </ul> | None    |

### Subelements of <typekey>

The <typekey> element contains the following subelements.

| <typekey> subelement | Description                                                                                                                                                         | Default |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| keyfilters           | Defines one or more <keyfilter> elements. There can be at most one <keyfilters> element in an entity. See “Dynamic filters” on page 176 for additional information. | None    |
| fulldescription      | See “<fulldescription>” on page 65.                                                                                                                                 | None    |
| tag                  | “<tag>” on page 71                                                                                                                                                  | None    |

### Subelements of <keyfilters>

The <keyfilters> element contains the following subelements.

| <keyfilters> subelement | Description                                                                                                                                                                                                                                                                                                                                                       | Default |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| <keyfilter>             | Specifies a keyfilter to use to filter the typelist. This element requires the <name> attribute. This attribute defines a relative path, navigable through Gosu dot notation, to a <i>physical</i> data field. Each element in the path must be a data model field.<br><b>Note:</b> You can include multiple <keyfilter> elements to specify multiple keyfilters. | None    |

## Interactions of ignoreForEvents attribute with data entity relationships

The interactions of the ignoreForEvents attribute with relationships between data entities produce what might appear to be unpredictable results. At the very least, the results are not intuitive.

However, an understanding of event generation and behavior, the definitions of the ignoreForEvents attribute, the direction of data entity relationships, and the behavior of the relationships in light of the attribute helps to predict such outcomes. Such an understanding will also help you to design predictable data entity relationships.

To this end, Guidewire recommends that you configure both sides of a data entity relationship with the same ignoreForEvents attribute value. For example, in an <array>-<foreignkey> pair and a <onetoone>-<foreignkey> pair, use the same ignoreForEvents attribute value for both pair members when possible. This topic assumes that you have followed this recommendation.

**IMPORTANT:** With the ignoreForEvents attribute, the application rules out specific entities or activities for event generation. The attribute eliminates events where they are not needed. Hence, the attribute results in improved performance. Set the ignoreForEvents attribute to true as appropriate to ensure optimal performance. This performance improvement is especially important for any entity shared among graph entities, such as Claim, Policy, or Account.

**Note:** As a shorthand, this topic sometimes refers to entities declaring foreign key relationships and one-to-one relationships as foreign key entities and one-to-one entities respectively. Similarly, as a shorthand, the topic

sometimes refers to the `<foreignkey>` and `<one-to-one>` elements themselves as foreign keys and one-to-ones. The topic refers to the entity to which a foreign key or one-to-one points as the target entity or target.

### Event generation and behavior

The application automatically generates standard events for most top-level objects. Generally, this event generation occurs upon any addition, modification, removal, or retirement of a top-level entity instance. The occurrence of an event results in the `EventFired` rule set executing. When this rule set is executed, the application generates messages using the context of the event.

### `ignoreForEvents` attribute definitions

The `ignoreForEvents` attribute has three definitions, a first for entities, a second for most relationships between entities, and a third for `<onetooone>` entity relationships. With respect to an entity, the value of the `ignoreForEvents` attribute determines whether to ignore the entity when generating events. With respect to an entity relationship, the `ignoreForEvents` attribute generally helps to rule out the relationship entity or an activity for event generation. With respect to a `<onetooone>` entity relationship, the `ignoreForEvents` attribute, when `true`, reverses the direction of the parent for a backing `<foreignkey>` element. This topic concerns only the interactions of the `ignoreForEvents` attribute with data entity relationships.

**Note:** The application implements `<onetooone>` entity relationships as arrays, not as a pair of singleton entities. Thus, if an entity B has a `<onetooone>` entity relationship with an entity A, a parent-child direction exists for that relationship. Moreover, B is the parent of A.

### Direction of data entity relationships

With respect to events, every relationship between two entities—foreign key, edge foreign key, one-to-one, and array—has a *direction*. This direction determines which entity in the relationship is the parent and which is the child.

For any given entity relationship, suppose that the parent entity generates events and the child entity does not generate events. In this scenario, when an instance of the child entity changes, the application raises a `ParentChanged` event. The key in such a scenario is that the relationship between the two entities must flow in the direction of an `EventAware` entity, a parent entity, to generate an `Event`. `EventAware` entities cannot be event children.

For a given entity relationship, first determine whether the preceding scenario is present. If so, the relationship is relevant. If not, the relationship is not relevant. Moreover, if the rules lead to a parent not being a `EventAware` entity and to the Child being an `EventAware` entity, no event relationship exists between the two entities. `EventAware` entities cannot have event parents because updates are not propagated through `EventAware` entities.

Second, determine which entity is the parent and which entity is the child in accordance with the following logic:

- An array entity is always the event parent. The entity with the array is the event parent, and the entity with the foreign key is the child.
- With a one-to-one relationship, the entity declaring the `<onetooone>` element is the event parent unless the value of the `ignoreForEvents` attribute on the one-to-one entity is `true`. The application implements one-to-one entities as arrays. In this implementation, the `ignoreForEvents` attribute only reverses the direction of the relationship between the array and a backing foreign key when the attribute has a value of `true`. When the `ignoreForEvents` attribute has a value of `false`, the one-to-one entity behaves exactly as an array with no reversal of ownership direction. Moreover, the one-to-one entity is the parent, and the foreign key entity is the child. If the `ignoreForEvents` attribute has a value of `true`, the foreign key entity is the parent, and the one-to-one entity is the child.
- With a foreign key relationship, the entity declaring a foreign key is the event parent, and the target entity of the foreign key is the event child. An exception to this scenario occurs if the foreign key backs an array. If the foreign key backs an array, the foreign key entity is the event child.
- With an edge foreign key relationship, the entity declaring an edge foreign key is the event parent, and the target entity of the edge foreign key is the event child.

### Behavior of data entity relationships in light of the ignoreForEvents attribute value

Determine the effects of the value of the `ignoreForEvents` attribute on data entity relationships in accordance with the following logic:

- If the value of the `ignoreForEvents` attribute on a relationship is `true`, ignore the relationship for event generation unless the relationship involves a one-to-one entity.

For a relationship involving a one-to-one entity, consider the relationship for event generation as you would an array with a reversed direction of events. In this case, events propagate from the one-to-one entity to the foreign key entity if the latter is `EventAware` and the former is not.

For a relationship involving a foreign key or edge foreign key, update events generally will propagate from the target object of the foreign key or edge foreign key. That is, the events generally will propagate by virtue of the target object alone being changed.

However, consider an important exception to this generalization. Suppose that the only fields changed on the target object of the foreign key or edge foreign key have a value of `true` for the `ignoreForEvents` attribute. In this case, the database reflects the changes. However, the application does not consider the target object updated for the purpose of event generation. Whether the foreign key backs an array is irrelevant for this behavior. This behavior allows you to update internal bookkeeping fields without raising events.

Suppose that you update only an entity that is not `EventAware`. Although that entity alone will not result in event generation, an `EventAware` parent or grandparent having an `ignoreForEvents` attribute with a value of `false` will. The `EventAware` parent or grandparent in this case will generate events on the basis of the changed child or grandchild that is not `EventAware`.

- If the value of the `ignoreForEvents` attribute is `false`, then evaluate the relationship for event generation in accordance with the following logic:
  - Events propagate from any changed entity that is not `EventAware` and that has an `ignoreForEvents` attribute with a value of `false`. The propagation proceeds along a chain of entities from child to parent so long as the last entity in the chain is `EventAware`.
  - For a relationship involving a one-to-one entity, consider the relationship for event generation as you would an array without a reversed direction of event propagation. In this case, events propagate from the foreign key entity to the one-to-one entity if the latter is `EventAware` and the former is not.

### Examples

#### 1. Array and foreign key

Suppose that an entity A has a foreign key to entity B and that the foreign key backs an array. Moreover, suppose that B has an array of A. In this case, B is the parent, and A is the child. The `ignoreForEvents` used is set on the array entity. In this case, the `ignoreForEvents` used is set on the array, but Guidewire recommends applying the same setting both to the array and to the backing foreign key.

If the foreign key does not back an array, then A is the parent, and B is the child.

#### 2. Array and multiple foreign keys

Suppose the following:

- `Claim` has an array of `ABC_NonEAEntity` with an `ignoreForEvents` value of `false`.
- `ABC_NonEAEntity` has a foreign key, `ClaimID`, to `Claim` with an `ignoreForEvents` value of `true`.
- `ABC_NonEAEntity` has a foreign key, `ABC_AnotherNonEAEntityID`, to `ABC_AnotherNonEAEntity` with an `ignoreForEvents` value of `true`.

Given this premise, the following is true:

- `Claim` is an event parent of `ABC_NonEAEntity`, because of the array.

That the foreign key of `ABC_NonEAEntity` to `Claim` has an `ignoreForEvents` attribute with a value of `true` does not matter. This irrelevance is because the foreign key backs an array.

In addition, under ordinary circumstances, `ABC_NonEAEntity` would be an event parent of `ABC_AnotherNonEAEntity`. However, the application does not treat `ABC_NonEAEntity` as an event parent because the foreign key has an `ignoreForEvents` attribute with a value of `true`.

- Given this configuration, the following actions will raise `ClaimChanged` events:
  - Changing a field on `Claim`
  - Adding a new `ABC_NonEAEntity` to `Claim`
  - Removing an existing `ABC_NonEAEntity` from `Claim`
  - Updating any combination of fields on an existing `ABC_NonEAEntity`, except any combination that includes only `ClaimID` or `ABC_AnotherNonEAEntityID`. The exclusion is due to the latter fields having an `ignoreForEvents` attribute with a value of `true`.
- The following independent actions will not raise `ClaimChanged` events:
  - Updating an existing `ABC_AnotherNonEAEntity`
  - Changing an existing `ABC_NonEAEntity` to point to a different `ABC_AnotherNonEAEntity`

The reason for no raising of `ClaimChanged` events is because the only field that changed, `ABC_AnotherNonEAEntityID`, has an `ignoreForEvents` attribute with a value of `true`.

### 3. Foreign key and array

Suppose the following:

- Entity `E` has a foreign key `F` with an `ignoreForEvents` value of `true`.
- An instance of `E` undergoes a change only to the field for `F`.
- No other change would cause the application to raise `EChanged` or `XYZChanged` events.

Given this premise, the following is true:

- The application will not consider the instance of `E` to have changed for event generation purposes.
- If `E` is an `EventAware` entity, the application will not raise an `EChanged` event.
- If `E` is not an `EventAware` entity, the application will not raise an `XYZChanged` event for any event parents of `E`.

Suppose further that foreign key `F` backs an array. Given this elaboration, the following is true:

- Only the array is used to determine the event propagation relationship between the foreign key and the array.
- If the array has an `ignoreForEvents` attribute with a value of `true`, the application will not propagate events between the two entities.
- Suppose that the foreign key has an `ignoreForEvents` attribute with a value of `true`. In this case, the application ignores changes to any instance of the foreign key for deciding whether a bean has changed for the purpose of event generation.

# Working with associative arrays

This topic describes the different types of associative arrays that Guidewire provides as part of the base data model configuration.

## Related concepts

“Overview of associative arrays” on page 77

“Subtype mapping associative arrays” on page 79

“Typelist mapping associative arrays” on page 82

## Overview of associative arrays

An associative array provides a mapping between a set of *keys* and *values* that the keys represent. A common example of such a mapping is a telephone book. A telephone book maps names of people in a jurisdiction to telephone numbers. Another common example is a dictionary. A dictionary maps terms to their definitions.

To expand on this concept, a telephone book contains a set of names. Each name in the set is a key, and each associated telephone number is a value. Using array-like notation, you can write:

```
telephonebook[peter] = 555-123-1234
telephonebook[shelly] = 555-234-2345
...
```

**Note:** When you modify an array such that it is an associative array, the array continues to exhibit its original properties as an indexed array. You can use an associative array just as you would an indexed array. For example, you can use an associative array in `for` loops.

InsuranceSuite uses associative arrays to expose array values as a type-safe map within Gosu code. The following example uses a typekey from a `State` typelist as a mapping index for an associative array of state capitals:

| State typekey index  | Maps to...  |
|----------------------|-------------|
| Capital[State.TC_AL] | Montgomery  |
| Capital[State.TC_AK] | Juneau      |
| Capital[State.TC_AZ] | Phoenix     |
| Capital[State.TC_AR] | Little Rock |

Two tasks are required to work with an associative array in Gosu:

- Exposing the key set to the type system
- Calculating the value from the key

This topic includes the following:

- “Associative array mapping types” on page 78
- “Scriptability and associative arrays” on page 78
- “Setting array member values” on page 79

## Associative array mapping types

An associative array must have a key that maps to a value. The mapping type describes what the application uses as the key and what value that key returns.

| Mapping type     | Key            | Value                               | More information                                 |
|------------------|----------------|-------------------------------------|--------------------------------------------------|
| Subtype mapping  | Entity subtype | Implicit subtype field on an entity | “Subtype mapping associative arrays” on page 79  |
| Typelist mapping | Typelist       | Typekey field on the entity         | “Typelist mapping associative arrays” on page 82 |

To implement an associative array, add one of the following elements to an `<array>` element in the data type definition file. The number of results that each returns—the cardinality of the result set—depends on the element type.

`<link-association>` Returns at most one element. The return type is an object of the type of the array.

`<array-association>` Returns an array of results that match the value. The number of results can be zero, one, or more.

Each `<array>` element in a data type definition file can have zero or one of each of these elements.

As an example, in the ClaimCenter Claim definition file (**configuration > config > Metadata > Entity > Claim.eti**), you see the following XML (simplified for clarity):

```
<entity xmlns="http://guidewire.com/datamodel"
  entity="Claim"
  table="claim"
  type="retireable">
  ...
  <array arrayentity="ClaimMetric"
    desc="Metrics related to this claim."
    exportable="false"
    ignoreforevents="true"
    name="ClaimMetrics"
    triggersValidation="false">
    <link-association>
      <subtype-map/>
    </link-association>
    <array-association>
      <typelist-map field="ClaimMetricCategory"/>
    </array-association>
  </array>
  ...
</entity>
```

See also

- “Subtype mapping associative arrays” on page 79.
- “Typelist mapping associative arrays” on page 82.

## Scriptability and associative arrays

It is possible to set the following attributes on each `<link-association>` and `<array-association>` element:

- `hasGetter`
- `hasSetter`

For example:

```
<link-association hasGetter="true" hasSetter="true">
  <typelist-map field="TAccountType"/>
</link-association>
```

For these attributes:

- If `hasGetter` is true, you can read the property.
- If `hasSetter` is true, you can update the property.

**Note:** If you do not specify either of these attributes, the default is `hasGetter="true"`.

See also

- “Data objects and scriptability” on page 28.

## Setting array member values

There are several restrictions on setting associative array member values, including:

- You can use a query builder expression to retrieve a specific entity instance. However, the result of the query is read-only. You must add the retrieved entity to a bundle to be able to manipulate its fields. To work with bundles, use one of the following:

```
var bundle = gw.transaction.Transaction.getCurrent()
gw.transaction.Transaction.runWithNewBundle(\ bundle -> ) //Use this version in the Gosu tester
```

- You can set array values only on fields that are database-backed fields, not on fields that are derived properties. To determine which fields are derived, consult the *Data Dictionary*.

See also

- *Plugins, Prebuilt Integrations, and SOAP APIs*

## Subtype mapping associative arrays

Use subtype mapping to access array elements based on their subtype. This type of associative array divides the elements of the array into multiple partitions. Each of these partitions contains array elements of only a particular object *subtype*.

For example, in PolicyCenter, you can define an associative array called `JobTypes` on the `Policy` object. In the `Policy` definition file (**configuration > config > Metadata > Entity > Claim.eti**), you can add the `<array>` element from the following example XML:

```
<entity xmlns="http://guidewire.com/datamodel"
  entity="Policy"
  table="policy"
  type="retireable">
  ...
  <array arrayentity="Job"
    desc="Types of jobs pertaining to a policy."
    exportable="false"
    ignoreforevents="true"
    name="JobTypes">
    <link-association>
      <subtype-map/>
    </link-association>
  </array>
  ...
</entity>
```

The `JobTypes` array contains a number of objects. Each of these objects is a subtype of a `Job` object. The data model defines the associative array using the `<link-association>` element. A link association returns at most one element. The return type is an object of the type of the array. In this case, the return type is an object of type `Job` or, more specifically, one of its subtypes.

The PolicyCenter data model defines a number of subtypes of the `Job` object, including:

- Audit
- Cancellation
- Issuance
- PolicyChange
- Reinstatement
- Renewal
- Rewrite
- RewriteNewAccount
- Submission

To determine the complete list of subtypes on an object, consult the *Data Dictionary*. The dictionary organizes the subtypes into a table at the top of the dictionary page with active links to sections that describe each subtype in greater detail.

## Working with array values by using subtype mapping

To retrieve an array value through subtype mapping, use the following syntax:

```
base-entity.subtype-map.property
```

Each field has the following meanings:

| Field       | Description                                                                                                                                                                                                                                                                        |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| base-entity | The base object on which the associative array exists. For example, consider the Claim entity for the ClaimMetrics array.                                                                                                                                                          |
| subtype-map | The array entity subtype, for example, AllEscalatedActivitiesClaimMetric (a subtype of ClaimMetric).                                                                                                                                                                               |
| property    | A field or property on the array object. For example, the AllEscalatedActivitiesClaimMetric object contains the following properties (among others): <ul style="list-style-type: none"> <li>• ClaimMetricCategory</li> <li>• DisplayTargetValue</li> <li>• DisplayValue</li> </ul> |

**Note:** To see a list of subtypes for any given object, consult the *Data Dictionary*. To determine the list of fields (properties) on an object, again consult the *Data Dictionary*.

The following table lists the attributes associated with the <subtype-map> element.

| <subtype-map> attributes | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 | Default |
|--------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| customAccessor           | <i>Internal.</i> Do not use.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | None    |
| propertyPrefix           | Optional attribute that specifies a prefix for every property that an array association generates. None This attribute is useful when you have two or more array associations with overlapping keys. For example, suppose that you have two array associations that both have a subtype map on the same field. In this case, you need to use the propertyPrefix attribute on at least one of the subtype maps. Providing a value for the attribute avoids the two array associations having the same set of property names. |         |



### Example 1

The following example code uses the sample data in the Guidewire ClaimCenter base configuration. It first retrieves a specific claim object using a query builder and then uses that object as the base entity from which to retrieve array member properties.

```
uses gw.transaction.Transaction
uses gw.api.database.Query

var clm = gw.api.database.Query.make(Claim).compare("Claim#ClaimNumber", Equals,
    "235-53-365870").select().getAtMostOneRow()

print("AllEscalatedActivitiesClaimMetric\tClaim Metric Category = "
    + clm.AllEscalatedActivitiesClaimMetric.ClaimMetricCategory.DisplayName)
print("AllEscalatedActivitiesClaimMetric\tDisplay Value = "
    + clm.AllEscalatedActivitiesClaimMetric.DisplayValue)
print("AllEscalatedActivitiesClaimMetric\tReach Yellow Time = "
    + clm.AllEscalatedActivitiesClaimMetric.ReachYellowTime)
```

The output of running this code in the Gosu Scratchpad looks similar to the following:

```
AllEscalatedActivitiesClaimMetric    Claim Metric Category = Claim Activity
AllEscalatedActivitiesClaimMetric    Display Value = 0
AllEscalatedActivitiesClaimMetric    Reach Yellow Time = null
```

### Example 2

The following sample code:

- Retrieves a read-only claim object.
- Adds the claim object to transaction bundle to make it writable.
- Sets a specific property on the AllEscalatedActivitiesClaimMetric object (a subtype of the ClaimMetric object) associated with the claim.

In the definition of the claim object, ClaimCenter associates an array of ClaimMetric objects—the ClaimMetrics array—with the Claim object. The metadata definition file also defines the ClaimMetrics array as being of type <link-association> using subtypes. Thus, you can access array member properties by first accessing the array member of the proper subtype.

```
uses gw.api.database.Query
uses gw.transaction.Transaction

var todaysDate = java.util.Date.CurrentDate
var clm = gw.api.database.Query.make(Claim).compare(Claim#ClaimNumber, Equals,
    "235-53-365870").select().AtMostOneRow

//Query result is read-only, need to get current bundle and add object to bundle
Transaction.runWithNewBundle(\bundle -> {
    if (clm != null) {
        clm = bundle.add(clm)

        print("AllEscalatedActivitiesClaimMetric\tReach Yellow Time = "
            + clm.AllEscalatedActivitiesClaimMetric.ReachYellowTime)
        clm.AllEscalatedActivitiesClaimMetric.ReachYellowTime = todaysDate

        print("\nAfter modifying the ReachYellowTime value...\n")
        print("AllEscalatedActivitiesClaimMetric\tReach Yellow Time = "
            + clm.AllEscalatedActivitiesClaimMetric.ReachYellowTime)
    }
}, "su")
```

The output of running this code in the Gosu Scratchpad looks similar to the following:

```
AllEscalatedActivitiesClaimMetric Reach Yellow Time = null

After modifying the ReachYellowTime value...

AllEscalatedActivitiesClaimMetric Reach Yellow Time = 2018-03-08
```

[See also](#)

- *Gosu Reference Guide*

## Typelist mapping associative arrays

You use a typelist map to partition array objects based on a typelist field (typecode) in the <array> element. In the ClaimCenter base configuration, the ClaimMetrics array on Claim contains an example of a typelist mapping.

```
<entity xmlns="http://guidewire.com/datamodel"
  entity="Claim"
  table="claim"
  type="retireable">
  ...
  <array arrayentity="ClaimMetric"
    desc="Metrics related to this claim."
    exportable="false"
    ignoreforevents="true"
    name="ClaimMetrics">
    ...
    <array-association>
      <typelist-map field="ClaimMetricCategory"/>
    </array-association>
  </array>
  ...
</entity>
```

The <typelist-map> element requires that you set a value for the field attribute. This attribute specifies the typelist to use to partition the array.

---

**IMPORTANT:** It is an error to specify a typelist mapping on a field that is not a typekey.

---

Associative arrays of type <array-association> are different from those created using <link-association> in that they can return more than a single element. In this case, the code creates an array of ClaimMetric objects named ClaimMetrics. Each ClaimMetric object as well as all subtype objects of the ClaimMetric object contain a property called ClaimMetricCategory. The array definition code utilizes that fact and uses the ClaimMetricCategory typelist as a partitioning agent.

The ClaimMetricCategory typelist contains three typecodes, which are:

- ClaimActivityMetrics
- ClaimFinancialMetrics
- OverallClaimMetrics

Each typecode specifies a category. This category contains multiple ClaimMetric object subtypes. For example, the OverallClaimMetrics category contains two ClaimMetric subtypes:

- DaysInitialContactWithInsuredClaimMetric
- DaysOpenClaimMetric

In another example from the ClaimCenter base configuration, you see the following defined for ReserveLine.

```
<entity entity="ReserveLine"
  xmlns="http://guidewire.com/datamodel"
  ...
  table="reserveline"
  type="retireable">
  ...
  <array arrayentity="TAccount"
    arrayfield="ReserveLine"
    name="TAccounts"
    ...
    setterScriptability="hidden">
    <link-association hasGetter="true" hasSetter="true">
      <typelist-map field="TAccountType"/>
    </link-association>
  </array>
  ...
</entity>
```

In this case, the array definition code creates a <link-association> array of TAccount objects and partitions the array by the TAccountType typelist typecodes.

This topic includes the following:

- “Working with array values by using typelist mapping” on page 83
- “Example: Mapping an entity to entities of a different type” on page 85

## Working with array values by using typelist mapping

To retrieve an array value through typelist mapping, use the following syntax:

```
entity.typecode.property
```

Each field has the following meaning:

| Field    | Description                                                                                                                                                                                                                                   |
|----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| entity   | The object on which the associative array exists. For example, this object might be the ReserveLine entity on which the Taccounts array exists.                                                                                               |
| typecode | The typelist typecode that delimits this array partition, for example, OverallClaimMetrics (a typecode from the ClaimMetricCategory typelist).                                                                                                |
| property | A field or property on the array object. For example, the ClaimMetric object contains the following properties (among others): <ul style="list-style-type: none"> <li>• ReachRedTime</li> <li>• ReachYellowTime</li> <li>• Skipped</li> </ul> |

The following table lists the attributes associated with the <typelist-map> element.

| <typelist-map> attributes | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              | Default |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| customAccessor            | <i>Internal.</i> Do not use.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | None    |
| field                     | <i>Required.</i> Name of the property to which the array associates.                                                                                                                                                                                                                                                                                                                                                                                                                                                     | None    |
| propertyPrefix            | Optional attribute that specifies a prefix for every property that an array association generates. This attribute is useful when you have two or more array associations with overlapping keys. For example, suppose that you have two array associations that both have a typelist map on the same field. In this case, you need to use the propertyPrefix attribute on at least one of the typelist maps. Providing a value for the attribute avoids the two array associations having the same set of property names. | None    |

### Example 1

The following example code uses sample data in the Guidewire ClaimCenter base configuration. It iterates over the members of the ClaimMetrics array that fall into the OverallClaimMetrics category.

```
uses gw.api.database.Query
uses gw.transaction.Transaction

var clm = Query.make(Claim).compare(Claim#ClaimNumber, Equals, "235-53-365870").select().AtMostOneRow
for (time in clm.OverallClaimMetrics) {
    print(time.Subtype.DisplayName + ": ReachYellowTime = " + time.ReachYellowTime)
}
```

The output of running this code in the Gosu Scratchpad looks similar to the following:

```
Days Open: ReachYellowTime = 2018-08-24
Initial Contact with Insured (Days): ReachYellowTime = 2018-02-12
```

## Example 2

The following example code also uses the sample data in the Guidewire ClaimCenter base configuration. It first retrieves a specific Claim object and then retrieves a specific ReserveLine object associated with that claim.

```
uses gw.api.database.Query
uses gw.transaction.Transaction

var clm = gw.api.database.Query.make(Claim).compare(Claim#ClaimNumber, Equals,
    "235-53-365870").select().AtMostOneRow
var thisReserveLine = clm.ReserveLines.first()

print(thisReserveLine)
print(thisReserveLine.cashout.CreateTime)
```

The output of running this code in the Gosu Scratchpad looks similar to the following:

```
(1) 1st Party Vehicle - Ray Newton; Claim Cost/Auto body
Thu Feb 22 15:35:02 PST 2018
```

## Setting array member values

The following example code also uses the sample data in the Guidewire ClaimCenter base configuration. It uses a query builder expression to retrieve a specific claim entity. Because the result of the query is read-only, you must first retrieve the current bundle, then add the claim to the bundle to make its fields writable. The retrieved claim is the base entity on which the ClaimMetrics array exists.

The following sample code:

- Retrieves a read-only claim object.
- Adds the claim object to transaction bundle to make it writable.
- Sets specific properties on the ClaimMetric object associated with the claims that are in the OverallClaimMetrics category.

In the definition of the claim object, ClaimCenter associates an array of ClaimMetric objects—the ClaimMetrics array—with the Claim object. The metadata definition file also defines the ClaimMetrics array as being of type <array-association> using the ClaimMetricCategory typelist. Thus, you can access array member properties by first accessing the array member of the proper category.

```
uses gw.transaction.Transaction
uses gw.api.database.Query

var todaysDate = java.util.Date.CurrentDate
var thisClaim = Query.make(Claim).compare(Claim#ClaimNumber, Equals, "235-53-365871").select().AtMostOneRow
//Query result is read-only, need to get current bundle and add entity to bundle

Transaction.runWithNewBundle(\bundle -> {
    if (thisClaim != null) {
        thisClaim = bundle.add(thisClaim)

        //Print out the current values for the ClaimMetric.ReachYellowTime field on each subtype
        for (color in thisClaim.OverallClaimMetrics) {
            print("Subtype - " + color.Subtype.DisplayName + ": ReachYellowColor = " + color.ReachYellowTime)
        }

        print("\nAfter modifying the values...\n")

        //Modify the ClaimMetric.ReachYellowColor value and print out the new values
        for (color in thisClaim.OverallClaimMetrics) {
            color.ReachYellowTime = todaysDate
            print("Subtype - " + color.Subtype.DisplayName + ": ReachYellowColor = " + color.ReachYellowTime)
        }
    }
}, "su")
```

The output of running this code in the Gosu Scratchpad looks similar to the following:

```
Subtype - Days Open: ReachYellowColor = 2017-11-08
Subtype - Initial Contact with Insured (Days): ReachYellowColor = 2017-10-30

After modifying the values...
```

```
Subtype - Initial Contact with Insured (Days): ReachYellowColor = 2018-03-08
Subtype - Days Open: ReachYellowColor = 2018-03-08
```

See also

- *Gosu Reference Guide*

## Example: Mapping an entity to entities of a different type

When creating a database entity that refers to multiple entities of a different type, a best practice is to use an associative array with typelist mapping.

### About this task

A database entity often requires references to multiple entities of a different type.

One way to implement such references is to use `<onetoone>` links. In this case, you must have a separate `<foreignkey>` element that corresponds to each such link. These separate `<foreignkey>` elements must be on each entity to which the original database entity refers. The separate elements must also point back to the original database entity. In addition, on each `<onetoone>` link, you must specify its `linkField` attribute to associate the link with its corresponding `<foreignkey>` element.

However, this method has a downfall. A `<onetoone>` link generally requires a unique relationship to the `<foreignkey>` element that backs the link. The relationship must have a unique index on the `<foreignkey>` element. Such a relationship and its unique index cannot be present if the `<foreignkey>` element is nullable. In this case, nothing enforces the constraint that the `<onetoone>` link implies.

A preferable option or best practice would be to use an `<array>` element and a `<link-association>` subelement. This configuration produces virtual properties for the original database entity. You can use these virtual properties to access the entity instances to which the original database entity refers.

To illustrate how to do so, consider an example of an automation that checks three kinds of results related to an insurance claim—coverability, handleability, and payability. Take the following steps to map a database entity, `Ext_Automation`, to three entities of type `Ext_AutomationCheckResult`—`CoverabilityResult`, `HandleabilityResult`, and `PayabilityResult`:

### Procedure

1. Create an entity, `Ext_Automation`.
2. Create another entity, `Ext_AutomationCheckResult`.
3. Add to `Ext_Automation` an array of the `Ext_AutomationCheckResult` entity.
4. Create one `<foreignkey>` element for the `Ext_AutomationCheckResult` entity that points back to the `Ext_Automation` entity.
5. Create a typelist named `Ext_AutomationResultType`.
6. Add three type codes to the `Ext_AutomationResultType` typelist: `CoverabilityResult`, `HandleabilityResult`, `PayabilityResult`.
7. Add a `<typekey>` element to the `Ext_AutomationCheckResult` entity.
8. With this `<typekey>` element, refer to the `Ext_AutomationResultType` typelist.
9. Under the array of `Ext_AutomationCheckResult` in the `Ext_Automation` entity definition, declare a `<link-association>` subelement.
10. Under the `<link-association>` subelement, declare a `<typelist-map>` that refers to your `<typekey>` field from step 7.
11. Add an `<index>` subelement to the `Ext_AutomationCheckResult` entity.
12. To ensure unique index entries, set the `unique` attribute on the `<index>` subelement to `true`.

13. Add as index columns the Ext\_Automation foreign key and the typekey corresponding to the Ext\_AutomationResultType typelist.

### Results

You now have three new virtual properties on the Ext\_Automation database entity with which to access the three Ext\_AutomationCheckResult types—CoverabilityResult, HandleabilityResult, and PayabilityResult.

## Constant mapping associative arrays

You use a constant map to allow the data model itself to specify key-value pairs. To indicate this mapping type, add the `<constant-map>` subelement to an `<array-association>` element or a `<link-association>` element. Then, enter key-value pairs in the definition file.

For example, the following extension code extends the Account entity type in PolicyCenter, and does the following:

- It defines an array of AccountLocation objects, with each contact associated with a specific role.
- It adds this array to the Account entity.
- It defines individual address types directly as properties on a `<constant-map>` subelement of the `<link-association>` and `<array-association>` elements.

```
<?xml version="1.0"?>
<extension xmlns="http://guidewire.com/datamodel" entityName="Account">
  <array name="AccLocs" arrayentity="AccountLocation">
    <link-association>
      <constant-map field="AddressType">
        <property name="BillingLoc" nullOk="true" value="billing"/>
      </constant-map>
    </link-association>

    <array-association>
      <constant-map field="AddressType">
        <property name="HomeLocs" value="home"/>
        <property name="BusinessLocs" value="business"/>
        <property name="OtherLocs" value="other"/>
      </constant-map>
    </array-association>
  </array>
</extension>
```

The structure of the constant-map association allows you to specify multiple properties under a constant. Notice that you specify array-association constants and link-association constants in two separate groups.

### See also

- “Subtype mapping associative arrays” on page 79
- “Typelist mapping associative arrays” on page 82

## Working with array values by using constant mapping

To retrieve a member of the Role array, use the following syntax:

```
base-entity.defined-property
```

Each field has the following meanings:

| Field            | Description                                                                                                             |
|------------------|-------------------------------------------------------------------------------------------------------------------------|
| base-entity      | The base object on which the associative array exists. For example, consider the Account entity for the Contacts array. |
| defined-property | The property that you defined for the constant map array.                                                               |

The following tables list the attributes and subelements associated with the `<constant-map>` element.

| <constant-map> attributes | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             | Default |
|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|
| customAccessor            | <i>Internal.</i> Do not use.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            | None    |
| field                     | <i>Required.</i> Name of the property to which the array associates.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                    | None    |
| propertyPrefix            | Optional attribute that specifies a prefix for every property that an array association generates. This attribute is useful when you have two or more array associations with overlapping keys. For example, suppose that you have two array associations that both have a constant map on the same field. In this case, you need to use the propertyPrefix attribute on at least one of the constant maps. Providing a value for the attribute avoids the two array associations having the same set of property names.<br><br><b>Note:</b> Constant maps generally do not require a propertyPrefix attribute because the property subelement allows full control over property names. | None    |

| <constant-map> subelements | Description                |
|----------------------------|----------------------------|
| property                   | Key-value pair in the map. |





# Modifying the base data model

This topic discusses how to extend the base data model as well as how to create new data objects.

## Related concepts

- “Planning changes to the base data model” on page 89
- “Defining a new data entity” on page 92
- “Extending a base configuration entity” on page 94
- “Working with attribute and element overrides” on page 95
- “Extending the base data model” on page 98
- “Removing objects from the data model” on page 112
- “Deploying data model changes to the application server” on page 116

## Planning changes to the base data model

Before proceeding to modify the base data model, Guidewire strongly recommends that you first review the *Data Dictionary*. Verify that the existing data model does not provide the functionality that you need first before modifying the base application functionality.

---

**WARNING:** Do not directly modify the physical database that InsuranceSuite uses. Make changes to the data model only through Guidewire Studio. The database reports errors upon upgrade if it cannot synchronize the base database schema in the upgrade version with the data model in the existing version.

---

## Overview of data model extensions

Entity extensions are additions to the entities in the base data model. Although you cannot modify the base data type declaration files directly, you can define an extension to one in a separate .etx file. You can also define new data model objects that extend the data model in an .eti file. This allows new application releases to modify the base definitions without affecting your extensions, thus preserving an upgrade path.

By extending the base data model, you can:

- Add fields (columns) to an existing base entity through the use of the <column>, <typekey>, <foreignkey>, <array>, and similar elements. See “Data entity subelements” on page 49.
- Create a new entity with custom fields using any of the entity types listed in “XML root elements and related base data object types” on page 29.
- Modify a small subset of the attributes of an existing base entity using overrides.

- Remove (or hide) an extension to a base entity that exists in the **extensions** folder as an **.etx** declaration file.
- Remove (or hide) a base entity that exists in the **extensions** folder as an **.eti** declaration file.

However, using extensions, you cannot:

- Delete a base entity or any of its fields. If you do not use a particular base entity or one of its fields, then simply ignore it.
- Change most of the attributes of a base entity or any of its fields.
- Extend an entity that is in an **.eti** declaration file in the **extensions > entity** Studio folder.
- Extend an extension.

## Strategies for extending the base data model

Extending the data model means one of the following:

- You want to add new fields to an existing entity.
- You want to create a new entity.

During planning for data model extensions, you need to consider performance implications. For example, if you add hundreds of extensions to a major object, this can conceivably exceed a reasonable row size in the database.

### Adding fields to an entity

If an entity has almost all the functionality you need to support your business case, you can add one or more fields to it. In this sense, Guidewire uses the term *field* to denote one of the following:

- Column
- Typekey
- Array
- Foreign key

### Subtyping a non-final entity

If you want to find a new use for an existing entity, you can subtype and rename it. For instance, suppose that you want to track individuals who have already had the role of **IssueOwner**. In this case, it can be useful to create **PastIssueOwner**.

### Creating a new entity

Occasionally, careful review of the base application data model makes it clear that you need to create a new entity. There are several types of base entities within Guidewire applications. However, Guidewire recommends in general practice that you always use one of the following types if you create a new entity:

---

**retireable** This type of entity is an extension of the **editable** entity. It is not possible to delete this entity. It is possible to retire it, however.

---

**versionable** This type of entity has a version and an ID. It is possible to delete entities of this type from the database.

---

Guidewire recommends the following:

- If another entity contains a foreign key reference to the new entity, then make the new entity **retireable**. For example, any entity that contains an array must be **retireable**.
- If there is no foreign key reference, and you desire the entity to contain the fields **CreateTime**, **UpdateTime**, **CreateUser**, and **UpdateUser**, then make the new entity **editable**.
- Otherwise, make the entity **versionable**.

**Note:** To later change an existing entity from **retireable** to **versionable**, you must drop the database.

See “Data entities and the application database” on page 25 for more information on these data types.

In general, you typically want to create a new entity under the following circumstances:

- If your business model requires an object that does not logically exist in the application. Or, if you have added too many fields to an existing entity, and want to abstract away some of it into a new, logical entity.
- If you need to manage arrays of objects, as opposed to multiple objects, you can create an entity array.

#### Reference entities

To store some unchanging reference data, such as a lookup table that seldom changes, you can create a reference entity. An example of a business case for a reference entity is a list of typical reserve amounts for a given exposure. To avoid the overhead of maintaining foreign keys, make reference entities keyable. Unless you want to build in the ability to edit this information from within the application, set `setterscriptability = hidden`. This prevents Gosu code from accidentally overwriting the data.

Guidewire recommends that you determine that this is not really a case for creating a typelist before you create a reference entity.

## Add a file name suffix to data model extensions

Enable the use of a file name suffix when creating a new entity or typelist extension.

#### About this task

A file name suffix allows you to specify different names for multiple extensions to common entities and typelists. If different projects involve extensions to the same file, using different names keeps the extensions separate from each other and avoids the need to merge changes. For example, when registering new lines of business in PolicyCenter, you might use a different suffix for each extension of the `InstalledPolicyLine` typelist. If different teams implement the different policy lines, each can work with a uniquely named extension file and not worry about conflicts with other teams.

#### Procedure

1. Navigate to **File > Settings > Guidewire Studio**.
2. Set the **Show filename suffix on new extension dialog** check box.

#### What to do next

When you define a new entity or typelist extension, the dialog contains the **Filename Suffix** text box.

## What happens if you change the data model?

During server start up, the application analyzes the metadata for changes since the last build. If you have made extensions, the application merges this into the working data model, which is the composite of the base entities and your extensions.

After merging the base data model with any extensions, the application compares the startup layout to the physical schema in the current database. (Each application database stores schema version numbers and metadata checksums to optimize the analysis and comparison.)

If the application detects changes between the startup layout and the physical database schema, it initiates a database upgrade automatically. This keeps the physical schema synchronized with the schema defined by the XML metadata. By default, the application refuses to start until the two are synchronized.

The PolicyCenter application server conducts a number of additional tests on the product model data model as it starts. See the *Product Model Guide*

---

**WARNING:** Do not directly modify the physical database that InsuranceSuite uses. Make changes to the data model only through Guidewire Studio. The database will report errors upon upgrade if it cannot synchronize the base database schema in the upgrade version with the data model in the existing version.

---

### Database upgrade triggers

The upgrade utility initiates a database upgrade automatically at application server startup if there are additions, modifications, or extensions to any of the following:

- Data model version
- Extensions version
- Platform version
- Data model
- Field encryption
- Typelists

In addition to these generic changes, the following specific localization changes trigger a database upgrade:

- In file `localization.xml`, any change to the `<LinguisticSearchCollation>` subelement on the `<GWLocale>` element of the default application locale forces a database upgrade at application server startup.
- In file `collations.xml`, any change to the source definition of the `DBJavaClass` definition forces a database upgrade at application server startup.

### Decimal database column upgrade triggers

A decimal database column will upgrade automatically at application server startup in the following Guidewire Studio scenario. For the scenario, note the following concepts:

- The characteristic of a database column is the set of digits to the left of the decimal point.
- The mantissa of a database column is the set of digits to the right of the decimal point.
- The precision of a database column is the total number of digits.
- The scale of a database column is the number of digits in the mantissa.
- The number of digits in the characteristic is the difference between the precision and the scale (precision - scale).

In the scenario, suppose that a user increases the number of digits in the characteristic (precision - scale) and/or the mantissa (scale) of a database column. Suppose also that the user maintains the number of digits in the other part of the database column, if any. The modified database column part or parts will automatically convert to the appropriate number of digits in this case.

---

**WARNING:** Do not decrease the number of digits in either the characteristic (precision - scale) or the mantissa (scale) of a database column. If you do, the application server will throw an exception during startup.

---

## Naming restrictions for extensions

The application uses the names of extensions as the basis for several other internally-generated structures, such as database elements and Java classes. Because of this, it is important that you adhere to the naming requirements and guidelines described in this section.

---

**IMPORTANT:** Deviations from these guidelines can result in product errors or unexpected behavior.

---

An extension name cannot start with a number; it must start with a letter. Other than that, an extension name can contain letters, numbers, or underscores (`_`). Guidewire does not permit any other characters in extension names.

## Defining a new data entity

You define all new data entity objects in declaration files that end with the `.eti` extension. You do this through Guidewire Studio. Studio automatically manages the process and stores the `.eti` file in the correct location in the application (in the **configuration > config > extensions > entity** folder).

## Create a new entity

### Procedure

1. In Guidewire Studio, in the **Project** tool window, navigate to **configuration > config > Extensions > Entity**.
2. Right-click **Entity**, and then click **New > Entity**.
3. In the **Entity** dialog, specify the entity definition values.

| Data entity           | More information                                                          |
|-----------------------|---------------------------------------------------------------------------|
| <delegate>            | "<delegate> elements and related data object types" on page 30            |
| <entity>              | "<entity> elements and related data object types" on page 33              |
| <extension>           | "<extension> elements and related data object types" on page 40           |
| <nonPersistentEntity> | "<nonPersistentEntity> elements and related data object types" on page 42 |
| <subtype>             | "<subtype> elements and related data object types" on page 43             |
| <viewEntity>          | "<viewEntity> elements and related data object types" on page 46          |
| <viewEntityExtension> | "<viewEntityExtension> elements and related data object types" on page 48 |

4. Add fields to your new data entity. In the **Field** drop-down list, select the field type to add, and then click **Add** . For example:

| XML tag      | Use to add                         |
|--------------|------------------------------------|
| <array>      | An array of entities               |
| <column>     | A field with a simple data type    |
| <foreignkey> | A field referencing another entity |
| <typekey>    | A field with a typelist            |

For information on the possible XML elements that you can add to your new entity definition, see "Data entity subelements" on page 49.

5. Deploy your changes to the application server. You must redeploy the application after you make any change to the data model. See "Deploying data model changes to the application server" on page 116 for details.

## Default properties on a new entity

The following table contains the entity properties that the application generates automatically when you create a new entity:

| Property      | Description                                                                                                                                                                                                       |
|---------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ID            | Internally managed primary key                                                                                                                                                                                    |
| CreateTime    | Timestamp indicating when the object was created. This property is applicable only for editable and retireable entity types, or in PolicyCenter for effdated, effdatedbranch, and effdatedcontainer entity types. |
| CreateUser    | User who created the object. This property is applicable only for editable and retireable entity types, or in PolicyCenter for effdated, effdatedbranch, and effdatedcontainer entity types.                      |
| LoadCommandID | Identifier for the command that loaded the object through the staging tables. This property is included for entities where the loadable attribute has a value of true. PolicyCenter doesn't use this property.    |
| New           | Indicator of whether this object is a new object that has not been committed to the database.                                                                                                                     |
| NewlyImported | Obsolete.                                                                                                                                                                                                         |

| Property     | Description                                                                                                                                                                                                                                                                                                                                                                                                                       |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PublicID     | ID or primary key of the row in the external system to which this row is mapped                                                                                                                                                                                                                                                                                                                                                   |
| Retired      | Derived property returning a Boolean value. The Boolean value is true if the RetiredValue property has a non-zero value. The Boolean value is false if the RetiredValue property has a zero value. The Retired property is applicable only for retireable entity types, or in PolicyCenter for effdated, effdatedbranch, and effdatedcontainer entity types.                                                                      |
| RetiredValue | Value indicating whether the object is still in active use. A zero value for the property means that the object is not retired. A non-zero value equal to the entity ID means that the object is retired from active use. Note that even if an object is retired, it can still be referred to by another object. The RetiredValue property is only applicable for effdatedbranch, effdatedcontainer, and retireable entity types. |
| UpdateTime   | Timestamp when the object was last updated. This property is only applicable for editable and retireable entity types, or in PolicyCenter for effdated, effdatedbranch, and effdatedcontainer entity types.                                                                                                                                                                                                                       |
| UpdateUser   | User who last updated the object. This property is only applicable for editable and retireable entity types, or in PolicyCenter for effdated, effdatedbranch, and effdatedcontainer entity types.                                                                                                                                                                                                                                 |

**Note:** New entities have constant properties that the data dictionary lists. The table in this topic does not list these properties. Instead of using these constant properties, use feature literals to refer to the static properties of entities in queries.

#### See also

- For more information on using feature literals to refer to entity properties, see *Gosu Reference Guide*.

## Extending a base configuration entity

You can define all your entity extensions in files that end with the .etx extension by using Guidewire Studio. Studio automatically manages the process and stores the .etx file in the correct location in the application.

**IMPORTANT:** Guidewire provides certain entity extensions as part of the base application configuration. Many of the extension index definitions address performance issues. Other extensions provide the ability to configure the data model in ways that would not be possible if the extension was part of the base data model. Do not overwrite a Guidewire extension with your own extension without understanding the full implications of the change.

Extensions enable you to add new fields to the base data entities. You can add custom fields to extendable entities only. Not all entities are extendable. However, most of the important business entities are. For example, Account, User, and Contact are extendable entities. You can determine if an entity is extendable by looking in the *Data Dictionary* to see if it supports the Extendable attribute. The *Data Dictionary* displays the list of attributes for that entity type directly underneath the entity name.

Use the <extension> XML root element to create an extension entity. Before creating a new extension file, first determine if one already exists.

- If an extension file for the entity does exist, edit that file to extend the entity.
- If an extension file for the entity does not exist, create the new extension file and populate it accordingly.

You can define multiple extensions for base configuration entities and typelist entities. Each extension file must have a unique name.

You can add a qualifier to a base entity or typelist file name to adhere to the uniqueness constraint. The following list contains valid entity extension file names with the first name being the base file name and the subsequent file names having qualifiers:

- Account.etx
- Account.Householder.etx
- Account.Renter.etx

The following list contains valid typelist extension file names with the first name being the base file name and the subsequent file names having qualifiers:

- Jurisdiction.ttx
- Jurisdiction.Canada.ttx
- Jurisdiction.Australia.ttx

## Create a new extension file

### Before you begin

If you need to use a file name suffix for your entity, see “Add a file name suffix to data model extensions” on page 91.

### About this task

Create a new extension file in Guidewire Studio.

### Procedure

1. In Guidewire Studio, in the **Project** tool window, navigate to **configuration > config > Metadata**, then expand **Entity**.
2. Right-click the entity that you want to extend, then click **New > Entity Extension**.

The file that you want to extend must have the .eti extension.

**Note:** If an extension file for the selected entity file already exists, Studio does not permit you to create another one. If Studio shows a message that the entity is not allowed, that might mean that an extension already exists. In that case, search in the **configuration > config > Extensions > Entity** folder for an existing extension file.

Studio creates a new extension file named *entity.etx*, places it in the **configuration > config > Extensions > Entity** folder, and opens it in a tab for editing.

3. Populate the extension with the required attributes.
4. Deploy your changes to the application server.

### What to do next

You must redeploy the application after you make any change to the data model. See “Deploying data model changes to the application server” on page 116.

## Working with attribute and element overrides

It is possible to override certain attribute values (fields) and elements on entity types that Guidewire defines in the base configuration. To make these changes, you add or edit overrides in extension files in the **configuration > config > Extensions > Entity** folder. You do not have write access to entity definition files in the **configuration > config > Metadata > Entity** folder.

To override attributes or other elements:

- If an entity extension file (.etx) already exists in the **Extensions** folder, add one of the specified override elements to the existing file.
- If an entity extension file (.etx) does not already exist in the **Extensions** folder, you create one and add an override element to that file.
- If an entity definition file (.eti) exists in the **Extensions** folder, you modify the original field definition instead of using an override element.

Add override elements only to files in the **configuration > config > Extensions > Entity** folder. Do not attempt to add an override element to a file in any other folder.

The following table lists the attributes that you can override in a file in the **Extensions > Entity** folder:

| Override type           | Attributes or elements that you can override                                     | More information                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|-------------------------|----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| array-override          | desc<br>triggersValidation                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| column-override         | createHistogram<br>default<br>desc<br>nullok<br>supportsLinguisticSearch<br>type | <p>You can override these attributes directly by using <code>column-override</code>. However, to modify data type attributes such as <code>size</code>, <code>logicalSize</code>, and <code>encryption</code>, add a <code>columnParam</code> subelement under a <code>column-override</code>. Then, set the data type attributes on the <code>columnParam</code>. See “Overriding data type attributes” on page 97 and “Using <code>&lt;column-override&gt;</code> to modify field validation” on page 157 for examples of this configuration.</p> <p><b>Note:</b> You can override the type attribute only to a data type that has the same value type as the previous data type. For example, if the value of the type attribute is <code>shorttext</code>, the value type is <code>java.lang.String</code>. The new value of the type attribute must also have a <code>java.lang.String</code> value type. For example, <code>longtext</code> is an acceptable data type, but <code>BigDecimal</code> is not.</p> |
| edgeForeignKey-override | desc<br>extractable<br>overlapTable                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| foreignkey-override     | desc<br>importableAgainstExistingObject<br>nullok<br>triggersValidation          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| keyfilters-override     | keyfilter                                                                        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| onetoone-override       | desc<br>triggersValidation                                                       |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| typekey-override        | default<br>desc<br>nullok<br>typefilter<br>keyfilters-override                   |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

#### See also

- “Changing the length of a text field” on page 158 for overriding the `size` data type attribute or the `logicalSize` data type attribute
- “`<columnParam>` subelement” on page 56 for more information on `size`, `logicalSize`, and other data type attributes.



- “The data types configuration file” on page 141 for global configuration of data type attributes
- “The Length attribute” on page 143 for global configuration of the character length of a column

## Overriding data type attributes

For each element type in an entity definition, the data model supports attributes that are specific to that element type. For example, all `<column>` elements have the same set of attributes. All `<typekey>` elements have the same set of attributes, but that set is not the same as the set that `<column>` elements have. You override those attributes by using the **Override** command. Besides the standard set of attributes on a column, the data model supports data type attributes on a column. These attributes are different for each supported column data type. You override these attributes by using nested **columnParam** subelements in the **column-override** element.

For example, for the Contact entity type, the base configuration defines a TaxID column in `Contact.eti` with the following attributes:

| Attribute name           | Attribute value                      |
|--------------------------|--------------------------------------|
| name                     | TaxID                                |
| type                     | ssn                                  |
| desc                     | Tax ID for the contact (SSN or EIN). |
| createhistogram          | true                                 |
| default                  |                                      |
| nullok                   | true                                 |
| supportsLinguisticSearch | false                                |

To encrypt the values in this column, which is a reasonable choice for this personally identifiable data, edit the Contact extension in the `Contact.etx` file. Use the **column-override > columnParam** command to set the encryption parameter on the column:

| Attribute name for the column parameter | Attribute value |
|-----------------------------------------|-----------------|
| name                                    | encryption      |
| value                                   | true            |

### See also

- See “`<columnParam>` subelement” on page 56 for a description of the `<columnParam>` element and the available column parameters for each data type.

## Override a column attribute

The entity editor in Guidewire Studio supports changing attributes of columns in an entity type.

### About this task

This task provides an example of how to override a column attribute by changing the description of the Name column on the Document entity type.

### Procedure

1. Open Guidewire Studio.
2. Navigate to **configuration > config > Extensions > Entity**.
3. Right-click **Entity**, then click **New > Entity Extension**.
4. In the **Entity Extension** dialog, in the list of entities to extend, select **Document**, then select **OK**. Studio creates a `Document.etx` file in the **configuration > config > Extensions > Entity** folder, and opens the file in the entity editor.

**Note:** If you see a message that the file already exists, open the existing `Document.etx` file in the editor.

5. In the **Primary Value** column, right-click **Name**, then click **Override**.
6. For the **desc** attribute, in the **Value** column, type a new value.
7. Stop and restart the application server.

The application server recognizes that there are changes to the data model and runs the upgrade utility on start up.

## Override an array attribute

The entity editor in Guidewire Studio supports changing attributes of arrays in an entity type.

### About this task

This task provides an example of how to override an array attribute by changing an attribute on the **RoleAssignments** array on the **Account** entity type.

### Procedure

1. Open Guidewire Studio.
2. Navigate to **configuration > config > Extensions > Entity**.
3. Right-click **Entity**, then click **New > Entity Extension**.
4. In the **Entity Extension** dialog, in the list of entities to extend, select **Account**, then select **OK**.  
Studio creates an **Account.etx** file in the **configuration > config > Extensions > Entity** folder, and opens the file in the entity editor.

**Note:** If you see a message that the file already exists, open the existing **Account.etx** file in the editor.

5. In the **Primary Value** column, right-click **RoleAssignments**, then click **Override**.
6. For the **triggersValidation** attribute, in the **Value** column, type or choose **false**.
7. Stop and restart the application server.

The application server recognizes that there are changes to the data model and runs the upgrade utility on start up.

### Results

After this change, the application no longer performs the validation that usually occurs on changes to elements of the **RoleAssignments** array.

## Overriding nested subelements

The only nested subelements that you can override are **<keyfilter>** subelements of a **<keyfilters>** subelement of a **<typekey>** element.

If you override a **<typekey>** element that contains a **<keyfilters>** element, Guidewire Studio copies the **<keyfilters>** element into the **<keyfilters>** element as a **<keyfilters-override>** element. You can override nested **<keyfilter>** subelements of the **<keyfilters-override>** element. To add new nested **<keyfilter>** subelements to the **<keyfilters-override>** element, right-click **keyfilters-override** in the **Element** column, and then click **Add new > keyfilter**. You can also edit or remove the existing **<keyfilter>** subelements from the **<keyfilters-override>** element.

The **<keyfilters-override>** element and its **<keyfilter>** subelements that you define override all of the **<keyfilter>** elements in the original **<keyfilters>** subelement of the **<typekey>** element. For example, suppose the original **<keyfilters>** subelement of a **<typekey>** element contains **<keyfilter>** subelements A, B, and C. If you override the **<typekey>** element and remove the **<keyfilter>** subelement A from the **<keyfilters-override>** subelement and add a **<keyfilter>** subelement D, the application ignores the original key filters A, B, and C and uses the overriding filters B, C, and D.

## Extending the base data model

As described in “Defining a new data entity” on page 92, you can define entirely new custom entities that become part of the entity model. You can then use these entities in your data views, rules, and Gosu classes in exactly the same way

as you use the base entities. The application makes no distinction between the usage of base entities and custom entities.

### Testing Your Work

After you make any change to the data model, Guidewire recommends that you do the following to test your work.

- First, stop and restart Guidewire Studio. Verify that there are no errors or warnings. If there are, do not proceed until you have corrected the issues. Guidewire does not strictly require that you always stop and restart Studio after a data model change. However, it is one way to test that you have not inadvertently made a typing error, for example.
- After starting Studio, start the application. As the application server starts, it recognizes that you have made changes to the database and runs the upgrade utility automatically. Verify that the application server starts cleanly, without errors or warnings.

### Related concepts

“Creating a new delegate object” on page 99

“Extending a delegate object” on page 101

“Defining a subtype” on page 107

“Defining a reference entity” on page 108

“Implementing a many-to-many relationship between entity types” on page 109

“Extending an existing view entity” on page 110

### Related tasks

“Define an entity array” on page 108

## Creating a new delegate object

Creating a delegate object and associating it with an entity is a relatively straightforward process. It does involve multiple steps, as do many changes to the data model. To create a new delegate, you need to do the following:

| Task                                          | Description                                                                                                                      | More information                                            |
|-----------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------|
| Step 1: Create the Delegate Object            | Define the delegate entity using the <delegate> element.                                                                         | “Step 1: Create the delegate object” on page 100            |
| Step 2: Define the Delegate Functionality     | Create a Gosu enhancement to provide any functionality that you want to expose on your delegate.                                 | “Step 2: Define the delegate functionality” on page 100     |
| Step 3: Add the Delegate to the Parent Entity | Use the <implementsEntity> element to associate the delegate with the parent entity.                                             | “Step 3: Add the delegate to the parent entity” on page 101 |
| Step 4: Deploy your Data Model Changes        | Deploy your data model changes. You may need to regenerate any Java API file or web service WSDL files after data model changes. | “Step 4: Deploy your data model changes” on page 101        |

### Applicability of base configuration delegates to custom entities

In some cases, Guidewire recommends that custom entities not implement base configuration delegates. To ascertain these cases, identify whether the delegate implements an interface designed for internal usage only. If the base configuration delegate implements an internal interface that is for only internal entities, Guidewire recommends against reusing the delegate.

Instead, create a delegate similar to the base configuration delegate. In addition, create interfaces similar to the interfaces that the base configuration delegate implements. Implement these new interfaces as needed.

An example of a delegate that Guidewire recommends against reusing for custom entities is the `Approvable` delegate. The `Approvable` delegate implements the `ApprovableInternalMethods` interface and indirectly implements the `ApprovalHandler` interface. Guidewire intends these interfaces for internal usage. The interface implementations are intended for base configuration objects.

Instead of reusing the `Approvable` delegate, Guidewire recommends creating another delegate similar to the `Approvable` delegate. In addition, create and implement the interfaces that the new delegate requires.

## Step 1: Create the delegate object

### About this task

The first step in defining a new delegate is to create the delegate file and populate it with the necessary code.

### Procedure

1. In Guidewire Studio, in the **Project** tool window, navigate to **configuration > config > Extensions > Entity**.
2. Right-click **Entity**, and then click **New > Entity**.
3. In the **Entity** text box, type the name of the delegate.
4. In the **Entity Type** drop-down list, click **delegate**.
5. Click **OK**.
6. Add fields to your new delegate. In the **Field** drop-down list, select the field type to add, and then click **Add** .

For information on the possible XML elements that you can add to your new entity definition, see “Data entity subelements” on page 49.

### What to do next

After completing this task, complete “Step 2: Define the delegate functionality” on page 100.

## Step 2: Define the delegate functionality

### Before you begin

Before beginning this task, complete “Step 1: Create the delegate object” on page 100.

### About this task

Next, you need to provide functionality for the delegate. While there are several ways to do this, you must use a Gosu enhancement implementation.

|                                 |                                                                                                                                                                                                                                                                                                                                                                                              |
|---------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Java class implementation       | In the base configuration, Guidewire provides a Java class implementation for each delegate to provide the necessary functionality. The <code>Delegate</code> object designates the Java class through the <code>javaClass</code> attribute. It is not possible for you to create and use a Java class for this purpose.                                                                     |
| Gosu enhancement implementation | You must implement the delegate functionality through a Gosu enhancement that defines any functionality associated with the fields on the delegate. By providing the name of the delegate entity to the enhancement as you create it, you inform Studio that you are adding functionality for that particular delegate. Studio automatically recognizes that you are enhancing the delegate. |

### Procedure

1. In Guidewire Studio, in the **Project** tool window, navigate to **configuration > config > gsrc > gw**.  
This is an example package. In actual practice, you can place the enhancement in a location that makes business sense.
2. Right-click **gw**, and then click **New > Gosu Enhancement**.
3. In the **Name** text box, type the name of the enhancement.  
As a best practice, Guidewire recommends that you use the name of the delegate followed by `Enhancement`.  
For example, if the delegate is named `MyDelegate`, then name the enhancement `MyDelegateEnhancement`.

4. In the **Enhanced type** list, click the name of the delegate entity that you created.
5. Click **OK**.
6. In the enhancement code window, enter code to provide the necessary functionality. The delegate automatically has access to all fields and members that you define in the Gosu enhancement.

#### What to do next

After completing this task, complete “Step 3: Add the delegate to the parent entity” on page 101.

## Step 3: Add the delegate to the parent entity

#### Before you begin

Before beginning this task, complete “Step 2: Define the delegate functionality” on page 100.

#### About this task

The next step is to associate a delegate with an entity using the `<implementsEntity>` element in the entity definition.

- If you are creating a new entity, then you need to add the `<implementsEntity>` element to the entity definition.
- If you are working with an existing entity, then you need to add the `<implementsEntity>` element to the entity extension.

#### Procedure

1. In Guidewire Studio, in the **Project** tool window, navigate to the entity to modify, or create a new entity.
2. In the **Field** drop-down list, click **implementsEntity**, and then click **Add** .
3. Next to the **name** attribute, click the **Value** drop-down list, and click the name of the delegate.

#### What to do next

After completing this task, complete “Step 4: Deploy your data model changes” on page 101.

## Step 4: Deploy your data model changes

#### Before you begin

Before beginning this task, complete “Step 3: Add the delegate to the parent entity” on page 101.

#### About this task

After completing the previous steps, you need to deploy your data model changes. If necessary, see “Deploying data model changes to the application server” on page 116 for details. Depending on whether you are working in a development or production environment, you need to perform different tasks. You may need to regenerate any Java API file or web service WSDL files after data model changes.

## Extending a delegate object

**Note:** A Delegate data object is a reusable entity that contains an interface and a default implementation of that interface. See “`<delegate>` elements and related data object types” on page 30 for more information.

Typically, you extend existing delegate objects to provide additional fields and behaviors on the delegate. Through extension, you can add the following to a delegate object:

- `<column>`
- `<foreignkey>`
- `<description>`

- <implementsEntity>
- <implementsInterface>
- <index>
- <typekey>

You cannot remove base delegate fields. However, you can modify them to a certain extent—for example, by making an optional field non-nullable (but not the reverse). You cannot replace the `requires` attribute on the base delegate (which specifies the required adapter), but you can implement other delegates.

In ClaimCenter, you can extend the following base configuration delegates:

- CopyOnWriteMetricLimitDelegate
- DecimalMetricDelegate
- DecimalMetricLimitDelegate
- EFTDataDelegate
- ISOMatchReport
- ISOReportable
- IntegerMetricDelegate
- IntegerMetricLimitDelegate
- MetricLimitTimeDelegate
- MoneyMetricDelegate
- MoneyMetricLimitDelegate
- PercentMetricDelegate
- PercentMetricLimitDelegate
- TimeBasedMetricDelegate
- TripAccommodationDelegate
- TripExpenseCancellationDelegate
- TripExpenseDelegate
- TripSegmentDelegate

In PolicyCenter, you can extend the following base configuration delegates:

- Auditable
- Cost
- Coverable
- Coverage
- Exclusion
- Modifiable
- Modifier
- PCAssignable
- PolicyCondition
- RateFactor
- Transaction
- UWIssueDelegate

**Note:** In addition to these application-specific delegates, you can extend the following system delegate: AddressAutofillable.

In BillingCenter, you can extend the system delegate AddressAutofillable.

You can extend a delegate only if the base configuration definition file for that delegate contains the following:

```
extendable="true"
```

The default for the extendable attribute on <delegate> is false. Therefore, if it is not set explicitly to true in the delegate definition file, you cannot extend that delegate.

Do not attempt to change the graph to which a Guidewire base entity belongs through extension. In other words, do not attempt to change the delegate that a Guidewire base entity implements through an extension entity using <implementsEntity>. If you attempt to do so, the application generates an error.

### Extend a delegate object

1. Navigate to **configuration > config > Extensions > Entity**.
2. Right-click and select **New > Entity Extension**.
3. Enter the name of the delegate that you want to extend and add the .etx extension. Studio opens an empty file.
4. Enter the delegate definition in the delegate extension file. If necessary, find an existing delegate file and use it as a model for the syntax. For details, see “Creating a new delegate object” on page 99.

## ClaimCenter EFT delegate example

To illustrate, in the base ClaimCenter configuration, ClaimCenter provides a delegate named EFTDataDelegate to use with Electronic Funds Transfer (EFT) data.

ClaimCenter defines the following EFTDataDelegate-related files in the base ClaimCenter configuration:

| File                | Location    | Description                                                       |
|---------------------|-------------|-------------------------------------------------------------------|
| EFTDataDelegate.eti | metadata/cc | Defines delegate EFTDataDelegate.                                 |
| EFTDataDelegate.etx | extensions  | Extends EFTDataDelegate and adds additional fields and behaviors. |
| EFTData.eti         | metadata/cc | Defines an EFTData object that implements EFTDataDelegate.        |
| Check.eti           | metadata/cc | Creates the Check object, which implements EFTDataDelegate.       |
| Contact.etx         | extensions  | Extends the Contact entity and adds an array of EFTData objects.  |

The following topics describe these files.

### The EFT data delegate

Guidewire defines the EFTDataDelegate in file EFTDataDelegate.eti. It looks similar to the following.

```
<delegate
  xmlns="http://guidewire.com/datamodel"
  extendable="true"
  javaClass="com.guidewire.cc.domain.contact.EFTDataDelegate"
  name="EFTDataDelegate">
  <fulldescription>
    <![CDATA[Delegate used by Contact and Check to store Electronic funds transfer data]]>
  </fulldescription>
</delegate>
```

Notice that, in this case, Java class com.guidewire.cc.domain.contact.EFTDataDelegate implements the plugin functionality. In actual practice, if you define your own delegate, you need to implement the delegate functionality in Gosu code.

Also, notice that this delegate is extendable.

## The EFT data delegate extension

Guidewire extends the definition of the EFTDataDelegate entity in file EFTDataDelegate.etx.

```
<extension
  xmlns="http://guidewire.com/datamodel"
  entityName="EFTDataDelegate">
  <column
    desc="The name on the account"
    name="AccountName"
    type="varchar">
    <columnParam name="size" value="100"/>
  </column>
  <column
    desc="The name of the bank"
    name="BankName"
    type="varchar">
    <columnParam name="size" value="100"/>
  </column>
  <typekey
    desc="The type of bank account e.g. checking, savings etc"
    name="BankAccountType"
    typelist="BankAccountType"/>
  <column
    desc="The bank account number"
    name="BankAccountNumber"
    type="varchar">
    <columnParam name="size" value="20"/>
    <columnParam name="encryption" value="true"/>
  </column>
  <column
    desc="The routing number is a nine digit bank code used in the United States"
    name="BankRoutingNumber"
    type="positiveinteger"/>
  <column
    desc="Indicates if this is the primary EFT record for the contact"
    name="IsPrimary"
    type="bit"/>
</extension>
```

Notice that the delegate extension adds a number of fields to the delegate that are accessible to any entity that implements this delegate, such as AccountName and BankAccountType, among others.

## EFTData implements EFTDataDelegate

In the base ClaimCenter configuration, the EFTData entity implements the EFTDataDelegate delegate. The following abbreviated code illustrates this.

```
<!-- Entity implementing the delegate -->
<entity name="EFTData">
  <implementsEntity name=" EFTDataDelegate "/>
  <foreignkey columnName="Contact"/>
</entity>
```

## Check implements EFTDataDelegate

In the base ClaimCenter configuration, the Check entity implements EFTDataDelegate directly, in a one-to-one relationship. The following abbreviated code illustrates this.

```
<!-- 1:1 relationship -->
<extension entityName="Check">
  <implementsEntity name=" EFTDataDelegate "/>
</extension>
```

## Contact contain array of EFTData objects

In the base ClaimCenter configuration, the Contact entity adds an array of EFTData objects. This is a one-to-many relationship. The following abbreviated code illustrates this.

```
<!-- 1:m relationship -->
<extension entityName="Contact">
  <array arrayentity="EFTData" />
</extension>
```



## PolicyCenter Modifier delegate example

To illustrate, in the base PolicyCenter configuration, PolicyCenter provides a **Modifier** delegate. PolicyCenter stores the delegate definition in file `Modifier.eti`, in **configuration > config > Metadata > Entity**. The following is a simplified version of this definition file.

```
<?xml version="1.0"?>
<delegate xmlns="http://guidewire.com/datamodel"
  effdatedOnly="true"
  extendable="true"
  javaClass="com.guidewire.pc.domain.policy.Modifier"
  name="Modifier"
  requires="gw.api.domain.ModifierAdapter">
  <fulldescription><![CDATA[A list of states and their rating factors (e.g. experience modification
  for workers' compensation).]]></fulldescription>
  <column desc="Boolean modifier"
    name="BooleanModifier"
    type="bit"/>
  ...
</delegate>
```

This delegate defines the following fields:

- BooleanModifier
- DateModifier
- Eligible
- Justification
- PatternCode
- RateModifier
- ReferenceDateInternal
- TypeKeyModifier
- ValueFinal
- State (type list)

Any entity that wants to use the **Modifier** delegate functionality must implement this delegate.

### The modifier delegate extension

Suppose that you want to extend the definition of the **Modifier** entity in file `Modifier.etx` to add additional columns to the delegate entity. For example:

- OverriddenRateModifier
- SuggestedRateModifier

The delegate extension code looks similar to the following.

```
<extension
  xmlns="http://guidewire.com/datamodel"
  entityName="Modifier">
  <column
    desc="The rate modifier that has been overridden."
    name="OverriddenRateModifier"
    type="rate">
  </column>
  <column
    desc="The rate modifier that the external system has suggested."
    name="SuggestedRateModifier"
    type="rate">
  </column>
</extension>
```

These fields are then accessible to any entity that implements this delegate. You use these fields to support user overrides of the modifier rates originally suggested by an external process. All modifiers for all lines of business use this override process. Thus, by placing this functionality in the **Modifier** delegate, each entity that implements the **Modifier** delegate inherits this functionality, instead of each entity implementing this functionality separately.

Each line of business (LOB) uses one or two LOB-specific modifier entities that implement the `Modifier` delegate. For example:

- `BAModifier`
- `BOPModifier`
- `CPModifier`
- `GLModifier`
- `PAModifier`
- `PAVehicleModifier`
- `ProductModifier`
- `WCModifier`

Because each of these LOB-specific modifier entities already implements the `Modifier` entity, each automatically inherits the `Modifier` column extensions.

If an entity does not already implement the `Modifier` delegate, then you need to do one of the following:

- If an extension for that entity already exists in the **Extensions** folder, then you need to modify that extension file so that it implements the required delegate. In this case, modify the extension entity and add the following, using the appropriate adapter name:

```
<implementsEntity adapter="xxx" name="Modifier"/>
```

- If an extension for that entity does not exist in the **Extensions** folder, then you need to create an extension and have that entity extension implement the required delegate. In this case, right-click the extensions folder and select **New > Other file**. Enter the entity name and add the `.etx` extension. Populate the extension file with something similar to the following, using the appropriate adapter and entity name (`entityName`):

```
<?xml version="1.0"?>
<extension xmlns="http://guidewire.com/datamodel" entityName="xxx">
  <implementsEntity adapter="xxx" name="Modifier"/>
</extension>
```

### Modifier delegates and arrays

Delegate entities do not support arrays, so the `Modifier` delegate does not include an array of `RateFactor` entities. Instead each concrete modifier type must define its own array of rate factors. Therefore, for each entity that implements the `Modifier` delegate (for example, `BAModifier`), there must also be a corresponding entity that implements the `RateFactor` delegate (for example, `BARateFactor`).

### Modifier delegates and adapters

Each entity that implements the `Modifier` delegate must specify an adapter class. Each adapter class must implement the `gw.api.domain.ModifierAdapter` interface. This interface implements generic modifier operations such as returning the owning `Modifiable` entity and working with the array of rate factors.

Similarly, entities that implement the `RateFactor` delegate must separately specify an adapter class that implements the `gw.api.domain.RateFactorAdapter` interface. This adapter needs to define one method only, one that returns the owning `Modifier`.

To support out-of-sequence and preemption handling, each modifier type must implement its own matcher class. Each modifier and rate factor entity must separately implement the `gw.api.effdate.MatchableEffDated` interface. Guidewire provides the following classes as starting points for configuration:

- `gw.api.effdate.matcher.AbstractModifierMatcher.gs`
- `gw.api.effdate.matcher.AbstractRateFactorMatcher.gs`

### See also

- “The application data model” on page 19

- “<delegate> elements and related data object types” on page 30
- “<implementsEntity>” on page 65
- “Creating a new delegate object” on page 99
- *Product Model Guide*

## Defining a subtype

A subtype is an entity that you base on another entity (its supertype). The subtype has all of the fields and elements of its supertype, and it can also have additional ones. You can also create subtypes of subtypes, with no limit to the depth of the hierarchy.

InsuranceSuite does not associate a unique database table with a subtype. Instead, the application stores all subtypes in the table of its supertype. The supertype table includes a subtype column. The subtype column stores the type values for each subtype. The application uses this column to resolve a subtype.

You define a subtype using the <subtype> element. You must specify certain attributes of the subtype, such as its name and its supertype (the entity on which the application bases the subtype entity). For a description of required and optional attributes, see “<subtype> elements and related data object types” on page 43.

Within the <subtype> definition, you must define its fields and other elements. For a description of the elements you can include, see “Data entity subelements” on page 49.

### Example

This example defines an `Inspector_Ext` entity as a subtype of `Person`. The `Inspector_Ext` entity includes a field for the inspector’s business license. To create the `Inspector_Ext.eti` file

1. In Studio, navigate to **configuration > config > Extensions > Entity**.
2. Right-click **Entity**, and then click **New > Entity**.
3. In the **Entity** text box, type `Inspector_Ext`.
4. In the **Entity Type** drop-down list, click **subtype**.
5. In the **Supertype** box, type `Person`.
6. Click **OK**.
7. In the **Field** drop-down list, click **column**.
8. Set the following properties for the new column:
  - **name:** `InspectorLicense_Ext`
  - **type:** `varchar`
  - **desc:** `Inspector's business license number`
9. In the **Field** drop-down list, click **columnParam**.
10. Set the following properties for the new column parameter:
  - **name:** `size`
  - **value:** `30`

Note that while `Inspector_Ext` is subtype of `Person`, `Person`, itself, is a subtype of `Contact`. The application automatically adds the new `Inspector_Ext` type to the `Contact` typelist. This is true, even though the application marks the `Contact` typelist as `final`.

To see this change:

- To see this change in the *Data Dictionary*, you must restart the application server.
- To see this change in the `Contact` typelist in Studio, you must restart Studio.

## Defining a reference entity

You use a reference entity to store reference data for later access from within the application without having to call out to an external application. For example, you can use reference entities to store:

- Medical payment procedure codes, descriptions, and allowed amounts
- Average reserve amounts, based on coverage and loss type
- PIP aggregate limits, based on state and coverage type

You can populate a reference entity by importing its data, and then you can query it using Gosu expressions. If you do not want the application to update the reference data, set `setterScriptability = hidden` during entity definition.

---

**IMPORTANT:** You can use any entity type as a reference entity. However, if you use the entity solely for storing and querying reference data, then Guidewire recommends that you use a keyable entity.

---

### Example

This example defines a read-only reference table named `ExampleReferenceEntityExt`.

```
<entity entity="ExampleReferenceEntityExt" table="exempleref" type="keyable"
  setterScriptability="hidden">
  <column name="StringColumn" type="shorttext"/>
  <column name="IntegerColumn" type="integer"/>
  <column name="BooleanColumn" type="bit"/>
  <column name="TextColumn" type="longtext"/>
  <index name="internal1">
    <indexcol name="StringColumn" keyposition="1"/>
    <indexcol name="IntegerColumn" keyposition="2"/>
  </index>
</entity>
```

## Define an entity array

### About this task

It is often useful to have a field that contains an array of other entities. For example, to represent that a contact can contain multiple address, the `Contact` entity contains the `Contact.ContactAddresses` field, which is an array of `ContactAddresses` entities for each `Contact` data object.

As you define the entity for the array, consider the type of entity to use. The general rule, is that if another entity does not refer to the new entity through a foreign key, then make the entity `versionable`. Otherwise, make the entity `retireable`.

### Procedure

1. Define the entity to use as a member of the array. Although you can use one of the application base entities for an array, it is often likely that you need to define a new entity for this purpose.
2. Define an array field in the entity that contains the array. You can give the field any name you want. It does not need to be the same name as the array entity.
3. Define a foreign key in the array entity that references the containing entity. The application uses this field to connect an array to a particular data object.

---

**For more information about See**

|                              |                                                         |
|------------------------------|---------------------------------------------------------|
| entity types                 | <a href="#">“Overview of data entities” on page 21</a>  |
| defining a new entity        | <a href="#">“Defining a new data entity” on page 92</a> |
| defining an array field      | <a href="#">“&lt;array&gt;” on page 50</a>              |
| defining a foreign key field | <a href="#">“&lt;foreignkey&gt;” on page 62</a>         |

---

### Example

The following example, defines a new retireable entity named `ExampleRetireableArrayEntityExt` and adds it as an array to the `Claim` entity.

The first step is to define the array entity:

```
<?xml version="1.0"?>
<entity entity="ExampleRetireableArrayEntityExt" table="exampleretarray" type="retireable"
  exportable="true">
  <column name="StringColumn" type="shorttext"/>
  <typekey name="TypekeyColumn" typelist="SystemPermissionType" desc="A test typekey column"/>
  <foreignkey name="RetireableFKID" fkentity="ExampleRetireableEntityExt"
    desc="FK back to ExampleRetireableEntity" exportable="false"/>
  <foreignkey name="KeyableFKID" fkentity="ExampleKeyableEntityExt"
    desc="FK through to ExampleKeyableEntity" exportable="false"/>
  <foreignkey name="ClaimID" fkentity="Claim" desc="FK back to Claim" exportable="false"/>
  <implementsEntity name="Extractable"/>
  <index name="internal1" unique="true">
    <indexcol name="RetireableFKID" keyposition="1"/>
    <indexcol name="TypekeyColumn" keyposition="2"/>
  </index>
</entity>
```

To make this example useful, suppose that you now add this array field to the `Claim` entity. It is possible that a `Claim` entity already exists in the base configuration. Verify that the data type declaration file does not exist before adding another one. To determine if a `Claim` extension file already exists, use `Ctrl+N` to search for `Claim.etx`.

- If the file does exist, then you can modify it.
- If the file does not exist, then you need to create one.

Add the following to `Claim.etx`.

```
<extension entityName="Claim" ...>
  ...
  <array arrayentity="ExampleRetireableArrayEntityExt"
    desc="An array of ExampleRetireableArrayEntityExt objects."
    name="RetireableArrayExt" />
  ...
</extension>
```

Next, modify the array entity definition so it includes a foreign key that refers to `Claim`:

```
<entity entity="ExampleRetireableArrayEntityExt" table="exampleretarray" ... >
  ...
  <foreignkey name="ClaimID" fkentity="Claim" desc="FK back to Claim" exportable="false"/>
</entity>
```

Finally, create the two referenced entities, `ExampleRetireableEntityExt` and `ExampleKeyableEntityExt`.

## Implementing a many-to-many relationship between entity types

To add a many-to-many relationship between entity types to the data model, you need to do the following:

- First, create a separate entity with a type attribute of `versionable` or a child of `versionable`.
- Add non-nullable foreign keys to each end of the many-to-many relationship.
- Add a unique index on each of the foreign keys.

These steps create a classic join entity.

The following example illustrates how to create a many-to-many relationship between `Claim` and `Contact` entity types.

- It first creates a `versionable` entity, `ClaimContact`, by setting the type attribute to `retireable`.
- It then defines foreign keys to `Claim` and `Contact`.
- Finally, it adds indexes to these foreign keys.

The code looks similar to the following:

```
<entity xmlns="http://guidewire.com/datamodel"
  entity="ClaimContact"
  table="claimcontact"
  type="retireable"
  desc="Join entity modeling many-to-many relationship between Claim and Contact entities">
  <foreignkey columnName="ClaimID"
    fkentity="Claim"
    name="Claim"
    nullok="false"/>
  <foreignkey columnName="ContactID"
    fkentity="Contact"
    name="Contact"
    nullok="false"/>
  <index name="claimcontacts" unique="true">
    <indexcol keyposition="1" name="ClaimID"/>
    <indexcol keyposition="2" name="ContactID"/>
  </index>
</entity>
```

To access the relationship, you need to add an array of the new join entities to either end or to both ends of the relationship. For example:

```
<extension xmlns="http://guidewire.com/datamodel" entityName="Claim">
  <array arrayentity="ClaimContact"
    desc="All the ClaimContact entities related to Claim."
    name="ClaimContacts"/>
</extension>
```

This provides an array of ClaimContact entities on Claim.

## Extending an existing view entity

InsuranceSuite uses `viewEntity` entities to improve performance for list view pages in rendering the application interface. (See “<viewEntity> elements and related data object types” on page 46 for details.) Some default PCF pages make use of list view entities and some do not. If you add a new field to an entity, then you need to decide if you want to extend a `viewEntity` to include this new field. This can potentially avoid performance degradation.

The following example illustrates a case in which you add an extension both to a primary entity and its corresponding `viewEntity`. First search for `Activity.etc` to determine if one exists. (Use Ctrl+N to open the search dialog.)

Add the following to `Activity.etc`:

```
<extension entityName="Activity">
  ...
  <column type="bit"
    name="validExt"
    default="true"
    nullok="true"
    desc="Sample bit extension, with a default value."/>
  ...
</extension>
```

Next, search for `ActivityDesktopView.etc`. Suppose that you do not find this file, but you see that `ActivityDesktopView.eti` exists. As this is part of the base configuration, you cannot modify this declaration file. However, find the highlighted file in Guidewire Studio™ and select **New > Entity Extension** from the right-click submenu. This opens a mostly blank file.

Enter the following in `ActivityDesktopView.etc`:

```
<viewEntityExtension entityName="ActivityDesktopView">
  <viewEntityColumn name="validExt" path="validExt"/>
</viewEntityExtension>
```

**Note:** The path attribute is always relative to the primary entity on which you base the view.

These data model changes add a `validExt` column (field) to the `Activity` object, which is also accessible from the `ActivityDesktopView` entity.

## Extending an existing view entity with a currency column in ClaimCenter

If you create or extend a view entity that references a column that is of type="currencyamount", you must handle the view entity extension in a particular manner. If the view entity extension references a currencyamount column, then you also need to define the currencyProperty that the original column definition specifies on the view entity as well.

For example, suppose that in ClaimCenter, you extend the PriorClaimView view entity by adding an OpenReserves field that has a path reference to ClaimRpt.OpenReserves:

```
<viewEntityColumn name="OpenReserves" path="ClaimRpt.OpenReserves"/>
```

Looking at the definition of ClaimRpt, you see the following:

```
<column default="0" desc="The open reserves." name="OpenReserves" nullok="false" type="currencyamount">
  <columnParam name="currencyProperty" value="ClaimCurrency"/>
</column>
```

Notice that ClaimRpt.OpenReserves is of type="currencyamount". Thus, if you extend PriorClaimView with this field as indicated, you also need to add the following to PriorClaimView.etx:

```
<viewEntityTypekey name="ClaimCurrency" path="Currency"/>
```

By defining a ClaimCurrency column on the view entity, the view entity loads the claim currency as a part of the view entity. This makes the claim currency available to the currencyamount column and the application avoids loading the whole entity while dereferencing Claim.Currency.

The complete extension of the PriorClaimView view entity for this example is as follows:

```
<viewEntityExtension entityName="PriorClaimView">
  <viewEntityColumn name="OpenReserves" path="ClaimRpt.OpenReserves"/>
  <viewEntityTypekey name="ClaimCurrency" path="Currency"/>
</viewEntityExtension>
```

## Extending an existing view entity with a compound data type in PolicyCenter

If you create or extend a view entity with a column that references a compound data type, you must handle the view entity extension in a particular manner. If the view entity extension column references a compound data type, then you need to append the appropriate suffix on the path for the column. This suffix represents the relevant data for the compound data type.

For example, suppose that in PolicyCenter, you extend the PolicyPeriodSummary view entity. In doing so, you wish to add a field from the PolicyPeriod entity, TotalPremiumRPT. Looking at the definition of PolicyPeriod, you see the following:

```
<monetaryamount
  amountColumnName="TotalPremiumRPT" desc="Total amount of all premium (but not taxes or any other costs)
    for the entire policy period. The total is denormalized for higher performance UI display and
    reporting support."
  name="TotalPremiumRPT"
  nullok="true"/>
```

Notice that PolicyPeriod.TotalPremiumRPT is a monetaryamount element. As a consequence, if you extend PolicyPeriodSummary with this field, you need to define a column for the field with a particular suffix on the path attribute. Add the following to PolicyPeriodSummary.etx:

```
<viewEntityColumn name="TotalPremiumRPT" path="TotalPremiumRPT_amt"/>
```

By defining a TotalPremiumRPT column on the view entity, the view entity loads the total reported premium.

The complete extension of the PolicyPeriodSummary view entity will also have a currency type for the TotalPremiumRPT column. Adding this currency type requires a type key. The name and path for the type key must

have the respective suffixes Cur and \_cur. The complete extension of the PolicyPeriodSummary view entity is as follows:

```
<viewEntityExtension xmlns="http://guidewire.com/datamodel" entityName="PolicyPeriodSummary">
  <viewEntityColumn name="TotalPremiumRPT" path="TotalPremiumRPT_amt"/>
  <viewEntityTypekey name="TotalPremiumRPTCur" path="TotalPremiumRPT_cur"/>
</viewEntityExtension>
```

## Removing objects from the data model

It is possible to safely remove certain objects from the base configuration data model. You can do this only if the object definition is either an .eti file or an .etx file in the **configuration > config > Extensions > Entity** folder. You can also remove extensions that you have defined in changes to the base configuration.

The following table lists the objects that you can remove or hide:

| Object to remove                                                                                                                                | Location              | File extension | More information                                    |
|-------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------|----------------|-----------------------------------------------------|
| Base configuration extension entity type<br>An extension entity type that you added as part of your configuration process                       | extensions/<br>entity | .eti           | "Remove an extension entity type" on page 112       |
| Base configuration extension to an entity type<br>An extension to a data model entity type that you added as part of your configuration process | extensions/<br>entity | .etx           | "Remove an extension to an entity type" on page 113 |

Guidewire recommends that you review the material in "Implications of modifying the data model" on page 114 before you remove an object from the data model.

---

**IMPORTANT:** InsuranceSuite provides certain entity extensions in the base application. Many of the index definitions in these extensions address performance issues. Other extensions provide the ability to configure the data model in ways that would not be possible if the extension was part of the base data model. Do not modify an extension without understanding the full implications of the change.

---



---

**WARNING:** Do not attempt to remove a base configuration data definition from the **configuration > config > Metadata** folder. Also, do not attempt to remove any extension that is marked as internal. Any attempt to do so can invalidate your Guidewire installation, causing the application server to refuse to start.

---

### Related concepts

"Implications of modifying the data model" on page 114

### Related tasks

"Remove an extension entity type" on page 112

"Remove an extension to an entity type" on page 113

## Remove an extension entity type

### About this task

It is possible to remove a data model extension entity type. From the base configuration, you can remove only an extension entity that is defined in the **configuration > config > Extensions > Entity** folder as an .eti file.

For example, the base configuration includes entity extension files in the **Extensions** folder, including:

- Adjudicator (ClaimCenter)
- AutoRepairShop (ClaimCenter)
- PremiumIncentive (BillingCenter)



- RateGLClassCodeExt (PolicyCenter)
- RateWCClassCodeExt (PolicyCenter)
- SimpleDelinquency (BillingCenter)

To remove an extension entity type, you delete the .eti file that defines the extension.

#### Procedure

1. Navigate to the **Extensions > Entity** folder and locate the .eti file for the entity extension type that you want to remove.
2. Delete the .eti file for the entity extension type that you want to remove.
3. Stop and restart the application server.  
At start up, the application server recognizes a data model change and upgrades the database. For an extension entity type in the base configuration, the upgrade processing removes the database columns for the data model columns in the extension entity.

#### What to do next

If you encounter error messages, or the application server refuses to start, examine your code and correct any issues before you attempt to continue.

## Remove an extension to an entity type

#### About this task

It is possible to remove an extension to a data model entity type. From the base configuration, you can only remove an entity extension that is defined in the **configuration > config > Extensions > Entity** folder as an .etx file.

You handle the removal of entity extensions in the following ways:

- If you want to remove some of the columns, you edit the extension file. Remove any unwanted columns from the .etx file.
- If you want to remove the whole extension, you delete the .etx file.

---

**WARNING:** You cannot delete an entity extension that is marked as internal. Any attempt to do so can invalidate your Guidewire installation, causing the application server to refuse to start.

---

#### Procedure

1. Navigate to the **Extensions > Entity** folder and locate the .etx file for the entity extension that you want to remove.
  - If you want to remove the whole extension, delete the file. Omit the next step and continue to step 3.
  - If you want to remove some extension columns from that extension, open the .etx file and continue to the next step.
2. Delete any unwanted columns from the .etx file.  
A partial deletion retains the column definitions that you have left in place.
3. Stop and restart the application server.  
At start up, the application server recognizes a data model change and upgrades the database to remove the database columns for the extension columns that you deleted.

#### What to do next

If you encounter error messages, or the application server refuses to start, examine your code and correct any problems before you attempt to continue.

## Marking personally identifiable information (PII)

You can mark columns in the data model as containing personally identifiable information (PII). For example, some columns contain sensitive data such as tax IDs, passport numbers, or financial information. Data in PII columns is not included in logs, error messages, database reports, or other areas where user data can be shown.

**Note:** Marking a column as containing PII does not encrypt its data.

For a column that contains PII, add the following tag to the definition of the column:

```
<tag
  name="PersonalIdentityInfo"
  value="PII"/>
```

For example, the following defines a column for a password and marks it as containing PII:

```
<column
  desc="User password."
  name="Password"
  nullable="false"
  type="varchar">
  <tag
    name="PersonalIdentityInfo"
    value="PII"/>
  <columnParam
    name="size"
    value="30"/>
</column>
```

## Implications of modifying the data model

Any change to a data object modifies the underlying database. Typically, each entity type has a corresponding table in the database and each column in the entity definition maps to a table column. For each data object, the possibility exists that the object contains data such as rows in an entity table or data in a column. Typically, application code or your own configuration code contains references to data objects.

If you remove data objects from the data model, the database upgrade processing writes a message to inform you that there is a mismatch between the data model and the database structure. You must consider whether existing data is valuable enough to retain. You must also ensure that no code refers to any data objects that you remove.

Some changes to the data type of a column in the data model cannot be handled by the database upgrader.

## Are changes to a data model extension valid?

Removing a data model extension or a column in an extension retains the data for the table or column in the database. You can create an upgrade trigger to remove the data if you are certain that it has no value. Removing a data object is typically valid only in your development environment. If you build a new configuration, it can sometimes be necessary to remove an extension rather than to drop and recreate the database. Dropping the database destroys any data that currently exists. If you share a database instance with multiple developers, removing the extension is less disruptive for the development team.

During server start up, the application checks for configuration changes, such as modified extensions, that require a database upgrade. If the database structure does not match the data model configuration, a warning message is written to the log file.

For some modifications to a data object, the application upgrade process cannot make the corresponding database modification for you. The database upgrade tool is unable to implement extension modifications that require it to do any of the following:

- Delete a column.
- Add a non-nullable column.
- Change a column from nullable to non-nullable if null values exist in the database column or if there is not a default value.

- Change the underlying data type of a column, such as changing a varchar column to clob or varchar column to int.
- Shorten the length or type of a varchar or other text-based column, if this change truncates data in the column, such as changing a mediumtext column to shorttext. If shortening the length does not require truncating existing data, the upgrader can handle both shortening the length of a varchar column and increasing the length of a varchar column.

## Writing code to support changes to the data model

For many of the changes that you make to the data model, Guidewire provides version triggers that the database upgrader uses to make the applicable changes to the database. These changes are made on server start up if the application detects data model changes. The database upgrader cannot make some changes because there is no straightforward way to revise the data in affected columns. For example, if your change reduces the size of a varchar column, you might need to reduce the length of over-sized content by abbreviating standard phrases rather than by a simple truncation.

You can implement the `IDatamodelUpgrade` plugin and write `IDatamodelChange` upgrade triggers to synchronize the database with the data model. Guidewire supports the functions `dropTable` and `dropColumn` to assist in this process. Guidewire recommends using this approach to making changes to the database.

---

**WARNING:** Be very careful of making changes to the data model on a live production database. Some changes might invalidate your installation.

---

## Strategies for handling extension removal

In many development environments, multiple developers all use the same database instance. Before modifying the data model, first you need to communicate with your team to make them aware of your planned changes. A good way to communicate your intentions is to provide the team with a list of files that refer to the affected data model extensions. After communicating with your team, follow a process similar to the following to remove a data model extension:

1. Remove the extension entity or entity extension using the methods outlined in the following sections:
  - “Remove an extension entity type” on page 112
  - “Remove an extension to an entity type” on page 113
2. Remove any references to the object in other parts of your configuration. If you do not remove these references, the application displays error messages when attempting server start up and the server does not start.
3. Check in your changes.
4. Write code that supports the changes to the database. Implement the `IDatamodelUpgrade` plugin and write `IDatamodelChange` upgrade triggers.  
The code in the upgrade trigger runs when you start the server.
5. Run the `gwb compile` command to update the resources that the server uses.
6. Start the server.

In a production environment, Guidewire recommends that you include formal testing and quality assurance before removing or modifying an extension. Also, involve your company database administrator (DBA) and any impacted departments. Guidewire recommends also that you document your change and the reasons for it.

## Troubleshooting modifications to the data model

It is possible to make changes to the data model that cause the data values in the database to become invalid. For example, you can change an integer column to a typekey column or the reverse, but integer values in the database might not map to a valid ID in the referenced typelist table. Similarly, removing typecodes from a typelist, instead of retiring them, might also cause data inconsistencies. If you have data that references a non-existent typecode, the database upgrade does not complete and the server refuses to start. Instead of removing typecodes, retire them instead.

You can remove an extension field or the entire entity from the data model. If you do this, the server logs an informational message to the console such as:

```
pcx_ex_ProviderServicedStates: mismatch in number of columns - 5 in data model, 6 in physical database
```

## Deploying data model changes to the application server

How you deploy changes to the data model depends on if you are working in a *development* or *production* environment.

### Development environment

If you are working in a development environment, then do the following:

1. Use the following command to regenerate the *Data Dictionary* so that it reflects your data model changes:

```
gwb genDataDictionary
```

2. Stop and restart both the application server and Studio. As the application server and Studio share the same file structure in the development environment, you need only restart the development application server to pick up these changes.

If necessary (and it is almost always necessary if you change the data model), the application runs the database upgrade tool during application start up.

### Production environment

If you are working in a production environment, then do the following:

1. Use the following command to regenerate the *Data Dictionary* so that it reflects your data model changes:

```
gwb genDataDict
```

2. Create a `.war` or `.ear` file using one of the build commands.

For information on how to use these commands, see the *Developer Setup Guide*.

3. Copy this file to the application server. The target location of the file is dependent on the application server.

If necessary (and it is almost always necessary if you change the data model), the application runs the database upgrade tool during application start up.

## Entity best practices

You can change the base data model of the application to accommodate your business needs with *data model extensions*. Extensions let you add fields to existing data entities and add entirely new data entities to the data model of your application. The application uses the data model and your extensions to create and manage the tables, columns, and indexes in the relational database that supports your application.

As a best practice, Guidewire recommends that you edit the metadata definition files of the data model by using the Data Model Extensions editor in Guidewire Studio.

## Observe entity declaration naming conventions

Entity declarations have three names: a metadata definition file name, an entity attribute name, and a `displayName` attribute name. Generally, entity declaration names begin with a capital letter. Medial capitals separate words in compound entity names.

Entity declaration names differ with respect to allowing a space to separate words in compound entity names and with respect to requiring a suffix. `displayName` attribute names permit spaces to separate words. Metadata definition file names and entity attribute names do not. Unlike the other entity declaration names, metadata definition file names require a three letter suffix. These suffixes include `.eti`, `.eix`, and `.etx`.

Metadata definition file names and entity attribute names are required programmatic names. They are for use in the Gosu programming language. These names prohibit spaces altogether. In addition, the base of a metadata definition file name must match the value of the corresponding entity attribute name.

While `displayName` attribute names are optional, Guidewire recommends that you use such names when two conditions are true. If you create a custom entity, and the programmatic entity declaration name for the entity exceeds one word, use a `displayName` attribute name. For example, an entity with an entity attribute name of `Activity` does not require a `displayName` attribute name. However, consider a `displayName` attribute name when you have an entity attribute name such as `ContactAddress` or `AllEscalatedActivitiesClaimMetric`. In these cases, use `Contact Address` and `All Escalated Activities Claim Metric` as respective `displayName` attribute names. The graphical user interface uses the `displayName` attribute name when displaying the corresponding entity.

## Add a suffix to entity extensions

In future releases, Guidewire may add or change base entities. If you make your own changes to entities, there is the possibility of a naming conflict with one of these future updates. For example, you may add new entities, or new properties to existing entities. To avoid possible naming conflicts, Guidewire recommends that you append the suffix `_Ext` to your new entities and properties.

If you add a new entity, add `_Ext` to the endings of entity names so that Guidewire Studio and the *Data Dictionary* list them next to any entities that they extend. For example, `CreditHistory_Ext`. The property names in your new entity do not need the suffix `_Ext` because the name of the entity already has the `_Ext` suffix.

If you add new properties to an existing Guidewire entity, add the `_Ext` suffix to their names. Follow this recommendation for properties, and also for virtual properties and functions added through entity enhancements.

**Note:** Because the application automatically prefixes extension table names with `xxx_`, if you run into limits on the length of the table name, you can consider removing the `_Ext` suffix from the table name.

As an example of property extensions to a base entity, the following sample metadata file extends the base `Policy` entity with an additional property (column) and an additional typekey.

```
<extension entityName="Policy">
...
  <column desc="Description of the column"
    name="MyCustomColumn_Ext"
    nullok="true"
    default="abc"
    type="varchar"
    <columnParam
      name="size"
      value="60" />
  </column>

  <typekey desc="Description of the typekey"
    name="MyCustomTypekey_Ext"
    typelist="myCustomTypeList_Ext"
    nullok="true" />
...
</extension>
```

## Use singular words for field names except for arrays

Guidewire recommends that you name most fields with a singular word such as `Phone` or `Age`. However, because array fields reference a list of objects, Guidewire recommends that you name them with a plural word.

For example, `Claim.Description` and `Claim.Policy` are single fields on a `Claim` entity, but `Claim.Notes` is an array of multiple notes. Also, for arrays fields that are extensions, make the primary name plural and not the `Ext` prefix or suffix. For example, use `Ext_MedTreatments` or `MedTreatments_Ext`, and not `MedTreatment_Exts`.

## Add ID as a suffix to column names for foreign keys

Guidewire recommends that you add ID as a suffix to the column names of foreign keys. By default, the column name of foreign keys have the same name as the foreign key names. Use the `columnName` attribute of `foreignkey` elements to override their default column names. For example:

```
<foreignkey
  columnName="ClaimID"
  ...
  name="Claim"/>

<foreignkey
  columnName="PolicyID"
  ...
  name="Policy"/>
```

Adding the suffix ID to the column names of foreign keys helps database administrators identify columns in the database that Guidewire uses as foreign keys.

If you add a foreign key as an extension to a base entity, follow the best practice of adding a prefix or suffix to the name. For example:

```
<foreignkey
  columnName="Claim_ExtID"
  ...
  name="Claim_Ext"/>

<foreignkey
  columnName="Policy_ExtID"
  ...
  name="Policy_Ext"/>
```

# Example: Creating assignable entities

This topic describes how to modify the Guidewire data model by showing how to construct a data entity and make it assignable. This topic includes the following:

- “Creating an assignable extension entity type” on page 119
- “Making your extension entity eligible for round-robin assignment” on page 123
- “Creating an assignable extension entity type that uses foreign keys” on page 129

## Creating an assignable extension entity type

To create an assignable extension entity, perform the following steps:

- “Step 1: Create extension entity type `UserAssignableEntity_Ext`” on page 119
- “Step 2: Create assignment class `NewAssignableMethodsImpl`” on page 121
- “Step 3: Test assignment of your extension entity instance” on page 122

---

**IMPORTANT:** The code and objects in this example refer to Guidewire ClaimCenter. However, the principles involved in extending the data model apply to all Guidewire applications.

---

### Step 1: Create extension entity type `UserAssignableEntity_Ext`

The first step in creating an assignable entity type is to create the entity or extend the entity if it already exists.

#### About this task

The assignable entity type implements the following:

- `CCAssignable` delegate class
- `CCAssignableMethods` interface

The entity type must link to a Gosu class that defines the necessary assignment methods and properties. The following procedure creates a new extension entity that implements `NewAssignableMethodsImpl`, a delegate class.

#### Procedure

1. Create the new extension entity file.
  - a) In Studio, navigate to **configuration > config > extensions > Entity**.

- b) Right-click **Entity** and select **New > Entity**.
2. Provide the required information for the new entity definition.
  - a) Enter the name of the extension entity in the text field.  
To follow the rest of this procedure, use the name `UserAssignableEntity_Ext`.
  - b) In **Entity Type**, select **entity**.
  - c) In **Desc**, type:  
Assignable extension entity, for testing
  - d) In **Type**, select **retireable**.
  - e) Select the check boxes **Extendable** and **Final**.
  - f) Click **OK**.
3. In the entity editor, add the following items to `UserAssignableEntity_Ext`.

| Element type        | Attribute   | Value                                                                       |
|---------------------|-------------|-----------------------------------------------------------------------------|
| fulldescription     |             | Simple entity that you can assign. This entity does not have a foreign key. |
| implementsEntity    | name        | CCAssignable                                                                |
| implementsInterface | iface       | gw.api.assignment.CCAssignableMethods                                       |
|                     | impl        | gw.assignment.NewAssignableMethodsImpl                                      |
| column              | name        | InstanceUpdateTime                                                          |
|                     | type        | datetime                                                                    |
|                     | nullok      | true                                                                        |
| column              | name        | VarcharColumn                                                               |
|                     | type        | varchar                                                                     |
|                     | nullok      | true                                                                        |
| columnparam         | name        | size                                                                        |
|                     | value       | 60                                                                          |
| typekey             | name        | Country                                                                     |
|                     | typelist    | Country                                                                     |
|                     | nullok      | true                                                                        |
| typekey             | name        | PhoneType                                                                   |
|                     | typelist    | PhoneType                                                                   |
|                     | nullok      | true                                                                        |
| index               | name        | internal1                                                                   |
| indexcol            | name        | VarcharColumn                                                               |
|                     | keyposition | 1                                                                           |

## Results

Notice the following:

- Entity `UserAssignableEntity_Ext` implements the `CCAssignable` delegate.
- Entity `UserAssignableEntity_Ext` uses class `NewAssignableMethodsImpl`, which implements the `CCAssignableMethods` interface, to define the necessary assignment methods and properties.

## What to do next

- “Step 2: Create assignment class `NewAssignableMethodsImpl`” on page 121



### See also

- For information on how to extend the data model, see “Modifying the base data model” on page 89.
- For information on the data model in general, see “The application data model” on page 19.

## Step 2: Create assignment class `NewAssignableMethodImpl`

Any new assignable entity must implement the `CCAssignableMethods` interface.

### Before you begin

- “Step 1: Create extension entity type `UserAssignableEntity_Ext`” on page 119

### About this task

The following procedure defines a delegate class that provides an implementation of the methods in the `CCAssignableMethods` interface. In the example, the `NewAssignableMethodImpl` class extends the abstract superclass `CCAssignableMethodsBaseImpl`. The `CCAssignableMethodsBaseImpl` abstract superclass provides standard implementations of some of the functionality for which the `CCAssignableMethods` interface provides method signatures.

### Procedure

1. Create a new class file in the package `gw.assignment` and name it `NewAssignableMethodImpl`.
  - a) In Studio, navigate to **configuration > gsrc > gw > assignment**.
  - b) Right-click **assignment** and select **New > Gosu Class**.
  - c) Enter the name of the class in the text box.

For this example, enter the following class name.

```
NewAssignableMethodImpl
```

- d) Click **OK**.

Studio creates a skeleton class file in the **assignment** folder and opens the file in an editor tab.

2. Have your class do one of the following.
  - Implement the interface `gw.api.assignment.CCAssignableMethods`.
  - Extend the abstract superclass `gw.api.assignment.CCAssignableMethodsBaseImpl`.

Which of these options you implement depends on your business needs. In either case, you must provide method bodies for all unimplemented methods. To determine what methods you need to implement, you need merely to have your class implement the interface or extend the abstract superclass.

In this example, you extend the abstract superclass. Make the class declaration look like the following line.

```
class NewAssignableMethodImpl extends CCAssignableMethodsBaseImpl {
```

The editor shows an error at `CCAssignableMethodsBaseImpl`.

3. Press **Alt-Enter** and click **Import class**.
4. Press **Alt-Enter** and click **Implement methods**. Then, click **OK**.

Studio provides skeleton method overrides for all the abstract methods in the `gw.api.assignment.CCAssignableMethodsBaseImpl` class. In later steps, you also override other methods that the abstract class implements.

5. Add a constructor to the class by entering the following code before the first overridden method.

```
construct(testEntity : UserAssignableEntity_Ext) {  
    super(testEntity)  
}
```

6. Add the following methods that override methods in `CCAssignableMethodsBaseImpl`.

```

override property get Owner() : UserAssignableEntity_Ext {
    return super.Owner as UserAssignableEntity_Ext
}

override function shouldCascadeAssignment(parent : Assignable, defaultOwnerUserId : Key,
                                          defaultGroupId : Key) : boolean {
    return false
}

```

7. To resolve the error indication, press **Alt-Enter** and click **Import class**. Import the class `gw.pl.persistence.core.Key`.

## Results

Your new class looks like the following code.

```

package gw.assignment

uses gw.api.assignment.CCAssignableMethods
uses gw.api.assignment.CCAssignableMethodsBaseImpl
uses gw.entity.ILinkPropertyInfo
uses gw.pl.persistence.core.Key

class NewAssignableMethodsImpl extends CCAssignableMethodsBaseImpl {

    construct(testEntity : UserAssignableEntity_Ext) {
        super(testEntity)
    }

    override property get AssignmentReviewActivitySubject() : String {
        return null
    }

    override property get AssignmentReviewActivityLinkProperty() : ILinkPropertyInfo {
        return null
    }

    override property get OwningClaim() : Claim {
        return null
    }

    override function pushAssignmentPopup(list : List<CCAssignableMethods>) {
    }

    override property get ChildrenForCascadeAssignment() : List<CCAssignableMethods> {
        return null
    }

    override property get Owner() : UserAssignableEntity_Ext {
        return super.Owner as UserAssignableEntity_Ext
    }

    override function shouldCascadeAssignment(parent : Assignable, defaultOwnerUserId : Key,
                                          defaultGroupId : Key) : boolean {
        return false
    }
}

```

## What to do next

- “Step 3: Test assignment of your extension entity instance” on page 122

## Step 3: Test assignment of your extension entity instance

### Before you begin

- “Step 2: Create assignment class `NewAssignableMethodsImpl`” on page 121

### About this task

After you create an assignable entity type, test your work to verify that your entity definition and associated Gosu code are correct. Because you have changed the data model, you first restart the server to force a database upgrade.

### Procedure

1. Stop the server, if it is running.
2. Start the server from within Studio.
3. Correct any problems that occur and repeat the previous steps until the server starts without errors.
4. If you have not already done so, load the demonstration data into the database.

The next step requires this data.

5. Click **Tools > Gosu Scratchpad** to open the Studio Gosu Scratchpad.
6. Enter the following code in the **Gosu Scratchpad** tab.

```
uses gw.api.database.Query
uses gw.api.database.Relop
uses gw.transaction.Transaction

var _users = Query.make(User).join(User#Credential).compare(Credential#UserName, Equals, "aapplegate")
var _usr = _users.select().AtMostOneRow
var _group = _usr.GroupUsers[0].Group
Transaction.runWithNewBundle(bundle -> {
  var newExtEntity = new UserAssignableEntity_Ext()
}, "su")
```

7. Click **Run**.  
Output similar to the following lines appears in the **Console** tab.

```
true           The assign method succeeded. Thus, it returns true.
Andy Applegate The user that is assigned the newExtEntity (UserAssignableEntity_Ext)
object.
Auto1 - TeamA  The group that is assigned the newExtEntity (UserAssignableEntity_Ext)
object.
```

## Making your extension entity eligible for round-robin assignment

To assign an entity instance through round-robin assignment, you invoke the `assignUserByRoundRobin` method on the entity. To implement round-robin assignment on the assignable entity, you need to add extra tracking properties for the round-robin state to the following entities:

- `DynamicAssignmentState`
- `GroupAssignmentState`
- `GroupUserAssignmentState`

To do so, you add these properties as an extension to a base configuration entity. If the base entity does not exist, you create an extension entity with the necessary properties.

Finally, you must create a new Gosu class that extends (subclasses) the `AssignmentType` class. You add methods to this subclass to instruct the round-robin infrastructure how to find the newly added properties.

The following procedures assume the existence of an assignable entity type named `UserAssignableEntity_Ext`. The steps in “Creating an assignable extension entity type” on page 119 describe how to create this entity.

This example includes the following steps:

- “Step 1: Extend the `AssignmentState` entity types” on page 124
- “Step 2: Subclass class `AssignmentType`” on page 126
- “Step 3: Create class `UserAssignableEntityExtEnhancement`” on page 127
- “Step 4: Test round-robin assignment” on page 128

---

**IMPORTANT:** The code and objects in this example refer to Guidewire ClaimCenter. However, the principles involved in extending the data model apply to all Guidewire applications.

---

## Step 1: Extend the AssignmentState entity types

You extend AssignmentState entity types to add fields on the object that ClaimCenter uses to track information for the round robin state.

### Before you begin

- “Creating an assignable extension entity type” on page 119

### About this task

The first step in creating an entity type that you can use with round-robin assignment is to extend the following base configuration entity types:

- DynamicAssignmentState
- GroupAssignmentState
- GroupUserAssignmentState

### Procedure

1. In Guidewire Studio, open DynamicAssignmentState.etx for editing or create a new entity extension.
  - a) Press Ctrl+N and start entering the entity type name.  
Studio displays a number of matches because files with the name DynamicAssignmentState already exist.
  - b) If DynamicAssignmentState.etx appears in the list, double-click that file.
  - c) If DynamicAssignmentState.etx is not in the list, create a new extension by navigating to **configuration > config > extensions > Entity**. Right-click **Entity** and select **New > Entity Extension**. Enter DynamicAssignmentState and click OK.
2. In the entity editor, add the following items to the DynamicAssignmentState extension.

| Element type | Attribute  | Value          |
|--------------|------------|----------------|
| foreignkey   | name       | LastTEAEUser   |
|              | fkentity   | User           |
|              | nullok     | true           |
|              | columnName | LastTEAEUserId |
| foreignkey   | name       | LastTEAEGrp    |
|              | fkentity   | Group          |
|              | nullok     | true           |
|              | columnName | LastTEAEGrpId  |
| foreignkey   | name       | LastTCAUser    |
|              | fkentity   | User           |
|              | nullok     | true           |
|              | columnName | LastTCAUserId  |
| foreignkey   | name       | LastTCAGrp     |
|              | fkentity   | Group          |
|              | nullok     | true           |
|              | columnName | LastTCAGrpId   |

3. Add fields to the file GroupAssignmentState.etx. If this file does not exist, you need to create it. Follow the instructions in “Step 1” to search for GroupAssignmentState.etx.
4. In the entity editor, add the following items to the GroupAssignmentState extension.

| Element type | Attribute  | Value          |
|--------------|------------|----------------|
| foreignkey   | name       | LastTEAEUser   |
|              | fkentity   | User           |
|              | nullok     | true           |
|              | columnName | LastTEAEUserId |
| foreignkey   | name       | LastTEAEGrp    |
|              | fkentity   | Group          |
|              | nullok     | true           |
|              | columnName | LastTEAEGrpId  |
| foreignkey   | name       | LastTCAUser    |
|              | fkentity   | User           |
|              | nullok     | true           |
|              | columnName | LastTCAUserId  |
| foreignkey   | name       | LastTCAGrp     |
|              | fkentity   | Group          |
|              | nullok     | true           |
|              | columnName | LastTCAGrpId   |
| column       | name       | TEAELoad       |
|              | type       | integer        |
|              | nullok     | true           |
| column       | name       | TCALoad        |
|              | type       | integer        |
|              | nullok     | true           |

5. Add fields to the file GroupUserAssignmentState.etx. If this file does not exist, you need to create it. Follow the instructions in “Step 1” to search for GroupUserAssignmentState.etx.
6. In the entity editor, add the following items to the GroupUserAssignmentState extension.

| Element type | Attribute | Value    |
|--------------|-----------|----------|
| column       | name      | TEAELoad |
|              | type      | integer  |
|              | nullok    | true     |
| column       | name      | TCALoad  |
|              | type      | integer  |
|              | nullok    | true     |

#### What to do next

- “Step 2: Subclass class AssignmentType” on page 126

## Step 2: Subclass class AssignmentType

The AssignmentType class contains getter and setter methods that inform the round-robin mechanism of the location of fields in the AssignmentState entity types.

### Before you begin

- “Step 1: Extend the AssignmentState entity types” on page 124

### About this task

Now, you need to create a subclass of the AssignmentType class to access the entity fields that you created in the previous step.

### Procedure

1. Create a new class file in the package `gw.assignment` and name it `UserAssignableEntityExtAssignmentType`.
  - a) In Studio, navigate to **configuration > gsrc > gw > assignment**.
  - b) Right-click **assignment** and select **New > Gosu Class**.
  - c) Enter the name of the class in the text box.

For this example, enter the following class name.

```
UserAssignableEntityExtAssignmentType
```

Studio creates a skeleton class file in the **assignment** folder and opens the file in an editor tab.

2. Enter the following code in your class file.

```
package gw.assignment

uses gw.api.assignment.AssignmentType
uses gw.entity.IEntityType
uses gw.pl.persistence.core.Key

class UserAssignableEntityExtAssignmentType extends AssignmentType {
    construct() {}

    override function getLastAssignedUser(groupState : GroupAssignmentState) : Key {
        return groupState.LastTEAEUser.ID
    }

    override function getLastAssignedUser(dynamicState : DynamicAssignmentState) : Key {
        return dynamicState.LastTEAEUser.ID
    }

    override function setLastAssignedUser(groupState : GroupAssignmentState, user : Key) {
        groupState.LastTEAEUser = groupState.Bundle.loadBean(user) as User
    }

    override function setLastAssignedUser(dynamicState : DynamicAssignmentState, user : Key) {
        dynamicState.LastTEAEUser = dynamicState.Bundle.loadBean(user) as User
    }

    override function getUserLoad(groupUserState : GroupUserAssignmentState) : int {
        return groupUserState.TEAELoad
    }

    override function setUserLoad(groupUserState : GroupUserAssignmentState, load : int) {
        groupUserState.TEAELoad = load
    }

    override function getLastAssignedGroup(groupState : GroupAssignmentState) : Key {
        return groupState.LastTEAEGrp.ID
    }

    override function getLastAssignedGroup(dynamicState : DynamicAssignmentState) : Key {
        return dynamicState.LastTEAEGrp.ID
    }

    override function setLastAssignedGroup(groupState : GroupAssignmentState, group : Key) {
        groupState.LastTEAEGrp = groupState.Bundle.loadBean(group) as Group
    }

    override function setLastAssignedGroup(dynamicState : DynamicAssignmentState, group : Key) {
```

```

    dynamicState.LastTEAEGrp = dynamicState.Bundle.loadBean(group) as Group
  }

  override function getGroupLoad(groupState : GroupAssignmentState) : int {
    return groupState.TEAELoad
  }

  override function setGroupLoad(groupState : GroupAssignmentState, load : int) {
    groupState.TEAELoad = load
  }

  protected override property get AssignableClass() : IEntityTypes {
    return UserAssignableEntity_Ext
  }
}

```

Studio indicates that the code is invalid in the `get AssignableClass` method definition because you have not yet created the necessary `UserAssignableEntity_Ext` Gosu enhancement. You create this entity enhancement in the next step.

#### What to do next

- “Step 3: Create class `UserAssignableEntityExtEnhancement`” on page 127

## Step 3: Create class `UserAssignableEntityExtEnhancement`

You create an enhancement to the `UserAssignableEntity_Ext` entity type for use in creating instances of that type.

#### Before you begin

- “Step 2: Subclass class `AssignmentType`” on page 126

#### Procedure

1. Create a new enhancement file and name it `UserAssignableEntityExtEnhancement`.

- a) In Studio, navigate to **configuration > gsrc > gw > assignment**.
- b) Right-click **assignment** and select **New > Gosu Enhancement**.
- c) Enter the `UserAssignableEntity_Ext` in the **Enhanced type** text box.

The text that you enter filters the available entity types. If multiple selections are shown, ensure that you select `UserAssignableEntity_Ext`. This entity type is the one that you need to enhance.

- d) Enter the name of the class in the **Type to enhance** text box.

For this example, enter the following class name.

```
UserAssignableEntityExtEnhancement
```

Studio creates a skeleton class file in the **assignment** folder and opens the file in an editor tab.

- e) Click **OK**.

2. Enter the following code in the enhancement file.

```

package gw.assignment

uses gw.api.assignment.AssignmentType

enhancement UserAssignableEntityExtEnhancement : UserAssignableEntity_Ext {
  static property get ASSIGNMENT_TYPE() : AssignmentType {
    return new UserAssignableEntityExtAssignmentType()
  }
}

```

#### What to do next

- “Step 4: Test round-robin assignment” on page 128

## Step 4: Test round-robin assignment

After you create an assignable entity type that is suitable for use with round-robin assignment, you must test your work to verify that you have performed the previous steps correctly.

### Before you begin

- “Step 3: Create class UserAssignableEntityExtEnhancement” on page 127

### About this task

Because you have changed the data model, you first restart the server to force a database upgrade.

### Procedure

1. Stop the server, if it is running.
2. Start the server from within Studio.
3. Correct any problems that occur and repeat the previous steps until the server starts without errors.
4. If you have not already done so, load the demonstration data into the database.

The next step requires this data.

5. Click **Tools > Gosu Scratchpad** to open the Studio Gosu Scratchpad.
6. Enter the following code in the **Gosu Scratchpad** tab.

```
uses gw.api.database.Query
uses gw.api.database.Relop
uses gw.transaction.Transaction

var _users = Query.make(User).join(User#Credential).compare(Credential#UserName, Equals, "aapplegate")
var _usr = _users.select().AtMostOneRow
var _group = _usr.GroupUsers[0].Group
Transaction.runWithNewBundle(\bundle -> {
  for (i in 1..3) {
    print("Pass " + i + " :")
    var newExtEntity = new UserAssignableEntity_Ext()
    // Assign the entity to the specified user and group
    print(newExtEntity.assign(_group, _usr) + "\t\t\tThe assign method succeeded. Thus, it returns true.")
    print(newExtEntity.AssignedUser + "\t\t\tThe user that is assigned the newExtEntity (UserAssignableEntity_Ext)
object.")
    print(newExtEntity.AssignedGroup + "\t\t\tThe group that is assigned the newExtEntity (UserAssignableEntity_Ext)
object.")
    var assignedToUser = newExtEntity.assignUserByRoundRobin(false, _group)
    print("\t" + assignedToUser + "\t\t\tThe assignUserByRoundRobin method succeeded. Thus, it returns true.")
    print("\t" + newExtEntity.AssignedUser + "\t\t\tThe assignUserByRoundRobin method assigns the object to the next
person in the group.")
  }
}, "su")
```

7. Click **Run**.

Output similar to the following lines appears in the **Console** tab.

```
Pass 1:
true           The assign method succeeded. Thus, it returns true.
Andy Applegate The user that is assigned the newExtEntity (UserAssignableEntity_Ext)
object.
Auto1 - TeamA  The group that is assigned the newExtEntity (UserAssignableEntity_Ext)
object.
true           The assignUserByRoundRobin method succeeded. Thus, it returns true.
Andy Applegate The assignUserByRoundRobin method assigns the object to the next
person in the group.
Pass 2:
true           The assign method succeeded. Thus, it returns true.
Andy Applegate The user that is assigned the newExtEntity (UserAssignableEntity_Ext)
object.
Auto1 - TeamA  The group that is assigned the newExtEntity (UserAssignableEntity_Ext)
object.
true           The assignUserByRoundRobin method succeeded. Thus, it returns true.
Sue Smith      The assignUserByRoundRobin method assigns the object to the next
person in the group.
```



```

Pass 3:
true      The assign method succeeded. Thus, it returns true.
Andy Applegate object. The user that is assigned the newExtEntity (UserAssignableEntity_Ext)
Auto1 - TeamA object. The group that is assigned the newExtEntity (UserAssignableEntity_Ext)
true      The assignUserByRoundRobin method succeeded. Thus, it returns true.
Betty Baker The assignUserByRoundRobin method assigns the object to the next
person in the group.

```

## Creating an assignable extension entity type that uses foreign keys

To create an assignable extension entity type with foreign key links to Claim and Activity, perform the following steps:

- “Step 1: Create extension entity type TestClaimAssignable\_Ext” on page 129
- “Step 2: Add foreign keys to assignable entity types” on page 130
- “Step 3: Create new assignment type for extension entity type TestClaimAssignable\_Ext” on page 131
- “Step 4: Create enhancement class TestClaimAssignableExtEnhancement” on page 132
- “Step 5: Create delegate class TestClaimAssignableMethodsImpl” on page 133
- “Step 6: Add the necessary permission for the extension entity type” on page 134
- “Step 7: Test assignment of the assignable entity type” on page 136

---

**IMPORTANT:** The code and objects in this example refer to Guidewire ClaimCenter. However, the principles involved in extending the data model apply to all Guidewire applications.

---

### Step 1: Create extension entity type TestClaimAssignable\_Ext

#### Before you begin

- “Making your extension entity eligible for round-robin assignment” on page 123

#### About this task

The first step is to create the assignable entity type. As with any assignable entity type, it must implement the following:

- CCAssignable delegate class
- CCAssignableMethods interface

#### Procedure

1. Create the new extension entity type.
  - a) In Studio, navigate to **configuration > config > extensions > Entity**.
  - b) Right-click **Entity** and select **New > Entity**.
2. Provide the required information for the new entity type definition.
  - a) Enter the name of the extension entity type in the text field.  
To follow the rest of this procedure, use the name `TestClaimAssignable_Ext`.
  - b) In **Entity Type**, select **entity**.
  - c) In **Desc**, type:

CCAssignable extension entity, which has a foreign key back to Claim, used for testing

- d) In **Type**, select **retireable**.
- e) Select the check boxes **Extendable** and **Final**.
- f) Click **OK**.

3. In the entity editor, add the following items to `TestClaimAssignable_Ext`.

| Element type               | Attribute         | Value                                                                                                                                                                                                                                                                                                                                                                |
|----------------------------|-------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                            | <b>exportable</b> | <b>true</b>                                                                                                                                                                                                                                                                                                                                                          |
| <b>fulldescription</b>     |                   | Simple test assignable with a foreign key back to Claim. This foreign key, together with the <code>CCAssignableMethods</code> delegate in <code>TestClaimAssignableMethodImpl</code> , allows this entity type to participate in manual assignment, assignment to claim owner and cascading. An instance of this type also creates history events as it is assigned. |
| <b>implementsEntity</b>    | <b>name</b>       | <b>Extractable</b>                                                                                                                                                                                                                                                                                                                                                   |
| <b>implementsEntity</b>    | <b>name</b>       | <b>CCAssignable</b>                                                                                                                                                                                                                                                                                                                                                  |
| <b>implementsInterface</b> | <b>iface</b>      | <code>gw.api.assignment.CCAssignableMethods</code>                                                                                                                                                                                                                                                                                                                   |
|                            | <b>impl</b>       | <code>gw.assignment.TestClaimAssignableMethodImpl</code>                                                                                                                                                                                                                                                                                                             |
| <b>foreignkey</b>          | <b>name</b>       | <b>Claim</b>                                                                                                                                                                                                                                                                                                                                                         |
|                            | <b>fkentity</b>   | <b>Claim</b>                                                                                                                                                                                                                                                                                                                                                         |
|                            | <b>nullok</b>     | <b>false</b>                                                                                                                                                                                                                                                                                                                                                         |
|                            | <b>columnName</b> | <b>ClaimID</b>                                                                                                                                                                                                                                                                                                                                                       |
|                            | <b>desc</b>       | The Claim for this test assignable                                                                                                                                                                                                                                                                                                                                   |
|                            | <b>exportable</b> | <b>false</b>                                                                                                                                                                                                                                                                                                                                                         |

## Results

Notice the following:

- Entity `TestClaimAssignable_Ext` implements the `CCAssignable` delegate.
- Entity `TestClaimAssignable_Ext` uses class `TestClaimAssignableMethodImpl`, which implements the `CCAssignableMethods` interface, to define the necessary assignment methods and properties.
- Entity `TestClaimAssignable_Ext` has a foreign key back to `Claim`.

## What to do next

- “Step 2: Add foreign keys to assignable entity types” on page 130

## Step 2: Add foreign keys to assignable entity types

To link your assignable entity type to a base configuration assignable entity type, you need a foreign key from that base configuration entity type to your assignable entity type.

### Before you begin

- “Step 1: Create extension entity type `TestClaimAssignable_Ext`” on page 129

### Procedure

1. In Guidewire Studio, open `Claim.etx` for editing or create a new entity extension.

- a) Press **Ctrl-N** and start entering the entity type name.

Studio displays a number of matches because files with the name `Claim` already exist.

- b) Find `Claim.etx` in the list and double-click that file.
2. In the entity editor, add the following items to the `Claim` extension.

| Element type | Attribute   | Value                           |
|--------------|-------------|---------------------------------|
| array        | name        | TestClaimAssignables            |
|              | arrayentity | TestClaimAssignable_Ext         |
|              | desc        | Test assignables for this claim |

3. Add fields to the file `Activity.etx`. If this file does not exist, you need to create it. Follow the instructions in “Step 1” to search for `Activity.etx`.
4. In the entity editor, add the following items to the `Activity` extension.

| Element type | Attribute   | Value                   |
|--------------|-------------|-------------------------|
| foreignkey   | name        | TestClaimAssignable     |
|              | arrayentity | TestClaimAssignable_Ext |
|              | nullok      | true                    |

#### What to do next

- “Step 3: Create new assignment type for extension entity type `TestClaimAssignable_Ext`” on page 131

## Step 3: Create new assignment type for extension entity type `TestClaimAssignable_Ext`

The `AssignmentType` class contains getter and setter methods that inform the round-robin mechanism of the location of fields in the `AssignmentState` entity types.

#### Before you begin

- “Step 2: Add foreign keys to assignable entity types” on page 130

#### Procedure

1. Create a new class file in the package `gw.assignment` and name it `TestClaimAssignableExtAssignmentType`.
  - a) In Studio, navigate to **configuration > gsrc > gw > assignment**.
  - b) Right-click **assignment** and select **New > Gosu Class**.
  - c) Enter the name of the class in the text box.  
For this example, enter the following class name.

```
TestClaimAssignableExtAssignmentType
```

Studio creates a skeleton class file in the **assignment** folder and opens the file in an editor tab.

- d) Click **OK**.

Studio creates a skeleton class file in the **assignment** folder and opens the file in an editor tab.

2. Enter the following code in the class file.

```
package gw.assignment

uses gw.api.assignment.AssignmentType
uses gw.entity.IEntityType
uses gw.pl.persistence.core.Key

class TestClaimAssignableExtAssignmentType extends AssignmentType {
    construct() { }

    override function getLastAssignedUser(groupState : GroupAssignmentState) : Key {
```

```

    return groupState.LastTCAUser.ID
}

override function getLastAssignedUser(dynamicState : DynamicAssignmentState) : Key {
    return dynamicState.LastTCAUser.ID
}

override function setLastAssignedUser(groupState : GroupAssignmentState, user : Key) {
    groupState.LastTCAUser = groupState.Bundle.loadBean(user) as User
}

override function setLastAssignedUser(dynamicState : DynamicAssignmentState, user : Key) {
    dynamicState.LastTCAUser = dynamicState.Bundle.loadBean(user) as User
}

override function getUserLoad(groupUserState : GroupUserAssignmentState) : int {
    return groupUserState.TCALoad
}

override function setUserLoad(groupUserState : GroupUserAssignmentState, load : int) {
    groupUserState.TCALoad = load
}

override function getLastAssignedGroup(groupState : GroupAssignmentState) : Key {
    return groupState.LastTCAGrp.ID
}

override function getLastAssignedGroup(dynamicState : DynamicAssignmentState) : Key {
    return dynamicState.LastTCAGrp.ID
}

override function setLastAssignedGroup(groupState : GroupAssignmentState, group : Key) {
    groupState.LastTCAGrp = groupState.Bundle.loadBean(group) as Group
}

override function setLastAssignedGroup(dynamicState : DynamicAssignmentState, group : Key) {
    dynamicState.LastTCAGrp = dynamicState.Bundle.loadBean(group) as Group
}

override function getGroupLoad(groupState : GroupAssignmentState) : int {
    return groupState.TCALoad
}

override function setGroupLoad(groupState : GroupAssignmentState, load : int) {
    groupState.TCALoad = load
}

protected override property get AssignableClass() : IEntityTypes {
    return TestClaimAssignable_Ext
}
}

```

#### What to do next

- “Step 4: Create enhancement class TestClaimAssignableExtEnhancement” on page 132

## Step 4: Create enhancement class TestClaimAssignableExtEnhancement

Create an enhancement to the TestClaimAssignable\_Ext entity type for use in creating entities of that type.

#### Before you begin

- “Step 3: Create new assignment type for extension entity type TestClaimAssignable\_Ext” on page 131

#### Procedure

1. Create a new enhancement file and name it TestClaimAssignableExtEnhancement.
  - a) In Studio, navigate to **configuration > gsrc > gw > assignment**.
  - b) Right-click **assignment** and select **New > Gosu Enhancement**.
  - c) Enter the TestClaimAssignable\_Ext in the **Enhanced type** text box.
  - d) Enter the name of the class in the **Type to enhance** text box.

For this example, enter the following class name.

```
TestClaimAssignableExtEnhancement
```

Studio creates a skeleton class file in the **assignment** folder and opens the file in an editor tab.

e) Click **OK**.

2. Enter the following code in the enhancement file.

```
package gw.assignment

uses gw.api.assignment.AssignmentType

enhancement TestClaimAssignableExtEnhancement : TestClaimAssignable_Ext {
    static property get ASSIGNMENT_TYPE() : AssignmentType {
        return new TestClaimAssignableExtAssignmentType()
    }
}
```

#### What to do next

- “Step 5: Create delegate class TestClaimAssignableMethodImpl” on page 133

## Step 5: Create delegate class TestClaimAssignableMethodImpl

Any new assignable entity type must implement the CCAssignableMethods interface.

#### Before you begin

- “Step 4: Create enhancement class TestClaimAssignableExtEnhancement” on page 132

#### About this task

The TestClaimAssignable\_Ext entity type that you created specified the delegate class TestClaimAssignableMethodImpl to implement the CCAssignableMethods interface.

The following procedure defines a delegate class that provides an implementation of the methods in the CCAssignableMethods interface. In this example, the TestClaimAssignableMethodImpl class extends the abstract superclass CCAssignableMethodsBaseImpl. The CCAssignableMethodsBaseImpl abstract superclass provides standard implementations of some of the functionality for which the CCAssignableMethods interface provides method signatures.

#### Procedure

1. Create a new class file in the package gw.assignment and name it TestClaimAssignableMethodImpl.
  - a) In Studio, navigate to **configuration > gsrc > gw > assignment**.
  - b) Right-click **assignment** and select **New > Gosu Class**.
  - c) Enter the name of the class in the text box.

For this example, enter the following class name.

```
TestClaimAssignableMethodImpl
```

d) Click **OK**.

Studio creates a skeleton class file in the **assignment** folder and opens the file in an editor tab.

2. Enter the following code in the file.

```
package gw.assignment

uses gw.api.assignment.CCAssignableMethods
uses gw.api.assignment.CCAssignableMethodsBaseImpl
uses gw.entity.ILinkPropertyInfo
uses gw.pl.persistence.core.Key

/**
 * Example CCAssignableMethods implementation for an assignable entity that is related to a claim,
 * and which can be manually assigned, assigned to claim owner, or cascaded. Also creates a history
 * event as it is assigned.
 */
```

```

class TestClaimAssignableMethodsImpl extends CCAssignableMethodsBaseImpl {
    construct(testEntity : TestClaimAssignable_Ext) {
        super(testEntity)
    }

    override property get AssignmentReviewActivitySubject() : String {
        return "Test Claim Assignable Assignment Review"
    }

    override property get AssignmentReviewActivityLinkProperty() : ILinkPropertyInfo {
        return Activity.Type.TypeInfo.getProperty("TestClaimAssignable") as ILinkPropertyInfo
    }

    override property get Owner() : TestClaimAssignable_Ext {
        return super.Owner as TestClaimAssignable_Ext
    }

    override property get OwningClaim() : Claim {
        return Owner.Claim
    }

    override function pushAssignmentPopup(assignables : List<CCAssignableMethods>) {
        // Needed for user interface
        AssignmentPopupUtil.pushAssignTestClaimAssignables
            (assignables.whereTypeIs(TestClaimAssignable_Ext).toTypedArray())
    }

    override property get ChildrenForCascadeAssignment() : List<CCAssignableMethods> {
        return null
    }

    override function createAssignmentHistoryEvent(assignable : Assignable) : History {
        var result = super.createAssignmentHistoryEvent(assignable)
        result.Description = "Test Claim Assignable History Event"
        return result
    }

    override function shouldCascadeAssignment(parent : Assignable,
        defaultOwnerUserId : Key,
        defaultGroupId : Key) : boolean {
        return true
    }
}

```

Studio indicates that your newly created class contains a serious error because there is no method definition for `AssignmentPopupUtil.pushAssignTestClaimAssignables`. You need to provide at least a skeleton body of this method. The next step resolves this issue.

3. Open `gw.assignment.AssignmentPopupUtil` for editing and add the following code.

```

static function pushAssignTestClaimAssignables(testclaimassignables : TestClaimAssignable_Ext[]) {
    // Implementation to do
}

```

#### What to do next

- “Step 6: Add the necessary permission for the extension entity type” on page 134

## Step 6: Add the necessary permission for the extension entity type

Each extension entity that has a foreign key to another entity must set up and handle the permissions for users to assign instances of this entity type.

#### Before you begin

- “Step 5: Create delegate class `TestClaimAssignableMethodsImpl`” on page 133

#### About this task

The `TestClaimAssignable_Ext` extension entity type has foreign keys to the entity types `Claim` and `Activity`. You must do the following tasks so that the application can ensure that only authorized users assign `TestClaimAssignable_Ext` instances.

- Define a permission of `permKey="own"` in `security-config.xml`.

- Add this permission to the `SystemPermissionType` typelist.
- Assign this permission to a user role.
- Assign this role to any user that needs to assign the new assignable entity.

If you do not perform these steps correctly, you see the following error if you attempt to assign your assignable entity:

```
com.guidewire.pl.system.exception.InsufficientPermissionException: No user defined
at com.guidewire.pl.system.security.PermCheck.setAndCheckAssign(PermCheck.java:81)
at com.guidewire.pl.domain.assignment.impl.AssignableImpl.assign(AssignableImpl.java:236)
...
```

### Procedure

1. Navigate to **configuration > config > Security**.

2. In this folder, double-click `security-config.xml`.

The file `security-config.xml` opens in an editor tab. You use this file to define system permissions for the data model entities.

3. Inside the `SecurityConfig` element, add the following code.

This code defines a permission that you can assign to a user to enable that person to assign an entity instance of types `TestClaimAssignable_Ext` and `UserAssignableEntity_Ext`.

```
<!-- Permission key needed for assigning extension entity in testing-->
<StaticHandler entity="UserAssignableEntity_Ext" permKey="own">
  <SystemPermType code="ownsensclaim"/>
  <SystemPermType code="InternalTools"/>
  <SystemPermType code="ext_entity_perms"/>
</StaticHandler>

<StaticHandler entity="TestClaimAssignable_Ext" permKey="own">
  <SystemPermType code="ownsensclaim"/>
  <SystemPermType code="InternalTools"/>
  <SystemPermType code="ext_entity_perms"/>
</StaticHandler>
```

4. In Guidewire Studio, open `SystemPermissionType.ttx` for editing.

You need to add the new system permission to the `SystemPermissionType` typelist.

- a) Press **Ctrl-N** and start entering the typekey name.

Studio displays a number of matches because files with the name `SystemPermissionType` already exist.

- b) Double-click the `DynamicAssignmentState.etx` file.

The typelist extension opens in the editor.

5. Add the following typecode.

| Field | Entry                                 |
|-------|---------------------------------------|
| code  | ext_entity_perms                      |
| name  | Own ext entity                        |
| desc  | Permission to access extension entity |

6. To see your new system permission in the application, stop and restart the application server.

- a) Stop the server, if it is running.
- b) Start the server from within Studio.
- c) Correct any problems that occur and repeat the previous steps until the server starts without errors.

7. Log into the application using an administrative account.

This example assumes the use of Guidewire ClaimCenter. You must be logged in as an administrator to be able to access the **Administration** tab. Additionally, that administrator account must have a role with the **Manage Roles** and **View Roles** permissions to be able to view and edit the Roles screen.

8. If you have not already done so, load the demonstration data into the database. The next step requires this data.
9. Click the **Administration** tab and navigate to **Users & Security > Roles**.
10. Assign the **Own ext** entity system permission to a specific user role such as *Adjuster*.
11. Assign that role with the necessary permission to a specific user such as *aapplegate*.

#### What to do next

- “Step 7: Test assignment of the assignable entity type” on page 136

## Step 7: Test assignment of the assignable entity type

After you create your assignable entity that uses a foreign key, you must test your work to verify that you have performed the previous steps correctly.

#### Before you begin

- “Step 6: Add the necessary permission for the extension entity type” on page 134

#### Procedure

1. In Studio, click **Tools > Gosu Scratchpad** to open the Studio Gosu Scratchpad.
2. Enter the following code in the **Gosu Scratchpad** tab.

```
var _users = Query. make(User).join(User# Credential).compare(Credential# UserName, Equals, "aapplegate")
var _usr = _users.select(). AtMostOneRow
var _group = _usr. GroupUsers[ 0]. Group
//Print out the members of the chosen group
print("\nChosen group " + _group. DisplayName + " contains the following members:")
_group. Users.each{\name -> print("\t" + name. User. DisplayName))

var _claim = gw.api.database.Query. make(Claim).compare(Claim# ClaimNumber, Equals,
"235-53-365870").select(). AtMostOneRow
print( "Claim number = " + _claim. ClaimNumber)
Transaction.runWithNewBundle(\bundle -> {
  for (i in 1.. 5) {
    var testAssignable = new TestClaimAssignable_Ext()
    testAssignable = bundle.add(testAssignable)
    testAssignable.Claim = _claim
    testAssignable.assign(_group, _usr)
    print("Pass " + i + ": Assigning claim (with claim number = " + testAssignable. Claim +
      ") to the test assignment object ")
    print ("\tThe assigned user is " + testAssignable. AssignedUser)
    print ("\tThe assigned group is " + testAssignable. AssignedGroup)

    if (testAssignable.assignUserByRoundRobin( false, _group) != true) {
      print("Cannot perform round-robin assignment.")
      break
    }
    print("\tFor assignUserByRoundRobin, AssignedUser = " +
      testAssignable.AssignedUser)
  }
}, "su")
```

3. Click **Run**.  
Output similar to the following lines appears in the **Console** tab.

```
Chosen group Auto1 - TeamA contains the following members:
Elizabeth Lee
Charles Shaw
Heidi Johnson
Cathy Clark
Felicity Wagner
Gary Wang
Dan Henson
Eugene Nyugen
Betty Baker
```



```
Chris Craft
Andy Applegate
Sue Smith
Claim number = 235-53-365870
Pass 1: Assigning claim (with claim number = 235-53-365870) to the test assignment
object
  The assigned user is Andy Applegate
  The assigned group is Auto1 - TeamA
  For assignUserByRoundRobin, AssignedUser = Dan Henson
Pass 2: Assigning claim (with claim number = 235-53-365870) to the test assignment
object
  The assigned user is Andy Applegate
  The assigned group is Auto1 - TeamA
  For assignUserByRoundRobin, AssignedUser = Eugene Nyugen
Pass 3: Assigning claim (with claim number = 235-53-365870) to the test assignment
object
  The assigned user is Andy Applegate
  The assigned group is Auto1 - TeamA
  For assignUserByRoundRobin, AssignedUser = Felicity Wagner
Pass 4: Assigning claim (with claim number = 235-53-365870) to the test assignment
object
  The assigned user is Andy Applegate
  The assigned group is Auto1 - TeamA
  For assignUserByRoundRobin, AssignedUser = Gary Wang
Pass 5: Assigning claim (with claim number = 235-53-365870) to the test assignment
object
  The assigned user is Andy Applegate
  The assigned group is Auto1 - TeamA
  For assignUserByRoundRobin, AssignedUser = Heidi Johnson
```



# Data types

This topic describes the Guidewire data types, what they are, how to customize a data type, and how to create a new data type.

In the Guidewire data model, a *data type* is an augmentation of an object property, along three axes:

| Axis         | Description                                                                                                                                                                                                                                                                 |
|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Constraint   | A data type can restrict the range of allowable values. For example, a <code>String</code> data type can restrict values to a maximum character limit.                                                                                                                      |
| Persistence  | A data type can specify how a value is stored in the database and in the object layer. For example, one <code>String</code> data type can store values as CLOB (Character Large Object) objects. Another <code>String</code> data type can store values as VARCHAR objects. |
| Presentation | A data type can specify how the user interface treats a value. For example, a <code>String</code> data type can specify an input mask to use in assisting the user with data entry.                                                                                         |

Guidewire stores the definitions for the base configuration data types in `*.dti` files in the `datatypes` directory. Each file corresponds to a separate data type, which the file name specifies.

Every data type has an associated Java or Gosu type (defined in the `valueType` attribute). For example, the associated type for the `datetime` data type is `java.util.Date`. Thus, you see the following XML code in the `datetime.dti` file.

```
<DataTypeDef xmlns="http://guidewire.com/datatype"
  type="com.guidewire.pl.metadata.datatype2.impl.DateTimeDataTypeDef"
  valueType="java.util.Date">
  ...
</DataTypeDef>
```

In a similar manner, the `decimal` data type has an associated type of `java.math.BigDecimal`.

```
<DataTypeDef xmlns="http://guidewire.com/datatype"
  type="com.guidewire.pl.metadata.datatype2.impl.DecimalDataTypeDef"
  valueType="java.math.BigDecimal">
  ...
</DataTypeDef>
```

## Related concepts

“Money data types”

## Working with data types

In working with data types, you can do the following:

| Operation                          | Description                                                                                                                                                                                                                                                                                                                                               |
|------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Customize an existing data type    | Modify the data type definition in file <code>datatypes.xml</code> , which you access through Studio. You can modify only a select subset of the base configuration data types.<br>See “Customizing base configuration data types” on page 143.                                                                                                           |
| Create a new data type             | Create a <code>.dti</code> definition file and place it in <b>modules &gt; configuration &gt; config &gt; datatypes</b> . You also need to create Gosu code to manage the data type.<br>See “Define a new data type” on page 146.                                                                                                                         |
| Override the data type on a column | Override the parameterization of the data type on individual columns (fields) on an entity. For example, you can set the maximum size of a column in the base data model by extending the entity and setting the <code>size</code> parameter on a <code>&lt;columnParam&gt;</code> element:<br><pre>&lt;columnParam   name="size"   value="50"/&gt;</pre> |

## Using data types

You can use any of the data types for data fields (except for those that are internal). This includes data types that are part of the base configuration or data types that you create yourself. If you add a new column to an entity or create a new entity, then you can use any data type that you want for that column. You do this by setting the `type` attribute on the column. For example:

```
<extension entityName="Claim">
  <column name="NewCompanyName" type="CompanyName" nullof="true" desc="Name for the new company."/>
</extension>
```

If you add too many large fields to any one table, you can easily reach the maximum row size of a table. In particular, this is a problem if you add a large number of long text or `VARCHAR` fields. Have your company database administrator (DBA) determine the maximum row size and increase the page size, if needed.

## Guidewire-reserved data types

Guidewire reserves the right to use the following data types exclusively. Guidewire does not support the use of these data types except for its own internal purposes. Do not attempt to create or extend an entity using one of the following data types:

- `foreignkey`
- `key`
- `typekey`
- `typelistkey`

## Database data types

Guidewire bases its base configuration data types on the following database data types:

- `BIT`
- `BLOB`
- `CLOB`
- `DECIMAL`
- `INTEGER`
- `TIMESTAMP`

- VARCHAR

## Data types and database vendors

It is possible to see both VARCHAR and varchar in the Guidewire documentation. This usage has the following meanings.

### Standard data types

In general, the standard database data types such as VARCHAR and CLOB (Character Large Object) use all uppercase characters. For the supported database types:

- H2 uses uppercase data type names.
- PostgreSQL uses lowercase data type names.

To view the entire set of database data types, consult the database vendor's documentation.

### Guidewire data types

Guidewire data types generally use all lower case characters, varchar and text, for example. You can determine the set of Guidewire data types by viewing the names of the data type metadata definition files (\*.dti) in **config > datatypes**.

## Defining a data type for a property

Guidewire associates data types with object properties using the following annotation:

```
gw.datatype.annotation.DataType
```

The annotation requires you to provide the name of the data type along with any parameters that you want to supply to the data type.

- You associate a data type with a metadata property by specifying the type attribute on the <column> element.
- You specify any parameters for the data type with <columnParam> elements, which are children of the <column> element.

Each data type has a value type. You can associate a data type only with a property that has a feature type that matches the data type of the value type. For example, you can only associate a String data type with String properties.

**Note:** InsuranceSuite does not enforce this restriction at compile time. (However, the application does check for any exception to this restriction at application server start up.) Guidewire permits annotations on any allowed feature, as long as you supply the parameters that the annotation requires. Therefore, you need to be aware of this restriction and enforce it yourself.

## The data types configuration file

---

**IMPORTANT:** If you make changes to the datatypes.xml file, you must increment the version number in extensions.properties.

---

InsuranceSuite lets you modify certain attributes on a subset of the base configuration data types by using the datatypes.xml configuration file. You can access this file in Guidewire Studio™ from **configuration > config > fieldvalidators**. You can modify the values of certain attributes in this file to customize how these data types work in the application.

This datatypes.xml file contains the following elements:

| XML element   | Description                                                                                                                                                                    |
|---------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <DataTypes>   | Top XML element for the datatypes.xml file.                                                                                                                                    |
| <...DataType> | Subelement that defines a specific <i>customizable data type</i> (for example, PhoneDataType, YearDataType, MoneyDataType) and assigns one or more default values to each one. |

**WARNING:** Modify the datatypes.xml file with caution. If you modify the file incorrectly, you can invalidate your installation.

This topic includes the following:

- “<...DataType>” on page 142
- “Deploying modifications to the data types configuration file” on page 142

## <...DataType>

The <...DataType> element is the basic element of the datatypes.xml file. It assigns default values to base configuration data types that you can customize. This element starts with the specific data type name. For example, the element for the PercentageDec data type in the datatypes.xml file is <PercentageDecDataType>.

The <...DataType> element has the following attributes:

| Attribute | Description                                                                                                                                                                                                                                                                                                                                                           |
|-----------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| length    | Assigns the maximum character length of the data type.                                                                                                                                                                                                                                                                                                                |
| validator | Binds the data type to a given validator definition. It must match the name attribute of the validator definition.                                                                                                                                                                                                                                                    |
| precision | Used for DECIMAL types only.                                                                                                                                                                                                                                                                                                                                          |
| scale     | <ul style="list-style-type: none"> <li>• precision is the total number of digits in the number.</li> <li>• scale is the number of digits to the right of the decimal point. The default value is 2.</li> </ul> <p>The value of scale must be less than the value of precision.</p> <p>For more information, see “The Precision and Scale attributes” on page 143.</p> |
| appscale  | Optional attribute for use with money data types.                                                                                                                                                                                                                                                                                                                     |

## Deploying modifications to the data types configuration file

If you change the datatypes.xml file, then you need to deploy those changes to the application server. Most modifications to the datatypes.xml file take effect the next time the server reboots.

- The application reloads the validator attribute for data type definitions upon server reboot. This is so that you can rebind different validators to data types.
- The application does not reload other data type attributes such as length, precision, and scale. This is because the application applies these attributes only during the initial server boot. (It uses them during table creation in the database.) The application ignores any changes to these attributes unless something triggers a database upgrade. For example, if you modify a base entity, then the application triggers a database upgrade at the next server restart.

## Recommendations for modifying data types

Guidewire recommends the following:

- Make modifications to the data types before creating the application database for the first time.
- Make modifications to the data types before performing a database upgrade that creates a new extension column.
- Use the defined text data types (for example, shorttext, mediumtext, and longtext) instead of varchar. You may choose to use varchar if a fixed width is known from or required by an external system.

The application looks at the data type definitions only at the time it creates a database column. Thus, it ignores any changes after that point. However, any differences between the type definition and the actual database column can cause upgrade errors or failure warnings. Therefore, Guidewire recommends that you exercise extreme caution in making changes to type definitions.

## Customizing base configuration data types

You can customize the behavior of the data types listed in `datatypes.xml`. To see exactly what you can customize for each data type, see “List of customizable data types” on page 144. In general, though, you can customize some or all of the following attributes on a listed data type (depending on the data type):

- length
- precision
- scale
- validator

For information on configuring national field validation, see the *Globalization Guide*.

### Related concepts

“Field validation” on page 153

### Related reference

“List of customizable data types” on page 144

## The Length attribute

Data types based on the VARCHAR data type have a `length` attribute that you can customize. This attribute sets the maximum allowable character length for the field (column).

## The Precision and Scale attributes

Data types based on the DECIMAL data type have `precision` and `scale` attributes that you can customize. These attributes determine the size of the decimal. The `precision` value sets the total number of digits in the number and the `scale` value is the number of digits to the right of the decimal point.

In some cases, there are special requirements for these attributes in working with monetary amounts. For more information, see the *Globalization Guide*.

## The Validator attribute

Most data types have a `validator` attribute that you can customize. This attribute binds the data type to a given validator definition. For example, `PhoneDataType` (defined in `datatypes.xml`) binds to the Phone validator by its `validator` attribute. This matches the name attribute of a `<ValidatorDef>` definition in file `fieldvalidators.xml`.

```
//File datatypes.xml
<DataTypes xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="../../platform/pl/xsd/datatypes.xsd">
  ...
  <PhoneDataType length="30" validator="Phone"/>
  ...
</DataTypes>

//File fieldvalidators.xml
<FieldValidators>
  ...
  <ValidatorDef description="Validator.Phone" input-mask="###-###-#### x####" name="Phone"
    value="[0-9]{3}-[0-9]{3}-[0-9]{4}( x[0-9]{0,4})?">
  ...
</FieldValidators>
```

## List of customizable data types

The following table summarizes the list of the data types that you can customize. The application defines these data types in `datatypes.xml`. If a data type does not exist in `datatypes.xml`, then you cannot customize its attributes.

The application builds all of its data types on top of the base database data types of CLOB, TIMESTAMP, DECIMAL, INTEGER, VARCHAR, BIT, and BLOB.

**Note:** Only decimal numbers use the precision and scale attributes. The precision attribute defines the total number of digits in the number. The scale attribute defines the number of digits to the right of the decimal separator. Therefore, precision must be greater than or equal to scale.

| Guidewire data type      | Built on | Customizable attributes |
|--------------------------|----------|-------------------------|
| ABContactMatchSetKey     | VARCHAR  | length                  |
| Account                  | VARCHAR  | length, validator       |
| AddressLine              | VARCHAR  | length, validator       |
| ClaimNumber              | VARCHAR  | length, validator       |
| CompanyName              | VARCHAR  | length, validator       |
| ContactIdentifier        | VARCHAR  | length, validator       |
| CreditCardNumber         | VARCHAR  | length, validator       |
| DaysWorkedWeek           | DECIMAL  | precision, validator    |
| DriverLicense            | VARCHAR  | length, validator       |
| DunAndBradstreetNumber   | VARCHAR  | length, validator       |
| EmploymentClassification | VARCHAR  | length, validator       |
| ExchangeRate             | DECIMAL  | precision, validator    |
| Exmod                    | DECIMAL  | precision, validator    |
| FirstName                | VARCHAR  | length, validator       |
| HoursWorkedDay           | DECIMAL  | precision, validator    |
| LastName                 | VARCHAR  | length, validator       |
| MediumText               | VARCHAR  | length                  |
| PercentageDec            | DECIMAL  | precision               |
| Phone                    | VARCHAR  | length, validator       |
| PolicyNumber             | VARCHAR  | length, validator       |
| PostalCode               | VARCHAR  | length, validator       |
| ProrationFactor          | DECIMAL  | precision, validator    |
| Rate                     | DECIMAL  | precision, validator    |
| RatingLineBasisAmount    | DECIMAL  | precision, validator    |
| Risk                     | DECIMAL  | precision, validator    |
| Speed                    | INTEGER  | validator               |
| SSN                      | VARCHAR  | length, validator       |
| VIN                      | VARCHAR  | length, validator       |



| Guidewire data type | Built on | Customizable attributes |
|---------------------|----------|-------------------------|
| Year                | INTEGER  | validator               |

## The percentage decimal data type

Guidewire builds the PercentageDec data type on top of the DECIMAL (3,0) data type. Only use decimal values from 0 to 100 inclusive.

## The Money data type

ClaimCenter uses the Money data type as the basis for the currencyamount column type in the data model.

PolicyCenter and BillingCenter use the Money data type as the basis for the <monetaryamount> subelement in the data model. The <monetaryamount> subelement is a compound field type, with a Money data type as one component and a typekey to the Currency typelist as the other component.

## The data type API

The classes in `gw.datatype` form the core of the data type API. Most of the time, you do not need to use data types directly, as Guidewire uses these internally in the system. However, there can be cases in which you need to access a data type, typically to determine the constraints information.

## Retrieving the data type for a property

To retrieve the data type for a property, you could look up the annotation on the property. You could then look up the data type reflectively, using the name and parameters properties of the annotation. However, this is a cumbersome process. As a convenience, use the following method instead:

```
gw.datatype.DataTypes.get(gw.lang.reflect.IAnnotatedFeatureInfo)
```

For example:

```
var property = Claim.Type.TypeInfo.getProperty("ClaimNumber")
var claimNumberDataType = DataTypes.get(property)
```

The `gw.datatype.DataTypes.get(gw.lang.reflect.IAnnotatedFeatureInfo)` method also has provides some performance optimizations. Therefore, Guidewire recommends that you use this method rather than looking up the annotation directly from the property.

## Retrieving a particular data type in Gosu

If you need an instance of a particular data type, use the corresponding method on `gw.datatype.DataTypes`. A static method exists on this type for each data type in the system. Some data types have two methods:

- One method that takes all parameters
- One method that takes only the required parameters

For example:

```
var varcharDataType = DataTypes.varchar(10)
var encryptedVarcharDataType = DataTypes.varchar(10,
    /* validator */ null,
    /* logicalSize */ null,
    /* encryption */ true,
    /* trimwhitespace */ null)
```

## Retrieving a data type reflectively

In rare cases, you may need to look up a data type reflectively. To do this, you need the name of the data type, and a map containing the parameters for the data type. For example:

```
var varcharDataType = DataTypes.get("varchar", { "size" -> "10" })
```

## Using the IDatatype methods

After you have a data type, you can access its various aspects using one of the `asXXXXDataType` methods, which are:

- `asConstrainedDataType()` : `IConstrainedDataType`
- `asPersistentDataType()` : `IPersistentDataType`
- `asPresentableDataType()` : `IPresentableDataType`

For example, suppose that you want to determine the maximum length of a property:

```
var claim : Claim = ...
var claimNumberProperty = Claim.Type.TypeInfo.getProperty("ClaimNumber")
var claimNumberDataType = DataTypes.get(claimNumberProperty)
var maxLength = claimNumberDataType.asConstrainedDataType().getLength(claim, claimNumberProperty)
```

It may seem odd that the `getLength(java.lang.Object, gw.lang.reflect.IPropertyInfo)` method (in this example) takes the claim and the claim number property. The reason for this is that the constraint and presentation aspects of data types are dynamic, meaning that they are based on context.

Many of the methods on `gw.datatype.IConstrainedDataType` and `gw.datatype.IPresentableDataType` take a context object, representing the owner of the property with the data type, along with the property in question. This allows the implementation to provide different behavior, based on the context. If you do not have the context object or property, then you can pass `null` for either of these arguments.

If you implement a data type, then you must handle the case in which the context is unknown.

## Define a new data type

### About this task

To define a new data type, create a `.dti` file named `xxx.dti`, with `xxx` as the name of the new data type. Data type names have no minimum or maximum allowable length. In this case, create a file named `age.dti`. To do this:

### Procedure

1. Create a `.dti` file (data type declaration file) to register the data type within the application. In Guidewire Studio, do the following:
  - a) In the **Project** tool window, navigate to **configuration > config > datatypes**.
  - b) Right-click **datatypes**, and then click **New > File**.
  - c) Enter the name of the data type, followed by the `.dti` extension.  
For example, to create a data type named `Age`, type `age.dti`.

You must enter definitions for the following items for the data type. If necessary, view other samples of datatype definition files to determine what you need to enter.

- Name
- Value type
- Parameters
- Implementation type

2. Create a data type definition class that implements the `gw.datatype.def.IDataTypeDef` interface. This class must include writable property definitions that correspond to each parameter that the data type accepts.
3. Create data type handler classes for each of the three aspects of the data type (constraints, persistence, and presentation). These classes must implement the following interfaces:
  - `gw.datatype.handler.IDataTypeConstraintsHandler`
  - `gw.datatype.handler.IDataTypePersistenceHandler`
  - `gw.datatype.handler.IDataTypePresentationHandler`

Guidewire provides a number of implementations of these three interfaces for the standard data types. For example, you can create your own CLOB-based data types by defining a data type that uses the `ClobPersistenceHandler` class. To access the handler interface implementations or to view a complete list, enter the following within Gosu code:

```
gw.datatypes.impl.*
```

### Results

After you create the data type, you will want to use the data type in some useful way.

### Example

For example, you can create an entity property that uses that data type and then expose that property as a field within the application.

### What to do next

For a discussion of constraints, persistence, and presentation as it relates to data types, see “Data types” on page 139.

## Defining a new tax identification number data type

The following examples illustrates the steps involved in defining a new data type and using it. The example defines a new data type for *Tax Identification Number* objects, called `TaxID`. The data type has one required property, the name of the property on the context object. This property, `countryProperty`, identifies which country is in context for validating the data.

This example contains the following steps:

- “Step 1: Register the Data Type”
- “Step 2: Implement the `IDataTypeDef` Interface”
- “Step 3: Implement the Data Type Aspect Handlers”

## Step 1: Register the data type

### About this task

To register a new data type, create a file named `xxx.dti`, with `xxx` as the name of the new data type. In this case, create a file named `TaxID.dti`. To do this:

### Procedure

1. In Guidewire Studio, in the **Project** tool window, navigate to **configuration > config > datatypes**.
2. Right-click **datatypes**, and then click **New > File**.
3. Enter `TaxID.dti` as the file name. This action creates an empty data type file and places it in the **datatypes** folder.

#### 4. Enter the following text in the file:

```
<?xml version="1.0"?>
<DataTypeDef xmlns="http://guidewire.com/datatype" type="gw.newdatatypes.TaxIDDataTypeDef"
  valueType="java.lang.String">
  <ParameterDef name="countryProperty" desc="The name of a property on the owning entity,
    whose value contains the country with which to validate and format values."
    required="true" type="java.lang.String"/>
</DataTypeDef>
```

The root element of `TaxID.dti` is `<DataTypeDef>` and the namespace is `http://guidewire.com/datatype`.

This example defines the following:

|                     |                                  |
|---------------------|----------------------------------|
| data type name      | TaxID                            |
| value type          | String                           |
| parameter           | contactType                      |
| implementation type | gw.newdatatypes.TaxIDDataTypeDef |

#### What to do next

After completing this task, complete “Step 2: Implement the `IDataTypeDef` Interface” on page 148.

#### See also

- For details on the attributes and elements relevant to the data type definition, see “The application data model” on page 19.

## Step 2: Implement the `IDataTypeDef` Interface

#### Before you begin

Before beginning this task, complete “Step 1: Register the data type” on page 147.

#### About this task

The implementation class that you create to handle the `TaxID` data type must do the following:

- It must implement the `gw.datatype.def.IDataTypeDef` interface.
- It must have a no-argument constructor.
- It must have a property for each of the data type parameters.

For example, suppose that you have a new data type that has a `String` parameter named `someParameter`. The implementation class (specified in the `type` attribute) must define a writable property named `someParameter`, so that the data type factory can pass the argument values to the implementation. The implementation can then use the parameters in the implementation of the various handlers, which are:

- `gw.datatype.handler.IDataTypeConstraintsHandler`
- `gw.datatype.handler.IDataTypePersistenceHandler`
- `gw.datatype.handler.IDataTypePresentationHandler`

#### Class `TaxIDDataTypeDef`

For our example data type, the `gw.newdatatypes.TaxIDDataTypeDef` class looks similar to the following. To create this file, first create the package, then the class file, in the Studio **Classes** folder.

```
package gw.newdatatypes

uses gw.datatype.def.IDataTypeDef
uses gw.datatype.handler.IDataTypeConstraintsHandler
uses gw.datatype.handler.IDataTypePresentationHandler
uses gw.datatype.handler.IDataTypePersistenceHandler
uses gw.lang.reflect.IPropertyInfo
```

```

uses gw.datatype.handler.IDataValueHandler
uses gw.datatype.def.IDataDefValidationErrors
uses gw.datatype.impl.VarcharPersistenceHandler
uses gw.datatype.impl.SimpleValueHandler

class TaxIDDataTypeDef implements IDataTypeDef {
    private var _countryProperty : String as CountryProperty

    override property get ConstraintsHandler() : IDataTypeConstraintsHandler {
        return new TaxIDConstraintsHandler(CountryProperty)
    }

    override property get PersistenceHandler() : IDataTypePersistenceHandler {
        return new VarcharPersistenceHandler(/* encrypted */ false,
                                              /* trimWhitespace */ true,
                                              /* size */ 30)
    }

    override property get PresentationHandler() : IDataTypePresentationHandler {
        return new TaxIDPresentationHandler(CountryProperty)
    }

    override property get ValueHandler() : IDataValueHandler {
        return new SimpleValueHandler(String)
    }

    override function validate(prop : IPropertyInfo, errors : IDataTypeDefValidationErrors) {
        // Check that the CountryProperty names an actual property on the owning type, and that
        // the type of the property is typekey.Country.
        var countryProp = prop.OwnersType.TypeInfo.getProperty(CountryProperty)

        if (countryProp == null) {
            errors.addError("Property \" + CountryProperty + "\" does not exist on type " +
                            prop.OwnersType)
        } else if (not typekey.Country.Type.isAssignableFrom(countryProp.Type)) {
            errors.addError("Property " + countryProp + " does not resolve to a " + typekey.Country)
        }
    }
}

```

Note that the class defines a property named `CountryProperty`, which the system calls to pass the `countryProperty` parameter. Also notice how the implementation reads the value of `CountryProperty` as it constructs its constraints and presentation handlers. Guidewire guarantees to fill the implementation parameters before calling the handlers.

In the example code, the class refers to constraints and presentation handlers created specifically for this data type. However, it also reuses a Guidewire-provided persistence handler, the `VarcharPersistenceHandler`. You do not usually need to create your own persistence handler, as Guidewire defines persistence handlers for all the basic database column types.

#### What to do next

After completing this task, complete “Step 3: Implement the data type aspect handlers” on page 149.

## Step 3: Implement the data type aspect handlers

#### Before you begin

Before beginning this task, complete “Step 2: Implement the `IDataTypeDef` Interface” on page 148.

#### About this task

As you define a new data type, it is possible (actually likely) that you need to define one or more handlers for the data type. These handler interfaces are different than the Data Type API interfaces. For example, clients that use the Data Type API use the following:

```
gw.datatype.IConstrainedDataType
```

However, if you define a new data type, you must implement the following:

```
gw.datatype.handler.IDataTypeConstraintsHandler
```

This separation of interfaces allows the definition of a caller-friendly interface for data type clients and a implementation-friendly interface for data type designers.

The example data type defines a handler for both constraints and presentation.

### Class `TaxIDConstraintsHandler`

This class looks similar to the following:

```
package gw.newdatatypes

uses gw.datatype.handler.IStringConstraintsHandler
uses gw.lang.reflect.IPropertyInfo
uses java.lang.Iterable
uses java.lang.Integer
uses java.lang.CharSequence
uses gw.datatype.DataTypeException

class TaxIDConstraintsHandler implements IStringConstraintsHandler {
    var _countryProperty : String

    construct(countryProperty : String) {
        _countryProperty = countryProperty
    }

    override function validateValue(ctx : Object, prop : IPropertyInfo, value : Object) {
        var country = getCountry(ctx)

        switch (country) {
            case "US": validateUSTaxID(ctx, prop, value as java.lang.String)
                break
            // other countries ...
        }
    }

    override function validateUserInput(ctx : Object, prop : IPropertyInfo, strValue : String) {
        validateValue(ctx, prop, strValue)
    }

    override function getConsistencyCheckerPredicates(columnName : String) : Iterable<CharSequence> {
        return {}
    }

    override function getLoaderValidationPredicates(columnName : String) : Iterable<CharSequence> {
        return {}
    }

    override function getLength(ctx : Object, prop : IPropertyInfo) : Integer {
        var country = getCountry(ctx)

        switch (country) {
            case "US": return ctx typeis Person ? 11 : 10
            // other countries ...
        }

        return null
    }

    private function getCountry(ctx : Object) : Country {
        return ctx[_countryProperty] as Country
    }

    private function validateUSTaxID(ctx : Object, prop : IPropertyInfo, value : String) {
        var pattern = ctx typeis Person ? "\\d{3}-\\d{2}-\\d{4}" : "\\d{2}-\\d{7}"
        if (not value.matches(pattern)) {
            throw new DataTypeException("${value} does not match required pattern ${pattern}", prop,
                "Validation.TaxID", { value })
        }
    }
}
}
```

### Class `TaxIDPresentationHandler`

This class looks similar to the following:

```
package gw.newdatatypes

uses gw.lang.reflect.IPropertyInfo
uses gw.datatype.handler.IStringPresentationHandler

class TaxIDPresentationHandler implements IStringPresentationHandler {
```

```
private var _countryProperty : String
construct(countryProperty : String) {
    _countryProperty = countryProperty
}

function getEditorValue(ctx : Object, prop : IPropertyInfo) : Object {
    return null
}

override function getDisplayFormat(ctx : Object, prop : IPropertyInfo ) : String {
    return null
}

override function getInputMask(ctx : Object, prop : IPropertyInfo) : String {
    switch (getCountry(ctx)) {
        case "US": return ctx.typeis Person ? "###-##-####" : "##-#####"
        // other countries ...
    }

    return null
}

override function getPlaceholderChar(ctx : Object, prop : IPropertyInfo) : String {
    return null
}

private function getCountry(ctx : Object) : Country {
    return ctx[_countryProperty] as Country
}
}
```

Notice how each of these handlers makes use of the context object in order to determine the type of input mask and validation string to use.





# Field validation

*Field validators* handle simple validation for a single field. A validator definition defines a *regular expression*, which a data field must match to be valid. It can also define an optional *input mask* that provides a visual indication to the user of the data to enter in the field.

Each field in the application has a default validation based on its data type. For example, integer fields can contain only numbers. However, it is possible to use a field validator definition to override this default validation.

- You can apply field validators to simple data types, but not to typelists.
- You can modify field validators for existing fields, or create new validators for new fields.

For complex validation between fields, use validation-specific Gosu code instead of simple field validators.

## Specifying the properties of a specific field

Field validators specify only the validation properties for a general kind of input (for example, any postal code). They do not specify the properties of a specific field in a particular data view. Instead, detail views and editable list views include additional validation attributes in their configuration files.

## Specifying field validators on a delegate entity

Apply any field validators for elements existing on a delegate entity to the delegate entity. Do not apply any field validators to the entities that inherit the elements from the delegate. Applying a field validator to an element on the delegate entity ensures that the application applies the field validator uniformly to that data element in whatever code utilizes the delegate.

# Field validator definitions

InsuranceSuite stores field validator definitions in the `fieldvalidators.xml` file, located in Guidewire Studio under **configuration > config > fieldvalidators**. The `fieldvalidators.xml` file contains a list of validator specifications for individual fields in the application. The `fieldvalidators.xml` file contains the following sections:

| XML element                          | Description                                                                                                          | More information                                     |
|--------------------------------------|----------------------------------------------------------------------------------------------------------------------|------------------------------------------------------|
| <code>&lt;FieldValidators&gt;</code> | Top XML element for the <code>fieldvalidators.xml</code> file.                                                       | " <code>&lt;FieldValidators&gt;</code> " on page 154 |
| <code>&lt;ValidatorDef&gt;</code>    | Subelement that defines all of the validators. Each validator must have a unique name by which you can reference it. | " <code>&lt;ValidatorDef&gt;</code> " on page 154    |

Using the `fieldvalidators.xml` file, you can do the following:

- You can modify existing validators. For example, it is common for each insurer to represent claim numbers differently. You can define field validation to reflect these changes.
- You can add new validators for existing fields or custom extension fields.

The following XML example illustrates the structure of the `fieldvalidators.xml` file:

```
<FieldValidators>
  <ValidatorDef name="Email"
    description="Validator.Email"
    input-mask=""
    value=".+@.+" />
  <ValidatorDef name="SSN"
    description="Validator.SSN"
    input-mask="###-##-####"
    value="[0-9]{3}-[0-9]{2}-[0-9]{4}|[0-9]{2}-[0-9]{7}?"
    />
  ...
</FieldValidators>
```

In the previous example, each validator definition specifies a `value` and an `input-mask`. These attributes have different uses, as follows:

|            |                                                                                                                                                                                                                                                                                                               |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| value      | A value is a regular expression that the field value must match for the data to be valid. the application persists this value to the database, including any defined delimiters or characters other than the # character.                                                                                     |
| input-mask | An input-mask, which is optional, assists the user in entering valid data. The application displays the input mask when the field opens for editing. For example, a # character indicates that the user must enter a digit for this character. These characters disappear when the user starts to enter data. |

The input mask guides the user to enter to valid sequences for the regular expression defined in the `value` attribute. After the user enters a value, the application uses the regular expression to validate the field data as it sets the field on the object.

This topic includes the following:

- “<FieldValidators>” on page 154
- “<ValidatorDef>” on page 154

[Related reference](#)

- “<FieldValidators>” on page 154
- “<ValidatorDef>” on page 154

## <FieldValidators>

The `<FieldValidators>` element is the root element in the `fieldvalidators.xml` file. It contains the XML subelement `<ValidatorDef>`.

## <ValidatorDef>

The `<ValidatorDef>` subelement of `<FieldValidators>` is the beginning element for the definition of a validator. This element has the following attributes:

- description
- floor, ceiling
- input-mask
- name
- placeholder-char
- validation-level
- validation-type

- value

## description

The `description` attribute specifies the validation message to show to a user who enters bad input. The description refers to a key in the `display_LanguageCode.properties` file that contains the actual description text. The naming convention for this display key is `Validator.validator_name`.

In the display text in the properties file, `{0}` represents the name of the field in question. The application determines this at runtime dynamically.

## floor, ceiling

The `floor` and `ceiling` attributes are optional attributes that specify the minimum (`floor`) and maximum (`ceiling`) values for the field. For example, you can limit the range to 100-200 by setting `floor="100"` and `ceiling="200"`.

Use floor and ceiling range values for numeric fields only. For example, use the floor and ceiling attributes to define a Money validator:

```
<ValidatorDef description="Validator.Money"
  input-mask="" name="Money"
  ceiling="9999999999999999.99"
  floor="-9999999999999999.99"
  value="."/ >
```

## input-mask

The `input-mask` attribute is optional. It specifies a visual indication of the characters that the user can enter. The application displays the input mask temporarily to the user during data entry. When the user starts to enter text, the input mask is no longer visible.

The input mask definition consists of the `#` symbol and other characters:

- The `#` symbol represents any character the user can type.
- Any other character represents itself in a non-editable form. For example, in an input mask of `###-###-##`, the two hyphen characters are a non-editable part of the input field.
- Any empty input mask of `""` is the same as not having the attribute at all.
- A special case is a mask with fixed characters on the end. The application displays those characters outside of the text field. For example `####mph` appears as a field `####` with `mph` on the outside end of it.

## name

The `name` attribute specifies the name of the validator. A field definition uses this attribute to specify which validator applies to the field.

## placeholder-char

The `placeholder-char` attribute specifies a replacement value for the input mask display character, which defaults to a period (`.`). For example, use the `placeholder-char` attribute to display a dash character instead of the default period.

## validation-level

This attribute can be set to one of the following values:

| Attribute | Description                             |
|-----------|-----------------------------------------|
| none      | Disables the validator.                 |
| relaxed   | Ignores warnings or partial validation. |

| Attribute        | Description                                        |
|------------------|----------------------------------------------------|
| strict (default) | Treats partially validation or warnings as errors. |

## validation-type

Specifies whether to use a regular expression or Gosu class for the validation.

This attribute can be set to one of the following values:

| Attribute       | Description                                                                                                                                                                                                                                                                                                           |
|-----------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| gosu            | The value of the <ValidatorDef> is a Gosu class. The Gosu class must extend <code>FieldValidatorBase</code> and override the <code>validate</code> method. See <code>gw.api.validation.PhoneValidator</code> for an example. Ensure that any Gosu validators that you define are low-latency for performance reasons. |
| regex (default) | The value of the <ValidatorDef> defines a regular expression that the application uses to validate data entered into a field that uses the field validator.                                                                                                                                                           |

## value

The `value` attribute specifies the acceptable values for the field. It is in the form of a regular expression. The application does not persist this value (the regular expression definition) to the database.

Use regular expressions with `String` values only. Use floor and ceiling range values for numeric fields, `Money`. for example.

The application uses the Java `regex` package described in the following location for regular expression parsing:

<https://docs.oracle.com/javase/8/docs/api/java/util/regex/package-summary.html>

The following list describes some of the more useful items:

|     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|-----|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| ()  | <i>Parentheses</i> define the order in which the application evaluates an expression, just as with any parentheses.                                                                                                                                                                                                                                                                                                                                                                                     |
| []  | <i>Brackets</i> indicate acceptable values. For example: <ul style="list-style-type: none"> <li>[Mm] indicates the letters M or m.</li> <li>[0-9] indicates any value from 0 to 9.</li> <li>[0-9a-zA-Z] indicates any alphanumeric character.</li> </ul>                                                                                                                                                                                                                                                |
| { } | <i>Braces</i> indicate the number of characters. For example: <ul style="list-style-type: none"> <li>[0-9]{5} allows five positions containing any character (number) between 0 and 9.</li> <li>{x} repeats the preceding value x times. For example, [0-9]{3} indicates any 3-digit integer such as 031 or 909, but not 16.</li> <li>{x,y} indicates the preceding value can repeat between x and y times. For example, [abc]{1,3} allows values such as cab, b, or aa, but not rs or abca.</li> </ul> |
| ?   | A <i>question mark</i> indicates one or zero occurrences of the preceding value. For example, [0-9]x? allows 3x or 3 but not 3xx. ([Mm][Pp][Hh])? means mph, MpH, MPH, or nothing.                                                                                                                                                                                                                                                                                                                      |
| ()? | Values within parentheses followed by a question mark are optional. For example, (-[0-9]{4})? means that you can optionally have four more digits between 0 and 9 after a dash -.                                                                                                                                                                                                                                                                                                                       |
| *   | An <i>asterisk</i> means zero or more of the preceding value. For example, (abc)* means abc or abcab but not ab.                                                                                                                                                                                                                                                                                                                                                                                        |
| +   | A <i>plus sign</i> means one or more of the preceding value. For example, [0-9]+ means any number of integers between 0 and 9 (but none is not an option).                                                                                                                                                                                                                                                                                                                                              |
| .   | A <i>period</i> is a wildcard character. For example:                                                                                                                                                                                                                                                                                                                                                                                                                                                   |

- `.*` means anything.
- `.+` means anything but the empty string.
- `...` means any string with three characters.

## Modifying field validators

You configure field validation in Guidewire Studio by editing `fieldvalidators.xml` files in various locations in the `fieldvalidators` folder. In Studio, navigate in the **Project** tools window to **configuration > config > fieldvalidators**.

- You define global field validators once in the `fieldvalidators.xml` file located in the root of the `fieldvalidators` folder.
- You define national field validators in `fieldvalidators.xml` files located in country-specific packages in the `fieldvalidators` folder.

You can, for example:

- Create a new field validator. For example:

```
<!-- Create a new validator -->
<ValidatorDef name="ExampleValidator" value="[A-z]{1,5}" description="Validator.Example"
  input-mask="#####" />
```

- Modify attributes of an existing validator. For example:

```
<!-- Modify a validator definition. Adding a ValidatorDef element with the same name as one defined
  in the base fieldvalidators.xml file replaces the base validator. -->
<ValidatorDef name="Claim Number" value="[0-9]{3}-[0-9]{5}" description="Validator.ClaimNumber"
  input-mask="###-#####" />
```

- Define field validators for a specific country.

## Using <column-override> to modify field validation

You use the `<column-override>` element in an extension file to override attributes on a field validator or to add a field validator to a field that does not contain one.

This topic includes the following:

- “Adding a field validator to a field” on page 157
- “Changing the length of a text field” on page 158

### Adding a field validator to a field

#### About this task

To add a validator to an application field that currently does not have one, use a `<column-override>` element in the specific entity extension file. Use the following syntax:

```
<extension entityName="SomeEntity">
  <column-override name="SomeColumn">
    <columnParam name="validator" value="SomeCustomValidator"/>
  </column-override>
</extension>
```

Suppose that you want to create a validator for a Date of Birth field (`Person.DateOfBirth`). To create this validator, you need to perform the following steps in Studio.

#### Procedure

1. Create a `Person.etx` file if one does not exist and add the following to it.

```
<extension entityName="Person">
  <column-override name="DateOfBirth">
```

```
<columnParam name="validator" value="DateOfBirth"/>
</column-override>
</extension>
```

2. Add a validation definition for the DateOfBirth validator to fieldvalidators.xml.

For example:

```
<ValidatorDef description="Validator.DateOfBirth" ... name="DateOfBirth" .../>
```

In this case, you can potentially create different DateOfBirth validators in different country-specific fieldvalidators files.

## Changing the length of a text field

### About this task

You can use the <column-override> element to change the size (length) of the text that a user can enter into a text box or field. Guidewire makes a distinction between the size attribute and the logicalSize attribute.

- The size attribute is the length of the database column (if a VARCHAR column).
- The logicalSize attribute is the maximum length of the field that the application permits. It must not be greater than size attribute (if applicable).

In this case, you set the logicalSize parameter, not a size parameter. This parameter does not change the column length of the field in the database. You use the logicalSize parameter simply to set the field length in the user interface. For example:

```
<column-override name="EmailAddressHome">
  <columnParam name="logicalSize" value="42"/>
</column-override>
```

The use of the logicalSize parameter does not affect the actual length of the column in the database. It merely affects how many characters a user can enter into a text field.

# Working with typelists

A *typelist* represents a predefined set of possible values, with each separate value defined as a *typecode*. Typically, you experience a typelist as a drop-down list that presents the set of available choices. You define and manage typelists through Guidewire Studio.

For information on localizing typecodes, see the *Globalization Guide*.

## Related concepts

- “What is a typelist?” on page 159
- “Terms related to typelists” on page 160
- “Typelists and typecodes” on page 160
- “Typelist definition files” on page 161
- “Different kinds of typelists” on page 162
- “Working with typelists in Studio” on page 163
- “Typekey fields” on page 167
- “Removing or retiring a typekey” on page 170
- “Typelist filters” on page 171
- “Static filters” on page 171
- “Dynamic filters” on page 176
- “Typecode references in Gosu” on page 179
- “Mapping typecodes to external system codes” on page 179

## What is a typelist?

---

**IMPORTANT:** Ensure that you fully understand the dependencies between typelists and other application files before you modify a typelist. Incorrect changes to a typelist can cause damage to the data model.

---

InsuranceSuite applications display many fields in the interface as drop-down lists of possible values. The list of available values for a drop-down field is called a *typelist*. Typelists limit the acceptable values for many fields within the application. Thus, a typelist represents a predefined set of possible values, with each separate value defined as a *typecode*. Whenever there is a drop-down list in the user interface, it is usually a typelist.

For example, the ClaimCenter **Loss Details** page that you access as you enter claim information contains several different typelists. One of these is the **Loss Cause** typelist that provides the available values from which you can choose as you enter claim information.

Typelists are very common for coding fields on the root objects of an application. They are also common for status fields used for application logic. Some typelist usage examples from the *Data Dictionary* include:

- `Contact.Country` uses a simple list.
- In `ClaimCenter`, `Check.Status` uses a list with a simple static filter, since only a subset of all transaction statuses make sense in this context.
- In `ClaimCenter`, `Claim.LossCause` uses a list filtered by `LossType` (that is, choices for this loss cause depend on the value of the loss type).
- In `PolicyCenter`, `PersonalVehicle.BodyType` uses a list filtered by `VehicleType` (that is, choices for this body type depend on the value of the vehicle type).
- In `BillingCenter`, `Collateral.Compliance` uses a list with a simple static filter, since only a subset of all compliance statuses make sense in this context.
- In `BillingCenter`, `ActivityPattern.Category` uses a list filtered by `ActivityType` (that is, choices for this category depend on the value of the activity type).

Besides displaying the text describing the different options in a drop-down list, typelists also serve a very important role in integration. Guidewire recommends that you design your typelists so that you can map their typecodes (values) to the set of codes used in your legacy applications. For example, make sure that you code a claim in `ClaimCenter` to values that can be understood by other applications within your company. This is a very important step.

## Terms related to typelists

There are several terms related to customizing drop-down lists within the application. Since they sound quite similar, it is easy to confuse the meaning of each term. The following is a quick definition list for you to refer back to at any time for clarification purposes:

| Term       | Definition                                                                                                      |
|------------|-----------------------------------------------------------------------------------------------------------------|
| typelist   | A defined set of values that are usually shown in a drop-down list.                                             |
| typecode   | A specific value in a typelist.                                                                                 |
| typefilter | A typelist that contains a static (fixed) set of values.                                                        |
| keyfilter  | A typelist that dynamically filters another typelist.                                                           |
| typekey    | The identifier for a field in the data model that represents a direct value chosen from an associated typelist. |

### Related concepts

- “Different kinds of typelists” on page 162
- “Typelists and typecodes” on page 160
- “Static filters” on page 171
- “Dynamic filters” on page 176
- “Typekey fields” on page 167

## Typelists and typecodes

A *typelist* represents a predefined set of possible values, with each separate value defined as a *typecode*. If Guidewire defines a typelist as `final`, it is not possible to add or delete typecodes from the typelist.

### Typelist size recommendation

Limit the size of typelists to 2,000 typecodes. Because of Java's limit on the size of the class initialization method, the application generates nested classes to implement typelists that have more than 2,048 typecodes. These nested classes affect how you refer to individual typecodes.

The maximum supported number of typecodes in a typelist is 8,192.



**Note:** For a `SystemPermissionType` typelist, the upper limit on the number of typecodes is 7,000 rather than 8,192.

If you have more than 2,048 typecodes in a typelist, the application creates nested classes that can each contain an additional 2,048 typecodes. A new nested class is created if you create a new typecode and the currently used class contains 2,048 typecodes. To keep the number of nested classes in the base typelist class to a minimum, the entity code generator adjusts the nested classes as you add and subtract typecodes. These adjustments can change the path that you use to refer to a particular typecode.

Adhering to a soft limit of 2,000 typecodes provides some room to grow your typelist. This soft limit also aids in avoiding splitting the typelist into multiple typelists or a nested-class typelist. To adhere to this soft limit, consider breaking up large typelists by prominent characteristics of your business.

---

**IMPORTANT:** If at all feasible, keep the size of your typelists under 2,048 typecodes, the maximum number of typecodes for a single-class typelist. A typelist with more than 2,048 typecodes uses nested classes. In this case, you must maintain proper references to the typecodes as their allocation among the nested classes changes. If you fail to maintain appropriate typecode references, the application does not compile.

---

### Using a lookup table instead of a typelist

If your implementation requires a typelist with more than 2,048 typecodes, Guidewire strongly recommends that you use a lookup table, such as a custom entity type, instead of a typelist. By using a custom entity type, you can ensure referential integrity by using foreign keys to reference the codes in other entity types. If you add or remove codes from the list, you need to update only the lookup table. You do not need to change your code that accesses the codes through the lookup table.

If you use a database table to perform the look up, you can use a query to retrieve the codes.

Alternatively, you can use other lookup structures such as XML files.

### Internal typecodes

Some typelists contain required internal typecodes that the application references directly. Therefore, the application requires that these typecodes exist. Guidewire Studio displays internal typecodes in non-editable cells so that you cannot edit or delete an internal typecode.

### Localized typecodes

InsuranceSuite supports localizing the individual typecodes in a typelist. See the *Globalization Guide* for more information.

### Mapping typecodes to external system codes

See the following:

- “Mapping typecodes to external system codes” on page 179
- *Plugins, Prebuilt Integrations, and SOAP APIs*

## Typelist definition files

Similar to entity definitions, typelist definitions are stored in XML files. There are three types of typelist files:

| File type | Contains...                                                                                                                                                                                                                         |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| t ti      | A single typelist declaration. The name of the file prior to the extension corresponds to the name of the typelist. This can be either a Guidewire base configuration typelist or a custom typelist that you create through Studio. |
| t tx      | A single typelist extension. This can be a Guidewire-exposed base application extension or a custom typelist extension that you create.                                                                                             |

| File type | Contains...                                                                                                                                                                          |
|-----------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| tix       | A single typelist extension for use by Guidewire only. These are generally Guidewire internal extensions to base application typelists, for use by a specific Guidewire application. |

Always create, modify, and manage typelist definition files through Guidewire Studio. Do not edit the XML typelist files directly.

#### See also

- “Data entity metadata files” on page 21

## Different kinds of typelists

InsuranceSuite organizes typelists into the following categories:

| Category   | Description                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                | More information                   |
|------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------|
| Internal   | <p>Typelists that Guidewire controls because InsuranceSuite requires these typelists for proper application operation. InsuranceSuite depends on these lists for internal application logic. Guidewire designates internal typelists as <i>final</i> (meaning non-extendable). Thus, Guidewire limits what you can modify for these typelists.</p> <p>You can override the following attribute values on these types of typelists:</p> <ul style="list-style-type: none"> <li>• name</li> <li>• description</li> <li>• priority</li> <li>• retired</li> </ul>                              | “Internal typelists” on page 162   |
| Extendable | <p>Typelists that you can customize. These typelists come with a set of example typecodes, but it is possible to modify these typecodes and to add your own typecodes. In some cases, these extendable typelists have internal typecode values that must exist for InsuranceSuite to function properly. You cannot remove the internal typecodes, but you can modify any of the example typecodes.</p> <p>InsuranceSuite designates internal typecodes by placing their code values in gray, non-editable cells. This makes these values inaccessible, and thus, impossible to modify.</p> | “Extendable typelists” on page 163 |
| Custom     | Typelists that you add for specific purposes, such as working with a new custom field. These typelists are not part of the Guidewire base configuration. Guidewire Studio automatically makes all custom typelists non-final (meaning extendable).                                                                                                                                                                                                                                                                                                                                         | “Custom typelists” on page 163     |

## Internal typelists

A few of the typelists in the application are internal. Guidewire controls these typelists because the application needs to know the list of acceptable values in advance to support application logic. Guidewire makes these typelists final by setting the `final` attribute to `true` in the data model. For example, `ActivityType` is an internal typelist because the application implements specific behavior for known activity types.

Guidewire Studio disables your ability to add additional typecodes to internal typelists.

The following are examples of internal typelists that you can't change:

- `ActivityStatus`
- `AgencyBillCycle` (BillingCenter)
- `ArrangementType` (PolicyCenter)
- `CancellationTarget` (BillingCenter)
- `ChargePattern` (BillingCenter)

- AggregateLimitType (ClaimCenter)
- AssignmentStatus (ClaimCenter)
- BindOption (PolicyCenter)
- BOPCost (PolicyCenter)
- Coverage (ClaimCenter)

### Overriding attributes on internal typelists

While you cannot change an internal typelist, you can override the following attributes on the typecodes of an internal typelist:

- name
- description
- priority
- retired

Studio does not permit you to add additional items (typecodes) to an internal typelist. However, you can create a filter for the typelist.

To override a modifiable typelist attribute, first open the typelist in Guidewire Studio by selecting it under **configuration > config > Extensions > Typelist**. Then, select the typecode cell that applies and enter the data. You cannot change the typecode itself. You can change only the attributes associated with the typecode.

## Extendable typelists

Many of the existing typelists are under your control. You cannot delete them or make them empty, but you can adjust the values (typecodes) within the list to meet your needs. InsuranceSuite includes default typelists with sample typecodes in them. You can customize these typelists for your business needs by adding additional typecodes, if you want.

The ActivityCategory typelist is an example of an extendable typelist. If you want, you can add additional typecodes other than the sample values that Guidewire provides in the base configuration.

## Custom typelists

If you add a new field to the application, then it is possible that you also need to add an associated typelist. You can only access these typelists through new extension fields. For more information on how to add a new field to the data model, see “Extending a base configuration entity” on page 94.

To create a custom typelist, in the **Project** tools window, navigate to **configuration > config > Extensions > Typelist**. Right-click on **Typelist**, and then click **New > Typelist**. Enter a name for the typelist, and then define your typecodes. InsuranceSuite limits the number of characters in a typecode to 50 or less.

## Working with typelists in Studio

You create, manage and modify typelists using Guidewire Studio:

- To work with an existing extendable typelist, expand the **Typelist** folder in the **Project** tools window and open the typelist from the list of existing typelists. The typelist opens in its editor, in which you can change non-internal values or define new typecodes and filters.
- To view the values set for an internal typelist, open the typelist in the **Typelist** editor.
- To create a new custom typelist, navigate to **configuration > config > Extensions > Typelist**. Right-click on **Typelist**, and then click **New > Typelist**. Enter its name, and then define typecodes and filters for the typelist.

You cannot add a new typecode to or modify an existing typecode of a final typelist. However, you can create filters for the typelist that modify its behavior.

## The typelist editor

If you modify an existing typelist, ensure that you thoroughly understand which other typelists depend on the typecode values in the typelist being modified. You must also update any related typelists.

For example, in ClaimCenter, any modifications that you make to the `PolicyType` typelist can potentially affect the `InsuranceLine` and `CoverageType` typelists that the `PolicyType` typelist filters. Therefore, you must update all of the related typelists.

After you open a typelist, Studio opens a typelist editor showing configuration options for that typelist.

This topic includes the following:

- “The Studio typelist editor interface” on page 164
- “Search for an existing typelist definition” on page 165
- “Create a new typelist definition” on page 165
- “Extend an existing typelist definition” on page 166

## The Studio typelist editor interface

The right side of the **Typelist** editor contains the following fields:

- **Description**
- **Table name**
- **Final**

### The Description field

The application transfers the value that you enter in the **New > Typelist** dialog for the type list name to the **Description** field in the typelist editor. It is possible to edit this field.

Guidewire recommends that you add an `_Ext` suffix to the value that you enter for the type list name. This ensures that the name of any typelist that you create does not conflict with a Guidewire typelist implemented in a future database upgrade.

### The Table Name field

By default, Guidewire uses `xxtl_typelist-name` as the name of the typelist table, where `xx` is the product code (for example, `cc` for ClaimCenter). However, if you want a different table name, you can override the default value by specifying a value in the **Table name** field for that typelist in Studio. If you override the default value, the table name becomes `xxtl_table-name`.

Guidewire restricts the typelist table name to ASCII letters, digits, and underscore. Guidewire also places limits on the length of the name. However, if you choose, you can override the name of the typelist, which, in turn, overrides the table name stored in the database.

Therefore:

- If you do not provide a value for the **Table name** field, then the application uses the **Name** value and limits the table name to a maximum of 25 characters.
- If you do provide a value for the **Table name** field, then this overrides the value that you set in the **Name** field. However, the maximum table name length is still 25 characters.

| Field | Value entered in... | Maximum length | Database table name                                                              |
|-------|---------------------|----------------|----------------------------------------------------------------------------------|
| Name  | New Typelist dialog | 25 characters  | <b>BillingCenter</b><br><code>bctl_typelist-name</code><br><b>ContactManager</b> |

| Field                       | Value entered in... | Maximum length | Database table name                                                                                                                                                    |
|-----------------------------|---------------------|----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                             |                     |                | abtl_typelist-name<br><b>ClaimCenter</b><br>cctl_typelist-name<br><b>PolicyCenter</b><br>pctl_typelist-name                                                            |
| Table name Typelists editor |                     | 25 characters  | <b>BillingCenter</b><br>bctl_table-name<br><b>ContactManager</b><br>abtl_table-name<br><b>ClaimCenter</b><br>cctl_table-name<br><b>PolicyCenter</b><br>pctl_table-name |

### The Final field

A final typelist is a typelist to which you cannot add additional typecodes. However, you can override the name, description, priority, and retired attributes. All custom typelists that you create are non-final.

## Search for an existing typelist definition

### Procedure

1. In Guidewire Studio, press **Ctrl+Shift+N**.
2. In the **Enter file name** dialog, start typing the name of the typelist that you want to find. Studio displays a list of matching entries that begin with the character string that you type.
3. In the list, click the name of the typelist that you want to view.

Pay attention to the typelist file extension. For example, if you type **AccidentPremises**, Studio displays a list that includes all components whose name contains that text. Typelists will have a file extension of **.tti**, **.tix**, or **.ttx**. Look for the typelist that ends with the appropriate extension.

### Results

Studio opens the file in the appropriate editor.

## Create a new typelist definition

### Procedure

1. In the **Project** tools window, navigate to **configuration > config > Extensions > Typelist**.
2. Right-click on **Typelist**, and then click **New > Typelist**.
3. Enter the typelist name in the **New Typelist** dialog. The application uses this name to uniquely identify this typelist in the data model.
4. Enter a description. Use the **Description** field to create a longer text description to identify how the application uses this typelist. This text appears in places like the *Data Dictionary*.
5. Verify that the (Boolean) **Final** field is set to **false**. Studio automatically sets this field to false for any typelist that you create. You have no control over this setting. This field has the following meanings:

|      |                                                                                                       |
|------|-------------------------------------------------------------------------------------------------------|
| True | You cannot add or delete typecodes from the typelist. You can only override certain attribute fields. |
|------|-------------------------------------------------------------------------------------------------------|

False You can modify or delete typecodes from this typelist, except for typecodes designated as internal, which you cannot delete. (You cannot remove internal typecodes, but you can modify their name, description, and other fields.)

## Extend an existing typelist definition

### Before you begin

If you need to use a file name suffix for your typelist, see “Add a file name suffix to data model extensions” on page 91.

### About this task

You can extend non-internal typelists that are in the following Guidewire Studio location: **configuration > config > Metadata > Typelist**.

### Procedure

1. In Guidewire Studio, in the **Project** tool window, navigate to **configuration > config > Metadata**, and then expand **Typelist**.
2. Right-click the non-internal typelist that you want to extend, and then click **New > Typelist Extension**.  
The file that you want to extend must be non-internal and have either a `.tti` or `.tix` file extension.
3. In the **Typelist Extension** dialog, Studio displays the name and location of the extension file it will create. Click **OK**.

### Results

Studio displays the name of your new file in the **Extensions > Typelist** folder and stores the new file at the following location:

```
configuration/config/extensions/typelist
```

Studio then opens your new file in the appropriate editor.

## Entering typecodes

Each typecode represents one value in the drop-down list. Every typelist must have at least one typecode.

### Creating a new typecode

You must open or create a non-internal typelist extension or a custom typelist in the **Typelist** editor to create a new typecode for a typelist. You cannot add a typecode to an internal typelist or typelist extension. See “Data entity metadata files” on page 21 and “Different kinds of typelists” on page 162 for more information.

In the **Typelist** editor, click **Add** .

With the new typecode selected, you can set the following values:

*code*

A unique identifier for internal Guidewire use. Enter a string containing only letters, digits, or the following characters:

- Dot (.)
- Colon (:) )

Do not include white space or use a hyphen (-).

Use this code to map to your legacy systems for import and export of data. The code must be unique within the list. The maximum number of allowed characters in a typecode is 50.

See also “Mapping typecodes to external system codes” on page 179.

**name**

The text that is visible in the drop-down lists in the application. You can use white space and longer descriptions. However, limit the number of characters so the drop-down list does not become too wide on the screen. The maximum name size is 256 characters.

**desc**

A longer description of this typecode. The maximum description size is 512 characters. The *Data Dictionary* displays the text in this field.

**identifierCode**

Determines how typelist codes become Gosu programmatic identifier codes. See the *Gosu Reference Guide*.

**priority**

A value that determines the sort order of the typecodes (lowest priority first, by default). You use the priority to sort the codes in the drop-down list and to sort a list by priority. If you omit this value, the application sorts the list alphabetically by name. You can specify priorities for some typecodes but not others, causing the application to order the prioritized typecodes at the top of the list with the unprioritized typecodes alphabetized afterwards.

**retired**

A Boolean flag that indicates that a typecode is no longer in use. The value is still valid, but is not offered as a choice in the drop-down list as a new value. The application does not make changes to any existing objects that reference this typecode. If you do not enter a value, the application uses the default value of `false`.

### Naming new typecodes

Guidewire recommends that you add the `_Ext` suffix to the **code** value for any new typecodes that you create. Do this only if the **code** value is legal on any external system that needs to use the value. If that value is not legal, then omit the `_Ext` suffix.

### Maximum typelist size

Guidewire strongly recommends that you limit the size of a typelist to 2,000 typecodes. More than 2,000 typecodes in a typelist can cause performance issues. If you need more than 2,000 typecodes, use a lookup (reference) table. Guidewire does not support the use of more than 8,192 typecodes on a typelist.

#### See also

- “Typelist size recommendation” on page 160

### Typelists and the data model

Guidewire recommends that you regenerate the *Data Dictionary* after you add or modify a typelist. Regenerating the *Data Dictionary* is an excellent way to identify any flaws with your new or modified typelist.

During application start up, Guidewire upgrades the application database if there are any changes to the data model, which includes any changes to a typelist or typecode.

#### See also

- “Typelists and typecodes” on page 160
- “Mapping typecodes to external system codes” on page 179
- *Plugins, Prebuilt Integrations, and SOAP APIs*

## Typekey fields

A *typekey field* is an entity field that the application associates with a specific typelist in the user interface. The typelist determines the values that are possible for that field. Thus, the specified typelist limits the available field values to those defined in the typelist. (Or, if you filter the typelist, the field displays a subset of the typelist values.)

For a field to use a typelist to set values requires the following:

1. The typelist must exist. If it does not exist, then you must create it using Guidewire Studio.
2. The typelist must exist as a <typekey> element on the entity that you use to populate the field. If the <typekey> element does not exist, then you must extend the entity and manually add the typekey.
3. The PCF file that defines the screen that contains your typelist field must reference the entity that you use to populate the field.

This topic includes the following:

- “Example: Typekey field” on page 168

## Example: Typekey field

The following example illustrates how to use the **Priority** typelist to set the priority of an activity that you create.

This example requires the following steps:

- “Step 1: Define the typelist in Studio” on page 168
- “Step 2: Add typekeys to the entity definition file” on page 168
- “Step 3: Reference the typelist in the PCF file” on page 169
- “Step 4: Update the data model” on page 170

### Step 1: Define the typelist in Studio

It is possible to set a priority on an activity, a value that indicates the priority of this activity with respect to other activities. In the base configuration, the **Priority** typelist includes the following typecodes:

- High
- Low
- Normal
- Urgent

You define both the **Priority** typelist and its typecodes (its valid values) through Guidewire Studio, through the **Typelist** editor. For information on using the **Typelist** editor, see “Working with typelists in Studio” on page 163.

After completing this step, complete “Step 2: Add typekeys to the entity definition file” on page 168.

### Step 2: Add typekeys to the entity definition file

Before beginning this step, complete “Step 1: Define the typelist in Studio” on page 168.

For an entity to be able to access and use a typelist, you need to define a <typekey> element on that entity. Use the <typekey> element to specify the typelist in the entity metadata.

For example, in the base configuration, Guidewire declares a number of <typekey> elements on the **Activity** entity (**Activity.eti**), including the **Priority** typekey:

```
<entity entity="Activity" ... >
  ...
  <typekey default="task"
    desc="The class of the activity."
    name="ActivityClass"
    nullok="false"
    typelist="ActivityClass"/>
  <typekey desc="Priority of the activity with respect to other activities."
    name="Priority"
    nullok="false"
    typelist="Priority"/>
  <typekey default="open"
    desc="Status of the activity."
    exportable="false"
    name="Status"
    nullok="false"
```



```

    typelist="ActivityStatus"/>
  <typekey default="general"
    desc="Type of the activity."
    name="Type"
    nullok="false"
    typelist="ActivityType"/>
  <typekey desc="Validation level that this object passed (if any) before it was stored."
    exportable="false"
    name="ValidationLevel"
    typelist="ValidationLevel"/>
  ...
</entity>

```

Notice that the `<typekey>` element uses the following syntax:

```
<typekey desc="DescriptionString" name="FieldName" typelist="Typelist" />
```

After completing this step, complete “Step 3: Reference the typelist in the PCF file” on page 169.

#### See also

- For information on the `<typekey>` element, see “`<typekey>`” on page 71.
- For information on how to create data model entities, see “The application data model” on page 19.
- For information on how to modify existing data model entities, see “Modifying the base data model” on page 89.

## Step 3: Reference the typelist in the PCF file

Before beginning this task, complete “Step 2: Add typekeys to the entity definition file” on page 168.

You can create a new activity. As you do so, you set a number of fields, including the priority for that activity. To render a **Priority** field on the screen, it must exist in the PCF file used to render the screen.

Thus, as the following image illustrates, the ClaimCenter **NewActivityDV** PCF file contains a **Priority** field with a value of **Activity.Priority**:

The screenshot displays the 'NewActivityDV.pcf' file in the Guidewire InsuranceSuite interface. The form contains several input fields: 'Subject \*' (text input), 'Description' (text area), 'Related To \*' (dropdown menu), 'Due Date' (date picker), 'Start Date' (date picker), 'End Date' (date picker), 'Escalation Date' (date picker), 'Priority \*' (dropdown menu), and 'Mandatory \*' (radio buttons for Yes/No). The 'Priority \*' field is highlighted with a blue border, and its dropdown menu is open, showing 'Activity.Priority'. Below the form, the 'PCF' editor is visible, showing the configuration for 'TypeKeyInput: Activity\_Priority'. The 'Basic properties' section includes: 'editable' (true), 'id\*' (Activity\_Priority), 'label' (displaykey.NVV.Activity.NewActivity.Activity.Priority), 'required' (true), and 'value\*' (Activity.Priority). The 'Advanced properties' section is also visible but not expanded.

After completing this step, complete “Step 4: Update the data model” on page 170.

## Step 4: Update the data model

Before beginning this task, complete “Step 3: Reference the typelist in the PCF file” on page 169.

Guidewire recommends that you regenerate the *Data Dictionary* before proceeding. If you have made any mistakes in the previous steps, regenerating the *Data Dictionary* helps to identify those mistakes.

In any case, you need to stop and restart the application server before you can view your changes in the user interface. Restarting the application server forces it to upgrade the data model in the application database.

## Removing or retiring a typekey

Ensure that you fully understand the dependencies between typelists and other application files before you modify a typelist.

In general, Guidewire does not recommend that you make changes to existing typelists other than the following:

- Extending a non-final typelist to add additional typekeys.
- Retiring a typekey, which makes it invisible in the user interface, but leaves the typekey in the application database.

Be very careful of removing typekeys from a typelist as it is possible that multiple application files reference that particular typekey. Removing a typekey incorrectly can cause the application server to not start. Guidewire recommends that you retire a typekey rather than remove it.

It is not possible to remove a typekey from a typelist marked as final. It is also not possible to remove a typekey marked as internal. The application indicates internal typekeys by placing their typecode values in gray, non-editable cells. This makes these typecode values inaccessible and impossible to modify.

## Remove a typekey

If you remove a typekey, you must ensure that the application does not contain code that refers to that typekey.

### About this task

Suppose that you delete the `email_sent` typekey from the base configuration `DocumentType` typelist. If you remove this typekey, you must also update all other parts of the application and prohibit the production of documents of that type. In particular, you must remove references to the typekey from any `.descriptor` file that references that typekey. In this case, a search of the document template files finds that the `CreateEmailSent.gosu.htm.descriptor` file references `email_sent`.

### Procedure

1. Navigate to the typelist that contains the typekey that you want to remove.
2. Click the typekey, and then click **Remove** .
3. Search for additional references to the typekey in the application files and remove any that apply. Pay particular attention to `.descriptor` files. To remove a typekey reference:
  - a) Perform a case-insensitive text search throughout the application files to find all references to the deleted typekey.
  - b) Open these files in Studio and modify as necessary.

## Retire a typekey

### Procedure

1. Navigate to the typelist that contains the typekey that you want to retire.

2. Select the **Retired** cell of the typekey that you want to retire.
3. Set the cell value to **true**.

### What to do next

If you retire a typekey, Guidewire recommends that you perform the steps outlined in “Remove a Typekey” to identify any issues with the retirement:

- Verify all Studio resources.
- Perform a case-insensitive search in the application files for the retired typekey.

## Typelist filters

It is possible to configure a typelist so that the application filters the typelist values so that they do not all appear in the drop-down list (typelist) in the user interface. InsuranceSuite divides typelist filters into the following categories:

| Type    | Creates                                                                                                                                                                                                                                                                                                                                                                               | More information              |
|---------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------|
| Static  | <p>A fixed (static) subset of the values on a typelist. You can create filters that:</p> <ul style="list-style-type: none"> <li>• Include certain specific typecodes on the typelist only.</li> <li>• Include certain specific categories of typecodes on the typelist.</li> <li>• Exclude certain specific typecodes from the full list of the typecodes on the typelist.</li> </ul> | “Static filters” on page 171  |
| Dynamic | <p>A dynamic subset of the values on a typelist. You can create filters that:</p> <ul style="list-style-type: none"> <li>• Associate one or more typecodes on a parent typelist with one or more typecodes on a child typelist.</li> <li>• Associate all the typecodes on a parent typelist with one or more typecodes on a child typelist.</li> </ul>                                | “Dynamic filters” on page 176 |

## Static filters

A *static* typelist filter causes the typelist to display only a subset of the typecodes for that typelist. Therefore, a static filter narrows the list of typecodes to show in the typelist view in the application. Guidewire calls this kind of typelist filter a static *typefilter*.

In the **Typelist** editor of Guidewire Studio™, you define a static filter at the level of the typelist by defining a typefilter for that particular typelist.

Studio manages the typelist XML file for you automatically. If you examine this file, you see that Studio uses the following XML syntax to define a static typelist filter. (In this case, a static filter that defines—or includes—a subset of the available typecodes.)

```
<typelistextension xmlns="http://guidewire.com/typelists" desc="Yes, no or unknown" name="YesNo">
  <typecode code="No" desc="No" name="No" priority="2"></typecode>
  <typecode code="Yes" desc="Yes" name="Yes" priority="1"></typecode>
  <typecode code="Unknown" desc="Unknown" name="Unknown" priority="3"/>
  <typefilter desc="Only display Yes and No typelist values" name="YesNoOnly">
    <include code="Yes"/>
    <include code="No"/>
  </typefilter>
</typelistextension>
```

The XML in this example declares each typecode on the typelist (Yes, No, and Unknown). It then defines a filter named YesNoOnly that limits the available values to simply Yes and No. This filter is the static (fixed) filter.

### See also

- For information on the <typefilter> element, see “<typekey>” on page 71
- “Create a static filter” on page 172

## Create a static filter

### About this task

See “Working with typelists in Studio” on page 163.

### Procedure

1. Open Guidewire Studio™.
2. Define the typecodes for this typelist in the **Typelist** editor.
3. In the selector next to **Add** , click **typefilter**.
4. Set the following information for your static filter:

| Attribute         | Description                                                                                                                                                                                                                                                                                                                                                                                                                                           |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>name</b>       | The name of the filter. The application uses this value to determine if a field uses this filter.                                                                                                                                                                                                                                                                                                                                                     |
| <b>desc</b>       | Description of the context for which to use this typefilter.                                                                                                                                                                                                                                                                                                                                                                                          |
| <b>includeAll</b> | (Boolean) Typically, you only set this value to <b>true</b> if you use the exclude functionality. <ul style="list-style-type: none"> <li>• <b>true</b> indicates that the typelist view starts with the full list of typecodes. You then use exclusions to narrow down the list.</li> <li>• <b>false</b> (the default) instructs the application to use values set in the various subpanes to modify the typelist view in the application.</li> </ul> |

5. In the selector next to **Add** , click one of the following:

| Subpane                    | Use to                                                                                      | More information                                        |
|----------------------------|---------------------------------------------------------------------------------------------|---------------------------------------------------------|
| <b>Add Categories</b>      | Specify one or more typecodes to include by category within the filtered typelist view.     | “Creating a static filter using categories” on page 173 |
| <b>Include Into Filter</b> | Specify one or more typecodes to include within the filtered typelist view.                 | “Creating a static filter using includes” on page 174   |
| <b>Exclude From Filter</b> | Specify one or more typecodes to exclude from the full list of typecodes for this typelist. | “Creating a static filter using excludes” on page 175   |

6. In the appropriate data model file, add a `<typefilter>` element to the child `<typekey>` for this typelist. To be useful, you must declare a static typelist filter (a `typefilter`) on that entity. Use the following XML syntax:

```
<typekey name="FieldName" typelist="Typelist" desc="DescriptionString" typefilter="FilterName"/>
```

You must manually add a typelist to an entity definition file. Studio does not do this for you.

For example:

- The following code adds an unfiltered YesNo typelist to an entity:

```
<typekey desc="Some Yes/No question." name="YesNoUnknown" typelist="YesNo"/>
```

- The following code adds a YesNoOnly filtered YesNo typelist to an entity:

```
<typekey desc="Some other yes or no question." name="YesNo" nullof="true" typefilter="YesNoOnly"
  typelist="YesNo"/>
```

7. (Optional) Regenerate the *Data Dictionary* and verify that there are no validation errors.
8. Stop and restart the application server to update the data model.

See also

- “Typekey fields” on page 167

## Creating a static filter using categories

Suppose that you want to filter a list of United States cities by state. (Say that you want to show a list of appropriate cities only if you select a certain state.) To create this filter, you need to first to define a **City** typelist (if one does not exist). You then need to populate the typelist with a few sample cities:

| City typecode | Location          |
|---------------|-------------------|
| ABQ           | Albuquerque, NM   |
| ALB           | Albany, NY        |
| LA            | Los Angeles, CA   |
| NY            | New York, NY      |
| SF            | San Francisco, CA |
| SND           | San Diego, CA     |
| SNF           | Santa Fe, NM      |

Then, for each **City** typecode, you need to set a category, similar to the following:

1. Click on a typecode.
2. In the selector next to **Add** , click **Add Categories**.
3. Select one or more typecodes to apply to this category.
4. Click **Next**.
5. In the **Typelist** pane, click the associated category typelist.
6. In the **Category** pane, click the associated category typecode.
7. Click **Finish**.

Do this for each of the following:

| City typecode | Associated typelist | Associated typecode |
|---------------|---------------------|---------------------|
| ABQ           | State               | NM                  |
| ALB           | State               | NY                  |
| LA            | State               | CA                  |
| NY            | State               | NY                  |
| SF            | State               | CA                  |
| SND           | State               | CA                  |
| SNF           | State               | NM                  |

To generalize this example to regions outside the United States, you could associate the **Jurisdiction** typelist and a specific jurisdiction with each city typecode instead.

After making your choices, you have something that looks similar to the following:

| Element  | Code | Name              | Priority | Name       | Value                          |
|----------|------|-------------------|----------|------------|--------------------------------|
| typelist | City | City              |          | name       | City                           |
| typecode | ABQ  | Albuquerque, NM   | -1       | desc       | City                           |
| category | NM   | State             |          | tableName  |                                |
| typecode | ALB  | Albany, NY        | -1       | final      | <input type="checkbox"/> false |
| category | NY   | State             |          | platform   | <input type="checkbox"/> false |
| typecode | LA   | Los Angeles, CA   | -1       | subpackage |                                |
| category | CA   | State             |          | xmlns      | http://guidewire.com/typelists |
| typecode | NY   | New York, NY      | -1       |            |                                |
| category | NY   | State             |          |            |                                |
| typecode | SF   | San Francisco, CA | -1       |            |                                |
| category | CA   | State             |          |            |                                |
| typecode | SND  | San Diego, CA     | -1       |            |                                |
| category | CA   | State             |          |            |                                |
| typecode | SNF  | Santa Fe, NM      | -1       |            |                                |
| category | NM   | State             |          |            |                                |

This neatly categorizes each typecode by state.

Now you can create a static filter that contains only cities that are in a certain state. Do the following:

1. In the selector next to **Add** , click **typefilter**.
2. Type **California** for the filter name and description.
3. Click the typefilter, and then in the selector next to **Add** , click **category**.
4. For the category, set the code to **CA**, and the typelist to **State**.

This action creates a static category filter that contains only cities that are in the state of California. Initially, the typelist contains Los Angeles, San Francisco, and San Diego. If you add additional cities to the list at a later time that also exist in California, then the typelist displays those cities as well.

To be useful, you need to also do the following:

- Add the typelist to the entity that you want to display the typelist in the user interface.
- Reference the typelist in the PCF file in which you want to display the typelist.

See “Typekey fields” on page 167 for more information on declaring a typelist on an entity and referencing that typelist in a PCF file. In general, though, you need to add something similar to the entity definition that want to display the typelist:

```
<typekey name="California" typelist="City" typefilter="California" nullok="true"/>
```

## Creating a static filter using includes

Suppose that you want to create a filtered typelist that displays zone codes that are in use only in Canada and not any other country. One way to create the filter is to use an **Includes** filter on the ZoneType typelist.

In this example, you want the typelist to display only the following:

- fsa
- province

The following table shows the typecodes in the ZoneType typelist:

| ZoneType typecode | Associated typelist | Associated typecode | Include in filter |
|-------------------|---------------------|---------------------|-------------------|
| city              | Country             | CA (Canada)         |                   |
|                   |                     | US (United States)  |                   |

| ZoneType typecode | Associated typelist | Associated typecode | Include in filter |
|-------------------|---------------------|---------------------|-------------------|
| county            | Country             | US                  |                   |
| fsa               | Country             | CA                  | •                 |
| postalcode        | Country             | AU                  |                   |
|                   |                     | CA                  |                   |
|                   |                     | DE                  |                   |
| province          | Country             | CA                  | •                 |
| state             | Country             | AU                  |                   |
|                   |                     | DE                  |                   |
|                   |                     | US                  |                   |
| zip               | Country             | US                  |                   |

#### To create an includes filter

1. Open the typelist that you want to filter in the Guidewire Studio™ **Typelists** editor.
2. In the selector next to **Add** , click **Include Into Filter**.
3. In the **Typecodes** list, select **fsa** and **province**.
4. Click **Next**.
5. On the **New Filter** tab, in the **Name** and **Description** text boxes, type **CAOnlyFilter**.
6. Click **Finish**.

| Element                                        | Code           | Name             | Priority |
|------------------------------------------------|----------------|------------------|----------|
| typelistextension                              | ZoneType       |                  |          |
| typecode                                       | country        | Country          | -1       |
| typecode                                       | unknown        | Unknown          | -1       |
| <input checked="" type="checkbox"/> typecode   | city           | City             | -1       |
| <input checked="" type="checkbox"/> typecode   | citykanji      | CityKanji        | -1       |
| <input checked="" type="checkbox"/> typecode   | county         | County           | -1       |
| <input checked="" type="checkbox"/> typecode   | state          | State            | -1       |
| <input checked="" type="checkbox"/> typecode   | prefecture     | Prefecture       | -1       |
| <input checked="" type="checkbox"/> typecode   | province       | Province         | -1       |
| <input checked="" type="checkbox"/> typecode   | postalcode     | Postal Code      | -1       |
| <input checked="" type="checkbox"/> typecode   | zip            | Zip code         | -1       |
| <input checked="" type="checkbox"/> typecode   | fsa            | FSA              | -1       |
| <input checked="" type="checkbox"/> typecode   | postcodearea   | Post Code Area   | -1       |
| <input checked="" type="checkbox"/> typecode   | postcoderegion | Post Code Region | -1       |
| <input checked="" type="checkbox"/> typefilter | CAOnlyFilter   | CAOnlyFilter     |          |
| include                                        | fsa            |                  |          |
| include                                        | province       |                  |          |

## Creating a static filter using excludes

Suppose that you want to create a filtered typelist that displays all of the zone codes except those that are in use in Canada. You want to display the complete list of typecodes in the ZoneType typelist except for the following:

- city
- fsa
- province

The following table shows the typecodes in the ZoneType typelist:

| ZoneType typecode | Associated typelist | Associated typecode | Include in filter |
|-------------------|---------------------|---------------------|-------------------|
| city              | Country             | CA (Canada)         |                   |
|                   |                     | US (United States)  |                   |
| county            | Country             | US                  | •                 |
| fsa               | Country             | CA                  |                   |
| postalcode        | Country             | AU                  |                   |
|                   |                     | CA                  |                   |
|                   |                     | DE                  |                   |
| province          | Country             | CA                  |                   |
| state             | Country             | AU                  | •                 |
|                   |                     | DE                  |                   |
|                   |                     | US                  |                   |
| zip               | Country             | US                  | •                 |

### To create an excludes filter

1. Open the typelist that you want to filter in the Guidewire Studio **Typelists** editor.
2. In the selector next to **Add** , click **Exclude From Filter**.
3. In the **Typecodes** list, select the typecodes that you want to exclude. For example, select **city** and **fsa**.
4. Click **Next**.
5. On the **New Filter** tab, in the **Name** and **Description** text boxes, type **ExcludesCanada**.
6. Set the **Include all typecodes** check box true. This ensures that you start with a full set of typecodes.
7. Click **Finish**.

| Element                                                                                        | Code           | Name             | Priority |
|------------------------------------------------------------------------------------------------|----------------|------------------|----------|
| typelistextension                                                                              | ZoneType       |                  |          |
| typecode                                                                                       | country        | Country          | -1       |
| typecode                                                                                       | unknown        | Unknown          | -1       |
|  typecode   | city           | City             | -1       |
|  typecode   | citykanji      | CityKanji        | -1       |
|  typecode   | county         | County           | -1       |
|  typecode   | state          | State            | -1       |
|  typecode   | prefecture     | Prefecture       | -1       |
|  typecode   | province       | Province         | -1       |
|  typecode   | postalcode     | Postal Code      | -1       |
|  typecode   | zip            | Zip code         | -1       |
|  typecode   | fsa            | FSA              | -1       |
|  typecode   | postcodearea   | Post Code Area   | -1       |
|  typecode   | postcoderegion | Post Code Region | -1       |
|  typefilter | ExcludesCanada | ExcludesCanada   |          |
| exclude                                                                                        | city           |                  |          |
| exclude                                                                                        | fsa            |                  |          |
| exclude                                                                                        | province       |                  |          |

## Dynamic filters

A *typecode filter* uses categories and category lists at the typecode level to restrict or filter a typelist. Typecode filters use a parent typecode to restrict the available values on the child typecode.

You define a typecode filter directly on a typecode in Guidewire Studio™. In the **Typelist** editor, you select a specific typecode and set a filter, or *category*, on that typecode.



There are two types of typecode filters that you can define:

| Filter type   | Use to                                                                                               | More information              |
|---------------|------------------------------------------------------------------------------------------------------|-------------------------------|
| Category      | Associate one or more typecodes on a parent typelist with one or more typecodes on a child typelist. | “Dynamic filters” on page 176 |
| Category list | Associate all of the typecodes on a parent typelist with one or more typecodes on a child typelist.  | “Dynamic filters” on page 176 |

### Category typecode filters

- You use a category filter to associate one or more typecodes from one or more typelists with a specific typecode on the filtered typelist.
- You define a category filter in the **Typelist** editor by adding a category to a typecode.

### Category list typecode filters

- You use a category list filter to associate all of the typecodes from one or more typelists with a specific typecode on the filtered typelist.
- You define a category list filter in the **Typelists** editor by adding a category list to a typecode.

This topic includes the following:

- “Creating a dynamic filter” on page 177

## Creating a dynamic filter

In general, to create a dynamic filter, you need to do the following:

- “Step 1: Set the Category Filter on Each Typecode”
- “Step 2: Declare the Category Filter on an Entity”
- “Step 3: Set the Field Value in the PCF File”
- “Step 4: Regenerate the Data Dictionary”

As the process of declaring a typecode filter on an entity can be difficult to understand conceptually, it is simplest to proceed with an example. Within Guidewire InsuranceSuite, a user with administrative privileges can define a new activity pattern (**Administration > Business Settings > Activity Patterns > New Activity Pattern**). Within the **New Activity Pattern** screen, you see the following drop-down lists:

- **Type**
- **Category**

BillingCenter and ClaimCenter automatically set the default value of **Type** to **General**. (You can't edit this field as the **editable** attribute of this field is set to **false** in the base configuration.) In PolicyCenter, you can also set **Type** to **Assignment Review**. The **Type** value determines the available choices that you see in the **Category** drop-down list. For example:

- Correspondence
- File Review
- General
- Interview
- ...

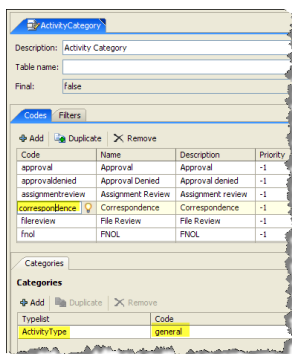
The **ActivityCategory** typelist is the typelist that controls what you see in the **Category** field in the application. If you open this typelist in the Studio **Typelist** editor, you can choose each typecode in the list one after another. As you select each typecode in turn, notice that the Studio associates each typecode with a category containing typelist and code

values. In this case, Studio associates each `ActivityCategory` typecode with an `ActivityType` typecode. Thus, the application filters each individual typecode in this typelist so that it is available for selection only if you first select the associated typelist and typecode.

## Step 1: Set the category filter on each typecode

The process is the same to create a category list typecode filter. In that case, you associate a single typelist (and all its typecodes) with each individual typecode on the dependent typelist. You make the association by selecting a typecode in the dependent typelist and adding a category.

In Studio for ClaimCenter, open the `ActivityCategory` typelist and select each typecode in turn. As you do so, you see that Studio associates each typecode with an `ActivityType` code value as a category. For example, if you select the correspondence typecode, you see that ClaimCenter associates this typecode with an `ActivityType` code value of `general`. You would need to duplicate this process if you create a custom filtered typelist or if you customize an existing typelist. The following graphic illustrates this process.



After completing this task, complete “Step 2: Declare the category filter on an entity” on page 178.

## Step 2: Declare the category filter on an entity

Before beginning this task, complete “Step 1: Set the category filter on each typecode” on page 178.

The question then becomes how do you set this behavior on the `ActivityPattern` entity. In other words, what XML code do you need to add to the `ActivityPattern` entity to enable the `ActivityType` typelist to control the values shown in the **Category** field? The following sample illustrates what you need to do. You must add a typekey for both the parent (`ActivityType`) typelist and the dependent child (`ActivityCategory`) typelist.

| ▼ Element  | Primary Value   | Secondary Value               |
|------------|-----------------|-------------------------------|
| entity     | ActivityPattern | An activity pattern is a t... |
| column     | AutomatedOnly   | bit                           |
| column     | Description     | mediumtext                    |
| column     | EscalationDays  | integer                       |
| column     | EscalationHours | integer                       |
| typekey    | Category        | ActivityCategory              |
| keyfilters |                 |                               |
| keyfilter  | Type            |                               |
| typekey    | ActivityClass   | ActivityClass                 |
| typekey    | Priority        | Priority                      |
| typekey    | Type            | ActivityType                  |

The sample defines a typekey element with name="Type" and typelist="ActivityType". This is the controlling (parent) typelist. The sample also defines a second typelist (`ActivityCategory`) with a keyfilter name="Type". It is the typelist referenced by the keyfilter element that controls the behavior of the typelist named in the typekey element. Thus, the value of the `ActivityType` code controls the associated typecode on the dependent `ActivityCategory` typelist.

For more information on the `<keyfilter>` element, see “`<typekey>`” on page 71.

After completing this task, complete “Step 3: Set the field value in the PCF file” on page 179.

## Step 3: Set the field value in the PCF file

Before beginning this task, complete “Step 2: Declare the category filter on an entity” on page 178.

After you declare these two typelists on the `ActivityCategory` entity, you need to link the typelists to the appropriate fields on the **New Activity Pattern** screen. To access an entity typelist, you need to use the `entity.Typelist` syntax. For example, to access the `ActivityCategory` typelist on the `ActivityPattern` entity, use `ActivityPattern.Category` with `Category` being the name of the typelist.

After completing this task, complete “Step 4: Regenerate the Data Dictionary” on page 179.

## Step 4: Regenerate the Data Dictionary

Before beginning this task, complete “Step 3: Set the field value in the PCF file” on page 179.

Guidewire recommends that you regenerate the *Data Dictionary* before proceeding. If you have made any mistakes in the previous steps, regenerating the *Data Dictionary* helps to identify those mistakes.

In any case, you need to stop and restart the application server before you can view your changes in the user interface. Restarting the application server forces it to upgrade the data model in the application database.

## Typecode references in Gosu

To refer to a specific typecode in Gosu, use the following syntax.

```
typekey.TypeList.TC_Typecode
```

For example, the default `State` typelist has typecodes for states in the US and provinces in Canada.

To refer to the typecode for the state of California in the typelist `State`, use the following Gosu expression.

```
typekey.State.TC_CA
```

You must prefix the code in the object path expressions for typecodes with `TC_`.

**Note:** In the typecode reference syntax, use the appropriate nested class name if and only if your typelist has over 2,048 typecodes. This modified syntax is as follows:

```
typekey.TypeList.[NestedClassName.]TC_Typecode
```

**Note:** Use code completion in Studio to build complete object path expressions for typecodes. Type “`typekey.`” to begin, and work your way down to the typecode that you want.

See also

- “Typelists and typecodes” on page 160

## Mapping typecodes to external system codes

Your InsuranceSuite application can share or exchange data with one or more external applications. If you use this functionality, Guidewire recommends that you configure the InsuranceSuite typelists to include typecode values that are one-to-one matches to those in the external applications. If the typecode values match, sending data to, or receiving data from, those applications requires no additional effort on the part of an integration development team.

However, there can be more complex cases in which mapping typecodes one-to-one is not feasible. For example, suppose that it is necessary to map multiple external applications to the same InsuranceSuite typecode, but the external applications do not match. Alternatively, suppose that you extend your typecode schema in InsuranceSuite. This can possibly cause a situation in which three different codes in InsuranceSuite represent a single (less granular) code in the other application.

To handle these more complex cases, edit the file `typecodemapping.xml` in the **configuration > config > typelists.mapping** folder. This file specifies a namespace for each external application. Then you identify the individual unique typecode maps by typelist.

### A typecode mapping example

The following code sample illustrates a simple `typecodemapping.xml` file:

```
<?xml version="1.0"?>
<typecodemapping>
  <namespacelist>
    <namespace name="accounting"/>
  </namespacelist>

  <typelist name="AccountSegment">
    <mapping typecode="PR" namespace="accounting" alias="ACT"/>
  </typelist>
</typecodemapping>
```

The `namespacelist` element contains one or more `namespace` elements—one for each external application. Then, to map the actual codes, specify one or more `typelist` elements as required. Each `typelist` element refers to a single internal or external typelist in the application. The `typelist`, in turn, contains one or more mapping elements. Each mapping element must contain the following attributes:

| Attribute              | Description                                              |
|------------------------|----------------------------------------------------------|
| <code>typecode</code>  | The application typecode.                                |
| <code>namespace</code> | The namespace to which the application maps the typecode |
| <code>alias</code>     | The code in the external application.                    |

In the previous example, the PR code maps to an external application named `accounting`. You can create multiple mapping entries for the same application typecode or the same name space. For example, the following specifies a mapping between multiple application codes and a single external code:

```
<typelist name="BoatType">
  <mapping typecode="AI" namespace="accounting" alias="boat"/>
  <mapping typecode="HY" namespace="accounting" alias="boat"/>
</typelist>
```

After you define the mappings, from an external system you can use the `TypelistToolsAPI` web service to translate the mappings. See *Plugins, Prebuilt Integrations, and SOAP APIs*.

## Qualified and unqualified typelist extensions

*Qualified extensions* are those typelist `.ttx` files that have a *qualifier* in the file name. For example, the typelist extension file `Foo.bar.ttx` extends the `Foo` typelist, and has a qualifier `bar` in the name.

*Unqualified extensions* are those typelist `.ttx` files that do not have a *qualifier* in the file name. For example, `Foo.ttx` extends the `Foo` typelist and has no qualifier.

All extendable (that is, non-final) typelists may have up to one unqualified extension file and any number of qualified extension files.

Qualified typelist extensions may refer to typecodes defined in the unqualified typelist extension.

## Typelist best practices

A *typelist* represents a set of allowed values for specific fields on entities in the data model. A *typecode* represents an individual value within a typelist. A typecode comprises:

- A code that the database stores as a column value

- A name that the user interface displays
- A priority setting that drop-down lists use to order the typecode names that they display

As a best practice, Guidewire recommends that you edit typelist definitions by using the Typelist editor in Guidewire Studio.

## Observe typelist naming conventions

The components of a typelist have the following naming conventions:

| Typelist component | Naming conventions                                                                                       | Example          |
|--------------------|----------------------------------------------------------------------------------------------------------|------------------|
| Typelist           | Mixed case, with the first letter uppercase, and with the first letter of each internal word capitalized | ActivityCategory |
| Code               | Only lowercase letters. Underscores (_) separate words.                                                  | approval_pending |
| Name               | Each word in a name begins with a capital letter. Spaces separate words.                                 | Approval Pending |

## Add a suffix to new typelists and typecode extensions

In future releases, Guidewire may add or change base typelists. If you make your own changes to typelists, there is the possibility of a naming conflict with one of these future updates. For example, you may add new typelists, or new typecodes to existing typelists. To avoid possible naming conflicts, Guidewire recommends that you append the suffix `_Ext` to your new typelists and typecodes.

If you add a new typelist, its typecode names do not need the suffix `_Ext` because the name of the typelist already has the `_Ext` suffix.

If you add new typecodes to an existing base Guidewire typelist, add the `_Ext` suffix to their names.

As an example of a new typelist, name one that represents types of medical procedures `MedicalProcedureType_Ext`. Name the typecodes in your new typelist without the suffix `_Ext`.

As an example of new typecodes in a base typelist, the following `AddressType` typelist has a new typecode for service entrances.

| Code                 | Name             | Description      | Priority | Retired |
|----------------------|------------------|------------------|----------|---------|
| -----                | -----            | -----            | -----    | -----   |
| billing              | Billing          | Billing          | -1       | false   |
| business             | Business         | Business         | -1       | false   |
| home                 | Home             | Home             | -1       | false   |
| other                | Other            | Other            | -1       | false   |
| service_entrance_Ext | Service Entrance | Service Entrance | -1       | false   |

## Do not rely on implicit conversion of typecodes to strings

Guidewire recommends that you do not send typecodes as arguments to display keys, but explicitly send the `Name` properties of `typekey`.



# Customizing the database upgrade

The `IDatamodelUpgrade` plugin interface provides hooks for custom code that you want to run during the database upgrade. When configuring the data model, you might need to perform operations such as:

- Prefilling a newly added column.
- Transferring data from one entity to another (which might involve the creation of a temporary table or entity via the database triggers).

You can implement the `IDatamodelUpgrade` plugin to:

- Execute custom version checks to test data or the data model itself before starting the upgrade.
- Make custom database changes before or after the database upgrade.
- Make data model changes to archived entities.

For example, you might fix a consistency check failure issue, correct issues reported by version checks, or delete a custom extension that you are no longer using.

## Creating custom upgrade version checks and version triggers

Use the `IDatamodelUpgrade` plugin to run custom version checks and triggers before and after the database upgrade. The `IDatamodelUpgrade` plugin interface defines method signatures for two methods that you must define in your plugin. These signatures are:

- property `getBeforeUpgradeDatamodelChanges()` :  
`List<IDataModelChange<BeforeUpgradeVersionTrigger>>`
- property `getAfterUpgradeDatamodelChanges()` :  
`List<IDataModelChange<AfterUpgradeVersionTrigger>>`

Each method returns a list of `IDataModelChange` entities, each taking a `BeforeUpgradeVersionTrigger` or `AfterUpgradeVersionTrigger` type parameter. The `IDataModelChange` interface has two methods that you use to make data model changes. The `getDatabaseUpgradeVersionTrigger` method is for changes to the database. The `getArchivedDocumentUpgradeVersionTrigger` method is for changes to archived entities. If your organization has not implemented archiving or you do not want to make changes to archived entities, return null for `getArchivedDocumentUpgradeVersionTrigger`.

The `getAfterUpgradeDatamodelChanges` method runs after the Guidewire upgrade version triggers. You can use this method to move data into extension tables or columns that did not exist prior to upgrading.

You can return an empty list from either `getBeforeUpgradeDatamodelChanges` or `getAfterUpgradeDatamodelChanges`. For example, if you only have triggers to run before the upgrade, you can return an empty list from `getAfterUpgradeDatamodelChanges`.

## Modifying tables using a custom version trigger

Both `BeforeUpgradeVersionTrigger` and `AfterUpgradeVersionTrigger` base classes provide a protected `getTable` method that accepts a string parameter. The `getTable` method returns an `IBeforeUpgradeTable` or `IAfterUpgradeTable` object that provides a number of methods for DDL and DML operations, such as:

- `create` – Create the table if it does not already exist. The table must be related to an entity defined in the data model. This method is available only for `IBeforeUpgradeTable`.
- `delete` – Deletes rows from a table. Returns a builder (`IBeforeUpgradeDeleteBuilder` for `IBeforeUpgradeTable`, `IDeleteBuilder` for `IAfterUpgradeTable`) that has methods for comparing data to restrict which rows are deleted.
- `drop` – Drops the table.
- `dropColumns` – Drops multiple columns from the table.
- `getColumn` – Returns an `IBeforeUpgradeColumn` or `IAfterUpgradeColumn` object that has methods to perform DDL operations on the column such as create, drop, rename, and more.
- `insert` – Returns a builder (`IBeforeUpgradeInsertBuilder` for `IBeforeUpgradeTable`, `IInsertBuilder` for `IAfterUpgradeTable`) to perform an insert operation.
- `insertSelect` – Returns a builder (`IBeforeUpgradeInsertSelectBuilder` for `IBeforeUpgradeTable`, `IInsertSelectBuilder` for `IAfterUpgradeTable`) for SQL to perform an insert operation using data selected from a table.
- `rename` – Renames the table.
- `update` – Returns a builder (`IBeforeUpgradeUpdateBuilder` for `IBeforeUpgradeTable`, `IUpdateBuilder` for `IAfterUpgradeTable`) for SQL to perform an update operation.

For DML operations, call the `execute` method on the builder to actually perform the operation. The `execute` method runs in its own transaction. You do not need to handle transactions and `TransactionManager`.

There are more methods on the `IBeforeUpgradeTable` and `IAfterUpgradeTable` classes documented in the Guidewire Gosu API documentation. To generate the Guidewire Gosu API documentation, run the `gwb gosudoc` command from the application installation directory. Then, open the file `build/gosudoc/index.html`.

The methods for `BeforeUpgradeVersionTrigger` intentionally take strings but not entities or properties. This is because the name of the column could change in the future. Consider `PropertyA` on `EntityE` which corresponds to column `A` in the database. Suppose you use `PROPERTYA_PROP` in a version trigger at minor version 200, but at minor version 250, you decide to rename the backing column from `A` to `B`. The version trigger you wrote in the past would break because it would execute before the rename operation and would try to use the new column name.

`AfterUpgradeVersionTrigger` is very similar to `BeforeUpgradeVersionTrigger`. A few differences include:

- The `AfterUpgradeVersionTrigger` DML builders use the query builder, `IQueryBuilder`.
- In an `AfterUpgradeVersionTrigger` you can use properties and types in addition to strings.
- Some DDL operations are not provided on the `IAfterUpgradeTable` object, including creating a table or adding a column.

**Note:** Guidewire recommends you use `BeforeUpgradeVersionTrigger` for custom version triggers unless you have a special case requiring one of the unique capabilities of `AfterUpgradeVersionTrigger`. `BeforeUpgradeVersionTrigger` has many more capabilities than `AfterUpgradeVersionTrigger`. One example of a limitation of `AfterUpgradeVersionTrigger` is that you cannot set `MonetaryAmount` fields from an `AfterUpgradeVersionTrigger`.



## Changing the nullability of subtype columns after a database upgrade

For entity definitions, an automatic database upgrade converts nullable columns to non-nullable columns. For the converted columns, the upgrade process sets a default value successfully.

However, the automatic database upgrade process does not convert column nullability for columns in subtype definitions. Instead, the application implements non-nullable columns on subtypes in the database as nullable columns. The product behaves this way because columns must have null values for rows that represent instances of other subtypes.

To make a subtype column non-nullable, write a custom version trigger. Program the version trigger to populate the column with an appropriate default value for existing subtype rows. After you perform this step, the application makes new column values for the subtype non-nullable. The product does this by setting the default value for new subtype rows automatically.

See “Setting a column value for a specific subtype using a version trigger” on page 191 and “Create a custom version check or version trigger” on page 186 for more information on creating a version trigger in this manner.

## Version checks

You can check for a certain condition in the database before the upgrade proceeds. This is referred to as a version check. Only read operations are available in version checks. For example, you can implement a version check to query a table or check the existence of a table or column, but the check cannot insert new rows. The `BeforeUpgradeVersionTrigger` class includes a `hasVersionCheck` method that you must define to return true or false. If the trigger does include a version check, overwrite the `createVersionCheck` method to define your custom version check. For standalone version checks that are not associated with a version trigger, you can use `BeforeUpgradeVersionCheckWrapper`.

The upgrade executes all custom version checks before custom version triggers. The upgrade runs Guidewire version checks after all custom `BeforeUpgradeVersionTrigger` implementations, so you can create a `BeforeUpgradeVersionTrigger` to correct issues detected by the Guidewire version checks.

If a custom version check fails, the upgrade stops before running any upgrade triggers. Correct the issue and restart the upgrade.

## Order of execution

The following table describes failure cases that are caused by data issues. If the upgrade fails for other reasons, such as a disruption of the database server, fix the issue causing the disruption, restore the database, and restart the upgrade. During an upgrade, InsuranceSuite performs database upgrade actions in the following order:

| Step | Action                                                          | In the event of failure due to a data issue...                                                                                                                                                                                                                                                                                                                                                                                               |
|------|-----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | Custom version checks                                           | Correct the data issue. Restart the upgrade. You do not need to restore the database because the upgrade has not made any changes.                                                                                                                                                                                                                                                                                                           |
| 2    | Custom <code>BeforeUpgradeVersionTrigger</code> implementations | Restore the database from a backup. Correct the data issue. Consider adding custom version checks to test for other instances of the data issue.                                                                                                                                                                                                                                                                                             |
| 3    | Guidewire version checks                                        | <p>If you do not have any custom <code>BeforeUpgradeVersionTrigger</code> implementations, correct the data issue and restart the upgrade.</p> <p>If you do have custom <code>BeforeUpgradeVersionTrigger</code> implementations, restore the database from a backup. Then, correct the data issue.</p> <p>In either case, consider creating a custom <code>BeforeUpgradeVersionTrigger</code> implementation to correct the data issue.</p> |
| 4    | Guidewire version triggers                                      | A failure due to data issues at this stage is unlikely. Contact Guidewire Support.                                                                                                                                                                                                                                                                                                                                                           |

| Step | Action                                                                         | In the event of failure due to a data issue...                                                                                                                           |
|------|--------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 5    | Automated data model upgrade to update the database to the defined data model. | A failure due to data issues at this stage is unlikely. Contact Guidewire Support.                                                                                       |
| 6    | Guidewire version triggers that require the updated data model in the database | A failure due to data issues at this stage is unlikely. Contact Guidewire Support.                                                                                       |
| 7    | Custom AfterUpgradeVersionTrigger implementations                              | Restore the database from a backup. Correct the data issue. Consider creating a custom BeforeUpgradeVersionTrigger implementation to correct the data issue if possible. |

## Create a custom version check or version trigger

### About this task

You can implement a `IDatamodelUpgrade` plugin with custom version triggers and version checks. When you start the server to perform the database upgrade from a previous release, the application calls the plugin and runs your custom methods.

Each `BeforeUpgradeVersionTrigger` and `AfterUpgradeVersionTrigger` instance requires a minor version number, passed as an integer. If the data model version number is less than or equal to the number passed to the instance, then the trigger executes. Whenever you make a data model change, or you want to force an upgrade, increment the version number in `extensions.properties`.

### Procedure

1. Create a new package, such as `companyName.upgrade`, to store your custom version triggers.
  - a) Open Studio.
  - b) In the Studio **Project** window, expand **configuration**.
  - c) Right-click **gsrc** and click **New > Package**.
  - d) Enter a package name for upgrade purposes, such as `companyName.upgrade`.
2. Right-click the upgrade package and click **New > Gosu Class**.
3. Enter a name for the class and click **OK**.
4. Create a new Gosu class that extends `CustomerDatamodelUpgrade` and implements `IDatamodelUpgrade`. The class you create must define the `getBeforeUpgradeDatamodelChanges` and `getAfterUpgradeDatamodelChanges` methods. This class is the container from which you call custom version trigger classes.

For example:

```
package companyName.upgrade
uses gw.plugin.upgrade.IDatamodelUpgrade
uses java.lang.Iterable
uses gw.api.database.upgrade.before.BeforeUpgradeVersionTrigger
uses gw.api.database.upgrade.after.AfterUpgradeVersionTrigger
uses java.util.ArrayList
uses gw.api.datamodel.upgrade.CustomerDatamodelUpgrade
uses gw.api.datamodel.upgrade.IDatamodelChange
uses gw.api.database.upgrade.DatamodelChangeWithoutArchivedDocumentChange

class TestDatamodelUpgradeImpl extends CustomerDatamodelUpgrade implements IDatamodelUpgrade {

    override property get BeforeUpgradeDatamodelChanges() :
    List<IDatamodelChange<BeforeUpgradeVersionTrigger>> {

        var list = new ArrayList<IDatamodelChange<BeforeUpgradeVersionTrigger>>()
        list.add(DatamodelChangeWithoutArchivedDocumentChange.make(new BeforeVersionTrigger1()))
        list.add(DatamodelChangeWithoutArchivedDocumentChange.make(new BeforeVersionTrigger2()))
        return list
    }

    override property get AfterUpgradeDatamodelChanges() :
    List<IDatamodelChange<AfterUpgradeVersionTrigger>> {
```

```

    var list = new ArrayList<IDatamodelChange<AfterUpgradeVersionTrigger>>()
    list.add(DatamodelChangeWithoutArchivedDocumentChange.make(new AfterVersionTrigger1()))
    return list
  }
}

```

5. Create your custom `BeforeUpgradeVersionTrigger` and `AfterUpgradeVersionTrigger` Gosu classes. See “IDatamodelUpgrade API examples” on page 187.
6. Implement the `IDatamodelUpgrade` plugin with the new class.
  - a) Start Guidewire Studio by entering `gwb studio` from the application directory.
  - b) In Studio, expand **configuration > config > Plugins**.
  - c) Right-click **registry** and click **New > Plugin**.
  - d) In the **Plugin** dialog, enter the name `IDatamodelUpgrade`. For this plugin, the name must match the interface.
  - e) In the **Plugin** dialog, click the ... button.
  - f) In the **Select Plugin Class** dialog, type `IDatamodelUpgrade` and select the `IDatamodelUpgrade` interface.
  - g) In the **Plugin** dialog, click OK. Studio creates a GWP file under **Plugins > registry** with the name you entered.
  - h) Click the **Add Plugin** icon (a plus sign) and select **Add Gosu Plugin**.
  - i) For **Gosu Class**, enter your class, including the package.
  - j) Save your changes.

## IDatamodelUpgrade API examples

The `IDatamodelUpgrade` plugin interface provides hooks for custom code that you want to run during the database upgrade. You can implement the `IDatamodelUpgrade` plugin to:

- Execute custom version checks to test data or the data model itself before starting the upgrade.
- Make custom database changes before or after the database upgrade.
- Make data model changes to archived entities.

The following topics contain examples of how you can customize your database upgrade by implementing an `IDatamodelUpgrade` plugin.

[See also](#)

“Creating custom upgrade version checks and version triggers” on page 183

## BeforeUpgradeVersionTrigger Structure

A custom `BeforeUpgradeVersionTrigger` subclass has the following structure. Define the `execute` method to perform your custom trigger actions.

```

package companyName.upgrade.before

uses gw.api.database.upgrade.before.BeforeUpgradeVersionCheck
uses gw.api.database.upgrade.before.BeforeUpgradeVersionTrigger

class myBeforeUpgradeTrigger extends BeforeUpgradeVersionTrigger {
  construct() {
    super(dataModelVersionNumber)
  }

  override function execute() {
    // Perform actions here.
  }

  override function hasVersionCheck() : boolean {
    // return true if creating a version check to determine whether the trigger can run.
  }
}

```

```

    } // return false if you are not implementing a version check.
    }

    override property get Description() : String {
        return "Description of the version trigger."
    }

    // Override the createVersionCheck method if you are implementing a version check.
    override function createVersionCheck() : BeforeUpgradeVersionCheck {
        return new BeforeUpgradeVersionCheck(dataModelVersionNumber) {

            override function verifyUpgradability() {
                if (condition to detect) {
                    addVersionCheckProblem("description of issue")
                }
            }

            override property get Description() : String {
                return "Description of the version check."
            }
        }
    }
}

```

## AfterUpgradeVersionTrigger Structure

A custom AfterUpgradeVersionTrigger subclass has the following structure.

```

package companyName.upgrade.after

uses gw.api.database.upgrade.after.AfterUpgradeVersionTrigger

class myAfterUpgradeTrigger extends AfterUpgradeVersionTrigger{

    construct() {
        super(dataModelVersionNumber)
    }

    override function execute() {
        // Perform actions here.
    }

    override property get Description() : String {
        return "Description of the version trigger."
    }
}

```

## Altering columns to match the data model using a version trigger

In most cases, you do not need to alter a column to match a change to the column type in the logical data model. InsuranceSuite automatically applies data model changes to the database during the upgrade. However, this occurs *after* all custom BeforeUpgradeVersionTrigger instances have run, so Guidewire provides methods to alter database columns to match the data model.

## Modify a single column using a version trigger

### About this task

If you need to modify a single column for use in a BeforeUpgradeVersionTrigger, use the following procedure.

### To modify a single column

#### Procedure

1. Modify the data model file.
2. Use the alterColumnTypeToMatchDatamodel method of IBeforeUpgradeColumn to make your changes.

### Example

For example:

```
var table = getTable("TableName")
var column = table.getColumn("ColumnName")
column.alterColumnTypeToMatchDatamodel()
```

## Modify multiple columns using a version trigger

### About this task

If you need to modify more than one column for use in a `BeforeUpgradeVersionTrigger`, use the following procedure.

### To modify multiple columns

#### Procedure

1. Modify the data model file.
2. Use the `alterMultipleColumnsToMatchDatamodel` method of `IBeforeUpgradeTable`.  
For example:

```
var table = getTable("TableName")
var columnsToChange = new IBeforeUpgradeColumn[2]
columnsToChange[0] = table.getColumn("column1")
columnsToChange[1] = table.getColumn("column1")
table.alterMultipleColumnsToMatchDatamodel(columnsToChange)
```

## Alter a non-nullable column to nullable using a version trigger

### About this task

To alter a column from non-nullable to nullable, use the `IBeforeUpgradeColumn` method `alterColumnToNullable`.

### Example

For example:

```
var table = getTable("TableName");
table.getColumn("ColumnName").alterColumnToNullable();
```

## Creating columns using a version trigger

The database upgrader automatically creates a column that is added to the data model if the column meets one of the following criteria:

- Nullable
- Non-nullable with a default value specified in the metadata
- Non-nullable without a default value if there are no rows in the table
- The column is an editable field

However, you might want to explicitly create the column in your upgrade trigger if you want the trigger to perform an action on the column such as populating it.

In the data model, the column must be defined as a property on an entity. The database upgrade will determine the correct datatype and nullability from the data model.

Creating a new column is moderately expensive in terms of performance of the upgrade.

## Creating a column

To create a column, invoke the `create` method on the `IBeforeUpgradeColumn`.

For example:

```
var table = getTable("TableName")

// Create column with given name.
// Column must be backed by a property on an entity.
// Upgrader will figure out the correct datatype and nullability.

table.getColumn("ColumnName").create()
```

## Creating a non-nullable column with an initial value

The upgrader throws an exception if you try to add a new non-nullable column without a default value and there are rows in the table. For non-nullable columns, either specify a default value, or create a version trigger that will populate the column.

To create a new column as non-nullable with an initial value, use the `createNonNullableWithInitialValue()` method. In the data model, the column must be defined as non-nullable.

For example:

```
IBeforeUpgradeTable table = getTable("TableName")
table.getColumn("ColumnName").createNonNullableWithInitialValue(Initial value)
```

The initial value must be of the appropriate type for the column's datatype. You can alter this value in later steps as needed.

## Creating a temporary column

Use the `createTempColumn` method of `IBeforeUpgradeTable` to add a temporary column to the table. The `createTempColumn` method takes two parameters, a `String` for the column name and an `IDataType` for the column data type. `createTempColumn` creates a new nullable column with the given name and datatype to hold temporary data. You must explicitly drop the temporary column during the upgrade. The schema verifier will report an error during server startup if the column has not been dropped. You can create the temporary column in a `BeforeUpgradeVersionTrigger` and drop it in an `AfterUpgradeVersionTrigger`. This approach is useful when you want to move data from a column that will be removed during the upgrade to a column that will be created during the upgrade.

In the following example, a `BeforeUpgradeVersionTrigger` adds a temporary `shorttext` column to an existing entity and populates it with data from another column on a different entity. An `AfterUpgradeVersionTrigger` moves the data to a new entity.

### BeforeUpgradeVersionTrigger Execute Method

```
// Add a temporary column to TableA.
var tableA = getTable("TableA")
var tempColumn = tableA.createTempColumn("tmp_column", DataTypes.shorttext())

// Get an IBeforeUpgradeUpdateBuilder for TableA.
var ub = tableA.update()

// Set the value of the temporary column to the value of ColumnA.
ub.set(tempColumn, ub.getColumnRef("ColumnA"))

ub.execute()
```

### AfterUpgradeVersionTrigger Execute Method

```
// Get an IUpdateBuilder for TableA.
var ub = getTable("TableA").update().withLogSQL(true)

var q = new Query(Account).withLogSQL(true)
q.compare("ID", Equals, ub.getQuery().getColumnRef("Account"))
var piDesc = PaymentInstrument.Type.TypeInfo.getProperty("Description") as IEntityPropertyInfo

ub.set(piDesc, q, q.getColumnRef(DBFunction.Expr({"tmp_xyz"}))) // tmp_xyz is the DB table column name
```

```
ub.execute()

var tempColumn = getTable("someTable").getColumn("tmp_xyz").drop()
```

## Dropping columns using a version trigger

The upgrader does not drop existing columns in order to prevent data loss. You can implement a version trigger to move the data (not shown in example) and then drop the column by using the `drop()` method of the `IBeforeUpgradeColumn`.

For example:

```
var table = getTable("TableName")
table.getColumn("ColumnName").drop()
```

There is a `dropColumns` method on `IBeforeUpgradeTable` to drop multiple columns in one statement. The `dropColumns` method takes an array of `IBeforeUpgradeColumn` objects.

For example:

```
var table = getTable("TableName")
table.dropColumns(table.getColumn("ColumnName2"), table.getColumn("ColumnName3"));
```

In Oracle, dropping a column usually has little effect on upgrade performance. Dropping a column actually marks the column as unused in the metadata. At a later point, the DBA is responsible for performing the necessary cleanup. You can override this functionality and force columns to be dropped right away.

In SQL Server, dropping a column is performance-intensive because the RDBMS has to do some clean up work.

## Renaming columns using a version trigger

To rename a column use the `rename` function on the column object.

```
override function execute() {
    getTable("TableName").getColumn("ColumnName").rename("NewColumnName")
}
```

## Setting a column value for a specific subtype using a version trigger

To set a column to a specific value for specific subtypes, use the `set` and `compare` methods of an `IBeforeUpgradeTable`. Get the typekey ID for comparison using the `BeforeUpgradeVersionTrigger` method `getTypekeyID`.

```
final var myTable = getTable("tableName")
final var myTypecode = getTypekeyID("typelist name", "typelist code")
final var updateBuilder = myTable.update()

updateBuilder
    .set("myColumn", "some value")
    .compare("subtype", Equals, myTypecode)

updateBuilder.execute()
```

## Adding a subtype to an extension entity using a version trigger

Extension entities are `final="true"` by default, and therefore do not allow adding subtypes.

To add a subtype to an existing extension entity, first edit the entity ETI file to set `final="false"`. Implement an upgrade trigger to add the subtype column and populate each row with an appropriate value.

The following example shows how to add a subtype using a `BeforeUpgradeVersionTrigger`, where all existing rows in a table are of the original root type.

```
final var myTable = getTable("MyExtensionEntityTableName")
```

```
// Create "Subtype" column, which must coincide with changing
// entity definition final = "false" in the MyExtensionEntity ETI file.

final var subTypeColumn = myTable.getColumn("Subtype")
final var myTypecode = getTypeKeyID(entity.MyExtensionEntity)

subTypeColumn.createNonNullableWithInitialValue(myTypecode)
```

## Using getTypeKeyID to get typekey values in a custom version trigger

The `BeforeUpgradeVersionTrigger` class includes a `getTypeKeyID` method with the following signature:

```
protected final Integer getTypeKeyID(IEntityType subtype)
```

**Note:** Protected methods do not appear in the Gosu documentation. Use CTRL + SPACE in Studio to show available methods and properties.

Use the `getTypeKeyID` method to get a typekey ID, either new or old, to insert into a typekey column.

The `getTypeKeyID` method returns the integer ID of the type code in the type list matching the given table name. This method checks both the existing typelist tables and the metadata files to determine what all typekey IDs will be after upgrade. Therefore, the `getTypeKeyID` method works as expected even before a new typekey or typelist table is created during the automatic schema upgrade phase.

This method also works for orphaned typecodes that have not yet been removed from the database. These are typecodes that still exist in the database table but not in the metadata file. You can use the `getTypeKeyID` method for remapping usages of orphaned typecodes.

## Creating tables using a version trigger

To add a new table to the database, define a new entity in the data model. The upgrade creates the table automatically. However, you might want to explicitly create the table in your upgrade trigger if you want the trigger to perform an action on the table such as populating it.

Creating a new table has negligible impact on upgrade performance.

You can create a regular table using the `create` method of `IBeforeUpgradeTable`. The table must first be defined in the data model.

For example:

```
var table = getTable("TableName").create()
```

### Creating temporary tables

You can add a temporary table to the database based on either the current database schema for a table or the data model definition of a table. You can also create a temporary table with a custom definition.

To create a temporary table based on the current table schema in the database, use the `createNewTempTableBasedOnCurrentSchema` method of `IBeforeUpgradeTable`. The table must be associated with an entity and exist in the database. The returned temporary table will contain the columns that this table has in the database currently. The columns may not match those specified in the entity metadata. For example, the metadata might contain a new column that has not yet been created. The `createNewTempTableBasedOnCurrentSchema` method is usually more appropriate than `createNewTempTableBasedOnThis` if you want to copy data from this table into the new temporary table as the columns will match exactly.

For example:

```
var table = getTable("TableName").createNewTempTableBasedOnCurrentSchema()
```

To create a temporary table based on the entity definition of a table in the data model, use the `createNewTempTableBasedOnThis` method of `IBeforeUpgradeTable`. Columns that do not exist in the table are not created on the temporary table, even if the metadata defines such a column. This table may not contain columns that



are going to be renamed. The metadata reflects the new name for the column but does not have an entry for the old name, so it would not be added to the temporary table.

For example:

```
var table = getTable("TableName").createNewTempTableBasedOnThis()
```

To create a temporary table with a custom definition, use the `createAsNewTempTable` method of `IBeforeUpgradeTable`. This method takes a Pair array in which the first object is a `String` defining the column name and the second object is an `IDataType` defining the column data type.

## Renaming tables using a version trigger

To rename a table use the `rename` function on the table object.

```
override function execute() {  
    getTable("extTableName").rename("TableName_EXT")  
}
```

## Deleting rows using a version trigger

To delete rows from a table, use the `delete` method of `IBeforeUpgradeTable` or `IAfterUpgradeTable`. The `delete` method returns a delete builder (`IBeforeUpgradeDeleteBuilder` that provides methods for comparing column data to restrict the rows that are deleted.

In the following example, all rows that have a `columnA` value of 0 are deleted.

```
var table = getTable("SomeTable")  
  
var deleteBuilder = table.delete()  
  
deleteBuilder.Query.compare("columnA", Equals, 0)  
  
deleteBuilder.execute()
```

## Inserting rows using a version trigger

To insert rows of data use the `insert` method of `IBeforeUpgradeTable` or `IAfterUpgradeTable`. The `insert` method returns a builder (`IBeforeUpgradeInsertBuilder` for `IBeforeUpgradeTable`, `IInsertBuilder` for `IAfterUpgradeTable`) for SQL to perform an insert operation.

In the following example, an `IBeforeUpgradeInsertBuilder` is used to add two rows with three columns to table `myTable`. The `IBeforeUpgradeInsertBuilder` includes a description.

```
var myTable = getTable("SomeTable")  
var insertBuilder = myTable.insert().withDescription("A custom insert  
trigger to add two rows.")  
  
insertBuilder  
    .mapColumn("columnA", "value of column A for first row")  
    .mapColumn("columnB", "value of column B for first row")  
    .mapColumn("columnC", "value of column C for first row")  
  
insertBuilder.execute()  
  
// add a second row  
insertBuilder  
    .mapColumn("columnA", "value of column A for second row")  
    .mapColumn("columnB", "value of column B for second row")  
    .mapColumn("columnC", "value of column C for second row")  
  
insertBuilder.execute()
```

## Inserting data selected from another table using a version trigger

To insert data selected from another table use the `insertSelect` method of `IBeforeUpgradeTable` or `IAfterUpgradeTable`. The `insertSelect` method returns a builder (`IBeforeUpgradeInsertSelectBuilder` for

IBeforeUpgradeTable, IInsertSelectBuilder for IAfterUpgradeTable). The builder includes a `mapColumn` method that can be passed explicit values, columns, or a query.

In the following example, the trigger sets `targetTable.column1` to an explicit value. The trigger sets `targetTable.column2` to the value of `sourceTable.sourceColumn`. Because there is no comparison being performed, the trigger will insert a row in the target table for each row in the source table:

```
var sourceTable = getTable("sourceTable")
var targetTable = getTable("targetTable")

var insertSelectBuilder = targetTable.insertSelect(sourceTable)

insertSelectBuilder.mapColumn("column1", "value") // sets a hard-coded value
.insertColumn("column2", sourceTable.getColumn("sourceColumn")) // sets column2 on target table to
                                                                // source table sourceColumn

insertSelectBuilder.execute()
```

In the next example, an existing table, `sourceTable`, is split into two tables, `targetTable1` and `targetTable2`.

```
var sourceTable = getTable("sourceTable")
var targetTable1 = getTable("targetTable1")
var targetTable2 = getTable("targetTable2")

var insertSelectBuilder1 = targetTable1.insertSelect(sourceTable)
var insertSelectBuilder2 = targetTable2.insertSelect(sourceTable)

insertSelectBuilder1.mapColumn("column1", sourceTable.getColumn("sourceColumn1"))
.mapColumn("column2", sourceTable.getColumn("sourceColumn2"))

insertSelectBuilder1.execute()

insertSelectBuilder2.mapColumn("column1", sourceTable.getColumn("sourceColumn3"))
.mapColumn("column2", sourceTable.getColumn("sourceColumn4"))

insertSelectBuilder2.execute()
```

## Updating rows using a before upgrade version trigger

To update rows in a table, use the `update` method of `IBeforeUpgradeTable` or `IAfterUpgradeTable`. This method returns a builder (`IBeforeUpgradeUpdateBuilder` or `IUpdateBuilder`). The builder includes methods to compare data to restrict which rows are updated. In the following example, table `SomeTable` is updated to set `column1` to `SomeValue` for each row where the subtype matches a certain entity type.

Before upgrade trigger example:

```
var table = getTable("SomeTable")

// get IBeforeUpgradeUpdateBuilder
var ub = table.update()

// set column 1 to SomeValue
ub.set("column1", "SomeValue")
// where
.compare("subType", Equals, getTypeKeyID(EntityType))

ub.execute()
```

## Upgrading archived or quote store entities using a version trigger

If you implement archiving in `ClaimCenter` or `PolicyCenter`, and you make custom data model changes to entities stored in the archive, then you can upgrade the retrieved XML of your archived entity using the `IDatamodelUpgrade` plugin.

You can also use this plugin to upgrade entities in the quote store created by a quote-only instance of `PolicyCenter` that handles high volume quote requests.

Simple data model changes do not require a custom trigger. These include:

- Adding a new entity
- Updating denormalization columns
- Adding editable columns such as `updatetime`
- Adding new columns
- Changing the nullability of a column

More complex transformations or those that could result in loss of data require a version trigger. These include:

- changing a datatype (other than just length)
- migrating data from one table or column to another
- dropping a column
- dropping a table
- renaming a column
- renaming a table

The application upgrades an archived entity as the entity is restored.

The `IDataModelChange` interface includes a `getArchivedDocumentUpgradeVersionTrigger` method that returns an `ArchivedDocumentUpgradeVersionTrigger`.

You can define custom `ArchivedDocumentUpgradeVersionTrigger` entities to modify archived XML. The `ArchivedDocumentUpgradeVersionTrigger` is an abstract class that you can extend to create your custom triggers.

Define the constructor of your custom `ArchivedDocumentUpgradeVersionTrigger` to call the constructor of the superclass and pass it a numeric value. For example:

```
construct() {  
    super(171)  
}
```

This numeric value is the extension version to which your trigger applies. If you run the upgrade against a database with a lower extension version, then your custom trigger is called. The current extension version is defined in `modules/configuration/config/extensions/extensions.properties`.

**Note:**

The constructors of `ArchivedDocumentUpgradeVersionTrigger` must be called in the ascending order of their extension number, within the scope of your

`BeforeUpgradeArchivedDocumentChanges` or `AfterUpgradeArchivedDocumentChanges`.

Provide an override definition of the `getDescription` property to return a `String` that describes the actions of your trigger.

Provide an override definition for the `execute` function to define the actions that you want your custom trigger to make on archived XML.

When the upgrade executes your custom trigger, it wraps each XML entity in an `IArchivedEntity` object. Each typekey is wrapped in an `IArchivedTypekey` object. The upgrade operates on a single XML document at a time.

`ArchivedDocumentUpgradeVersionTrigger` provides the following key operations:

- `getArchivedEntitySet(entityName : String)` – returns an `IArchivedEntitySet` object that contains all `IArchivedEntity` objects of the given type in the XML document.

`IArchivedEntitySet` provides the following key methods:

- `rename(newEntityName : String)` – renames all rows in the set to the new name.
- `delete()` – deletes all rows in the set.

- `search(predicate : Predicate<IArchivedEntity>)` – returns a List of IArchivedEntity objects that match the given predicate.
- `create(referenceInfo : String, properties : List<Pair<String, Object>>)` – returns a new IArchivedEntity with the given properties.

IArchivedEntity provides the following key methods:

- `delete()` – deletes just this row.
- `getPropertyValue(propertyName : String)` – returns the value of the property of the given name. If the property value is a reference to another entity, this method returns an IArchivedEntity.
- `move(newEntityName : String)` – moves this to a new entity type of the given name. The type is created if it did not exist. You must add any required properties to the type.
- `updatePropertyValue(propertyName : String, newValue : String)` – updates the property of the given name to the given value.
- `getArchivedTypekeySet(typekeyName : String)` – returns an IArchivedTypekeySet object that contains all IArchivedTypekey objects in the given typelist.
- `getArchivedTypekey(typelistName : String, code : String)` – Returns an IArchivedTypekey representing the typekey.

IArchivedTypekey provides the following key method:

- `delete()` – deletes the typekey from the XML.

More methods are available for IArchivedEntitySet, IArchivedEntity and IArchivedTypekey. See the Gosu documentation for a full listing.

## Incremental upgrade

When ClaimCenter or PolicyCenter archives an entity, it records the current data model version on the entity. The application upgrades an archived entity as the entity is restored. The upgrader executes the necessary archive upgrade triggers incrementally on each archived XML entity, according to the data model version of the archived entity.

An entity archived at one version might not be restored and upgraded until several intermediate data model upgrades have been performed. Therefore, do not delete your custom upgrade triggers.

Consider the following situation. Note that this example is for demonstration purposes only. The version numbers included do not represent actual application versions but are included to explain the incremental upgrade process. Each '+' after a version number indicates a custom data model change.

### Guidewire data model changes

v6.0.0 – Entity does not have column X.

v6.0.1 – Guidewire adds column X to the entity with a default value of 100.

v6.0.2 – Guidewire updates the default value of column X to 200.

### Implementation #1 upgrade path

v6.0.0+ – Start with version 6.0.0 data model with custom changes.

v6.0.0++ – More custom data model changes.

v6.0.0+++ – More custom data model changes.

v6.0.2+ – column X introduced with a default value of 200.

Result of upgrade of an entity archived at v6.0.0+: column X has value 200.

In this situation, version 6.0.1 was skipped in the upgrade path. Therefore, column X is added with the default value of 200 that it has in version 6.0.2.

### Implementation #2 upgrade path

v6.0.0+ – Start with version 6.0.0 data model with custom changes.

v6.0.0++ – More custom data model changes.

v6.0.1+ – column X introduced with a default value of 100.

v6.0.2+ – column X already exists.

Result of upgrade of an entity archived at v6.0.0+: column X has value 100.

In this situation, column X is added in version 6.0.1 with the default value of 100. During the upgrade from version 6.0.1 to version 6.0.2, column X already exists. Therefore, the upgrader does not add the column. A custom trigger would be required to update the value of X from 100 to 200 during the upgrade from version 6.0.1 to 6.0.2.

